University of Warsaw
Faculty of Mathematics, Informatics and Mechanics

Piotr Styczyński
Student no. 386038

An integrated and practical database solution for storage and efficient processing of large genomic datasets

Master's thesis
in COMPUTER SCIENCE

Supervisor:
mgr. XYZ

Warsaw, 2024-12-08

Abstract

Genomic analysis is commonly organised as a sequence of standalone programs connected by files. This model provides specialised tools, but it separates sequence data from relational metadata, repeats parsing and indexing work, and makes query planning unaware of the true cost of alignment. This thesis presents BioSeqDB, a PostgreSQL extension that stores compact biological sequences and executes a minimap2-derived seed–chain–extend pipeline inside database scans. The work is both algorithmic and systems-oriented. It develops alphabet-generic bit-packed sequence views, scalar and SIMD-accelerated minimizer extraction, Vulkan implementations of selected sketching and chaining operations, exact extension based on established dynamic-programming kernels, and optional SneakySnake and GateKeeper prefilters. It also develops persistent database-resident minimizer indexes, bounded sequence ingestion, external sequence providers, a cache with request coordination, and structural generation of the PostgreSQL SQL and function-manager boundary.

BioSeqDB does not expose alignment as an ordinary row-by-row scalar function. Planner support functions introduce custom paths whose execution nodes batch sequences, select an implementation according to data and runtime capabilities, and substitute alignment results into planned expressions. This design connects high-throughput kernels with PostgreSQL joins, filtering, metadata projection, and persistence. The thesis reconstructs the algorithms from their original publications, validates the description against the implemented C++26 and PostgreSQL code, and uses project benchmark evidence to explain which intermediate designs were retained or rejected. The result is a practical account of how a standalone mapper architecture can be transformed into an integrated, query-planned sequence data system without hiding the trade-offs introduced by batching, storage, and heterogeneous execution.

Keywords

sequence alignment,bioinformatics,minimap2,PostgreSQL,databases,genomic data management,SIMD,GPU computing

Thesis domain (Socrates-Erasmus subject area codes)

11.3 Informatics; Computer Science

Subject classification

Applied computing
Life and medical sciences

Bioinformatics

Tytuł pracy w języku polskim

Zintegrowane i praktyczne rozwiązanie bazodanowe do przechowywania i wydajnego przetwarzania dużych zbiorów danych genomicznych

Contents

Introduction 5

1. Biological foundations of sequence analysis 9
   1.1. Biological sequences as information-bearing observations 9
      1.1.1. DNA, RNA and protein alphabets 9
   1.2. From genomes to computational records 10
      1.2.1. Genomes, chromosomes and assemblies 10
      1.2.2. Reads are measurements, not perfect substrings 11
      1.2.3. Biological variation and measurement error 11
   1.3. Reference and query are computational roles 12
   1.4. Why biological sequence comparison requires alignment 13
   1.5. Why place sequence analysis inside a database? 14
   1.6. Transition from biology to algorithms 15

2. Sequence alignment and search strategies 17
   2.1. Alignment as a model of biological correspondence 17
      2.1.1. Global, local and free-end objectives 18
   2.2. Exact dynamic programming 19
      2.2.1. Linear-gap recurrence 19
      2.2.2. Affine and two-piece affine gaps 19
      2.2.3. Cost, banding and stopping criteria 20
   2.3. Seed–chain–extend 21
      2.3.1. Why candidate generation is necessary 21
      2.3.2. Seeds and minimizer sketches 21
      2.3.3. Anchors as points in a coordinate plane 22
      2.3.4. From chains to base-level alignment 23
   2.4. Graph-based sequence representations 23
      2.4.1. Why replace one reference string by a graph? 23
      2.4.2. Dynamic programming on a graph 25
      2.4.3. Relation to BioSeqDB 25
   2.5. Choosing an approach 26

3. Limitations of existing tools 29
   3.1. Why a fast mapper is not yet a database system 29
   3.2. Scientific limitations do not disappear with optimization 29
      3.2.1. Seed selection trades work for evidence 29
      3.2.2. Chaining and extension remain heuristic filters 30
      3.2.3. No processor path dominates every workload 30

3.3. The file-oriented workflow leaves systems work outside the mapper 31
3.4. Why an ordinary SQL function is insufficient 32
3.5. Representation and lifetime are part of performance 32
3.6. A persisted index is a database object, not merely a byte image 33
3.7. Bounded memory and parallel work must be designed together 34
3.8. Ingestion, metadata, and external data are part of the query 34
3.9. Evidence from intermediate results 35
3.10. Comparison of solution classes 36
3.11. Problem statement and design requirements 37
3.12. Research questions and contributions 37
3.13. Scope of the evaluation 38

4. The BioSeqDB alignment pipeline 41
4.1. From an alignment function to a database execution pipeline 41
4.2. Evolution of the architecture 43
4.2.1. The direct minimap2-derived port 43
4.2.2. Typed push stages 43
4.2.3. Containing template growth 44
4.2.4. Database-aware scheduling and accelerator residency 44
4.3. The logical alignment chain 45
4.3.1. Preparation before the stage chain 45
4.3.2. Minimizer and index hand-off 46
4.3.3. Seed-hit lookup 46
4.3.4. Chaining and region generation 46
4.3.5. Candidate reduction, divergence and prefiltering 47
4.3.6. Extension, exact filtering and reclassification 47
4.4. Compile-time algorithm selection and runtime routing 47
4.4.1. The dispatched tuple 47
4.4.2. The CPU and SIMD route 49
4.4.3. The mandatory Vulkan route 49
4.5. Batching, memory and parallel execution 50
4.5.1. The PostgreSQL dispatch buffer 50
4.5.2. Hydration waves and lazy fallback 50
4.5.3. Sharing a prepared target 51
4.6. Description and execution evidence 52
4.6.1. One structural description 52
4.6.2. Stable physical stages 52
4.6.3. Fail-closed validation 53
4.7. Comparison with minimap2 and G3SA 53
4.7.1. The minimap2 control structure 53
4.7.2. What was adopted from G3SA 54
4.7.3. Measured consequence of batching and residency 54
4.8. Final architecture, invariants and remaining boundaries 55

5. Minimizer extraction 59
5.1. From every base to a sparse, comparable sketch 59
5.1.1. Formal definition 60
5.1.2. The minimap2 scalar reference 60
5.2. Eliminating data-dependent rescans 62

5.2.1. Deque, rescan, and two-stack formulations . . . 63
5.2.2. Rolling ntHash . . . 64
5.2.3. Eight logical streams on variable-width SIMD . . . 65
5.3. BioSeqDB's CPU implementation history . . . 66
5.3.1. Why the literal SIMD port failed . . . 66
5.3.2. The V4–V15 branch: useful ideas and misleading names . . . 66
5.4. The current ExactSimd continuation . . . 68
5.4.1. Bounded state across arbitrary fragments . . . 68
5.4.2. The common $k = 15, w = 10$ B8 specialisation . . . 68
5.4.3. PNT1: turning the paper algorithm into a persistent BioSeq scheme . . . 69
5.4.4. Vulkan Phase C: exact extraction on the device . . . 70
5.4.5. Measurements, provenance, and backend choice . . . 73
5.4.6. The resulting selection policy . . . 75
5.4.7. Limitations and open measurements . . . 77
5.4.8. Implementation evidence map . . . 77
5.4.9. Conclusion . . . 78

6. Chaining seed anchors . . . 79

7. Chaining: from local seed hits to candidate mappings . . . 81
7.1. The geometric problem . . . 81
7.1.1. Canonical order is part of the contract . . . 82
7.2. The scalar dynamic program . . . 82
7.2.1. Why the control flow is semantically observable . . . 83
7.3. Recovering chains and competing mappings . . . 85
7.3.1. From competing chains to primary and secondary regions . . . 85
7.4. An augmented-tree RMQ formulation . . . 85
7.4.1. Current RMQ correctness boundary . . . 86
7.5. SIMD chaining on the CPU . . . 86
7.5.1. Structure-of-arrays scoring . . . 87
7.5.2. `exact_visited`: vector arithmetic, scalar decisions . . . 87
7.5.3. `bounded_batch_approximate`: a deliberate speed policy . . . 87
7.5.4. Vector backtracking . . . 88
7.6. From G3SA to the Vulkan implementation . . . 88
7.6.1. What the original G3SA implementation did . . . 88
7.6.2. The live Vulkan kernel . . . 89
7.6.3. Two projection-dependent completion paths . . . 89
7.7. How the implementation evolved . . . 90
7.8. Measured behavior . . . 91
7.8.1. Endpoint qualification . . . 92
7.9. The final execution policy . . . 92
7.10. Summary . . . 93

8. Final alignment and extension . . . 95
8.1. From a chain of seeds to a base-level alignment . . . 95
8.2. Affine-gap dynamic programming . . . 96
8.2.1. Why one gap-open cost is not enough . . . 96
8.2.2. Banding and Z-drop . . . 97
8.3. The minimap2 extension strategy and BioSeqDB's port . . . 97

8.3.1. Three different DP problems, not one matrix . . . 97
8.3.2. Where the port intentionally stops matching minimap2 . . . 98
8.4. KSW2 in BioSeqDB . . . 99
8.4.1. The Suzuki–Kasahara difference representation . . . 99
8.4.2. Endpoint and CIGAR semantics . . . 100
8.4.3. Port-specific memory and hot-loop changes . . . 100
8.5. Block Aligner: adaptive SIMD dynamic programming . . . 101
8.5.1. The original algorithm . . . 101
8.5.2. SIMD inside a block . . . 102
8.5.3. The current Highway port . . . 102
8.5.4. BioSeqDB-specific adaptations . . . 103
8.5.5. A proof that can avoid the DP entirely . . . 103
8.5.6. Why fail-closed attestation is part of the algorithm . . . 104
8.6. Traceback, CIGAR, and exact edit accounting . . . 104
8.7. Runtime dispatch and the batched-KSW2 experiment . . . 105
8.7.1. Enum outside, templates inside . . . 105
8.7.2. What was built for inter-sequence SIMD . . . 105
8.8. GPU extension and gridded traceback . . . 106
8.8.1. The G3SA idea . . . 106
8.8.2. BioSeqDB's Vulkan implementation . . . 106
8.8.3. What was inherited and what was not . . . 108
8.9. Intermediate measurements and current evidence . . . 109
8.9.1. CPU block-size sweep . . . 109
8.9.2. Resident GPU evidence . . . 109
8.9.3. Why the paper and project numbers should not be merged . . . 111
8.10. The final solution and its boundaries . . . 111
8.10.1. What was retained . . . 111
8.10.2. How the final design differs from the starting point . . . 112
8.10.3. Open technical limitations . . . 113

9. Alignment prefiltering . . . 115
9.1. Why filter after chaining but before alignment? . . . 115
9.1.1. The safe rejection domain in BioSeqDB . . . 116
9.1.2. Difference from the minimap2 baseline . . . 117
9.2. SneakySnake: routing through a chip maze . . . 117
9.2.1. Why the obstacle count can filter . . . 118
9.2.2. From the reference implementation to the BioSeqDB port . . . 119
9.2.3. SneakySnake on Vulkan . . . 119
9.3. GateKeeper: shifted masks, amendment, and an exact guard . . . 120
9.3.1. Why masks are amended . . . 120
9.3.2. The exact guard changes the correctness argument . . . 121
9.3.3. Clean-room, alphabet-generic implementation . . . 122
9.4. Dispatch and integration with the query pipeline . . . 123
9.5. Intermediate results and the final routing policy . . . 124
9.5.1. The first SneakySnake port: correct but usually slower . . . 124
9.5.2. Optimised SneakySnake CPU and Vulkan . . . 125
9.5.3. GateKeeper closes the small-window Vulkan gap . . . 126
9.6. Failure modes, limitations, and what the filter does not prove . . . 126
9.7. Conclusion . . . 127

10. Sequence representations and lazy evaluation . . . 129
10.1. One logical sequence, several physical forms . . . 129
10.2. The compile-time compact range . . . 130
10.2.1. Alphabet, container, and range concepts . . . 130
10.2.2. Ranks, symbols, and orientation . . . 131
10.3. Bit layout and reinterpretation . . . 132
10.3.1. Least-significant symbol first . . . 132
10.3.2. The bit-casted view . . . 133
10.4. A C-compatible packed object . . . 134
10.4.1. Standard layout and a zero-copy adapter . . . 134
10.4.2. Contrast with minimap2's split representation . . . 135
10.4.3. Runtime metadata becomes compile-time specialization . . . 135
10.4.4. Dispatch is not SQL code generation . . . 136
10.5. From fragmented storage to one lazy facade . . . 137
10.5.1. Why a packed pointer is not a sufficient interface . . . 137
10.5.2. Ownership is part of the abstraction . . . 139
10.6. Persistent SQL expressions as immutable piece trees . . . 139
10.6.1. Leaves, branches, and transforms . . . 139
10.6.2. Composition and complexity . . . 141
10.6.3. Bound, unbound, and letter values . . . 141
10.7. SQL slicing and logical transforms . . . 142
10.8. Access planning and materialization boundaries . . . 143
10.8.1. Consumers state what they require . . . 143
10.8.2. What "zero copy" does and does not mean . . . 144
10.8.3. Bounded source reads . . . 145
10.9. Evolution of the design . . . 145
10.10Measured consequences . . . 146
10.10.1An early boundary-placement failure . . . 146
10.10.2Small slices perform bounded work . . . 147
10.10.3Deferred composition versus eager intermediates . . . 147
10.11Practical contract for algorithm authors . . . 148

11. PostgreSQL API and custom scans . . . 149
11.1. Alignment as a PostgreSQL execution primitive . . . 149
11.2. The SQL surface is a rendezvous, not a hidden mode switch . . . 150
11.2.1. Two typed entry points . . . 150
11.2.2. Match projections . . . 151
11.3. Planner support, costs, and exact ownership . . . 151
11.3.1. Why the ordinary expression has a high cost . . . 151
11.3.2. A semantic operation cannot be selected by cost alone . . . 152
11.3.3. Relation and join hooks . . . 153
11.4. From CustomPath to CustomScan . . . 153
11.4.1. What crosses the planning boundary . . . 153
11.4.2. Plain EXPLAIN does not allocate the aligner . . . 155
11.5. Batch execution and zero–one–many output . . . 155
11.5.1. The write–process–read state machine . . . 155
11.5.2. Scalar and vectorized access converge before alignment . . . 155
11.5.3. Lazy sequence sources and tuple lifetimes . . . 157
11.5.4. A lower-bound prefilter does not become the SQL predicate . . . 157

11.6. Injecting match fields into planned expressions . . . 157
11.6.1. Mutable typed Const nodes . . . 157
11.6.2. Why joins need PlaceHolderVar . . . 158
11.6.3. Stable identity is more precise than function OID . . . 159
11.7. Join ownership and expression-tree surgery . . . 159
11.7.1. Four plan shapes . . . 159
11.7.2. Raw BioseqBatchJoin . . . 159
11.7.3. Parameterized scans and rescans . . . 160
11.7.4. Removing every executable copy . . . 160
11.7.5. Security and evaluation order . . . 160
11.8. What BioSeqDB borrowed from ParadeDB . . . 161
11.9. Evolution from proof of concept to the final owner protocol . . . 161
11.9.1. Why the true-returning stub was especially dangerous . . . 163
11.9.2. Why per-scan state was not optional . . . 163
11.9.3. Why an owner needs a versioned ABI . . . 163
11.10. SQL and EXPLAIN evidence . . . 163
11.10.1. Intermediate cardinality evidence . . . 163
11.10.2. Current plan invariants . . . 164
11.10.3. Structured pipeline descriptions . . . 165
11.11. GUCs, prepared plans, and resource safety . . . 165
11.11.1. GUC behavior . . . 165
11.11.2. Prepared statements and once-only evaluation . . . 166
11.11.3. Memory contexts and ResourceOwners . . . 166
11.11.4. Parallelism . . . 166
11.12. Limitations and engineering trade-offs . . . 167
11.13. Conclusion . . . 167

12. Physical sequence storage in PostgreSQL . . . 169
12.1. Storage is part of the execution model . . . 169
12.2. The Citus-derived column store . . . 170
12.2.1. Lineage and retained ideas . . . 170
12.2.2. Pages, logical offsets, and reservations . . . 170
12.2.3. Stripes, chunk groups, and column streams . . . 171
12.2.4. Skip metadata and generated catalogs . . . 172
12.2.5. Projection, pruning, and reconstruction . . . 173
12.3. Sequence payload indirection . . . 173
12.3.1. Why a locator is necessary . . . 173
12.3.2. The version-3 locator . . . 173
12.3.3. Length buckets, shards, and fragment rows . . . 174
12.3.4. Concurrency, shard rollover, and lifecycle . . . 174
12.3.5. Lazy fragment acquisition . . . 175
12.4. Caches with different scopes . . . 176
12.4.1. Backend-local decompression cache . . . 176
12.4.2. The separate query-result cache extension . . . 176
12.4.3. Claim state and request locking . . . 177
12.4.4. TTL, publication, and hybrid storage . . . 177
12.5. Vectorized processing of scalar metadata . . . 178
12.5.1. From chunk group to vector slot . . . 179
12.5.2. Planner eligibility and safe fallback . . . 179

12.6. What the final storage design changes . . . 179
12.7. Implementation reference map . . . 180

13. Persistent sequence indexes . . . 183
13.1. An index is derived database state . . . 183
13.1.1. The compile-time registry and SQL policy . . . 183
13.2. Two logical minimizer organizations . . . 185
13.2.1. Minimizers per fragment . . . 185
13.2.2. Minimizers per sequence . . . 185
13.2.3. Why both organizations remain useful . . . 186
13.3. Bounded construction and publication . . . 186
13.3.1. The failure of an apparently simple global sort . . . 186
13.3.2. Synchronous and identity-only asynchronous work . . . 187
13.4. Physical persistence in PostgreSQL . . . 188
13.4.1. Legacy sidecar relations . . . 188
13.4.2. Immutable generations . . . 188
13.4.3. The optional mapped-file provider . . . 189
13.5. Formats 5 and 6: from complete images to selective streams . . . 190
13.5.1. Shape-aware Format 5 . . . 190
13.5.2. Hash-major Format 6 . . . 191
13.5.3. A rejected per-query occurrence merge . . . 191
13.6. Compatibility is part of correctness . . . 192
13.7. Selective cold loading and memory accounting . . . 192
13.7.1. The useful numerator is admitted memory . . . 192
13.7.2. Backend-local residency caches . . . 194
13.8. Runtime selection . . . 194
13.9. Comparison with minimap2's `.mmi` index . . . 195
13.10Design evolution . . . 196
13.10.1What the current solution actually selects . . . 196
13.11Performance evidence . . . 198
13.11.1Early evidence: persistence could help, but did not solve building . . . 198
13.11.2Intermediate results must retain their persistence label . . . 198
13.11.3Current mapped-provider evidence . . . 198
13.12Operational consequences and limitations . . . 199
13.13Conclusions . . . 200

14. Reliable sequence ingestion . . . 203
14.1. Ingestion is part of the database contract . . . 203
14.2. Layered reader architecture . . . 204
14.2.1. Source selection and format selection . . . 204
14.2.2. Parallel gzip and the librapidarchive experiment . . . 205
14.3. Incremental parsing and direct compact encoding . . . 206
14.3.1. FASTA framing . . . 206
14.3.2. FASTQ framing . . . 207
14.3.3. Alphabet and storage-width dispatch . . . 207
14.4. Bounded delivery and backpressure . . . 208
14.4.1. General batches . . . 208
14.4.2. The PostgreSQL streaming contract . . . 208
14.4.3. Optional parser/writer overlap . . . 209

14.5. Header dialects and typed relational mapping . . . 209
14.5.1. Mapping is resolved once . . . 210
14.6. SQL integration, failure propagation, and transaction semantics . . . 211
14.6.1. Error domains . . . 211
14.6.2. Comparison with a conventional mapper input path . . . 212
14.7. The standalone writer is a different contract . . . 212
14.8. Measured evolution and current limits . . . 213
14.9. Conclusion . . . 214

15. External sequence providers . . . 217
15.1. Remote biological databases as relations . . . 217
15.1.1. What each relation means . . . 218
15.2. Evolution from generated calls to a streaming table provider . . . 218
15.3. Generated clients and handwritten adapters . . . 220
15.4. Planning and executing a remote table scan . . . 221
15.4.1. A table access method with no local tuples . . . 221
15.4.2. Predicate lowering and provider selection . . . 221
15.5. The NCBI streaming data plane . . . 222
15.5.1. Pulling bytes without buffering the body . . . 223
15.5.2. Backpressure and PostgreSQL cancellation . . . 223
15.6. From remote letters to a compact PostgreSQL sequence . . . 224
15.7. Authentication, transport policy and errors . . . 224
15.7.1. Authentication and configuration . . . 224
15.7.2. Rate limits and failure representation . . . 225
15.8. Measured behaviour and limits of the evidence . . . 225
15.9. Final architecture and remaining work . . . 226
15.10Implementation reference map . . . 227

16. C-driven SQL code generation . . . 229
16.1. Why the SQL boundary became generated code . . . 229
16.2. Architecture and authority flow . . . 230
16.3. The C++26 annotation model . . . 231
16.3.1. A typed vendor namespace . . . 231
16.3.2. Documentation and checked SQL . . . 232
16.4. Structural parsing from the real build . . . 233
16.4.1. Why text scanning is insufficient . . . 233
16.4.2. PREPARE, module closure, and PCM provenance . . . 233
16.5. Collection and normalized identities . . . 234
16.5.1. Stable and installed identities . . . 234
16.5.2. Dependencies and deterministic ordering . . . 235
16.6. Rendering SQL and PostgreSQL C++ boundaries . . . 236
16.6.1. SQL and module facets . . . 236
16.6.2. Generated fmgr entrypoints . . . 236
16.6.3. GUC registration . . . 237
16.7. Physical order, manifests, and provenance . . . 237
16.7.1. A compatibility ledger which owns no SQL . . . 237
16.7.2. Manifests as joinable evidence . . . 238
16.8. PostgreSQL errors across a C++ boundary . . . 238
16.8.1. Why a wrapper catch is not sufficient . . . 238

16.8.2. Closed-world CFG and lifetime analysis . . . 239
16.9. Deterministic generation and the golden workflow . . . 239
16.10 Migration outcome and evidence . . . 240
16.10.1 What changed from the original boundary . . . 240
16.10.2 Checked artifact and test evidence . . . 242
16.11 Lessons and remaining limitations . . . 242

17. Conclusions . . . 245
17.1. What was built . . . 245
17.2. From the original pipeline to the final architecture . . . 246
17.3. Answers to the research questions . . . 246
17.4. Limitations . . . 248
17.5. Future work . . . 249
17.6. Closing perspective . . . 249

Introduction

Sequencing instruments turn biological material into collections of strings. The useful interpretation of those strings rarely exists in isolation. A read has an instrument and sample provenance. A reference interval belongs to an assembly and annotation release. An alignment has a score, a strand, a position, an edit path, and a relationship to experimental metadata. Conventional workflows keep the strings in FASTA, FASTQ, SAM, BAM, or mapper-specific index files while keeping much of the surrounding information in tables or ad-hoc scripts. Each tool sees only the portion of the workflow represented by its command-line arguments.

Relational database systems solve a different part of the problem. PostgreSQL provides durable storage, transactions, typed values, indexing, joins, extensible operators, and a cost-based planner. It does not, by itself, provide the representations or execution kernels required for billions of packed symbols and computationally expensive sequence searches. A naive user-defined function is not sufficient. Evaluating one alignment function independently for every row prevents batching, hides accelerator opportunities, and gives the planner little information about a predicate that can dominate the query.

BioSeqDB investigates the boundary between these two systems. It is implemented as a PostgreSQL extension whose non-database core is written in C++26. Biological symbols are represented by generic compact ranges rather than by a DNA-only byte string. The alignment path is derived from minimap2: representative k-mers are extracted as minimizers, matching minimizers become anchors, compatible anchors are chained, and exact dynamic programming extends the selected regions. The port is not a line-for-line translation. The implementation introduces template-selected alphabets and layouts, Highway SIMD kernels, task-based parallel execution, Vulkan compute paths, persistent PostgreSQL-backed indexes, and prefilters for candidate regions. These changes must preserve the semantics that make minimap2 useful while operating on data that may be lazy, fragmented, cached, or obtained from an external provider.

The database boundary is equally important. An SQL expression such as `mmsearch(a, b)` has a conventional function definition so that PostgreSQL can type and plan it. Its ordinary implementation is deliberately expensive. Planner support code can instead construct a cheaper custom path. The resulting custom scan batches input, invokes the alignment pipeline, and makes its results available to planned expressions through injected PostgreSQL nodes. This is what permits alignment to participate in filters, projections, and joins while retaining the execution structure required by CPU vector units and GPUs.

The central research question is therefore not whether an existing aligner can be called from SQL. It is whether the aligner's data and control flow can become a database execution primitive without losing algorithmic fidelity, resource bounds, or the ability to exploit modern processors. The investigation is organised around four supporting questions:

1. Which parts of seed–chain–extend can be expressed over a common compact sequence abstraction without eager conversion?

2. Which scalar operations benefit from SIMD or GPU execution, and which overheads move the break-even point?

3. How can PostgreSQL plan and execute a set-valued, batch-oriented search while presenting an ordinary typed SQL interface?

4. How should sequences, indexes, ingestion, external data, caches, and generated boundary code fit around the kernels so that the resulting system remains inspectable and reliable?

This thesis answers these questions through implementation and measurement. Historical designs are included when they explain a constraint or a rejected optimisation. They are not presented as simultaneous production architectures. Where project documents disagree, the implementation and the most recent dated evidence take precedence. Numerical claims are tied to a named benchmark report and its configuration. Comparisons with minimap2, SIMD-Minimizers, G3SA, Block Aligner, SneakySnake, GateKeeper, ParadeDB, and Citus Columnar distinguish inherited mechanisms from BioSeqDB-specific changes.

The main contributions developed in the following chapters are:

1. an alphabet-generic sequence representation that supports packed, fragmented, PostgreSQL-backed, and lazily composed data through a common range contract;

2. a staged minimap2-derived mapper with scalar, SIMD, and Vulkan components, including explicit dispatch and evidence for intermediate optimisation decisions;

3. optional GPU-capable SneakySnake and GateKeeper candidate filters integrated without changing the meaning of an accepted alignment;

4. persistent sequence and minimizer-index storage derived from a columnar PostgreSQL access method, with bounded loading, caching, and selective cold-index reads;

5. a planner and custom-scan interface that connects SQL expressions and joins to batch-oriented alignment; and

6. structural generation and validation of SQL installation definitions and PostgreSQL function-manager boundary declarations from annotated C++ sources.

The document follows the same progression as a query. It first establishes the biological and algorithmic context, then states the shortcomings that motivate an integrated system. It follows data through minimizer extraction, chaining, extension, and prefiltering. It then moves outward through sequence abstractions, PostgreSQL planning, physical storage, persistent indexes, ingestion, external providers, and generated SQL boundary code.

- Chapter 1 introduces biological sequences; sequencing data; variation; and the computational questions that motivate database-resident sequence analysis.

- Chapter 2 develops exact dynamic programming; seed-and-extend mapping; and graph-based approaches so that the later engineering chapters share a precise algorithmic vocabulary.

- Chapter 3 separates algorithmic bottlenecks from data-management and integration problems; then states the design goals addressed by BioSeqDB.
- Chapter 4 presents the complete processing path; its stage contracts; its CPU and accelerator routes; and the evolution from the imported baseline to the final architecture.
- Chapter 5 explains minimizer sketching; rolling hashing; SIMD and Vulkan implementations; comparison with minimap2 and SIMD-Minimizers; and the optimisations selected for the final system.
- Chapter 6 derives the chaining objective; compares scalar; RMQ; SIMD; and GPU approaches; and documents the policy used to retain mapping quality while accelerating endpoint selection.
- Chapter 8 describes affine-gap alignment; KSW2 and block-based SIMD extension; traceback; accelerator integration; and the final runtime selection rules.
- Chapter 9 reconstructs SneakySnake and GateKeeper; explains their filtering guarantees and GPU mappings; and records how BioSeqDB deploys them before expensive alignment.
- Chapter 10 develops compact_sequence_range; bit-level views; fragmented and lazy sequences; PostgreSQL-backed adapters; and generated alphabet dispatch.
- Chapter 11 follows an mmsearch expression through planner support; custom paths and plans; node injection; batched execution; joins; and per-row result substitution.
- Chapter 12 explains the columnar-derived storage layout; fragment metadata; cache and request locking; and vectorised metadata processing.
- Chapter 13 describes index construction; storage; compatibility; selective cold loading; memory accounting; update semantics; and differences from standalone minimap2 indexes.
- Chapter 14 presents the bounded IO pipeline for FASTA and FASTQ data; compression; header parsing; typed PostgreSQL writes; failure handling; and measured throughput.
- Chapter 15 explains generated API clients and the provider abstraction that exposes remote sequence databases as PostgreSQL relations without weakening the core data plane.
- Chapter 16 documents structural source analysis; annotations; dependency graphs; SQL and fmgr generation; safety ledgers; reproducibility controls; and migration from handwritten boundary code.
- Chapter 17 answers the research questions; contrasts the final system with its minimap2-derived starting point; and identifies the principal limitations and next engineering steps.

Chapter 1

Biological foundations of sequence analysis

1.1. Biological sequences as information-bearing observations

The data processed by Bioseq originate in molecules, but an alignment algorithm does not receive a molecule. It receives an ordered sequence of symbols. This distinction is elementary and important. A physical DNA molecule has chemical structure, direction, complementary strands and a history inside a cell. A database value is a finite string together with metadata that describes how the string was obtained and how its symbols should be interpreted. A reliable computational system must preserve enough of that interpretation to distinguish, for example, a nucleotide sequence from a protein sequence and an unknown base from an ordinary base.

1.1.1. DNA, RNA and protein alphabets

DNA is a polymer whose monomers contain one of four canonical bases: adenine, cytosine, guanine and thymine. Their conventional one-letter symbols form the alphabet

$$\Sigma_{\mathrm{DNA4}} = \{\mathtt{A}, \mathtt{C}, \mathtt{G}, \mathtt{T}\}.$$

The order of symbols is normally written from the $5'$ end of a strand to its $3'$ end. DNA in a chromosome is usually double stranded. Adenine pairs with thymine, while cytosine pairs with guanine. Consequently, the sequence `ACGTT` on one strand determines `AACGT` on the opposite strand when that strand is also written in the $5'$-to-$3'$ direction. The second string is the reverse complement of the first. A mapper cannot assume that an observed read has the same orientation as the stored reference. It commonly has to consider both the forward sequence and its reverse complement.

Four symbols are sufficient for an unambiguous DNA string, but experimental data are not always unambiguous. The symbol `N` conventionally means that the nucleotide at a position is unknown or was not resolved. Other IUPAC codes describe sets of possible bases, such as a purine at a position. These are uncertainty symbols, not additional chemical bases. Treating `N` as if it were an ordinary fifth base would change the meaning of exact matches, hashes and alignment scores. A sequence type therefore needs an explicit alphabet and an explicit policy for symbols outside that alphabet. The current Bioseq sequence core follows this principle at the software level: its compact ranges are parameterized by an alphabet, and current code paths instantiate both four-symbol and five-symbol DNA representations. The later sequence-abstraction chapter explains how those symbols are packed and viewed.

RNA uses adenine, cytosine, guanine and uracil, so a basic RNA alphabet replaces `T` with `U`. Many RNA molecules are transcribed from DNA. Protein-coding transcripts can then

be translated in groups of three nucleotides, called codons, into amino acids. This flow is often summarized as DNA to RNA to protein, although it is not a claim that all RNA is translated. Ribosomal, transfer, regulatory and many other RNAs are functional without becoming proteins.[1]

A protein sequence is written over the twenty common amino-acid symbols. Protein comparison differs from nucleotide comparison in two ways that matter computationally. First, the alphabet is much larger. Second, substitutions are not naturally divided into only "same" and "different". Replacing an amino acid with one having similar chemical properties may be more plausible than a replacement with a very different residue. Protein aligners therefore often use empirical substitution matrices. Nucleotide aligners can also use a matrix, but a simpler match, mismatch and gap model is often appropriate.

Sequence class	Core symbols	Computational consequences
DNA	`A`, `C`, `G`, `T`, with optional ambiguity codes	Reverse-complement orientation is biologically meaningful. Unknown and degenerate symbols need explicit matching and hashing policies.
RNA	`A`, `C`, `G`, `U`, with optional ambiguity codes	Transcript sequences may cross splice junctions. A system must state whether it preserves `U` or normalizes it to `T`.
Protein	Twenty common amino-acid symbols, plus data-format conventions for ambiguity and exceptional residues	A larger alphabet requires more bits per symbol. Substitution scores normally encode biochemical or evolutionary similarity rather than equality alone.

Table 1.1: Biological alphabets and the distinctions that a sequence-processing system must not erase. The table describes biological data in general. It does not assert that every alphabet or ambiguity convention is exposed by every Bioseq SQL path.

The alphabet is thus part of the type of a sequence, not a decorative label. It determines the set of valid symbols, the minimum number of bits needed for storage, whether a complement operation exists and how comparison should behave. This observation connects basic biology to a central systems decision in Bioseq: algorithms should consume a common sequence abstraction while remaining explicit about the alphabet represented by that abstraction.

1.2. From genomes to computational records

1.2.1. Genomes, chromosomes and assemblies

A genome is the genetic material carried by an organism or biological sample. Depending on the organism and the question, it may be distributed among one or more chromosomes, plasmids or organellar molecules. The genome of a diploid individual contains homologous chromosome copies that need not be identical. A microbial sample can contain many species and strains at once. Consequently, "the genome" is not always one linear string.

Sequencing and assembly software nevertheless have to produce finite records. A contig is a contiguous consensus sequence assembled from overlapping evidence. Scaffolds can order

[1] F. Crick, "Central dogma of molecular biology", Nature 227 (1970), pp. 561–563, https://doi.org/10.1038/227561a0.

or orient contigs while retaining unresolved gaps. A reference assembly is a selected set of such sequences used as a coordinate system and comparison target. It is extremely useful, but it is not the unique biological truth for every sample. Regions absent from the reference, alternative haplotypes and repeated segments can all make a reference-centered explanation incomplete.

This distinction affects the interpretation of an alignment. Mapping a read to a reference says that the chosen reference interval is a plausible source or homologous locus under a scoring model. It does not prove that the sample is identical to the reference at that interval. It also does not prove that the reported location is unique when multiple loci are sufficiently similar. The alignment result must therefore retain coordinates, orientation, score and, when available, information about competing placements.

1.2.2. Reads are measurements, not perfect substrings

A sequencing instrument observes molecules and emits base calls. A base call is the instrument's interpretation of a physical signal. The calls for one observed molecule or molecule fragment form a read. Some protocols produce relatively short reads, others produce long reads, and paired-end protocols retain a relationship between two reads from opposite ends of a fragment. The categories are operational rather than absolute. What matters to an aligner is the joint profile of read length, error, orientation and expected biological divergence.

FASTA stores named sequence records. FASTQ additionally associates each called symbol with a quality character. The widely used Phred convention expresses an estimated error probability p as

$$Q = -10 \log_{10} p.$$

The FASTQ format and the historical variation in its quality encodings are documented by Cock et al.[2] A quality score is useful evidence, but it is neither a guarantee nor a biological property of the base itself. Calibration can vary by instrument, chemistry and processing pipeline.

Sequencing errors are commonly described as substitutions, insertions and deletions. A substitution reports one symbol in place of another. An insertion adds a symbol relative to the underlying molecule or comparison target. A deletion omits one. Long-read and short-read technologies can have different mixtures and contexts of these errors, which is why one universal scoring preset is insufficient for all data.[3]

1.2.3. Biological variation and measurement error

Two homologous biological sequences can differ even if both are measured perfectly. A single-nucleotide variant changes one nucleotide. An insertion or deletion changes sequence length. Larger structural variants can duplicate, invert, move or remove longer intervals. Recombination and horizontal transfer can join segments with different histories. In transcript data, splicing removes introns and joins exons, so a mature RNA or cDNA read can correspond to multiple separated intervals in a genomic reference.

[2] P. J. A. Cock, C. J. Fields, N. Goto, M. L. Heuer and P. M. Rice, "The Sanger FASTQ file format for sequences with quality scores, and the Solexa/Illumina FASTQ variants", Nucleic Acids Research 38(6), 2010, pp. 1767–1771, https://doi.org/10.1093/nar/gkp1137.

[3] S. Goodwin, J. D. McPherson and W. R. McCombie, "Coming of age: ten years of next-generation sequencing technologies", Nature Reviews Genetics 17 (2016), pp. 333–351, https://doi.org/10.1038/nrg.2016.49; G. A. Logsdon, M. R. Vollger and E. E. Eichler, "Long-read human genome sequencing and its applications", Nature Reviews Genetics 21 (2020), pp. 597–614, https://doi.org/10.1038/s41576-020-0236-x.

The algorithm sees many of these events through the same small vocabulary: matches, substitutions and gaps. A mismatch may represent a true variant or a base-calling error. A gap may represent a biological insertion or deletion, a sequencing error, or a difference between the reference assembly and the sample. A split alignment may represent a structural variant, a splice junction, an assembly boundary or an experimental chimera. Interpretation therefore requires metadata and context in addition to the aligned strings.

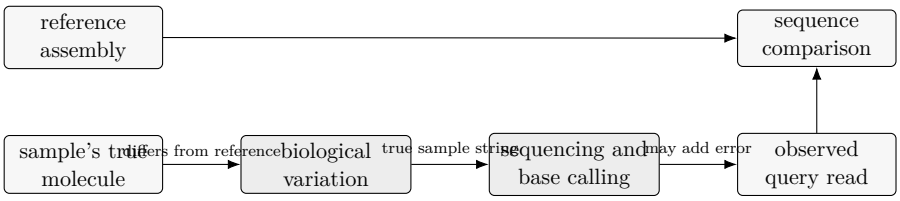


Figure 1.1: Two sources of disagreement meet at the same comparison. Biological variation separates the sample molecule from the reference, while sequencing and base calling can separate the observed read from that molecule. Alignment locates and describes disagreement. Metadata and experimental context are needed to interpret its cause.

Figure 1.1 explains why exact string equality is too strict and why alignment alone is not a variant caller. The comparison stage must tolerate plausible differences. A later stage must then combine the alignment with coverage, base qualities, sample relationships and other evidence before assigning a biological interpretation.

Observed pattern	Possible biological explanation	Possible observational or representation explanation	Alignment-level representation
One symbol differs	single-nucleotide variant or divergence between homologues	base-calling error, consensus error or unresolved ambiguity	mismatch or scored substitution
Extra or missing symbols	insertion or deletion in the sample lineage	insertion/deletion sequencing error or assembly gap	one or more gap operations
Long discontinuity	structural variant, rearrangement or splice junction	chimeric read, reference discontinuity or assembly error	long gap or split alignment
Several plausible locations	repeated or duplicated biological sequence	incomplete reference or query too short to disambiguate	competing mappings with similar scores

Table 1.2: The same alignment pattern can have several causes. The table is a reminder that similarity search generates evidence. It does not by itself determine the biological event.

1.3. Reference and query are computational roles

Throughout this thesis, the reference or target is the sequence collection that is prepared for search, often by building an index. The query is the sequence presented for comparison against that collection. These are roles in an operation, not fixed classes of molecule. A chromosome assembly can be a reference when reads are mapped to it and a query when it is compared with another assembly. A read set can be searched against a reference genome, against another read set to find overlaps, or against a database of known sequences. Bioseq's SQL-facing similarity predicate follows the same operational distinction by accepting a target sequence value and a query sequence value.

The role assignment changes the engineering cost. Work spent once on a stable reference index can be amortized over many queries. If both inputs change for every comparison, as in an all-versus-all overlap workload, index construction and reuse have a different balance. A database makes this reuse question explicit because sequence values, indexes and query plans have managed lifetimes rather than being unrelated files produced by a script.

Several coordinate conventions follow from the reference/query distinction. Positions are usually reported in the coordinate system of the stored target. The query can align in forward or reverse-complement orientation. Local alignment can consume only a portion of either input. Gaps advance one sequence without advancing the other. Split mappings can produce several target intervals for one query. These facts explain why an alignment result is richer than a Boolean answer and why downstream columns such as start, end, orientation, score and edit description are useful.

1.4. Why biological sequence comparison requires alignment

Exact equality answers whether two symbol strings are identical. Most biological questions ask something different: whether the strings can be explained as related sequences after a plausible number and arrangement of changes. A useful comparison must therefore answer three connected questions:

1. Where can the query be placed relative to the target?
2. How do their symbols correspond, including substitutions and gaps?
3. How credible is that correspondence compared with alternatives?

An alignment makes the correspondence explicit by placing symbols in columns and introducing gaps where one sequence advances without the other. A scoring model rewards compatible symbol pairs and penalizes substitutions and gaps. Global alignment attempts to explain both inputs end to end. Local alignment searches for high-scoring similar regions. Semi-global and free-end-gap variants model cases in which one input is expected to be contained within another. Spliced and split alignment add structure for discontinuous relationships.

The fully general dynamic-programming solution examines a matrix whose axes are the two sequences. That formulation is precise, but its work grows with the product of their lengths. Genome-scale tools therefore avoid evaluating most matrix cells. Modern mappers commonly identify short exact or near-exact anchors, connect mutually consistent anchors into chains, and perform detailed alignment only around promising chains. Minimap2 is a prominent example of this seed–chain–align organization.[4] Bioseq's current alignment pipeline is minimap2-based, while replacing or extending individual stages with CPU SIMD and GPU-oriented implementations. The following chapter introduces these algorithmic families without assuming that every method uses the same seed representation or the same final alignment kernel.

The biological discussion above determines what those algorithms must preserve. They must handle strand orientation, tolerate both substitutions and length-changing differences, avoid being dominated by repeated sequence, and retain coordinates that can be joined with annotations. They must also avoid assigning biological meaning that the sequence comparison alone cannot support.

[4] H. Li, "Minimap2: pairwise alignment for nucleotide sequences", Bioinformatics 34(18), 2018, pp. 3094–3100, https://doi.org/10.1093/bioinformatics/bty191.

1.5. Why place sequence analysis inside a database?

Sequence analysis is often presented as a file-to-file transformation. A FASTA or FASTQ file enters a command-line program and an alignment file leaves it. That model is effective for a single tool invocation, but a real study also contains sample identifiers, collection conditions, organism labels, cohort membership, reference versions, quality-control decisions and derived annotations. When each stage writes another loosely coupled file, the relationship between a sequence and its provenance is maintained by file names, scripts and conventions outside the data itself.

A database-resident design changes the unit of work. Instead of asking an aligner to process every record in a file, a relational query can first select the relevant samples, join them with the chosen reference or query set, invoke similarity search on the resulting sequence pairs, and aggregate or filter the results. A schematic workload is:

$$\text{metadata selection} \longrightarrow \text{candidate sequence pairs} \longrightarrow \text{alignment} \longrightarrow \text{relational result}.$$

This organization is valuable for five reasons.

1. Metadata and sequence remain addressable together. Alignment results can be related to sample, experiment and reference records without reconstructing those relationships from external file naming schemes.

2. Cheap predicates can reduce expensive work. Filtering by sample attributes or indexed identifiers before alignment can reduce the number of candidate comparisons. This is a planning opportunity, not a guarantee that every SQL formulation will be optimal.

3. Indexes acquire managed lifetimes. A sequence index can be associated with stored data and reused across queries. Its construction, invalidation and selection become part of the database system rather than an implicit side effect of a script.

4. Results compose with ordinary relational operations. Coordinates, scores and edit summaries can feed joins, grouping and aggregation. A biological workflow need not cross a process boundary after every comparison step.

5. Provenance and consistency have a common home. Transactions and typed columns can keep a sequence value, its metadata and derived state synchronized. External resources and large data transfers still exist, but their relationship to stored records can be made explicit.

Bioseq implements this idea as a PostgreSQL extension rather than as a standalone aligner wrapped by SQL text. Current source exposes alphabet-aware sequence values and an `mmsearch` similarity predicate. Integration tests exercise searches over stored columns, constants, slices and joins. PostgreSQL custom scans then provide execution paths that can batch sequence work and return relational tuples. Later chapters explain the planner rewrites, storage layout, index persistence, caching and vectorized aggregates that make this interface more than a scalar function call.

Database residence does not imply that all computation is free or that no data ever moves. Alignment remains expensive. GPU execution can require transfer to device memory. External sequence providers still require network and I/O boundaries. Large intermediate results can still dominate a query. The useful claim is narrower: sequence storage, candidate selection, alignment execution and downstream relational processing can participate in one managed system. That integration creates opportunities for reuse, batching and provenance that are difficult to obtain from a collection of unrelated file transformations.

1.6. Transition from biology to algorithms

The biological problem can now be stated in computational terms. The input is not simply "DNA". It is an alphabet-tagged reference or target, an alphabet-compatible query, orientation and coordinate conventions, and metadata describing the origin of both values. Differences can arise from evolution, within-population variation, transcript structure, sequencing error or incomplete representation. The required output is therefore not only a similarity decision. It is a defensible correspondence between positions, accompanied by enough score and coordinate information for later interpretation.

The next chapter develops the algorithms that approximate this correspondence at scale. It begins with exact dynamic programming to establish what an alignment optimizes. It then explains seed-and-extend methods, minimizer-based seeding and graph-based alternatives such as de Bruijn graphs. That progression connects the biological sources of disagreement described here to the design of Bioseq's processing pipeline: minimizer extraction proposes candidate anchors, chaining assembles them into coherent placements, prefilters reject unlikely pairs, and the final alignment step produces a base-level account of matches, substitutions and gaps.

Chapter 2

Sequence alignment and search strategies

2.1. Alignment as a model of biological correspondence

Sequence search and sequence alignment are related, but they are not the same operation. A search procedure asks where useful correspondences might exist. An alignment describes one such correspondence symbol by symbol. This distinction is the organizing idea behind modern mappers. Exhaustive dynamic programming can solve a precisely stated pairwise problem, but it is too expensive to apply to every possible reference interval. Seed–chain–extend methods first generate a small set of plausible intervals, then apply dynamic programming only there. Graph methods change the object against which the query is compared. They can represent several reference alternatives at once, but they do not remove the need to define alignment scores and paths.

Let $X = x_1 x_2 \dots x_n$ and $Y = y_1 y_2 \dots y_m$ be strings over an alphabet Σ. A pairwise alignment is a sequence of columns

$$A = ((a_1, b_1), \dots, (a_L, b_L)), \qquad (a_\ell, b_\ell) \in (\Sigma \cup \{-\})^2 \setminus \{(-,-)\},$$

such that deleting gaps from $a_1 \dots a_L$ recovers X, and deleting gaps from $b_1 \dots b_L$ recovers Y. A column containing two symbols is a match or substitution. A symbol aligned to a gap represents an insertion or deletion relative to the other sequence. The alignment is therefore not merely a visual formatting of two strings. It is a hypothesis about which symbols descend from corresponding positions and which differences are best explained as substitutions or gaps.

A scoring model turns that hypothesis into an optimization problem. Let $s(a, b)$ denote a substitution score and let $g(\ell)$ be the cost of a gap run of length ℓ. The score of an alignment can be written as

$$S(A) = \sum_{\text{symbol–symbol columns } \ell} s(a_\ell, b_\ell) - \sum_{\text{gap runs } r} g(|r|).$$

This chapter uses the affine convention

$$g(\ell) = q + \ell e, \qquad \ell \geq 1,$$

where q is a gap-open cost and e is a per-symbol extension cost. Some literature and software instead writes $q + (\ell - 1)e$. Neither notation is intrinsically preferable, but parameter values cannot be transferred between the conventions without adjustment. BioSeqDB follows the contracts of the selected alignment kernel, so later chapters state the convention whenever scores are compared.

The affine form captures an important biological prior. One insertion of ten bases is normally more plausible than ten independent one-base insertion events. Charging q once and e for each base makes a long run cheaper than repeatedly opening new gaps. Gotoh showed how to compute this model with a constant number of states per dynamic-programming cell rather than scanning all possible gap lengths.[1]

2.1.1. Global, local and free-end objectives

The recurrence alone does not determine the alignment problem. Boundary conditions and traceback endpoints determine which parts of the two strings must participate. Three objectives are especially important:

Global alignment aligns both strings from end to end. It is suitable when the inputs are expected to describe the same complete object. The classical Needleman–Wunsch formulation fixes the traceback at opposite corners of the dynamic-programming matrix.[2]

Local alignment chooses the best pair of substrings. Scores are allowed to restart at zero, and traceback begins at the best cell and ends where the score reaches zero. Smith–Waterman gives the standard exact recurrence.[3]

Semi-global and overlap alignment require selected ends but make other leading or trailing gaps free. A read-to-reference fit, for example, may require the entire read while allowing the selected reference interval to have unaligned flanks. Overlap alignment instead looks for a suffix of one sequence matching a prefix of the other.

These names are useful shorthand, but software APIs often provide finer combinations. Ends-free, extension, clipping and local-extension modes differ in which matrix borders are free and which endpoint is fixed. The current figure reconstruction specification, DOC0367, records eight such boundary patterns. The unifying rule is simple: a fixed corner or penalized border requires the alignment to account for that end, while a free border allows traceback to start or stop there without paying for the omitted flank.

This observation prevents a common conceptual error in mapper descriptions. Minimap2 and BioSeqDB do not solve one unconstrained local alignment over an entire chromosome. Anchors determine candidate regions. Gaps between anchors are mandatory global fills, while the two outer portions are extension problems with stopping rules such as Z-drop or X-drop. The final record is assembled from several related dynamic-programming problems.

[1] O. Gotoh, “An improved algorithm for matching biological sequences”, Journal of Molecular Biology 162 (1982), pp. 705–708, https://doi.org/10.1016/0022-2836(82)90398-9.

[2] S. B. Needleman and C. D. Wunsch, “A general method applicable to the search for similarities in the amino acid sequence of two proteins”, Journal of Molecular Biology 48 (1970), pp. 443–453, https://doi.org/10.1016/0022-2836(70)90057-4.

[3] T. F. Smith and M. S. Waterman, “Identification of common molecular subsequences”, Journal of Molecular Biology 147 (1981), pp. 195–197, https://doi.org/10.1016/0022-2836(81)90087-5.

Objective	Initialization	Traceback endpoints	Typical interpretation
Global	Penalize both first row and first column	(n, m) to $(0, 0)$	Complete sequence against complete sequence
Local	Zero borders and permit a zero restart in every cell	Best cell to the first zero	Best matching pair of substrings
Semi-global	Make selected leading borders free	Best cell on selected terminal border to an allowed initial border	Entire read fitted within a reference interval
Overlap	Make the permitted unaligned prefix and suffix borders free	One allowed border to another	Suffix–prefix or prefix–suffix overlap
Extension	Fix an anchor cell, then permit a free stopping position	Anchor to the best permitted endpoint	Extend a known seed until evidence deteriorates

Table 2.1: Alignment objectives are distinguished by boundary and endpoint rules, not by a different biological alphabet or a wholly different recurrence.

2.2. Exact dynamic programming

2.2.1. Linear-gap recurrence

For a linear gap cost d, let $H_{i,j}$ be the best score for prefixes $X[1..i]$ and $Y[1..j]$. A global recurrence is

$$H_{i,j} = \max \begin{cases} H_{i-1,j-1} + s(x_i, y_j), \\ H_{i-1,j} - d, \\ H_{i,j-1} - d. \end{cases}$$

With $H_{0,0} = 0$, $H_{i,0} = -id$, and $H_{0,j} = -jd$, this computes a global score. Adding 0 as a fourth candidate and initializing the borders to zero yields the local Smith–Waterman score. Free-end objectives alter the initialization and the set of cells from which traceback may begin.

Each cell represents a state (i, j), meaning that i symbols of X and j symbols of Y have been consumed. A diagonal transition consumes one symbol from each sequence. Vertical and horizontal transitions consume one symbol from only one sequence. A traceback from the chosen terminal cell therefore spells an edit path and can be run-length encoded as a CIGAR string. For example, `10M1I4M` denotes ten aligned symbol pairs, one insertion, then four more aligned pairs. Whether `M` is later refined to exact-match `=` and mismatch `X` is an output convention.

2.2.2. Affine and two-piece affine gaps

An affine model needs separate states because the cost of extending a gap is different from the cost of opening one. Let $M_{i,j}$ end with a symbol–symbol column, $I_{i,j}$ end with a gap in X, and $D_{i,j}$ end with a gap in Y. Under the convention in Equation 2.1,

$$\begin{aligned} M_{i,j} &= \max\{M_{i-1,j-1}, I_{i-1,j-1}, D_{i-1,j-1}\} + s(x_i, y_j), \\ I_{i,j} &= \max\{M_{i,j-1} - q - e,\ I_{i,j-1} - e\}, \\ D_{i,j} &= \max\{M_{i-1,j} - q - e,\ D_{i-1,j} - e\}, \\ H_{i,j} &= \max\{M_{i,j}, I_{i,j}, D_{i,j}\}. \end{aligned}$$

Appropriate sentinel values prevent impossible boundary transitions. Local alignment additionally permits a restart at zero. Traceback stores or reconstructs which candidate produced each state.

A single affine function is not equally realistic for short and very long gaps. Minimap2 therefore uses a two-piece affine cost

$$g_2(\ell) = \min(q_1 + \ell e_1,\ q_2 + \ell e_2).$$

The first component can model short insertions and deletions, while the second has a larger open cost and smaller extension cost for long events. The DP adds a second insertion state and a second deletion state. BioSeqDB's Highway KSW2 port exposes both single- and dual-affine modes through the unified `KswAlignerMode` template. Its module comment and types in `main/bioseq/backend/alignment_hwy/ksw2.cppm` document the Suzuki–Kasahara difference states used by the implementation. The alternative Block Aligner backend is deliberately single-affine.

Algorithm 1 Affine-gap global alignment with score and traceback states

Require: sequences $X[1..n]$, $Y[1..m]$, substitution score s, gap-open cost q, extension cost e

initialize M, I, D for the selected boundary objective
for $i = 1, \ldots, n$ do
    for $j = 1, \ldots, m$ do
        $M_{i,j} \leftarrow \max(M_{i-1,j-1}, I_{i-1,j-1}, D_{i-1,j-1}) + s(x_i, y_j)$
        $I_{i,j} \leftarrow \max(M_{i,j-1} - q - e, I_{i,j-1} - e)$
        $D_{i,j} \leftarrow \max(M_{i-1,j} - q - e, D_{i-1,j} - e)$
        store a predecessor code for each winning state

choose the terminal cell permitted by the objective
follow predecessor codes to an allowed initial cell
reverse and run-length encode the resulting operations
return score, endpoints and CIGAR

2.2.3. Cost, banding and stopping criteria

Filling an $n \times m$ matrix costs $O(nm)$ time. A score alone can be computed with $O(\min(n, m))$ memory because each row depends on only nearby rows. Direct traceback generally retains $O(nm)$ predecessor information, or trades memory for recomputation. Even quadratic time becomes prohibitive when one input is a read and the other is a large genome.

Several optimizations reduce the evaluated state space without changing the recurrence inside it. A banded aligner computes only cells satisfying $|i - j - \delta| \le b$ for an expected diagonal δ and half-width b. This changes work to roughly $O(b(n+m))$, but a band that excludes the true path changes the result. A mapper can choose the band from seed geometry, widening it when anchors imply an indel. Z-drop and X-drop terminate an extension when its current score has fallen sufficiently far behind the best score observed. They are endpoint heuristics for extension, not substitutes for the affine recurrence.

SIMD changes how cells are evaluated. It does not change the optimization objective. The Suzuki–Kasahara difference formulation stores bounded differences between neighboring scores, which makes narrow integer lanes practical and is the basis of KSW2.[4] BioSeqDB

[4] H. Suzuki and M. Kasahara, "Introducing difference recurrence relations for faster semi-global alignment of long sequences", BMC Bioinformatics 19, 45 (2018), https://doi.org/10.1186/s12859-018-2014-8.

ports that formulation to Google Highway so one source can dispatch to SSE, AVX, NEON, SVE, WebAssembly SIMD, and other supported targets. The Block Aligner backend uses a different geometric optimization. It maintains a square block around the promising part of the matrix and slides, grows, or shrinks that block instead of visiting the full rectangle. Its current implementation supports global gap fills and native two-confirmation X-drop extension. The details and measured trade-offs are developed in the final-alignment chapter.

2.3. Seed–chain–extend

2.3.1. Why candidate generation is necessary

Suppose a query of length m is aligned independently against every length-m interval of a reference of length N. Even ignoring indels, this involves $O(N)$ candidate intervals, each with an $O(m^2)$ dynamic program. Most intervals are clearly unrelated. A mapper therefore seeks a cheap signal that is unlikely to occur repeatedly by chance in a true alignment. Exact or hashed short substrings provide that signal.

The seed–chain–extend pattern separates the problem into three abstractions:

1. Seeding selects query substrings and finds compatible occurrences in the reference.
2. Chaining selects a collinear subset of those matches and turns isolated hits into candidate regions.
3. Extension performs base-level dynamic programming between and around the anchors, then emits coordinates, a score and optionally a CIGAR.

This is a heuristic search wrapped around an exact scoring kernel. Missing all seeds for a true locus causes a false negative before alignment begins. Repetitive seeds cause excess candidates and extra work. Chaining may also discard a useful set of anchors. Once a particular mandatory gap is passed to an exact DP backend with a sufficient band, however, its computed score is exact for that stated subproblem. Sensitivity must therefore be discussed stage by stage rather than by calling the whole mapper either exact or approximate.

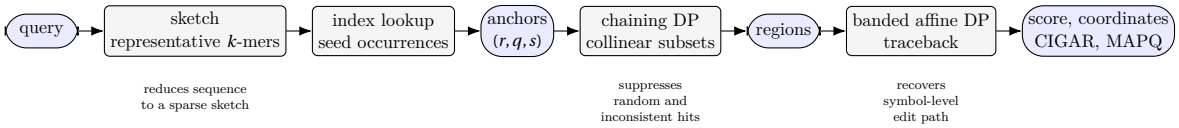


Figure 2.1: The seed–chain–extend decomposition. The diagram is reconstructed from the stage contracts in current BioSeqDB code and DOC0367 rather than copied from a published figure.

2.3.2. Seeds and minimizer sketches

An exact k-mer seed is a length-k word occurring in both sequences. Indexing every k-mer maximizes the available exact evidence, but creates a large index and many redundant neighboring hits. A minimizer sketch selects a representative k-mer from each window of w consecutive k-mers, usually the minimum under a hash order. Consecutive windows overlap, so the same representative often persists and is emitted once. This retains a locally

dense sampling property while reducing storage and lookup work. Roberts and colleagues introduced the minimizer idea to reduce storage for sequence comparison.[5]

Canonical DNA seeds treat a k-mer and its reverse complement as one key, while retaining a strand bit. The reference index maps each key to one or more positions. Looking up a query minimizer produces seed hits. Very frequent keys are often suppressed or down-weighted because they carry little location information and can dominate memory. Later chapters distinguish BioSeqDB's scalar, two-stack SIMD, SIMD-Minimizers-derived and Vulkan extraction routes. For the present chapter, the important invariant is that a hit becomes an anchor with a reference coordinate, query coordinate, orientation and span.

2.3.3. Anchors as points in a coordinate plane

Plot an anchor a_i at (x_i, y_i), where x_i is its reference end position and y_i its query end position. Hits from an ungapped alignment lie near one diagonal because $x_i - y_i$ is nearly constant. Insertions and deletions change the diagonal by a bounded amount. Random matches are scattered. Chaining is therefore a sparse path problem in this two-dimensional plane.

In minimap2 notation, an anchor has span w_i. After sorting anchors by reference and query order, a chaining score can be written

$$f(i) = \max\left(w_i,\ \max_{j<i}\{ f(j) + \alpha(j,i) - \beta(j,i) \} \right),$$

where α rewards newly covered matching bases and β penalizes large gaps and diagonal changes. A representative minimap2 reward is

$$\alpha(j,i) = \min\big(\min(y_i - y_j,\ x_i - x_j),\ w_i\big).$$

Pairs with reversed order, excessive distance, or excessive diagonal deviation are invalid. The gap penalty combines a linear component with a small logarithmic component. Backtracking predecessor pointers from high scoring endpoints yields ordered anchor chains.

The recurrence is exact only relative to its candidate predecessor set and its scoring function. Testing every $j < i$ is $O(A^2)$ for A anchors. Minimap2 scans a bounded recent window and stops after a configured number of unproductive predecessors, making the usual cost much smaller while retaining the same local scoring rule.[6] Range-minimum-query methods accelerate a related formulation for long gaps. SIMD and tiled GPU implementations instead evaluate several predecessor scores together. These are not interchangeable labels. One changes the data structure and search formulation, another changes the execution of the predecessor loop, and a bounded approximate policy may change which endpoint is selected.

BioSeqDB preserves this distinction in its current API. The `ChainingAlgorithmType` enum in `main/bioseq/backend/pipeline/minimap_port.cppm` exposes scalar minimap2, SIMD minimap2, RMQ and Vulkan tiled-DP routes. The shared `compute_chaining_score` function in `main/bioseq/backend/sequences/chaining/chaining_utils.cppm` computes 64-bit reference and query differences, rejects invalid geometry, rewards new coverage, then applies linear and logarithmic penalties. The current `Anchor` is a flat 24-byte object with 64-bit coordinates. This is an important difference from early April design notes, which discussed older packed layouts and must not be read as the present implementation.

[5] M. Roberts, W. Hayes, B. R. Hunt, S. M. Mount and J. A. Yorke, "Reducing storage requirements for biological sequence comparison", Bioinformatics 20 (2004), pp. 3363–3369, https://doi.org/10.1093/bioinformatics/bth408.

[6] H. Li, "Minimap2: pairwise alignment for nucleotide sequences", Bioinformatics 34 (2018), pp. 3094–3100, https://doi.org/10.1093/bioinformatics/bty191.

Algorithm 2 Conceptual bounded predecessor chaining

Require: anchors $a_1, \ldots, a_A$ in canonical coordinate order

for $i = 1, \ldots, A$ do
    $f[i] \leftarrow \text{span}(a_i)$
    $p[i] \leftarrow \perp$
    for candidate $j < i$ in the active distance and iteration window do
        if a_j and a_i have valid order, strand, gap and bandwidth then
            $v \leftarrow f[j] + \text{reward}(a_j, a_i) - \text{gapPenalty}(a_j, a_i)$
            if $v > f[i]$ then
                $f[i] \leftarrow v$, $p[i] \leftarrow j$
    optionally stop after the configured skip heuristic fires
backtrack from high-scoring endpoints through p
filter chains by score, seed count and overlap

2.3.4. From chains to base-level alignment

A chain supplies orientation and approximate query and reference intervals. It also divides the work into left extension, mandatory gaps between anchors, and right extension. The anchors themselves account for exact matching segments. Dynamic programming fills the intervening pieces and decides whether an apparent displacement is better represented by substitutions or gaps. Traceback concatenates the resulting operations with the anchored matches.

Minimap2's base-level route calls KSW2 with a band, two-piece affine gaps and Z-drop. Its source-level sequence is visible in the local comparison copy: `mm_sketch` in `minimap2-master-clean/sketch.c`, `mm_collect_matches` in `minimap2-master-clean/seed.c`, `mg_lchain_dp` in `minimap2-master-lchain.c`, then `mm_align_skeleton` and `ksw_extd2_sse` in `minimap2-master-clean/align.c`. BioSeqDB retains the same conceptual sequence but exposes several implementations at each stage. Current `main/bioseq/backend/pipeline/minimap_port.cppm` names nine high-level steps: target minimizer index construction, seed collection, chaining, region generation, primary/secondary classification, filtering, dual-affine alignment, mapping-quality calculation and result construction.

The port is therefore best described as minimap2-derived rather than as a line-for-line copy. The recurrence, stage meanings and output concepts provide the semantic baseline. BioSeqDB changes storage types, widens coordinates, uses C++ modules and explicit algorithm selectors, employs Highway SIMD, supports arena-backed stage data, adds Vulkan paths and integrates the work with database-resident indexes. Later chapters evaluate whether particular routes preserve bit-identical sketches or exact chain endpoints. This chapter only establishes the common algorithmic vocabulary needed to interpret those measurements.

2.4. Graph-based sequence representations

2.4.1. Why replace one reference string by a graph?

A linear reference chooses one sequence for each locus. Real populations contain alternative alleles, insertions, deletions and rearrangements. Reads carrying a non-reference allele may

Stage	Minimap2 reference path	BioSeqDB implementation surface	Meaning retained
Sketch	Scalar canonical minimizers from `mm_sketch`	Scalar minimap-int, reference SIMD, production two-stack ntHash, paper-compatible ntHash SIMD, and Vulkan minimap-int selectors	Sparse representative seeds
Seed lookup	Minimizer index lookup followed by anchor preparation and sorting	PostgreSQL-capable fragment index, seed-hit collection, canonical anchor ordering and CPU or device preparation paths	Matching keys become coordinate anchors
Chain	Bounded dynamic programming and optional RMQ path	Scalar, Highway SIMD, RMQ and Vulkan tiled-DP selectors over a shared anchor contract	Collinear anchors define candidate regions
Extend	SSE KSW2 two-piece-affine banded extension with Z-drop	Highway KSW2 single/dual-affine, batched KSW2 and adaptive Block Aligner selectors	Candidate regions become scored base-level paths
Output	PAF or SAM records with primary/secondary semantics	`MatchResult` records with coordinates, strand, score, MAPQ, divergence and optional CIGAR	Search evidence becomes an interpretable mapping

Table 2.2: Conceptual correspondence between minimap2 and current BioSeqDB. The table describes available implementation routes, not a claim that all routes execute simultaneously. Runtime and compile-time policy select a concrete path.

receive a worse score simply because the chosen linear string omits that allele. A sequence graph can retain shared context once and branch where alternatives differ. An alignment is then a mapping of the query to a path through the graph.

Two graph families are often conflated:

Variation or sequence graphs explicitly represent reference segments as nodes or node labels and alternative adjacencies as edges. A path spells a haplotype or another permitted sequence. The vg toolkit demonstrated a practical variation-graph framework for genome inference and mapping. [7]

de Bruijn graphs represent local substring adjacency. In an order-k construction, a common convention uses $(k-1)$-mers as nodes and observed k-mers as directed edges from their prefix to their suffix. A walk spells a sequence by appending one symbol per edge. These graphs are foundational in genome assembly because reads contribute local k-mer evidence without requiring a known global placement.[8]

[7] E. Garrison et al., "Variation graph toolkit improves read mapping by representing genetic variation in the reference", Nature Biotechnology 36 (2018), pp. 875–879, https://doi.org/10.1038/nbt.4227.

[8] P. A. Pevzner, H. Tang and M. S. Waterman, "An Eulerian path approach to DNA fragment assembly", Proceedings of the National Academy of Sciences 98 (2001), pp. 9748–9753, https://doi.org/10.1073/pnas.171285098.

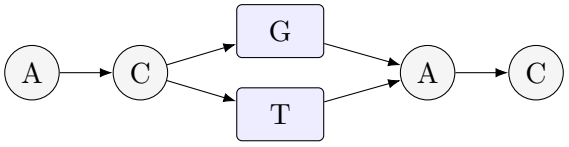
The de Bruijn construction changes assembly from choosing an ordering of whole reads to finding a walk that uses local adjacency evidence. Under ideal conditions, reconstruction can be stated as an Eulerian path problem over edges. This should not be confused with an overlap graph, in which reads are vertices and compatible suffix–prefix overlaps are edges. Nor should it be confused with pairwise alignment. The graph supplies a reference language. An aligner must still score how the query follows a graph path and where it uses substitutions, insertions or deletions.

2.4.2. Dynamic programming on a graph

For a linear target, the alignment state (i, j) identifies a query prefix and one target position. For a node-labeled directed acyclic graph, a corresponding state can be written (i, v, o), where i is the query position, v a graph node, and o an offset within its label. A symbol–symbol transition advances both the query and graph path. A query insertion advances only the query. A deletion advances only the graph path. At a branch, the recurrence considers incoming or outgoing graph edges.

If the graph is acyclic, nodes can be processed in topological order. Cycles from repeats or de Bruijn structure complicate this order and require additional restrictions, indexing, graph transformations or search algorithms. The state space scales with the represented graph, not with the length of one chosen path. Repeated k-mers collapse into the same de Bruijn nodes and can make origin ambiguous. Increasing k resolves some repeats but needs longer exact context and makes the graph more sensitive to coverage gaps and errors. Decreasing k improves local connectivity while collapsing more repeats. The value of a graph is therefore coupled to its construction and indexing policy.

Candidate generation remains essential. Graph mappers commonly find seeds in graph paths or a graph-derived index, cluster or chain the corresponding positions, then align against a local subgraph. The search space is richer than in a linear reference because one seed may correspond to many paths. Graph representation can reduce reference-allele bias, but it introduces path selection, index size, update and output-coordinate questions. A database system must additionally decide how graph nodes, named paths and annotations participate in relational joins and versioning.



linear reference chooses one path
ACGAC or ACTAC

graph retains the shared prefix and suffix
while representing both alleles

Figure 2.2: A minimal variation-graph branch. A query alignment chooses a path and an edit path together. A de Bruijn graph uses a different node and edge construction, although both replace one linear target string by a set of possible paths.

2.4.3. Relation to BioSeqDB

BioSeqDB's current mapper is a linear-reference seed–chain–extend system. Its anchor contract stores one reference coordinate and a strand, and current code explicitly removes a reference identifier from the 24-byte anchor because the active fragment index is single-target.

The pipeline may process many stored sequences and many target fragments, but each mapping problem still chains against a linear coordinate system. This is an implementation fact, not a claim that graph references are unimportant.

A graph-native BioSeqDB mapper would require more than substituting a graph table for a text column. Seeds would need graph positions or path coordinates. Chaining would need a reachability-aware order rather than a single increasing reference coordinate. Extension would need graph DP or extraction of a local linearized subgraph. Output would need stable node, path and version identifiers that can be joined with relational metadata. Persistent indexes would need update semantics compatible with graph changes. These are valuable research directions, but they are not silently supplied by the present minimap2-derived pipeline.

2.5. Choosing an approach

No single method dominates for every task. Exhaustive dynamic programming is the clearest exact model for two moderate strings. Seed–chain–extend is the practical choice when searching long references or large collections, provided its sensitivity and repetitive-seed behavior are understood. Graph mapping is appropriate when representing alternatives is itself part of the problem, at the cost of a more complex reference, index and coordinate model.

Approach	Search object	Candidate reduction	Principal strength	Principal limitation
Exhaustive pairwise DP	Two explicit strings	None, except banding or stopping rules	Precisely defined optimum for the selected scoring and boundary model	Quadratic state space in the unbanded case
Seed–chain–extend	Usually a linear reference or sequence collection	Sparse seeds, occurrence filtering and coordinate chaining	Scales to long references while retaining exact base alignment in selected regions	Search sensitivity depends on seeds, chaining and bands
Variation-graph mapping	Named paths and alternatives in a sequence graph	Graph-aware seeds, clustering, chaining or local-subgraph extraction	Represents known alternatives and can reduce linear-reference bias	Larger and more ambiguous index, path-aware coordinates and DP
de Bruijn graph methods	Local k-mer adjacency graph	k-mer compaction, graph cleaning and path constraints	Natural representation of assembly evidence and local sequence language	Repeats collapse, errors branch, and a graph walk is not yet an alignment

Table 2.3: Comparison of the main algorithmic viewpoints introduced in this chapter. Candidate reduction makes large-scale search feasible, but adds assumptions that must be measured separately from the final DP kernel.

The rest of the thesis follows the current BioSeqDB choice. It first explains why existing standalone tools and ordinary SQL functions leave an integration gap. It then descends through the linear mapper's stages: minimizer extraction, anchor chaining, base-level extension and optional prefiltering. Those chapters compare each BioSeqDB implementation with

minimap2 and with the corresponding research prototype. The later systems chapters show how the same computation is made to operate on compact, lazy and PostgreSQL-backed sequences, how its indexes persist, and how its results become part of a database execution plan.

Chapter 3

Limitations of existing tools

3.1. Why a fast mapper is not yet a database system

The previous chapter introduced several ways to search biological sequences. A specialized mapper such as minimap2 is already a highly engineered answer to the algorithmic question. It avoids whole-matrix dynamic programming, reduces the seed set with minimizers, chains compatible anchors, and invokes expensive alignment only for selected regions. It also provides a persistent `.mmi` representation for reuse of a reference index. It would therefore be misleading to motivate BioSeqDB by claiming that existing mappers are generally slow or algorithmically naive.

The missing capability lies at a different boundary. A mapper normally owns a process, reads files or an index image, maps a stream of queries, and writes another file or stream. PostgreSQL owns relations, snapshots, transactions, typed expressions, joins, memory contexts, and a cost-based plan. Calling the first system from the second does not make their execution models coincide. The database still needs to know when alignment will run, which rows supply its operands, whether one input row can yield several outputs, which intermediate data can be reused, and how the work is bounded when a relation contains billions of bases.

BioSeqDB addresses this mismatch. Its problem is not merely "alignment from SQL". A scalar PostgreSQL function can invoke a mapper and satisfy that literal description. The stronger objective is to make sequence search a planned, batch-oriented, resource-accounted database operation while retaining a credible minimap2-derived algorithmic baseline. This chapter separates the scientific limitations of alignment heuristics from the systems limitations created by storage and query integration. This separation is important because the remedies are different. A missed seed needs an algorithmic remedy. Repeated detosting, cold sidecar reads, or one function call per tuple needs a systems remedy.

3.2. Scientific limitations do not disappear with optimization

3.2.1. Seed selection trades work for evidence

Minimizers make large-scale mapping feasible by retaining a sparse subset of the available k-mers. The reduction is valuable precisely because the aligner no longer examines every possible local match. It also means that the seed set is evidence for an alignment, not a proof that every biologically meaningful alignment has been retained. Sensitivity depends on k, the minimizer window, the hash and tie policy, the treatment of ambiguous symbols, and the

error pattern in the query. A short query may contain too few representatives. A repetitive region may produce a very large posting list. A mutation or sequencing error can remove a seed that would otherwise connect two parts of a chain.

No database integration can eliminate this trade-off. BioSeqDB must preserve it and make the selected policy explicit. A persisted index is usable only when its alphabet, k, window, layout, hash semantics, and source generation agree with the query path. Reading an incompatible index quickly would be a correctness failure. Recomputing every index for safety would discard the central benefit of persistence. The system therefore needs compatibility metadata and a fail-closed dispatch policy, not only serialized posting lists.

The same point applies to vectorized minimizer extraction. SIMD changes how several candidate positions are evaluated. It must not silently change which positions are emitted unless the alternative sketch is deliberately defined as a different algorithm. An early BioSeqDB incident illustrates the risk. A circular-buffer slot equality was interpreted as distance zero even though it represented one complete window. The resulting position shift then caused a later valid minimizer to be removed by position deduplication. Adjacent off-by-one and inactive-lane problems were exposed by the same test campaign.[1] The lesson is broader than that defect. An implementation is not semantically equivalent merely because it is a vector form of the same pseudocode.

3.2.2. Chaining and extension remain heuristic filters

Chaining ranks collinear anchors without evaluating every possible alignment path. Parameters such as maximum predecessor distance, occurrence filtering, chain score, skip limits, and gap penalties determine which evidence survives. Their useful values vary with read length, error profile, repeat structure, and reference scale. A policy that is effective for a long noisy read can remove all evidence for a short exact probe. A permissive policy can retain too many candidates and transfer the cost to sorting, chaining, or final alignment.

The final extension step is exact only within the regions and score model that reach it. Affine-gap dynamic programming can establish the best alignment inside a selected band, but it cannot recover a candidate that seeding or chaining removed. Banded or block-based methods introduce additional resource and boundary choices. Traceback and CIGAR construction can also require substantially more memory than score-only evaluation. Consequently, "alignment accuracy" is not a single counter. At minimum, evaluation must distinguish whether a result was found, whether its coordinate is correct within a stated tolerance, whether the expected strand was selected, and whether a requested CIGAR was produced correctly.

These properties explain why BioSeqDB keeps minimap2 as a comparison baseline and tests intermediate algorithms against biological result receipts. A faster stage is not a contribution when it moves an anchor, changes a primary result, or falls back to another implementation without reporting the fallback. Later chapters give the exact policies used for minimizer extraction, chaining, and extension.

3.2.3. No processor path dominates every workload

Scalar, SIMD, and GPU execution have different fixed and variable costs. SIMD is attractive when independent lanes can perform similar work with regular memory access. It is less attractive when lane-local rescans, divergent control flow, gathers, or conversion between array layouts dominates the useful arithmetic. A GPU adds command submission, buffer

[1] `DOC0068`, "Why kmers_test.cpp Was Failing", 2026-04-23.

management, synchronization, and transfer or residency requirements. A large resident batch can amortize these costs. One short query against one small target often cannot.

The current project reports demonstrate both sides of this boundary. A resident multi-query GPU path can enter GPU seeding, chaining, and bounded CIGAR construction with no CPU or SIMD fallback. The same reports state that the general G3SA traceback design is not reproduced in every case and that a small bounded-CIGAR workload remained faster in minimap2. Likewise, a one-query multi-gigabase workload is not valid evidence for a resident GPU batch when the batch branch is not admitted.[2]

BioSeqDB therefore needs an explicit dispatch contract. It must record which path was selected, why an accelerator declined work, whether a fallback occurred, and which correctness receipts belong to the selected path. "GPU enabled" or "SIMD case" is not sufficient evidence. This is an algorithmic and experimental limitation of heterogeneous execution, not a PostgreSQL-specific defect.

3.3. The file-oriented workflow leaves systems work outside the mapper

In a conventional workflow, relational metadata and biological sequences often meet only in orchestration code. A query selects samples or identifiers. A script exports sequences to FASTA or FASTQ. A mapper opens or constructs an index and emits PAF, SAM, or BAM. Another step parses the results and joins them with annotations. Workflow engines can make this process repeatable, but the mapper and the database still optimize different fragments of the computation.

This separation has several consequences.

1. The database planner cannot compare an alignment path with another relational plan because the expensive operation occurs after export. Selective predicates and joins may be applied too early, too late, or in duplicated scripts.

2. Sequence bytes cross representation boundaries. They may be decoded from a packed database value, rendered as text, parsed by the mapper, and parsed again when results return.

3. The lifetime of an index is external to the source relation. Scripts must decide whether an index is current, compatible, and safe to reuse after updates.

4. Failures span several durability domains. A committed database change, a partially written file, and a terminated mapper process do not automatically form one atomic operation.

5. Alignment results are detached from SQL expression semantics. Producing several matches for one sequence requires an explicit result table or file expansion before relational processing can resume.

These costs are not evidence that file formats are unnecessary. FASTA, FASTQ, PAF, SAM, BAM, and mapper indexes are valuable interchange and archival formats. The gap is that they become mandatory execution boundaries even when both the source data and the desired analysis already belong to one database query.

[2] `DOC0356`, batched resident GPU implementation report, and `DOC0369`, local end-to-end benchmark report dated 2026-07-08.

3.4. Why an ordinary SQL function is insufficient

PostgreSQL makes it straightforward to expose C or C++ code as a typed function. A direct implementation of mmsearch(*target, query*) could decode its two arguments, run the mapper, and return a Boolean or a result value. This form is a useful correctness fallback. It is a poor primary execution model for BioSeqDB.

First, PostgreSQL normally evaluates a scalar expression for one tuple at a time. That makes batching accidental. It prevents a scan from collecting many targets or queries before choosing an implementation, and it makes accelerator submission costs recur at tuple granularity. Second, one sequence pair can produce zero, one, or several matches. A Boolean predicate alone cannot express the result cardinality needed by the projection. Third, score, coordinates, strand, CIGAR, divergence, and mapping quality belong to the same alignment result. Re-evaluating one expensive function for each projected field would duplicate work. Fourth, the ordinary function is evaluated before a later batch could know that a row has no result.

The early custom-scan proof exposed these issues directly. It could replace one function call with one mutable constant and could buffer tuples, but it did not yet invoke the real aligner, return the complete result, filter a no-match row after batch execution, support the vectorized scan path, or emit multiple rows for one input.[3]

The present design keeps an ordinary function body so that PostgreSQL can resolve types and so that unsupported contexts have a defined fallback. Its planner cost is intentionally high. A support function and relation path hook offer a cheaper custom scan when the expression has a supported shape. The custom plan owns batching and replaces result-bearing expressions with nodes whose values are populated for each emitted match. The custom scan can therefore return no row for a target without a match or several rows for a target with several matches. Current project guidelines require this plan choice to be verified with `EXPLAIN (ANALYZE)` and require both vectorized and non-vectorized execution to be tested.[4]

This planner arrangement also creates obligations that a standalone mapper does not have. PostgreSQL may copy expression trees, rescan an inner plan, evaluate constants during planning, or reset a memory context. The extension must keep node identity, ownership, and rescan state valid under those operations. Plan costs and EXPLAIN annotations must describe the path that will actually execute. Hidden thread-local state is inappropriate because it changes function behavior without changing its arguments and interacts badly with constant folding and error unwinding.

3.5. Representation and lifetime are part of performance

A biological sequence is logically a range of symbols. At the storage boundary it may be a PostgreSQL varlena value, an inline packed payload, a fragmented column, a lazy slice, a concatenation, a reverse view, a mapped external page, or a materialized C++ container. Treating all of these as `std::string` would simplify an interface by hiding the most important cost.

The April integration design illustrates the problem. A Datum obtained from a tuple slot cannot be retained after the slot is reused. A toasted value may have to be detoasted

[3] `DOC0054`, "Wiring the real aligner batch through the Postgres custom scan", 2026-04-21. This archived design is used as history. The current contract is stated in `DOC0321`.

[4] `DOC0321`, Sections 7.2 to 7.4, current code and workflow guidelines.

into memory owned by a PostgreSQL context. The first real batch design therefore copied each complete varlena into an owned C++ byte vector and then constructed an owning `compact_sequence_range` for each row. This was a reasonable correctness step. The same document explicitly listed a non-owning compact view and batch-level target-index reuse as follow-up work.[5]

At scale, ownership conversion can dominate an algorithm that was optimized to avoid touching most symbols. Eagerly materializing a long fragmented sequence creates work proportional to the whole sequence even when minimizer extraction or alignment needs one subrange. Reversing by copying doubles traffic. Decoding a packed sequence to bytes discards the storage density that made database scans feasible. Rebuilding a contiguous target so worker threads can share it may be worthwhile, but that cost must be a deliberate stage with a memory budget rather than an implicit function argument conversion.

This motivates the common sequence abstraction developed later in the thesis. Algorithms consume a compact range contract and convert only the needed region at the point of use. PostgreSQL-specific ownership remains outside the numeric kernels. Packed, fragmented, and lazy views must preserve alphabet information, base-pair length, and iteration semantics. Zero copy is not an unconditional objective. It is useful only when the borrowed lifetime is proven and when irregular access does not cost more than one bounded materialization.

3.6. A persisted index is a database object, not merely a byte image

The ability to save an index is necessary but not sufficient for database-resident reuse. A PostgreSQL index representation must answer questions that an external reference image can leave to the caller:

1. Which exact source relation and generation produced the index?
2. Which alphabet, minimizer parameters, hash scheme, position width, and layout can consume it?
3. Does a transaction see the old generation or the new generation while an update is being published?
4. How are an aborted publication, a dropped table, a tablespace move, a crash, and an orphaned generation handled?
5. Can a cold reader select only the needed extents before allocating memory, or must it reconstruct the entire index?
6. How are checksum work, decoded buffers, posting lists, GPU residency, and temporary query projections charged to one memory budget?

The project's first answer was a shared sidecar keyed by relation and row number with a flat serialized target index. It was deliberately a proof-of-concept. It accepted orphan cleanup, native-layout serialization, and coarse shared-table locality. Later designs moved toward immutable generations, explicit wire formats, compatibility manifests, bounded selective loading, and PostgreSQL-owned publication. This progression is important. It shows why "store the minimizers in a bytea" describes the first write operation but not the lifecycle of an index.

[5] `DOC0058`, Sections 4, 7, and 11, archived 2026-04-21 design.

Cold loading also changes the performance question. A precomputed index can be slower than recomputation when the reader performs many small tuple fetches, validates the same payload twice, builds a query-projected facade, sorts data again, or repeatedly walks all selected extents for memory accounting. The July 15 diagnosis found both an $O(E^2)$ ownership-accounting path over selected extents and a duplicate 4.46 GB checksum traversal in the then-current Format 5 reader. It explicitly warned that its payload timer also included C++ routing and decode work, so it could not be interpreted as a pure PostgreSQL storage timer.[6]

The resulting requirement is stronger than persistence. The stored form must already support the dominant query access pattern, and the reader must authenticate and materialize it once under a linear memory ledger. The index chapter describes the current generation and loading design.

3.7. Bounded memory and parallel work must be designed together

Sequence workloads combine large immutable inputs with variable-size derived structures. Target minimizers, posting lists, query hashes, anchors, chain state, alignment matrices, traceback, and output rows can be live at overlapping times. A planner estimate based only on source sequence bytes misses this amplification. A GPU path adds device buffers and often a host staging representation.

The naive responses are both undesirable. Keeping every intermediate resident can exceed the backend or device budget. Spilling immediately can leave most CPU cores idle while storage becomes the bottleneck. BioSeqDB instead needs phase-aware admission. Before a task starts, the system estimates its source, derived, scratch, and output ownership. It admits parallel tasks while their combined peak remains within the budget. Buffers are released or their authority is transferred when a phase completes. External runs or recomputation remain fallbacks, not the default while memory is available.

PostgreSQL adds a thread-affinity constraint. Catalog access, tuple formation, snapshots, and many memory-context operations must remain on the backend thread. CPU workers can operate on immutable C++ views after the backend has acquired and validated them. Their tasks must join before the backend releases PostgreSQL-owned resources. This boundary prevents a direct translation of a standalone mapper's thread pool. Task parallelism is valuable only after input ownership is safe and the PostgreSQL calls have been removed from worker closures.

These concerns explain an otherwise surprising observation from the no-index investigation. A benchmark labeled parallel did not prove that PostgreSQL-backed target-index construction was parallel. The fast cells benefited from prebuilt indexes and backend-local residency, while the no-index path included serial loading and failed worker-shareability gates for lazy PostgreSQL-backed sequences.[7] Stage labels must therefore be supported by execution receipts and resource accounting.

3.8. Ingestion, metadata, and external data are part of the query

An integrated system must load data reliably before it can align it. FASTA and FASTQ records are not only symbol strings. They contain identifiers, descriptions, quality information, source-specific fields, and sometimes multiline records. Compression and remote delivery change how much input can be buffered. A database loader must turn this stream

[6] `DOC0381`, Sections 2 and 3, "Bioseq just_index cold-load acceleration", 2026-07-15.

[7] `DOC0318`, executive verdict and root-cause sections, 2026-06-07.

into typed columns while preserving record boundaries and reporting the record that caused a conversion error.

This task should not be embedded in the mapper. The non-PostgreSQL IO module parses and decompresses sequence records. A concept-constrained writer defines the boundary. The PostgreSQL implementation maps parsed fields to destination columns and invokes the appropriate type input functions. The current implementation report records a single-pass FASTA and FASTQ header parser, typed multi-column mapping, integration tests, and a non-empty `mmsearch` over data loaded through that path.[8]

Remote sequence databases create the same separation at another scale. An external provider can expose data as a relation, but network failure, pagination, authentication, retry policy, and bounded streaming remain provider responsibilities. The alignment pipeline should see the same sequence abstraction whether the source is a local column, a loaded file, or an external relation. Otherwise every new source multiplies the algorithm implementations and their correctness surface.

3.9. Evidence from intermediate results

The strongest illustration of the systems gap is that the ranking of two implementations changed when the index delivery path changed. Table 3.1 contains three dated measurements. They are not combined into one speedup claim because the reports use different artifacts and acceptance states.

Evidence	BioSeqDB measured region	minimap2 measured region	Interpretation
2026-07-08, 2 Gbp DNA4, no index, MPF	23.888 s	73.566 s	Both found the expected result. The BioSeqDB recompute path was faster in this case.
2026-07-08, 2 Gbp DNA4, forced cold restart, MPF persisted index	36.187 s	15.465 s	Both produced the expected match. BioSeqDB was 2.34 times slower, showing that persisted delivery dominated the saved computation.
2026-07-17 handoff, six-run lower-middle P50	5.368746 s	16.487527084 s	The report records 16 of 16 parity and a 3.071020 times advantage after a fragment-native and compact-direct redesign. Its final isolated rerun was still pending.

Table 3.1: Intermediate performance evidence for the integration problem. The first two rows come from `DOC0369`, reports 73 and 76. The third comes from the current summary of `DOC0384`. The rows demonstrate that storage, reconstruction, and batching can reverse the ranking of the same broad alignment pipeline. They do not establish a machine-independent final speedup.

The first two rows are especially useful because they reject a simple story. BioSeqDB could be faster than minimap2 while rebuilding minimizers, yet slower after being given a

[8] `DOC0319`, implementation report dated 2026-06-09.

persisted index. The alignment kernels alone cannot explain that ordering. The forced restart removed backend-local warmth, and the later investigation identified index reconstruction, query projection, checksum, tuple delivery, and memory-accounting work. The July 17 handoff then reported that a fragment-native and compact-direct route reversed the comparison on its paired framework samples.

The third row is reported with its open gate. A lower-middle P50 from six observations is valuable evidence for a design decision. It is not a license to claim that every BioSeqDB query is three times faster than minimap2. The report itself required a final contemporary rerun and separated the retained `just_index` result from later XXL memory work. This thesis follows that distinction.

3.10. Comparison of solution classes

Table 3.2 summarizes the roles of the principal solution classes. The wording is deliberately scoped. A blank in the BioSeqDB problem column does not mean that a tool is defective. It means that the concern belongs to another layer.

Solution class	Primary strength	Gap at the BioSeqDB boundary	Requirement derived for this thesis
Standalone minimap2	Mature seed, chain, and extend mapping with reusable reference indexes	Does not own PostgreSQL snapshots, relational joins, tuple cardinality, or extension memory contexts	Preserve a minimap2-derived baseline while adapting data and control flow to a custom scan.
File and workflow pipeline	Portable interchange, reproducible orchestration, and composition of specialized programs	Parsing, index validation, failure handling, and joins remain across process and file boundaries	Keep file IO but remove mandatory export and re-import for database-resident queries.
Scalar PostgreSQL function	Simple typed SQL surface with a well-defined fallback	Per-row evaluation hides batching, duplicates multi-field work, and cannot naturally emit several matches per input	Use planner support and a batch-owning custom scan with per-match result substitution.
Generic columnar storage	Compression, projection, and scan bandwidth for typed columns	Does not define sequence alphabets, minimizer compatibility, or alignment-specific derived data	Add compact sequence types, sequence metadata, and versioned persistent indexes.
Accelerator-first mapper	High throughput when enough regular work is resident on the device	Submission, transfer, memory admission, and fallback can dominate small or irregular SQL batches	Dispatch by measured workload shape and expose path and fallback receipts.
BioSeqDB target design	One relational surface for storage, filtering, joining, and batched alignment	Adds planner, lifecycle, ownership, compatibility, and resource obligations to every optimized kernel	Treat these obligations as first-class contributions and validate them separately from algorithm speed.

Table 3.2: Existing solution classes and the additional requirements introduced by database-resident alignment.

3.11. Problem statement and design requirements

The design problem can now be stated precisely:

> Given biological sequences and relational metadata managed by PostgreSQL, execute a minimap2-derived search as a planned, set-oriented operation that can reuse compatible persistent indexes, operate over compact or lazy representations, remain within explicit host and device memory budgets, and return zero or more typed alignment rows without sacrificing a correct ordinary SQL fallback.

This statement yields eight requirements.

1. Algorithmic fidelity. Scalar, SIMD, and GPU variants need explicit semantics and correctness receipts. A dispatch label is not proof of the executed algorithm.

2. Planner ownership. PostgreSQL must be able to choose a custom path whose cost reflects the expensive fallback. EXPLAIN must identify the chosen alignment, index, vectorization, and accelerator path at the level needed to audit a query.

3. Set-valued batching. The executor must collect enough work to amortize indexing and accelerator costs, then emit zero, one, or several results for each input row.

4. Representation independence. Numeric kernels must consume a common compact range instead of PostgreSQL Datum or an eager byte vector. Alphabet and layout compatibility remain explicit.

5. Transactional persistence. Derived indexes need a versioned format, source identity, atomic publication, invalidation, crash and abort behavior, and bounded cold loading.

6. Resource accounting. Source, decoded, index, scratch, output, and device allocations must belong to one phase-aware budget. Parallel tasks may use only resources whose ownership survives their lifetime.

7. Reliable data entry. Local and remote sources need bounded streaming, record-aware errors, typed metadata mapping, and a clean boundary between provider code and the mapper.

8. Reproducible evidence. Performance results must name the workload, index state, cache or restart state, selected implementation, fallback counters, biological result receipts, and measured region.

The requirements can conflict. A zero-copy view can prevent safe worker sharing. A persistent representation optimized for dense reading can waste work for a selective query. A GPU batch can improve throughput while increasing latency. An exact compatibility check can add cold startup work. The design must make these trade-offs visible instead of hiding them behind a single function.

3.12. Research questions and contributions

The thesis investigates four questions derived from the gap analysis.

1. How can a minimap2-derived pipeline operate over packed, fragmented, PostgreSQL-backed, and lazily composed sequences without converting every input to one owning byte representation?

2. How can PostgreSQL plan a familiar typed expression while a custom executor performs batch-oriented, potentially set-valued alignment and supplies all projected result fields from one execution?

3. How can minimizer indexes and derived metadata be stored, versioned, validated, loaded, cached, and replaced under PostgreSQL lifecycle rules without making cold reuse slower than recomputation?

4. Under which workload and resource conditions should scalar, SIMD, or Vulkan stages execute, and what evidence is sufficient to show that the selected path was both faster and biologically acceptable?

The contributions claimed in this thesis are correspondingly systems-oriented and algorithm-aware.

1. A common compact sequence interface connects owning, bit-packed, fragmented, PostgreSQL-backed, and lazy representations to the same pipeline while keeping alphabet dispatch explicit.

2. A custom-scan execution model converts supported SQL expressions into batch alignment, supports one-to-many results, and substitutes score, coordinate, strand, and CIGAR fields without re-running the mapper for every projection.

3. PostgreSQL-resident sequence and minimizer storage ties derived generations to source identity and supports compatibility checks, bounded loading, caching, and controlled recomputation.

4. CPU vector and Vulkan implementations are integrated behind explicit selection and fallback contracts rather than exposed as unconditional replacements for the baseline.

5. Bounded ingestion, typed header mapping, external providers, and generated PostgreSQL boundary code extend the same typed data plane beyond the alignment kernels.

These are not claims that BioSeqDB replaces every mapper, workflow engine, or sequence format. They establish a design for workloads whose data, metadata, and desired results already benefit from relational storage and query planning. They also do not imply that every experimental optimization became part of the final route. The later chapters identify which intermediate algorithms were retained, which were rejected, and which benchmark gates remained open at the documented cutoff.

3.13. Scope of the evaluation

Three cautions govern the evaluation in the remainder of the thesis. First, minimap2 is a baseline for comparable mapping work, not a proxy for all external workflow costs. A comparison must state whether loading, index construction, index opening, PostgreSQL restart, result parsing, and correctness audits are inside the timer. Second, one local machine cannot

establish portability or universal throughput. The July 8 report, for example, explicitly identifies its environment as Darwin on Arm and rejects conclusions about x86-64 Linux. Third, historical reports describe the implementation that produced them. A newer architecture can supersede their design conclusions without making the earlier measurement false.

For these reasons, the thesis uses numbers to answer bounded questions. A microbenchmark identifies a kernel bottleneck. A selective end-to-end case demonstrates SQL integration and a real result. A cold-index run tests persistence rather than backend-local cache residency. A GPU result needs branch, dispatch, and fallback receipts. No single chart is asked to prove all of these properties.

The next chapter follows one query through the resulting pipeline. It defines the stage boundaries that make these separate forms of evidence possible. The algorithm chapters then compare each retained stage with its minimap2 or research origin. The systems chapters return to the sequence, planner, storage, index, and IO requirements established here.

Chapter 4

The BioSeqDB alignment pipeline

4.1. From an alignment function to a database execution pipeline

BioSeqDB follows the same broad seed–chain–extend idea as minimap2, but "the pipeline" in BioSeqDB means more than the sequence of biological algorithms. A request begins as an SQL expression, passes through a PostgreSQL custom scan, is grouped with other requests, obtains or constructs a target minimizer index, and only then enters the mapper. The selected mapper can execute on scalar CPU code, vectorised CPU code, or a Vulkan device. Its results return to the scan as typed records which are projected into SQL rows. Consequently, there are two useful views of the system:

1. the logical alignment pipeline, which transforms sequence symbols into minimizers, anchors, chains, candidate regions and final matches; and

2. the physical execution pipeline, which acquires sequences and indexes, admits memory, creates CPU or GPU work waves, records which engine executed each stage, and delivers the results to PostgreSQL.

The distinction matters. A fast chaining kernel does not make an end-to-end mapper fast if every SQL row rebuilds the same target index. A GPU minimizer kernel does not establish a GPU pipeline if the anchors are copied back to the host and every later biological decision is made on the CPU. Conversely, a host I/O stage does not invalidate a GPU compute claim when the stage is explicitly identified as data transfer rather than biological computation. The final architecture therefore makes stage ownership, index provenance and backend execution observable rather than inferring them from configuration names.

The implementation described in this chapter is grounded in four source surfaces. The minimap2-derived algorithms and result structures are in `main/bioseq/backend/pipeline/minimap_port.cppm`. The typed stage composition and runtime-to-template dispatcher are in `main/bioseq/backend/pipeline/pipe.cppm`. PostgreSQL batching and typed dispatch are in `main/bioseq/backend/pg/scheduler/dispatch_buffer.cppm` and `main/bioseq/backend/pg/scheduler/dispatch_processor_typed_impl.inc`. Finally, memory admission and task scheduling are in `main/bioseq/backend/pipeline/parallelism`. These files take precedence over older design documents where the project history and the current code disagree.

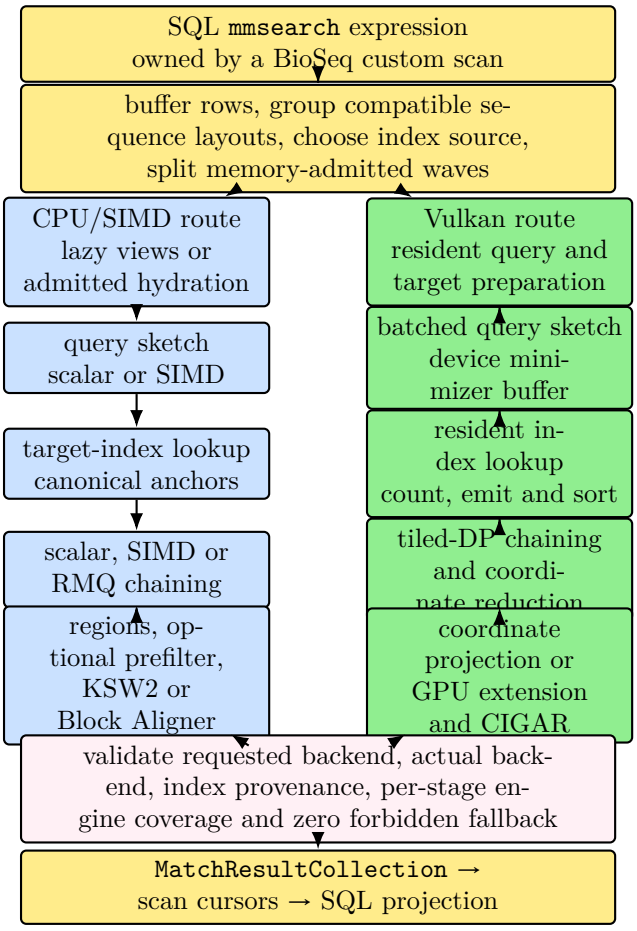


Figure 4.1: End-to-end BioSeqDB alignment dataflow. The blue branch is the scalar/SIMD route and the green branch is the selected Vulkan route. Both routes are subordinate to PostgreSQL batching and memory admission. They reconverge at an execution-evidence boundary before results become visible to SQL. A standalone Vulkan prefilter can be used within an otherwise CPU pipeline and therefore does not by itself select the full green branch.

4.2. Evolution of the architecture

4.2.1. The direct minimap2-derived port

The earliest useful implementation was organised like an ordinary mapper. A call acquired a target index, collected query minimizers, looked up seed hits, chained anchors, generated regions, classified primary and secondary hits, ran dynamic programming, calculated mapping quality and converted the internal region into an output record. This shape remains visible in `minimap_port.cppm`. For example, `collect_seed_hits` performs the minimizer-index lookup and canonical anchor sort, while `generate_regions`, `classify_primary_secondary` and the alignment helpers correspond to recognisable minimap2 operations.

This first form was valuable because it established semantic parity before optimisation. It was also insufficient as a database execution primitive. Intermediate values were owned by local containers, algorithm selection was interleaved with work, and a caller could observe only the final `MatchResultCollection`. The structure did not provide a common surface for an `EXPLAIN` description, for replacing one stage with a SIMD or GPU implementation, or for proving that a requested implementation actually ran.

The old benchmark documents illustrate why semantic and measurement foundations had to be fixed first. DOC0079 reported an early BioSeqDB result of approximately 74.1 ms against approximately 2.404 ms for minimap2. DOC0091 later found that some comparisons accidentally selected Block Aligner when they were intended to compare KSW2, while DOC0103 traced missing long-input results to unrealistic error profiles, run ordering and an RMQ defect. These values are not current performance claims. Their lasting contribution is methodological: a pipeline optimisation is meaningful only when the selected algorithms, correctness predicate, input regime and measured region are explicit.

4.2.2. Typed push stages

The May pipeline work, recorded in archived DOC0249, DOC0250 and DOC0254, changed the control structure. Instead of one large routine which repeatedly branches on runtime options, BioSeqDB gained a push-style stage chain. Each stage accepts a batch view, produces the next wire type and immediately emits it to its downstream sink. The variadic `make_pipeline` composer fixes one compile-time parameter object for the entire chain.

Three details made this more than a cosmetic refactor.

1. Stage-to-stage values are non-owning spans backed by an arena slab. Kernels such as seed collection and chaining write directly into the slab, so the staged API does not require a vector-to-vector copy at every boundary.

2. Algorithm enums are mapped to kernel traits. Once the dispatcher selects a tuple, the hot stage body calls the fixed kernel type rather than switching on the enum for every query or candidate.

3. `EmitToken` and `EmittedToken` make downstream emission part of a stage's type contract. A stage process method must return an emitted proof, and the only way to construct that proof is to consume the token through an `emit_*` operation. In debug builds, a second consumption also trips a runtime assertion.

The current definitions are at `pipe.cppm:3971`–`pipe.cppm:4345`. They support ordinary spans, paired spans, index-decorated spans and a `MinimizerBatchView`. The latter carries both a host minimizer span and an optional device span, which lets a Vulkan sketch stage pass resident query minimizers to GPU seed lookup without a second upload.

4.2.3. Containing template growth

Top-level template dispatch removes branches from the hot loop, but an uncontrolled Cartesian product can make the binary and build process impractical. The intermediate implementation instantiated every global combination of sequence-length bucket L, k-mer length K, base aligner, chainer and minimizer implementation. DOC0271 counted 5,292 `pipeline_run_impl_batch` bodies and 21 (K, L) representation cells, although many length buckets were invalid for a given K.

The retained solution is a pinned dispatch chain. It first selects K, then iterates only the length breakpoints registered for that K. The source comment in `pipe.cppm:10044` records the reduction from 21 global (K, L) cells to nine valid cells, $3+4+2$ for the currently registered K values. It then pins the base alignment, chaining and minimizer algorithms. The historical duplicate cross-query deferred-alignment pipeline was also removed. It was almost identical to the ordinary batch body and had no production caller. The deletion and the current flag-independent semantics are documented directly at `pipe.cppm:9002`–`pipe.cppm:9010` and `pipe.cppm:9286`–`pipe.cppm:9294`.

This episode explains an important final design choice. BioSeqDB does not specialise every conceivable runtime value. Common $w = 10$ and registered K and L combinations receive compile-time specialisations at higher optimisation levels. Other values use a typed generic fallback. The dispatcher therefore moves common decisions out of the hot path without requiring an unbounded binary.

4.2.4. Database-aware scheduling and accelerator residency

The next transition moved the unit of work from one sequence pair to a database batch. DOC0294 introduced the current TBB pool, coarse task regions, memory-admitted waves and PostgreSQL scheduler constraints. DOC0303 then analysed G3SA[1] against BioSeqDB and minimap2. At that date the project did not yet have an end-to-end GPU route. DOC0306, DOC0308 and DOC0311 successively added Vulkan tiled chaining, Vulkan minimizer and seed lookup, and the resident Phase-C front end. DOC0354 and the final addenda of DOC0356 moved the decisive boundary again: multiple SQL queries could share a resident target index and execute as one GPU work set.

The chronological result is instructive. Isolated stage offload was often slower than CPU execution because submission, fencing, allocation and host–device transfer dominated the kernel. Speed appeared only after the system grouped independent queries, reused target residency, eliminated repeated target-index construction and retained intermediate buffers across GPU stages. This is the systems counterpart of G3SA's central observation that a collection of fast kernels is not automatically a fast end-to-end mapper.

The current code also adopts a stricter failure policy than G3SA. G3SA can send exceptional reads to a CPU fallback thread. In BioSeqDB, an explicitly selected mandatory Vulkan route is fail-closed. An unsupported tuple, unavailable runtime, one-row wave, failed resident index or missing operation proof becomes an error. The scheduler may choose the CPU or SIMD route before entering the mandatory GPU path, but it may not silently change the meaning of a result labelled as GPU execution.

[1] Y. Han, S. Kim, S. Park and J. Lee, G3SA: A GPU-Accelerated Gold Standard Genomics Library for End-to-End Sequence Alignment, ICS 2025, DOI 10.1145/3721145.3729516.

4.3. The logical alignment chain

The complete generic chain is constructed in one function, `make_minimap_pipeline`, at `pipe.cppm:8573`. This function is used by both execution and description, so adding a stage does not require a separate hand-maintained `EXPLAIN` pipeline. Its current stage order is:

```
[Minimizer] -> Index -> SeedHit -> Chain -> Region
 -> PrimarySecondary -> SecondaryFilter -> Divergence
 -> Prefilter -> Align -> ExactEditFilter
 -> PrimarySecondary -> SecondaryFilter -> Mapq -> Sink
```

The bracketed minimizer stage exists only for `EntryPoint::FromSequence`. A caller with authenticated prebuilt query minimizers uses `EntryPoint::WithPrebuiltMinimizers` and begins at the index hand-off. This is structural omission, not an empty runtime branch.

Algorithm 3 Logical execution of one BioSeqDB query against one prepared target

Require: lazy query Q, lazy target T, target index I, parameters P
Ensure: ordered collection of typed mapping results

if the entry point starts from sequence symbols then
    $M \leftarrow \text{Sketch}(Q, P.w, P.k, P.minimizer_algorithm)$
else
    $M \leftarrow$ authenticated prebuilt query minimizers

$A \leftarrow \text{LookupAndCanonicalise}(M, I, P.max_occ)$
$(C, A') \leftarrow \text{Chain}(A, P.chaining_algorithm)$
$R \leftarrow \text{GenerateRegions}(C, A', |Q|)$
ClassifyAndFilterSecondaries(R, P)
if divergence was requested by the projection then
    EstimateDivergence$(R, M, P.k)$

if a conservative prefilter is enabled then
    RejectImpossibleGapJobs(R, A', Q, T, P)

if coordinates, score, edit distance or CIGAR require extension then
    ExtendRegions$(R, A', Q, T, P.base_alignment_algorithm)$

if an exact edit predicate is active then
    $R \leftarrow \text{ApplyExactEditPredicate}(R, P)$
    ClassifyAndFilterSecondaries(R, P)

if mapping quality was requested by the projection then
    ComputeMapq(R, P)

return ConvertToMatchResults$(R, |Q|, |T|)$

4.3.1. Preparation before the stage chain

Target preparation is deliberately outside the per-query stage chain. A target can arrive as an in-memory compact sequence, a lazy PostgreSQL recipe, a fragment locator, or a

persisted minimizer-index generation. The scheduler resolves an index policy and records whether the consumed target index was recomputed or persisted. With many queries against the same physical target, `CpuPreparedTarget` or `GpuPreparedShapeV5Target` can survive across dispatch waves. This prevents the fixed cost of target sketching and index construction from being charged once per query.

The persisted route is not just a cache hit. Its generation identifier, source identity and storage geometry must match the requested w, k, alphabet and minimizer representation. A cold index may be decoded or selectively read in bounded fragments. The chapter on persistent indexes develops this storage path in detail. At the pipeline level, the invariant is simple: seed lookup must see a complete, well-formed occurrence count and position set for every minimizer it is allowed to consume.

4.3.2. Minimizer and index hand-off

`MinimizerStage` reads the captured lazy query view. It computes a reserve estimate of approximately $|Q|/w + 64$, validates that query positions fit the selected minimizer representation, then calls the scalar, SIMD or Vulkan kernel fixed by its trait. CPU kernels write directly into an `ArenaSlab`. The Vulkan form can return host minimizers together with an owned device span. Empty queries and queries shorter than k are valid and naturally produce an empty batch.

`IndexStage` performs no biological work. It decorates the query minimizer batch with a target-index reference and an optional GPU cache identity. At optimisation level $O3$ this stage is intended to inline away. Keeping it explicit is nevertheless useful because the wire type says exactly when index provenance enters the mapper.

4.3.3. Seed-hit lookup

`SeedHitStage` turns each query minimizer into zero or more `Anchor` records. The scalar operation is implemented by `collect_seed_hits` at `minimap_port.cppm:1109`. It asks the target index for the global occurrence count, discards absent or over-repetitive minimizers using `max_occ`, rejects incomplete persisted postings, converts positions to a strand-consistent query coordinate, and finally establishes one canonical anchor order.

The ordering invariant is important. Chaining algorithms may use different internal traversal or vectorisation schemes, but they all begin from the same reference, query and full-position ordering. BioSeqDB retains 64-bit coordinates in this boundary, avoiding the truncation that would occur for a multi-gigabase target if an encoded position were narrowed to 32 bits.

On the selected Vulkan path, seed lookup requires a resident target index. The query device span from `MinimizerStage` can be reused directly. If index residency, hardware support or the accepted completion token is missing, the selected route throws instead of materialising scalar anchors. This rule is what allows the later execution report to interpret a GPU seed counter as evidence rather than as a request that might have fallen back.

4.3.4. Chaining and region generation

`ChainStage` accepts canonical anchors and emits two arena-backed spans: chains and anchors reordered into chain order. Its kernel trait can select minimap2-style scalar dynamic programming, SIMD dynamic programming, RMQ chaining or Vulkan tiled dynamic programming. The mathematical scoring and each implementation are developed in the chaining chapter.

`RegionStage` is the equivalent of minimap2's `mm_gen_regs`. It sorts chains by descending score, constructs one `AlignRegion` per chain and derives reference and strand-correct query coordinates from the first and last anchors. It also computes the fuzzy seeded match length and block length used by later heuristics. The output estimate is bounded by the chain estimate and `best_n` + 16.

4.3.5. Candidate reduction, divergence and prefiltering

The first `PrimarySecondaryStage` assigns parent relationships between overlapping regions. `SecondaryFilterStage` then compacts the surviving prefix according to the primary ratio, score difference and `best_n`. Both stages mutate the region slab in place.

`DivergenceStage` is projection-driven. If no SQL result expression needs divergence, the stage passes through without paying for the estimate. When enabled, it estimates divergence from seed coverage in the minimap2-derived manner. The same principle controls mapping quality, CIGAR and edit-distance work: optional output fields should not force computation for a query that does not project them.

`PrefilterStage` is a BioSeqDB-specific work-avoidance boundary. It is active only when filtering is enabled and an exact edit predicate makes the rejection useful. For every region it constructs mandatory seed-to-seed gap jobs. SneakySnake or GateKeeper supplies a conservative lower bound on edits. Unsupported geometry is accepted for full alignment, not rejected. A region is removed only when the accumulated lower bound already exceeds the exact threshold. Thus, prefiltering may save extension work but cannot create a false negative under its supported contract. The detailed bit-vector algorithms are reconstructed in the prefilter chapter.

4.3.6. Extension, exact filtering and reclassification

`AlignStage` invokes the fixed base-alignment trait. The available CPU choices are KSW2, Block Aligner and an inter-region batched KSW2 path. The batched path groups short fill problems into width buckets $\leq 32, \leq 64, \leq 128, \leq 256$, while longer fills use the intra-sequence KSW2 kernel. A selected Vulkan extension route uses GPU job records and returns coordinates or a packed CIGAR according to the admitted projection.

After traceback, `ExactEditFilterStage` applies the authoritative edit predicate. This position in the chain is intentional. A prefilter can only prove that some candidates are impossible. Exact edit distance can be known only after the extension result exists.

The primary/secondary classifier and secondary filter then run a second time. Without this second pass, a candidate rejected by the exact predicate could remain the parent of a survivor or influence the survivor's mapping quality. Reclassification makes the final candidate set self-consistent before `MapqStage`. `SinkStage` finally converts regions into `MatchResult` values with 64-bit coordinates, score, chain and seed metadata, optional divergence, MAPQ, edit distance and CIGAR.

4.4. Compile-time algorithm selection and runtime routing

4.4.1. The dispatched tuple

The runtime configuration is represented by `Minimap2Params`. It contains w, k, scoring and chaining limits, output requirements, prefilter settings, algorithm selectors and Vulkan policy flags. The dispatcher consumes the selectors at the top and fixes the tuple

Table 4.1: Contracts of the current logical stage chain. “Pass” means the stage can be present structurally while optional work is disabled.

Stage	Input	Output	Main invariant
Minimizer	lazy query symbols	minimizer span, optionally with device span	deterministic representation and position domain for the selected w, k; valid empty output for short queries
Index	minimizer batch	minimizers plus target-index reference and cache identity	zero-copy hand-off; identity cannot alias another target segment
SeedHit	minimizers and target index	canonical `Anchor` span	global occurrence pruning, complete postings, strand-correct and full-width coordinates
Chain	canonical anchors	`Chain` span plus reordered anchors	selected kernel consumes the same scoring parameters and emits a verifiable execution proof where required
Region	chains and reordered anchors	`AlignRegion` span	one region per retained chain, descending score, valid half-open query/reference coordinates
Primary/secondary	regions and anchors	same region span	parent relation reflects overlap and score among the current candidate set
Secondary filter	classified regions	compact survivor prefix	rejected tail is never observed downstream
Divergence	regions, anchors and query minimizers	annotated regions or pass-through	work occurs only when requested

Table 4.2: Contracts of the final half of the current logical stage chain.

Stage	Input	Output	Main invariant
Prefilter	regions, anchors and lazy sequences	compact candidate prefix	only a proven lower bound above the threshold rejects a region; unsupported jobs continue to exact alignment
Align	candidates, anchors and sequence windows	scored regions with optional edit path	requested coordinates, score and traceback fields are produced by the selected kernel, with no hidden backend substitution
Exact edit filter	aligned regions	exact-predicate survivors	exact edit distance, not the prefilter estimate, is authoritative
Reclassification	exact survivors	self-consistent primary/secondary survivors	removed regions cannot affect parent choice, secondary score or MAPQ
MAPQ	final classified regions	regions with optional MAPQ	work is projection-driven and uses only final survivors
Sink	final regions	`MatchResultCollection`	stable result order and 64-bit PAF-like coordinates, with owned optional CIGAR data

$$(\mathsf{Cep}, \mathsf{BAA}, \mathsf{CA}, \mathsf{MA}, \mathsf{EP}),$$

where Cep is the compile-time window, k-mer and sequence-length bucket, BAA is the base aligner, CA is the chainer, MA is the minimizer algorithm, and EP is the entry point. The concrete cascade is `dispatch_pipeline` at `pipe.cppm:10321`. The execution leaf uses a specialised $w = 10$ parameter object when the runtime values match a registered tuple and otherwise chooses a typed fallback.

Table 4.3: Implemented algorithm axes visible to the current generic pipeline.

Axis	Implementations
Minimizer extraction	scalar minimap2-compatible sketch, early SIMD minimap2 form, two-stack ntHash SIMD, Vulkan minimap-int, and paper-oriented ntHash SIMD
Chaining	scalar minimap2 DP, SIMD minimap2 DP, RMQ, and Vulkan tiled DP
Final alignment	KSW2, Block Aligner, and inter-region batched KSW2; the dedicated Vulkan route also admits GPU coordinate or CIGAR extension shapes
Prefilter	SneakySnake on scalar or SIMD CPU, plus Vulkan SneakySnake or GateKeeper in their supported domains

This design avoids virtual dispatch and repeated option checks within the high-frequency stage calls. It also localises unsupported combinations. A new algorithm must provide the required trait surface, including the kernel type, label and metadata. Static assertions at `pipe.cppm:9987`–`pipe.cppm:9999` verify that every registered enum value has a trait specialisation.

4.4.2. The CPU and SIMD route

Scalar and SIMD are not two completely separate pipelines. They share lazy sequence views, arena lifetimes, index providers and region semantics. A selected SIMD tuple changes the minimizer, chaining or final-alignment kernel while scalar control stages such as region generation and classification remain scalar. This matters when interpreting an "all SIMD" report. The term means that every stage for which a SIMD biological implementation is defined executed on the SIMD engine, not that host I/O or a small bookkeeping loop became a vector instruction.

At statement level the allowed concurrency modes are `SINGLE`, `PARALLEL` and `TASK_PARALLEL`. `PARALLEL` allows the scheduler to distribute independent rows. `TASK_PARALLEL` additionally enables coarse inner regions such as query batches, alignment regions, segment recomputation and fragment builds. The minimum triggers are four queries, four alignment regions, 64 seeds and four fragments. A global `TaskParallelScope` permits only one active coarse nested region. If a worker attempts to open another, the inner operation runs serially. This prevents nested parallel fan-out from multiplying the effective worker count.

4.4.3. The mandatory Vulkan route

Vulkan selection uses two predicates with deliberately different meanings. `vulkan_components_requested` asks whether a full biological GPU component, such as Vulkan sketching, seed lookup, tiled chaining or extension, was selected. `vulkan_accelerator_requested` also includes an independent Vulkan prefilter. The independent prefilter can accelerate one conservative test inside a CPU pipeline, so it must not relabel the entire statement as an all-GPU mapper.

The currently admitted dedicated GPU tuple is checked by `complete_all_gpu_tuple_selected` at `pipe.cppm:8692`. It requires Vulkan capability, Vulkan seed hits, the fused front end, GPU target-index build, Vulkan minimap-int, Vulkan tiled-DP chaining and the Block Aligner selector. Divergence, MAPQ, exact edit filtering and the independent prefilter are excluded from this specialised shape. CIGAR is allowed only when Vulkan extension is enabled.

The PostgreSQL typed scheduler imposes additional physical constraints. A selected GPU wave must contain at least two jobs. It must be coordinate-only or use an admitted bounded GPU CIGAR shape, and the GPU target-index-build route must be active. Runtime failure and an incomplete tuple are errors. For an authenticated persisted Shape-V5 target, the current direct route is coordinate-only. The source explicitly reports that no genuine GPU target-window reader is wired for persisted CIGAR projection and forbids CPU target hydration or CPU/SIMD alignment fallback (`dispatch_processor_typed_impl.inc:5291`–`dispatch_processor_typed_impl.inc:5302`).

This is the most important qualification to the phrase "all GPU". BioSeqDB has implemented Vulkan query sketching, seed lookup, tiled chaining, coordinate reduction and GPU CIGAR/gridded traceback for admitted in-memory or recomputed resident routes. The newer DOC0371 cold-index specification uses a stricter statement-level definition which also forbids host biological planning and target-index rebuild. DOC0374's authenticated Format-5 streaming graph remains the target for that strongest definition. A thesis claim must therefore name the index source and projected fields rather than saying only "the GPU pipeline was enabled".

4.5. Batching, memory and parallel execution

4.5.1. The PostgreSQL dispatch buffer

The custom scan does not call the mapper once per tuple. During its write phase, `TupleDispatchBuffer` copies eligible scan slots and records their query and target operands. Its fixed capacity is 8,192 records. At capacity or end of input, `run_batch_and_build_cursors` invokes the typed processor. The read phase then replays buffered slots, fills result placeholders from the matching cursor and executes the ordinary PostgreSQL projection.

The capacity is a bound, not a truncation rule. A full buffer is drained and the scan continues with the next wave. Each batch has a homogeneous alphabet and packed storage width. `BatchProcessor` dispatches that pair once into the generated typed implementation, avoiding a variant or alphabet switch in the per-row hot loop.

The scheduler supports a broadcast query, a query expression evaluated per row and additional independent query predicates. Additional predicates run over the same active wave and their matching row identifiers are intersected. This gives a database meaning to batching: it preserves the logical AND of SQL restrictions while sharing physical target preparation.

4.5.2. Hydration waves and lazy fallback

Sequence data can be flat, fragmented or represented by a lazy storage recipe. Some selected kernels can consume the lazy facade directly. Others require a flat payload. Before dispatch, the buffer estimates the bytes required to hydrate every row and divides the batch into row-span waves under the currently allocatable budget. It then requests a `MemoryPermit` for the wave.

If the permit is granted, materialised buffers and their permit share a carefully ordered lifetime: storage is destroyed before the permit is released. If hydration is not admitted and the selected route can stream, the target locators remain lazy and are served through `scanner_pg`. The system therefore degrades by reducing materialisation, not by ignoring the budget.

`MemoryAdmissionController::try_reserve` takes one live snapshot of process footprint, system availability, process soft cap, minimum free memory and the optional index cap. It then atomically accounts the reservation. The budgeted parallel helper attempts the requested fan-out, decreases it until a wave fits, and finally uses a serial fallback when even one admitted parallel task would exceed the budget. This is an admission fallback, not an algorithm fallback: the biological kernel remains the one selected by the tuple.

4.5.3. Sharing a prepared target

`pipeline_run_impl_batch` at `pipe.cppm:9018` runs N queries against one target index. It opens one outer arena chain for the batch and one nested arena scope per query, so query-local slabs are reclaimed before the next serial query. A GPU index cache is shared by the batch wave. When Phase-C GPU target-index build is active, the index is built once and inserted under the same identity that `SeedHitStage` will request.

On CPU, a batch of at least four worker-shareable lazy queries may use `parallel_for_pipeline`. A selected Vulkan accelerator disables this CPU query fan-out because the GPU scheduler needs to observe and batch the whole work set. Results retain input-query order in both cases.

The parameter `cross_query_batched` still exists in `Minimap2Params`, and the KSW2-batched trait advertises historical support. It must not be described as an active second scheduling mode. The duplicate deferred-align implementation was deleted and current runtime semantics do not depend on the flag. Cross-query acceleration today is provided by PostgreSQL work grouping and the dedicated GPU batch route, not by the deleted per-query KSW2 accumulator.

Algorithm 4 Memory-aware batch dispatch

Require: buffered SQL rows B, typed layout S, resolved parameters P

group rows by compatible query mode, target identity and index policy
estimate flat hydration bytes for every row
partition B into row-span waves under the live memory ceiling
for all waves W do
    $permit \leftarrow$ TryReserve(EstimatedBytes(W))
    if $permit$ exists then
        hydrate only operands required by the selected kernel
    else
        retain lazy locators and stream through the PostgreSQL scanner
    prepare or reuse one CPU/GPU target object for each physical target
    if MandatoryVulkanTupleIsSelected(P) then
        require at least two jobs, complete tuple and usable runtime
        sketch queries in batch and create query–segment jobs
        execute admitted resident GPU waves with operation proofs
    else
        run typed scalar/SIMD queries serially or through the TBB pool
    scatter result collections back to buffered row identifiers
    finish read sessions, destroy wave storage, release $permit$

4.6. Description and execution evidence

4.6.1. One structural description

Every stage can produce a `StageDescription` containing its name, algorithm, runtime parameters, computed properties and an optional reserve estimate. `PipelineDescription` is the ordered vector of these records. `describe_pipeline_for` uses the same dispatch chain and `make_minimap_pipeline` function as execution. This removes a common source of misleading plans: an explanatory renderer cannot silently retain an obsolete stage order after the executable pipeline changes.

The estimates are explicitly allocation hints, not biological cardinality claims. The sketch estimate is $|Q|/w + 64$. The anchor estimate is the minimum of twice the sketch count and a shared k-mer ceiling. Chain count uses the minimum seed count, region count is capped by `best_n` + 16, and result count uses the larger of a `best_n` margin and a coarse sequence-length floor. Arena slabs grow geometrically when an estimate is low, so correctness is not contingent on the heuristic.

4.6.2. Stable physical stages

The execution-evidence vocabulary is more detailed than the logical C++ stage classes. `PipelineStage` in `pipe.cppm:224` assigns stable ordinals to 20 operations:

```
query sketch, target sketch, target index build,
```

```
persisted index decode, partition selection, device index upload,
seed lookup, chain score, chain backtrack, region generation,
primary/secondary reduction, divergence, MAPQ,
target-window plan, target-window I/O, target-window upload,
CIGAR job generation, left extension, gap extension, right extension
```

This split answers questions the logical chain alone cannot answer. For example, a persisted CPU route should report host I/O for index decode, scalar CPU for partition selection and no target sketch or target-index build. A recomputed GPU route should report GPU target sketch and GPU index build, but no persisted decode. Target-window I/O remains host I/O even when the subsequent upload and extension are GPU work.

4.6.3. Fail-closed validation

For every target–query job, the evidence records requested and actual backend, requested and actual target-index source, persisted generation and source identity, environment identity and a per-stage engine count. `expected_execution_engine` maps each combination to exactly one allowed engine or to an ineligible state. The validator requires:

1. requested and actual backend are equal;
2. requested and actual index source are equal;
3. a persisted source has a non-zero generation and source identity;
4. every eligible stage is credited to only its allowed engine;
5. every ineligible stage has zero work;
6. the unconditional front end executed once; and
7. every fallback counter is zero.

The check in `validate_job_execution_stats` is deliberately fail-closed. Missing or mixed evidence does not become a successful proof. This makes benchmark comparisons stronger than comparing configuration labels. A result row called `vulkan_gpu` must contain the operation tokens, stage coverage and environment identity required for that label.

4.7. Comparison with minimap2 and G3SA

4.7.1. The minimap2 control structure

Upstream minimap2 organises the long-read mapper around `mm_map_frag_core` in `main/bench/tool_minimap2/minimap2/map.c:588`. It sketches the query, collects seed matches from an `.mmi` index, invokes DP or RMQ chaining, may re-chain under a different occurrence threshold, generates regions, performs post-chain processing and extension, then calculates MAPQ. CPU parallelism is primarily expressed as `kt_for` over input fragments, with a worker-local `mm_tbuf_t` allocation arena.

BioSeqDB preserves the recognisable backbone, but not every minimap2 mode. Its current focus is the database-resident long-read and single-target path. It should not be described as reproducing all paired-read, splice, rescue or multi-segment behaviours. The important differences are systems differences: packed lazy sequences rather than ASCII query strings,

PostgreSQL-resident or selectively loaded indexes rather than only an external `.mmi`, typed stages rather than one imperative routine, and query-plan batching rather than a command-line input batch.

4.7.2. What was adopted from G3SA

G3SA treats seed–chain–extend as a resident accelerator graph rather than three unrelated kernels. Its minimap2 route sketches batches of reads, looks minimizers up in a resident index, stores anchors in structure-of-arrays form, sorts them, applies tiled DP and RMQ chaining, packs extension jobs, runs an AGAThA-derived forward kernel and reconstructs the path with gridded traceback. The tiled chaining design places a tile of predecessor scores in shared memory and target scores in registers, reducing repeated global memory traffic on long predecessor ranges. Gridded traceback stores boundary scores rather than an entire direction matrix and recomputes only the grids crossed by the final trace.

BioSeqDB adopts the architectural lessons rather than copying the CUDA program. Its Vulkan route batches query sketches into a device buffer, groups jobs by exact target identity, builds or reuses resident target segment indexes, performs seed count and emission, constructs sorted anchor ranges, applies tiled chain/coordinate work, and optionally executes GPU CIGAR jobs. The public source symbols include `run_phase_c_seed_to_coordinate_multi_query_batch_rou` and the GPU alignment CIGAR wire records. The route is written for Vulkan/SPIR-V rather than CUDA and therefore also targets MoltenVK and Apple GPUs.

There are also deliberate differences. G3SA's double-elongation FM-index seeding belongs to its BWA-MEM short-read branch, not to its minimap2 long-read branch, so BioSeqDB does not attribute that technique to minimizer lookup. G3SA uses a large reusable CUDA memory pool and can send exceptional reads to CPU. BioSeqDB uses global admission permits, resident caches and bounded waves, and forbids silent CPU fallback for a selected GPU SQL statement. BioSeqDB must also preserve database index generation identity, lazy sequence lifetimes, interrupts and result-to-row correspondence, none of which is part of a command-line CUDA mapper.

4.7.3. Measured consequence of batching and residency

The final addendum of DOC0356 points to `main/reporting/bioseq_bench_e2e_batch_gpu_proof/report_47.json`. The artifact was generated on 28 June 2026 from commit `ac58f88500d4ad8652e26e163d2840e5713346b4`, with a dirty-tree marker. It contains no rule violation or regression, but it contains sanity warnings for constant metrics and only one sample per shown configuration. The two rows below are therefore implementation evidence for the named cases, not a general claim that GPU execution is always faster.

The 300 Mbp case executed 608 query–segment problems in 38 waves, built 38 segment indexes and reused them 570 times. Its GPU result set contained 233 rows while the SIMD result set contained 16. The correctness gate proves at least one in-tolerance hit for each of the 16 expected queries, not exact row-set parity. The many-gap case is stronger for final alignment: it reported exactly 4,096 matches and 4,096 correct CIGARs, executed 4,096 gridded jobs, reused one resident index 4,095 times, and recorded zero CPU fallback rows, zero SIMD fallback rows and zero Vulkan declines.

These results support the architectural story rather than an isolated kernel claim. In the 300 Mbp source configuration, query sketching and the seed–chain kernel are a small part of the measured GPU region, while target deduplication and index build dominate. Reusing residency across more queries is therefore as important as accelerating the recurrence. This

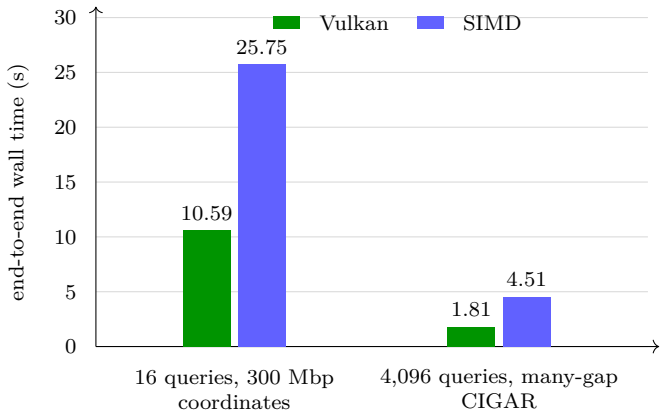


Figure 4.2: The two selected `report_47.json` cases. These one-sample source-configuration measurements demonstrate the effect of query batching and target-index reuse in their tested regimes. They do not establish a universal GPU break-even point.

is why the final solution batches at the database scheduler and why a one-row mandatory GPU wave is rejected.

4.8. Final architecture, invariants and remaining boundaries

The final pipeline is best summarised by its invariants.

1. One executable stage order. Execution and description both use `make_minimap_pipeline`. The exact-edit survivor set is reclassified before MAPQ.

2. One top-level algorithm choice. Runtime selectors are consumed by a pinned dispatch chain. Hot stages see fixed kernel traits, while uncommon shapes retain a typed fallback.

3. Borrowed wires with explicit ownership. Stage values are spans over arena slabs, lazy sequence providers or resident device handles. Their owners outlive downstream use, and permits are released after storage.

4. Target work is batch-scoped. Prepared CPU or GPU targets, persisted readers and resident indexes are reused across compatible query waves.

5. Parallelism is bounded twice. A single TBB arena controls worker concurrency, while memory admission controls the number and size of concurrent materialisations.

6. Accelerator selection is fail-closed. A selected mandatory Vulkan tuple cannot turn into CPU or SIMD biological work without being rejected.

7. Backend claims are evidence, not names. Per-job validation checks backend identity, index provenance, environment and exclusive per-stage engine coverage.

Several boundaries remain and should be stated plainly. BioSeqDB is derived from minimap2's long-read pipeline but does not claim all minimap2 modes. The generic CPU stage chain and the dedicated resident GPU batch route share algorithms and result contracts but are not literally the same instruction graph. Persisted Shape-V5 GPU execution is currently coordinate-only because a genuine GPU target-window reader for that storage form is not

wired. The strongest authenticated, host-biological-work-free Format-5 graph specified in DOC0374 is not yet the universal production route. Finally, small GPU batches can lose to fixed SQL and device overhead even when the kernel is faster.

These qualifications do not weaken the contribution. They explain why the final solution differs from the original direct port. BioSeqDB has turned a procedural mapper into a typed, inspectable and resource-bounded database operator. It keeps minimap2's seed–chain–extend semantics recognisable, while adding lazy packed inputs, persistent indexes, conservative prefiltering, compile-time algorithm traits, controlled CPU parallelism, resident Vulkan batches and execution attestation. The following chapters open the major stages individually, beginning with minimizer extraction.

Table 4.4: Pipeline-level comparison of upstream minimap2, G3SA's minimap2 route and BioSeqDB.

Aspect	minimap2	G3SA minimap2 route	BioSeqDB
Invocation unit	command-line read fragment	fixed CUDA read batch	PostgreSQL custom-scan wave
Sequence form	ASCII query and indexed reference	pinned ASCII input, device decoding and extension encodings	alphabet-aware packed lazy, fragmented or scanner-backed ranges
Index lifetime	external `.mmi`, shared by worker calls	whole minimap2 index copied and retained on device	in-memory, persisted, fragment-native or GPU-resident provider with generation and source identity
Control structure	imperative C routine with runtime option branches	CUDA wrappers around stage-specific batches	runtime-to-template dispatch followed by a typed push-stage chain
CPU parallelism	`kt_for`, worker-local buffer	I/O and exceptional-read fallback threads	TBB task arena with controlled coarse regions and memory-admitted fan-out
GPU execution	none in upstream	CUDA seed–chain–extend with exceptional CPU fallback	Vulkan computational routes with mandatory selected paths failing closed
Candidate filtering	minimap2 occurrence, chain and region heuristics	original pipeline heuristics ported to GPU	minimap2-derived heuristics plus optional conservative SneakySnake/GateKeeper and exact edit predicate
Result surface	PAF or SAM records	packed GPU results, CIGAR and SAM output	`MatchResult` fields substituted into SQL projection
Scope	mature general-purpose mapper	DNA-focused GPU long-read configuration	database-integrated long-read/single-target path with generic alphabets

Table 4.5: Selected end-to-end PostgreSQL cases from `report_47.json`. Speedup is SIMD wall time divided by GPU wall time.

Case	GPU (s)	SIMD (s)	Speedup	Correct output
16 queries, 300 Mbp segmented target, coordinates	10.585	25.746	2.432×	16/16 queries found
4,096 many-gap CIGAR queries	1.809	4.512	2.493×	4,096/4,096 CIGARs

Chapter 5

Minimizer extraction

5.1. From every base to a sparse, comparable sketch

The first computational stage of the BioSeqDB mapper converts a sequence into a much smaller stream of representative k-mers. This stream is called a minimizer sketch. The target sketch becomes an index; the query sketch is looked up in that index; and equal keys become the anchors consumed by Chapter 6. Sketching is therefore more than a sampling optimisation. Its hash function, strand convention, tie rule, invalid-symbol rule, and output encoding jointly define the language spoken by the query and its stored target index. Changing any one of them can turn a fast implementation into a perfectly fast producer of keys that the index can never find.

This observation explains an otherwise surprising property of the final implementation: BioSeqDB retains several minimizer backends instead of declaring one universally best algorithm. The current public backend enum in `main/bioseq/backend/sequences/kmers/index.cppm` contains `Scalar`, `ExactSimd`, `PaperNtHashSimd`, and `VulkanSegment`. Scalar and ExactSimd share the legacy minimap-style scheme. PaperNtHashSimd implements a different, explicitly identified 32-bit ntHash scheme. VulkanSegment reproduces the legacy minimap-style scheme on a device through the Phase C tiled kernel. Persisted index metadata records the selected scheme; dispatch is not allowed to substitute one scheme for another merely because both outputs are called "minimizers".

The implementation history is useful because it exposes three recurring lessons:

1. a scalar algorithm with irregular control flow is not made efficient merely by replacing scalar integers with vectors;

2. a mathematically attractive hash or tie policy is not a transparent replacement for a deployed index scheme; and

3. accelerator throughput matters only after fragment boundaries, ambiguous symbols, output order, memory admission, and host/device transfers have been included in the contract.

The discussion below first defines that contract, reconstructs minimap2 and SIMD-Minimizers, then follows the rejected and retained BioSeqDB designs. Code locations refer to the current checkout. Historical timing claims are tied to the archived report that measured them; they are not presented as current performance.

5.1.1. Formal definition

Let $S = s_0 s_1 \dots s_{n-1}$ be a sequence over an alphabet Σ. For nucleotide sketching, valid bases are normalised to minimap2's nt4 codes

$$\eta(A) = 0, \qquad \eta(C) = 1, \qquad \eta(G) = 2, \qquad \eta(T) = 3,$$

and an ambiguous base such as dna5's N maps to a sentinel. The k-mer beginning at i is $K_i = S[i, i+k)$. Its reverse complement is $\overline{K_i}$, and its canonical orientation is determined from the two encoded integers

$$F_i = \mathrm{enc}(K_i), \qquad R_i = \mathrm{enc}(\overline{K_i}).$$

For the minimap-style scheme, define

$$C_i = \min(F_i, R_i), \qquad z_i = \begin{cases} 0 & F_i < R_i, \\ 1 & R_i < F_i, \end{cases} \qquad h_i = H_{\mathrm{mm}}(C_i).$$

When $F_i = R_i$, the strand is undefined and the candidate is a sentinel rather than an emitted minimizer. A candidate record is

$$x_i = (h_i,\ i + k - 1,\ z_i),$$

where the position denotes the final base of the k-mer, matching minimap2. For every window of w consecutive candidates,

$$W_j = (x_j, x_{j+1}, \dots, x_{j+w-1}),$$

the basic minimizer is an argument attaining

$$m_j = \min_{x \in W_j} h(x).$$

Random hashes make the same candidate the minimum of several neighbouring windows; adjacent windows therefore usually repeat a position. A collector emits changes in the selected identity rather than one record per window. For ideal independent hashes the expected minimizer density is approximately $2/(w+1)$, so $w = 10$ retains roughly one candidate in every 5.5 positions.

The word "identity" is load-bearing. In BioSeqDB's compact `MinimizerRepr`, comparisons mask off position and strand and compare only the hash field, while `does_sketch_match` means "same hash but a different identity". Two equal hashes at different positions are therefore ties to be reported, not duplicate machine words to be discarded. This is why the final ExactSimd implementation carries both a hash vector and raw position/strand metadata through its rare tie path.

5.1.2. The minimap2 scalar reference

The bundled baseline, `main/bench/tool_minimap2/minimap2/sketch.c`, makes the compatibility contract concrete. It maintains two rolling two-bit integers:

$$F_i = ((F_{i-1} \ll 2) \mid c_i) \,\&\, (2^{2k} - 1),$$
$$R_i = (R_{i-1} \gg 2) \mid ((3 \oplus c_i) \ll 2(k-1)).$$

The smaller orientation is mixed by Thomas Wang's reversible integer scramble, with every arithmetic step masked to the $2k$-bit domain. In the C source this is `hash64`; the packed

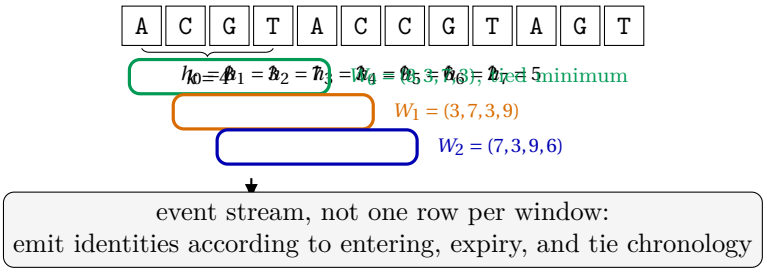


Figure 5.1: A minimizer is the minimum hashed k-mer in each sliding window. The example deliberately contains an equal-hash tie. An implementation must specify which endpoint wins and whether both distinct identities are emitted; "take the minimum" alone is not a complete interoperability contract.

record is `x = hash << 8 | span` and `y = rid << 32 | lastPos << 1 | strand`. BioSeqDB drops fields not needed by its pipeline and instead chooses a 16-, 32-, 64-, or 128-bit `MinimizerRepr` from the position and hash requirements. The common $k = 15$, maximum-position-2^{28} layout uses a single 64-bit word with one strand bit, 28 position bits, and 35 available hash bits.

The scalar window minimum is a circular buffer plus a saved minimum. A new candidate replaces the saved minimum on less than or equal; consequently the entering candidate is the rightmost equal minimum. When the saved candidate expires, the ring is scanned from oldest to newest with a greater than or equal comparison, again ending on the closest (rightmost) equal candidate. The first full window and expiry path also emit other equal-hash, different-identity candidates. This chronology is more specific than the often-quoted rule "deduplicate minimizer positions".

Algorithm 5 Minimap-style scalar event collector, including the semantic branches

Require: packed or decoded sequence S, k, w
initialise forward and reverse rolling encodings
ring$[0..w-1] \leftarrow \infty$; current $\leftarrow \infty$
for each base s_i do
    if s_i is ambiguous then
        reset valid-run length and both rolling encodings
        candidate $\leftarrow \infty$
    else
        roll (F, R); increment valid-run length
        if at least k clean bases and $F \neq R$ then
            candidate $\leftarrow (H_{\text{mm}}(\min(F, R)), i, \text{strand}(F, R))$
        else
            candidate $\leftarrow \infty$
    overwrite the next circular-buffer slot with candidate
    if this is the first full window then
        emit other identities tied with current
    if candidate.hash $\leq$ current.hash then
        emit old current when the stream is armed
        current $\leftarrow$ candidate
    else if the overwritten slot held current then
        emit old current; rescan the ring with $\geq$
        emit other identities tied with the rescanned minimum
unconditionally emit the final live current

An ambiguous input character resets the clean-base counter and rolling k-mer, ensuring that no emitted k-mer spans an N. The original minimap2 C loop skips a palindromic k-mer with `continue`; the BioSeqDB scalar reference instead advances the ring with a max-valued sentinel. The palindrome still cannot become a minimizer, but it consumes a window slot. This was chosen so scalar and vector lanes share one chronological window shape. It is a disclosed structural deviation from minimap2, not an accidental implementation detail.

5.2. Eliminating data-dependent rescans

The minimap2 ring is an excellent scalar design. Its rescan branch is taken only when the saved minimum expires, approximately once every w candidates. Lane-parallel execution changes that probability. With L independent streams and a scalar rescan probability near $1/w$, the probability that at least one lane requests the slow path in a vector step is

$$1 - \left(1 - \frac{1}{w}\right)^{L}.$$

For $w = 10$ and four NEON lanes this is approximately 34.4%, rather than 10%. Each divergent step must mask unaffected lanes or spill state to a temporary matrix. This is why BioSeqDB's first literal SIMD port performed more work even though it computed four hashes at once.

Property	minimap2 `mm_sketch`	BioSeq scalar / ExactSimd	PaperNtHash-Simd	Vulkan Phase C
Candidate hash	canonical two-bit integer, Wang scramble	same legacy scheme	32-bit rolling ntHash, wrapping forward+reverse sum	same legacy minimap-style scheme
Equal minimum	entering/rightmost winner; special duplicate emissions	same event chronology	keep left and right extremes; emit both identities when tied	same event chronology inside each replayed tile
Palindrome	iteration skipped	max sentinel consumes a slot	ntHash endpoint with recorded orientation	max sentinel
Ambiguity	reset clean run	reset; one invalid ring transition, no flush	production packed route is dna4-only and fails closed otherwise	rank LUT, rolling-state reset, k-base re-arm
Fragment boundary	whole byte string	bounded continuation, no reset or replay	bounded segments with explicit proof and boundary dedup	replay halo plus half-open tile ownership
Persisted identity	minimap2 index format	legacy-minimap-v1	`PNT1` (`0x504E5431`)	legacy-minimap-v1

Table 5.1: The four implementations are not interchangeable merely because they all select window minima. The final dispatcher couples a backend to its scheme identity and admissible input representation.

5.2.1. Deque, rescan, and two-stack formulations

There are three standard ways to maintain a sliding minimum:

Rescan. Keep a circular buffer and a current minimum; scan w candidates only when that minimum expires. Expected linear time and very small scalar state, but data-dependent work.

Monotone deque. Remove expired indices from the front and candidates no smaller than the entering value from the back. Amortised $O(n)$, but each candidate executes a data-dependent number of pops. It vectorises poorly for the same reason.

Prefix/suffix or two-stack minimum. Partition candidates into blocks of w. Store prefix minima from the start of the current block and suffix minima from the end of the previous block. A crossing window is the minimum of exactly two precomputed values. The suffix pass runs once every w candidates and always has the same length.

For a hash sequence $a_0, \ldots, a_{n-1}$, let block $B_q = (a_{qw}, \ldots, a_{qw+w-1})$. Define

$$P_i = \min(a_{qw}, \ldots, a_i), \qquad Q_i = \min(a_i, \ldots, a_{qw+w-1})$$

for $i \in B_q$. If a window begins at j and ends at $e = j + w - 1$,

$$\min(a_j, \ldots, a_e) = \min(Q_j, P_e).$$

Both input and control flow are therefore regular. A tie-aware combine can carry (h, p_ℓ, p_r) instead of a single hash, retaining the leftmost and rightmost positions that attain h.

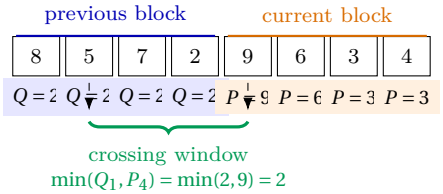


Figure 5.2: The two-stack sliding minimum reconstructed from the SIMD-Minimizers paper. A suffix minimum from the old block and a prefix minimum from the new block answer every crossing window. Rebuilding four suffixes is a fixed four-step operation, independent of the data.

5.2.2. Rolling ntHash

SIMD-Minimizers combines the data-independent minimum with ntHash. Associate each nucleotide with a pseudo-random 32-bit constant $f(b)$ and choose a rotation r (BioSeqDB freezes $r = 7$). For a k-mer beginning at i,

$$H_i^F = \bigoplus_{j=0}^{k-1} \mathrm{rotl}\big(f(s_{i+j}), r(k-1-j)\big),$$
$$H_i^R = \bigoplus_{j=0}^{k-1} \mathrm{rotl}\big(f(\overline{s_{i+j}}), r\,j\big).$$

After the $O(k)$ initialisation, the forward state advances in constant time:

$$H_{i+1}^F = \mathrm{rotl}(H_i^F, r) \oplus \mathrm{rotl}(f(s_i), rk) \oplus f(s_{i+k}).$$

The reverse-complement recurrence rotates in the opposite direction and replaces the outgoing and incoming complement contributions. In the BioSeqDB PNT1 scheme the canonical key is the wrapping 32-bit sum

$$h_i = H_i^F + H_i^R \pmod{2^{32}},$$

while the stored strand records whether $H_i^R < H_i^F$. This differs fundamentally from applying minimap2's integer scramble to $\min(F_i, R_i)$; the two schemes cannot share a persisted index.

The upstream SIMD-Minimizers implementation makes canonical window selection reverse-complement invariant by retaining both extreme equal minima and then choosing the left or right endpoint according to a canonical orientation of the full $(k+w-1)$-base window. Its orientation counter compares the number of G/T symbols with the number of A/C symbols and requires an odd window span to avoid a tie. BioSeqDB's packed PNT1 fork makes a different, explicit choice: it emits the left endpoint and, when the minimum hash is tied, also the right endpoint, then suppresses identical adjacent identities. Thus it keeps a reverse-complement-symmetric endpoint superset without importing the upstream window-orientation counter.

Algorithm 6 Data-independent paper-style ntHash minimizers

Require: dna4 sequence S, compile-time k, w

```
split S into eight logical streams with k + w − 2 bases of context
initialise vector forward/reverse ntHash states and a w-slot ring
for each lock-step packed-symbol vector do
    decode ranks directly from packed sequence words
    update forward and reverse ntHash with rotations/XOR/table lookup
    h ← H^F + H^R (mod 2^32); record per-k-mer strand
    update the block prefix minimum with (h, p_ℓ, p_r, z_ℓ, z_r)
    if the w-slot block wrapped then
        rebuild all suffix minima in a fixed reverse loop
    combine one prefix and one suffix minimum
    emit the left endpoint if its identity differs from the last output
    emit the right endpoint too when the minimum is tied
reconcile the first/last identities of adjacent logical streams
```

5.2.3. Eight logical streams on variable-width SIMD

The original Rust implementation fixes eight 32-bit lanes. It splits the sequence into eight contiguous chunks and overlaps neighbouring chunks by $k + w - 2$ bases so that each owned window has enough left and right context. Window minima are produced lane-wise, transposed back to sequence order, and deduplicated at chunk joins.

BioSeqDB preserves the eight-logical-stream algorithm but implements it with Highway. On AArch64 NEON, four physical 32-bit lanes require two independent vector registers per step; on an eight-lane target one register is enough. The execution proof requires

$$\text{registers-per-step} \times \text{physical-lanes} = 8.$$

Input is not converted to the Rust library's packed type. The kernel consumes the existing `compact_sequence_range` through the project's packed iterator, fixes the final-stream tail mask explicitly, computes segment-local positions, and lets the segmented caller add a checked global base offset. The first event of logical lanes 1–7 is withheld until the preceding lane's last identity is known, so cross-stream deduplication uses only bounded metadata.

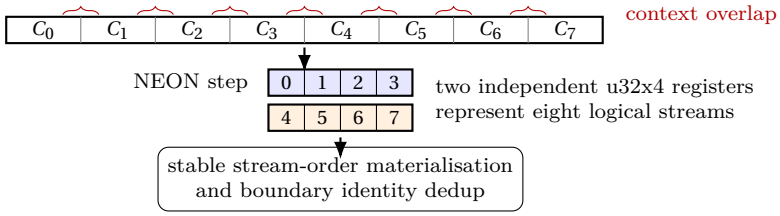


Figure 5.3: The SIMD-Minimizers chunk layout as adapted by BioSeqDB. Logical stream count is an algorithmic constant; physical vector width is a dispatch property. Context overlap makes windows complete, while explicit join metadata prevents a duplicated boundary record.

Platform	Algorithm in SIMD-Minimizers	ns/base range	CHM13 time
AVX2	packed forward ntHash minimizers	1.61–1.69	5.19 s
AVX2	packed canonical minimizers	2.16–2.28	6.71 s
AVX2	scalar/rescan comparison	5.61–11.65	–
NEON	packed forward ntHash minimizers	1.38–1.42	–
NEON	packed canonical minimizers	1.82–1.85	–
NEON	scalar/rescan comparison	2.82–5.95	–

Table 5.2: Published SIMD-Minimizers measurements reproduced from DOC0116 and the bundled paper notes. These are upstream measurements on the paper's inputs and machines, not BioSeqDB benchmark results. They establish the potential of rolling hash plus data-independent minima; they do not prove compatibility with minimap2's hash or event stream.

5.3. BioSeqDB's CPU implementation history

5.3.1. Why the literal SIMD port failed

The earliest vector collector kept minimap2's rescan state independently in each lane. Two bugs arrived before performance could even be judged. First, the strand bit was not carried through every buffer update, so approximately half of reverse-oriented matches disappeared. DOC0107 records roughly 50% accuracy before the fix and 0.958/0.972 parity on its short-/medium fixtures after strand masks, strand buffers, and duplicate-emission propagation were corrected. Second, each lane's data-dependent rescan forced horizontal tests and masked buffer updates. The SIMD unit was busy, but the instruction stream did more bookkeeping than the scalar loop.

Micro-optimisations could not repair that algorithmic shape. DOC0108 replaced general complement work with dna4 XOR, hoisted palindrome masks, and fused lane predicates. Its measured collector moved only from approximately 40.8 to 39.6 ms while the scalar reference remained near 12.7–12.9 ms. Later direct packed access removed an eager conversion, but also exposed an LSB-first ordering error in the first helper. Correcting that error restored identity, not speed.

The long-sequence report DOC0113 gives the honest intermediate result. For the 903 kbp case, whole-pipeline minimap2, scalar BioSeqDB, and SIMD BioSeqDB were 185.6, 158.4, and 170.7 ms; for 5 Mbp they were 1062.1, 994.5, and 1047.2 ms. The vector route was still 5–8% slower than BioSeqDB scalar. The report attributes the gap to horizontal `AllFalse` checks, spills, output-buffer traffic, and runtime shifts. These archived figures should not be compared numerically with the current XXL benchmark: they use different inputs, binaries, and whole-pipeline configurations.

5.3.2. The V4–V15 branch: useful ideas and misleading names

DOC0114 and DOC0118 then moved the collector to a prefix/suffix minimum inspired by SIMD-Minimizers. This branch produced several techniques that remain useful:

- direct traversal of packed sequence storage;
- static $w = 10, k = 15$ specialisation;
- two independent vector streams on NEON to expose instruction-level parallelism;
- explicit transposition and stable compression of lane outputs;

- forced inlining of the packed-iterator callback; and

- preallocation and bounded boundary deduplication.

It also produced an important naming trap. The public benchmark name `two_stack_nthash` was used while some versions ordered canonical two-bit words lexicographically, mixed them with a golden-ratio multiplier, or deferred the minimap scramble until emission. A non-order-preserving scramble cannot be deferred while retaining the same minimizer set. Later DOC0370 forensics found that the exceptionally fast report-76 row was the older resident eight-stream prefix/suffix collector: it disabled palindrome detection on its long dual-stream route and had stream-topology-dependent boundaries. It is therefore historical approximate evidence, not a byte-for-byte baseline for today's ExactSimd continuation.

The optimisation sequence was nevertheless informative. The V8 ordering change roughly halved the expensive inner work, V11 interleaved two independent streams, and V12 removed a surviving out-of-line lambda; the recorded inner phase fell to approximately 9.23 ms on that report's case. V13's largest win was actually downstream: replacing a sort in `mm_est_err` with prefix counts reduced that region from 7.88 to 0.85 ms. A destructive `prefix_kmer` experiment collapsed the effective hash to 14 bits and caused a 36-fold pipeline regression, so it was reverted. V14 implemented a true ntHash table lookup, adding about 2 ms of sketch time while saving only 0.38 ms in seed lookup on its measurement. At that checkpoint it was correctly rejected as a net loss. The later PNT1 backend succeeds because it changes the entire hash/window scheme coherently and records that new scheme, not because a table lookup was dropped into a legacy index.

Checkpoint	Change	What was learned	Reported evidence
V1	strand propagation repaired	semantic parity must precede speed claims	$\sim 50\% \rightarrow 0.958/0.972$
V2	complement/mask micro-tuning	instruction cleanup cannot fix divergent rescans	$40.8 \rightarrow 39.6$ ms
V3	packed storage, smaller state	conversion removal helped but SIMD still lost end to end	170.7 vs 158.4 ms at 903 kbp
V4	first two-stack route	regular control flow is necessary, not sufficient	28.02 vs 14.31 ms sketch
V8–V12	deferred work, two streams, forced inline	callback shape and independent streams materially affect NEON	inner phase ~ 9.23 ms
V13	sort-free error estimate	a minimizer project can be dominated by the next consumer	7.88 to 0.85 ms region
V14	true ntHash experiment	changing hash has both sketch and lookup costs	$+2.0$ ms sketch, -0.38 ms lookup

Table 5.3: Selected historical milestones from DOC0107–DOC0120 and DOC0118. Rows are deliberately heterogeneous: the table reports what each experiment measured, not a fabricated common benchmark series.

5.4. The current ExactSimd continuation

The exact segmented kernel was written to answer a different question from the paper backend: can vector instructions reproduce BioSeqDB's legacy minimap-style event stream over a lazy, fragmented packed sequence without falling back to scalar sketching? Its public checked driver is in `main/bioseq/backend/sequences/kmers/two_stacks_minimizers.cppm`. Despite historical function names containing `nthash`, this path canonicalises the two-bit forward/reverse k-mer and applies the minimap integer scramble. It is not ntHash.

5.4.1. Bounded state across arbitrary fragments

For a static W, K, `TwoStackContinuationState` retains at most $K-1$ normalised nt4 codes, $W-1$ candidate hashes/raw records/ordinals, less than one pending SIMD block, the current minimum, counters, and execution proof. A fragment boundary neither resets nor replays the biological algorithm. The next call completes the pending chronological block and continues from the saved carries. The memory bound depends on K, W, and the vector width, not on sequence length or fragment count.

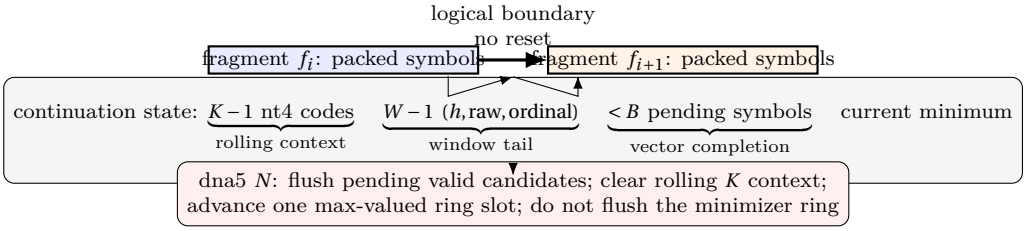


Figure 5.4: Fragment continuation and ambiguity are different events. A storage fragment boundary is invisible to the algorithm. An ambiguous symbol resets the valid k-mer run and advances a sentinel through the same chronological window state used by the scalar reference.

For dna5, packed symbols do not form a continuous bit stream across storage words because three bits per symbol leave unused high bits. The generic decoder therefore prepares source-word addresses and shifts per active lane, then performs rank extraction, rank-to-nt4 normalisation, and ambiguity classification as Highway vector operations. Scalar orchestration is reported separately and never hashes, canonicalises, selects a minimum, or emits a record. Unsupported alphabets, storage shapes, position ranges, or scalar Highway targets return a checked status and are converted into `ExactSimdRuntimeError`; the selected Exact backend does not silently call the scalar collector.

An ambiguous base is also not an implicit fragment finalisation. The kernel flushes any pending valid candidate block, clears the K-mer context and clean-run length, then advances one max-valued candidate through the ring. It does not clear the W-candidate tail, reset its chronological ordinal, or emit the current minimum. End-of-input alone performs minimap2's unconditional final-current flush.

5.4.2. The common $k = 15, w = 10$ B8 specialisation

The generic chronological kernel reconstructs overlapping canonical k-mers and scans W candidates for each vector group. DOC0370 showed why this exact approach regressed: on a 500 Mbp target an older two-lane version executed about 250 million vector batches but 9.75 billion scalar-orchestration steps, behaving like $O(N(K+W)/2)$ rather than a rolling hash plus amortised minimum.

The retained specialisation processes eight candidates at a time. It takes the $K+B-1=22$ needed dna4 codes, SWAR-packs them into 44 bits, extracts eight overlapping 30-bit K15 words through fixed shifts, forms forward and reverse orientations in two independent u32 streams, applies the exact minimap scramble, and carries 64-bit position/strand metadata separately. The $W=10$ reduction still scans ten chronological slots, deliberately using `<=` so each lane ends on the rightmost equal minimum. This is more work than the paper's prefix/suffix combine, but it preserves the deployed event chronology.

Most windows have no equal-hash, different-identity event. The common path therefore performs a stable `CompressStore` directly. A hash-only prefilter guards the expensive raw-aware tie matrix. In report 100 the target prefilter rejected 46,654,429 possible-tie lanes; only 1,051 batches reached the raw verifier and tie matrix. When that rare route is needed, the kernel constructs event 0 (the old minimum) plus up to W tied identities per lane, transposes the event matrix in registers, and stable-compresses rows in chronological order.

Algorithm 7 ExactSimd B8 block, simplified

Require: $K=15, W=10$, eight pending dna4 positions, continuation state
load $22=K+8-1$ context symbols and SWAR-pack them
extract eight overlapping forward K15 integers by fixed shifts
construct eight reverse-complement integers
$c \leftarrow \min(f,r)$; mark palindromes as max sentinels
$h \leftarrow H_{\mathrm{mm}}(c)$; pack (h,p,z) in raw metadata
concatenate the saved $W-1$ candidate tail and eight new candidates
for each of the W chronological slots do
    update eight lane minima using $\leq$ (rightmost equal wins)

derive entering-minimum, expiry, first-window, and final event masks
if no actual equal-hash/different-identity event then
    stable-compress ordinary events directly
else
    build and transpose the exact tie-event matrix
    stable-compress all chronological events

save the final $W-1$ candidates and current minima as continuation

The source still contains a separately attested full-resident `vector_stream` route for borrowed contiguous dna4/u32 data and the historical XXL shape. It is intentionally distinguished in `TwoStackExecutionProof` from the B8 continuation. Because later forensics found that historical route did not satisfy all exact palindrome and boundary semantics, it must be treated as an approximate resident control, not as proof that the segmented Exact scheme has reached report-76 speed. This remaining split is one reason the final performance-qualified SIMD story uses the separately named PNT1 backend rather than relabelling the resident control as ExactSimd.

5.4.3. PNT1: turning the paper algorithm into a persistent BioSeq scheme

The SIMD-Minimizers result was ultimately useful, but not as a drop-in faster implementation of the minimap2 sketch. Its canonical hash selects an extreme minimum using the orientation of the complete $(K+W-1)$-base window; minimap2 instead hashes every canonical K-mer and applies the rightmost-minimum event rules described above. Those definitions can choose different seeds even when both implementations are internally correct. BioSeqDB therefore gives the paper-derived implementation a distinct, versioned identity: `PackedPaperNthash`,

persisted as scheme identifier `PNT1` (`0x504e5431`). This decision converts a tempting hidden substitution into an explicit index contract.

The current PNT1 collector accepts packed dna4 input. It keeps eight logical streams because that is the decomposition used in the original algorithm, while mapping them onto either four or eight physical Highway lanes according to the target instruction set. Forward and reverse-complement rolling hashes are updated with wrapping addition and seven-bit rotations. Each stream retains both extreme equal minima. The upstream canonical algorithm chooses one of those extremes from the complete-window orientation; BioSeq instead emits the left extreme and, when its identity differs, the right extreme as well. This BioSeq tie contract is deliberate: it is deterministic, does not silently erase equal-hash identities, and is encoded in the PNT1 scheme rather than advertised as minimap2 compatibility.

Logical fragments do not reset the rolling state. The continuation contains the previous $K-1$ bases, the minimum candidates required by the eight streams, the absolute origin, and at most one pending block. A fragment is therefore an input-delivery boundary, whereas an ambiguous symbol is a biological reset boundary. The packed dna4 precondition means that ambiguity has already been excluded by the representation. If the proof of packed storage, position width, or supported parameters is missing, the backend fails closed; it does not reinterpret dna5 bytes as two-bit symbols and does not fall through to a differently named scheme.

Algorithm 8 PNT1 collection over a fragmented packed sequence

Require: scheme `PNT1`, packed dna4 fragments $F_0, \ldots, F_m$
verify storage proof, supported (K, W), and position width
initialise eight logical rolling streams and an empty continuation
for each fragment F_i in logical order do
    prepend only the saved $K-1$ context and the pending partial block
    for each complete eight-position block do
        decode the block directly from the packed words
        roll forward and reverse-complement ntHash values in all streams
        update each stream's left and right extreme minima
        emit deterministic PNT1 events owned by the block
    save the bounded continuation; do not flush merely at F_i's end
flush the one logical sequence end and return the scheme-tagged sketch

Table 5.4 separates four implementations that were easy to conflate during development. In particular, the V13 resident experiment is not the ancestor of the persisted PNT1 hash: V13's destructive prefix-hash optimisation collapsed effective entropy to roughly 14 bits and was reverted.

Two consequences are important for SQL users. First, a query sketch and the stored target index must name the same scheme; matching hashes from different families is meaningless even if both fields happen to be 32 bits wide. Second, changing a seed contract is an index-format migration, not a local micro-optimisation. The catalogue and on-disk header can reject a mismatch before the false-negative failure reaches the chaining stage.

5.4.4. Vulkan Phase C: exact extraction on the device

The GPU implementation follows a different optimisation path. Its purpose is not to reproduce the eight-stream PNT1 decomposition, but to execute the minimap-compatible scalar

Table 5.4: Contracts of the paper-derived and compatibility collectors.

Property	SIMD-Minimizers reference	BioSeq PNT1	BioSeq Exact-Simd	Historical resident V13
Hash family	rolling ntHash, 16-bit packed key	rolling ntHash-derived, 32-bit scheme key	minimap2 integer scramble	lexicographic/prefix experiments
Canonical choice	left/right extreme selected from window orientation	both distinct extremes retained	canonical K-mer before scrambling	experimental dual streams
Tie contract	one oriented extreme	left, then distinct right	minimap2 rightmost/event identity rules	not a compatibility contract
Input	packed two-bit DNA	proved packed dna4	one-word exact representations; generic fallback	borrowed resident dna4
Fragment handling	eight overlapped chunks	bounded streaming continuation	bounded streaming continuation	full-resident only
Persisted identity	none	`PNT1`	exact minimap-compatible scheme	none

state machine over device-resident packed sequences. It belongs to Phase C of the GPU pipeline: metadata, packed bases, sketch events, seed hits, and downstream work can remain on the device. This avoids making a fast kernel irrelevant by repeatedly materialising sequences or events on the host.

Relationship to G3SA

G3SA demonstrated that seed generation and alignment preparation can be structured as a resident CUDA pipeline. Its minimap2-derived sketch kernel assigns one block to a read, uses a 32-thread warp and 128-base tiles, and has thread zero compact the events. The supplied reference kernel reconstructs a K-mer from characters for each position, so its core work is $O(K)$ per candidate rather than an $O(1)$ rolling update. It also assumes a small DNA-plus-`N` alphabet and treats an invalid character as a local `continue`; that is not equivalent to resetting every window that spans the symbol.

BioSeqDB adopts the useful system-level lesson—keep the data and successive stages resident—but not those sketch semantics. The implementation uses Vulkan compute shaders, packed generic sequence descriptors, rolling forward/reverse encodings, an exact minimap scramble, and an explicit count/scan/emit protocol. Table 5.5 makes the distinction concrete.

Tiles are storage units, not independent sequences

An output event depends on the preceding $K-1$ bases and on a minimizer window of W candidate positions. Starting every tile with empty state would therefore invent resets and lose boundary events. Phase C solves this with replay and ownership. For a tile whose owned trigger interval is $[b, e)$, the shader begins earlier by a conservative overlap currently bounded by $K+W+1$, reconstructs the exact state, and emits only events whose trigger position lies in $[b, e)$. Neighbouring tiles may recompute context, but they cannot both own the same event.

Table 5.5: G3SA's reference sketch kernel and BioSeqDB Vulkan Phase C.

Dimension	G3SA reference	BioSeqDB Phase C
Programming model	CUDA, one block per read	Vulkan compute, grid-stride tile ownership
Source representation	character DNA/`N`	descriptor-proved packed dna4 or generic rank lookup
K-mer update	reconstruct K symbols per position	rolling forward/reverse state, $O(1)$ per position
Tile shape	128 bases, 32-thread warp	planner target about 256 bases; representation-aligned storage span
Event compaction	thread 0 after block work	count pass, device prefix scan, then bounded emit pass
Ambiguity	skip the current position	reset state and invalidate every crossing window
Boundary ownership	read-local	replay overlap plus half-open owned trigger interval
Compatibility target	minimap2-inspired	scalar BioSeq/minimap event stream, including ties and final flush

The planner targets approximately 256 logical bases and rounds storage according to the sequence representation; a dna5 packing shape therefore commonly produces 252-base logical tiles. The ABI bounds the supported span and currently admits up to 384 bases including replay. Those numbers are planner parameters, not algorithmic constants: correctness follows from replay coverage and half-open ownership, not from the nominal tile size.

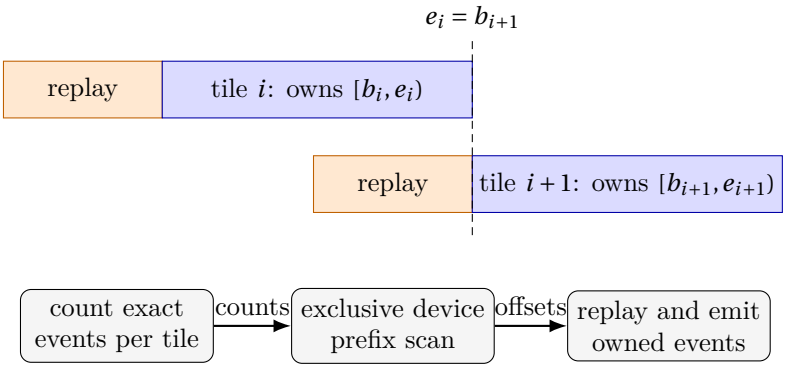


Figure 5.5: Phase C tile replay and ownership. Recomputed context overlaps, but the emitted trigger intervals are disjoint. The two kernel passes give every tile an exact, race-free output range.

The output cardinality is data dependent: ties and expiry events mean that a window does not imply exactly one stored record. A one-pass kernel would need a large pessimistic buffer or a contended global append counter. Phase C instead executes the same state machine twice. The first pass counts events per descriptor; an exclusive prefix scan assigns exact offsets; the second pass replays and writes only to the assigned interval. This provides deterministic event order and makes allocation failure visible before emission.

Algorithm 9 Phase C exact tiled minimizer extraction

```
Require: packed sequence descriptors, (K, W), half-open tile ownership
for pass ∈ {count, emit} do
    for all tiles in a grid-stride loop do
        start at the tile's replay origin with empty scalar-equivalent state
        for each logical symbol through the owned interval do
            decode rank from the representation-specific packed layout
            if the symbol is invalid then
                reset rolling encodings, ring, minimum, and valid-run length
            else
                roll forward and reverse encodings; canonicalise and scramble
                apply palindrome, rightmost-tie, expiry, and final-event rules
                for all events triggered at this position do
                    if trigger is in the half-open owned interval then
                        count it, or write it at the scanned tile offset

    if pass is count then
        exclusive-scan tile counts and allocate the exact output
```

Why the first GPU version was not the final one

The development record is instructive because each intermediate version was plausible in isolation:

1. Phase B used a mixture of host metadata and specialised GPU work. A 4096-base dna5 case produced 859 events where the scalar oracle produced 795. Changing padding and inserting `N` symbols did not remove the excess, pointing to ownership/state rather than input corruption (DOC0348).

2. The Phase C ABI made packed storage and descriptor metadata available everywhere, then DOC0351–DOC0352 retired the Phase-B route and forced one semantics-bearing path for all supported representations.

3. The first Phase C shader still reconstructed too much state and left work concentrated in too few lanes. DOC0353 replaced that structure with rolling hash updates and an all-lanes, grid-stride tile schedule. The output remained bit-exact while sketch throughput increased from about 73 to 494 Mbp/s in the recorded benchmark.

This sequence matters more than the name of a kernel. The final design is the combination of an ABI proof, explicit replay ownership, exact scalar-equivalent event semantics, and a parallel schedule. Removing any one of those properties reproduces a previously observed failure mode.

5.4.5. Measurements, provenance, and backend choice

Minimizer benchmarks are unusually easy to misread. A shorter sketch can make chaining faster while reducing sensitivity; a warm target index can hide construction cost; and a process exit status of one can mean either a crash or simply no complete end-to-end match. The tables below therefore report measurements only with their algorithm contract and experiment scope.

Final CPU comparison

The latest qualifying CPU comparison available while this chapter was written is report 103 in DOC0370. All three rows produced the expected result (one primary alignment and zero base-pair error). This was a controlled component/pipeline benchmark, not a promise of end-to-end ingestion throughput. The PNT1 rows use a PNT1-built target index; the minimap2 and ExactSimd rows use their compatible index contracts.

Table 5.6: Qualifying CPU minimizer results from DOC0370 report 103. Times are seconds unless marked otherwise; lower is better.

Configuration	Sketch	Target index	Wall	Alignment
ExactSimd + SIMD chain/backtrack + Block	9.256	7.123	17.291	not isolated
PNT1 + SIMD chain/backtrack + Block	2.923	included in run	4.046	345.548 ms
PNT1 + SIMD chain/backtrack + KSW2	2.939	included in run	4.342	not isolated
minimap2 reference route	4.733	included in run	11.577	reference

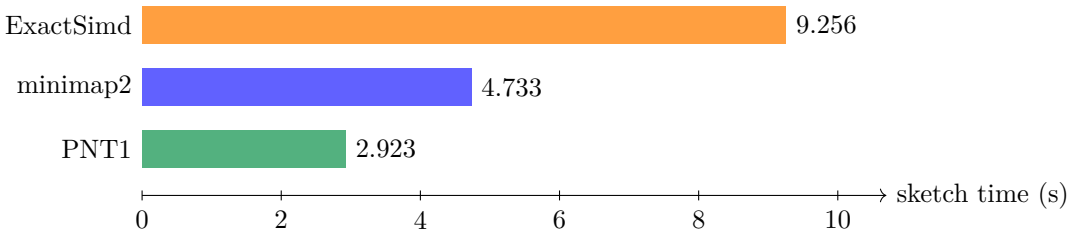


Figure 5.6: Sketch time in the report-103 qualifying comparison. PNT1 is a different persisted seed scheme, so this chart compares complete valid configurations rather than interchangeable function bodies.

The result closes the loop begun by the disappointing DOC0113 microbenchmark. Merely vectorising minimap2's scan did not help: it remained 5–8% slower than the scalar implementation on the long inputs reported there. The final speed came from changing the scheme explicitly, preserving the new scheme in the index identity, and optimising the downstream consumers for its output. PNT1 sketching is about $4.733/2.923 = 1.62\times$ faster than the recorded minimap2 route and $9.256/2.923 = 3.17\times$ faster than the exact segmented SIMD route in this experiment. These ratios must not be quoted as universal instruction throughput: they include the tested sequence shape, build, and target machine.

GPU kernel progression

DOC0353 records the controlled before/after measurement of the Phase C shader rewrite shown in Table 5.7. The primary evidence is bit-exact output together with the kernel improvement; the total probe also improved because sketching ceased to dominate the GPU pipeline.

This is a $6.75\times$ kernel improvement and a $1.87\times$ whole-probe improvement for that workload. It does not establish that Vulkan beats PNT1 on every host, nor that the entire 2/4 GB all-GPU pipeline has been qualified. DOC0369 explicitly records that the accepted 2/4 GB runs available at that point exercised the CPU minimizer variants; the bounded 4096-base Phase C proof established correctness but was then slower than the minimap2 control. The honest conclusion is narrower: Phase C removed the structural GPU sketch bottleneck and

Table 5.7: Phase C rolling/all-lanes rewrite from DOC0353.

Phase C shader	Per-call sketch	Throughput	Total probe
reconstructing/under-parallel version	272.0 ms	73 Mbp/s	21.83 s
rolling hash, all-lanes grid stride	40.3 ms	494 Mbp/s	11.70 s

produced exact device events, while large-scale cross-backend qualification remained future measurement work.

Why older “wins” are retained but not promoted

The archive contains faster-looking numbers which are valuable as engineering history but invalid as final claims. Report 76 measured a 1.982 s resident vector route against a 9.416 s scalar route, yet later forensic work found approximate lexicographic/golden-ratio hashes, missing dual-stream palindrome semantics, and topology-dependent boundaries. Report 90 restored exactness but regressed to 66.273 s; later specialisation reduced that to 13.413 s and eventually the report-103 9.256 s. Those experiments show why performance must be attached to an audited contract, not merely to the class name `TwoStackMinimizerAlgorithm`.

The same caution applies to the SIMD tie path. Report 100 recorded 40,464,168 candidate batches and 46,654,429 rejected ordinary lanes, with only 1,051 raw/tie-matrix batches reaching the expensive exact path. That demonstrates that the rare-path prefilter works; it does not by itself prove that the remaining ordinary path beats scalar code.

5.4.6. The resulting selection policy

No single collector is silently declared “best”. The pipeline chooses an algorithm only after it has chosen a compatible sketch/index scheme:

`Scalar`. The portability backend and semantic oracle. It is the reference for invalid-symbol resets, equal-hash identities, duplicate events, position widths, and final flushes.

`ExactSimd`. The segmented minimap-compatible implementation. It is selected when exact compatibility is required and its representation, K/W shape, and continuation proof are supported. The B8 path removes rescans and sends only genuine hash-identity collisions to the expensive tie matrix, but current measurements do not justify claiming a general scalar speedup.

`PaperNtHashSimd`. The performance-qualified CPU scheme for packed dna4 when both sides carry the PNT1 identity. It realises the eight-stream rolling idea while making BioSeq’s tie rule and hash width part of a persistent contract. It cannot open or query a minimap-compatible index by pretending that a minimizer is just a 32-bit integer.

`VulkanSegment`. The exact minimap-compatible device route when Phase C admission proves the representation, tile span, buffer capacity, and device features. It is favoured when the next stages remain resident. Admission failure returns a typed unsupported result or chooses an explicitly compatible host route; it never launches a best-effort shader.

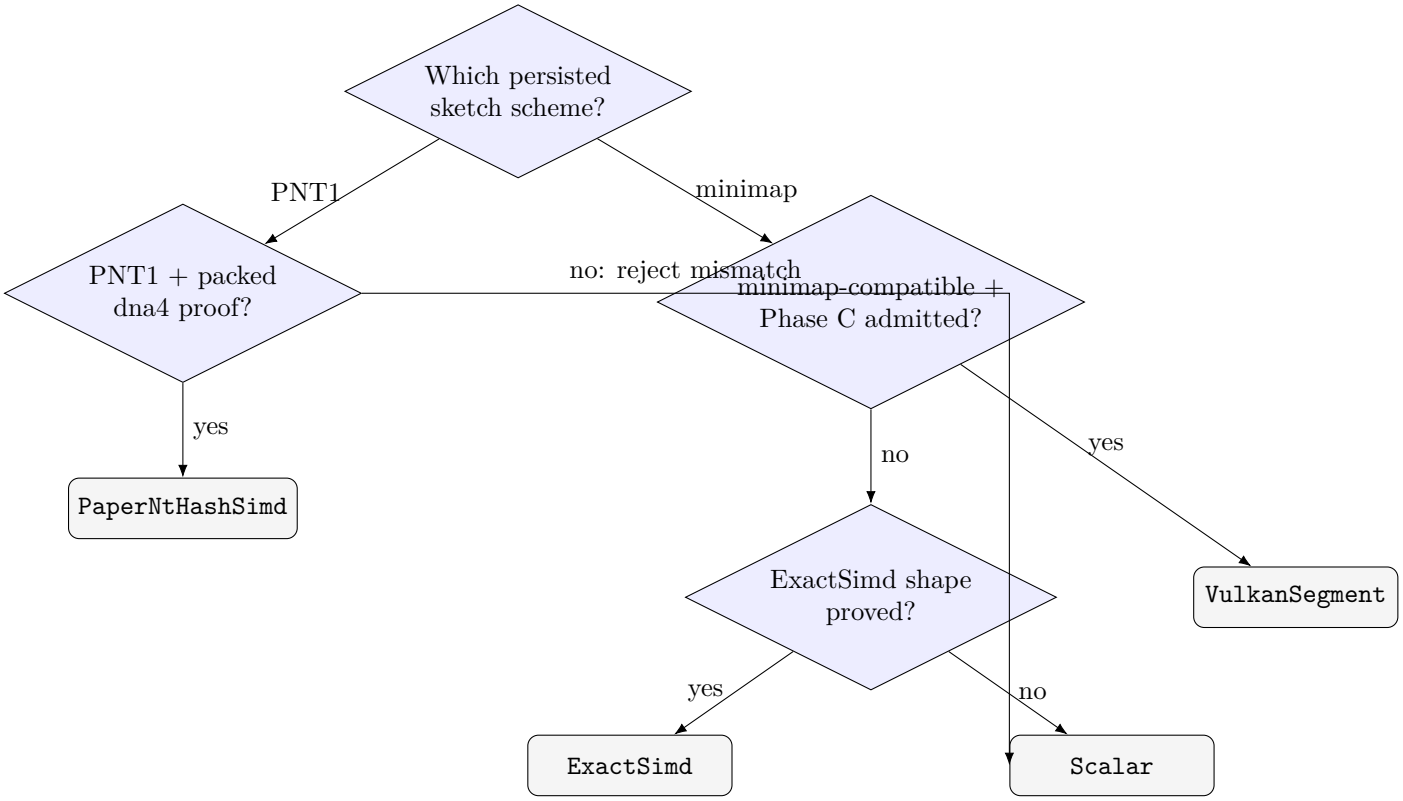


Figure 5.7: Backend selection is subordinate to the persisted scheme and admission proofs. A fast implementation is never substituted across an incompatible seed definition.

5.4.7. Limitations and open measurements

Several limitations remain visible rather than hidden behind automatic fallbacks. PNT1 currently requires proved packed dna4 and a supported position width. ExactSimd's best-specialised continuation is the common $K = 15, W = 10$, eight-position shape; less favourable parameters use a generic path or scalar code. The Phase C tile owns one serial scalar-equivalent state, so positions within a tile are not independently parallel; parallelism comes from many tiles and grid-stride work distribution. Count and emit deliberately run the state machine twice to obtain deterministic compact output. Finally, the historical full-resident route remains in the source for comparison and must not be mistaken for an exact compatibility proof.

The most useful next measurements are therefore not another isolated hash loop. They are: a preregistered 2/4 GB comparison of complete PNT1 and Phase-C configurations; cold and warm target-index builds reported separately; adversarial ambiguity at every fragment and tile boundary; tie-heavy synthetic inputs that force the exact identity path; and cross-machine throughput with the emitted seed stream compared byte-for-byte against the appropriate oracle.

5.4.8. Implementation evidence map

Table 5.8 records the authoritative code regions used for this chapter. Line numbers describe the version inspected for this draft and should be refreshed if the modules are reformatted. The archive was used to explain the path to the implementation; current call sites and scheme identifiers were used to decide what the implementation is now.

Table 5.8: Primary implementation evidence for minimizer extraction.

Source	Lines inspected	Evidence
`main/bioseq/backend/sequences/kmers/2bit_exact.cppm`	70–220; 776–968	layout widths, persisted sketch fields, backend enumeration, scalar oracle, ambiguity and tie/final-event rules
`main/bioseq/backend/sequences/kmers/2bit_stacks_minimizers.cppm`	557–629; 2367–3435; 4036–4085; 5461–6020	compile-time proofs, exact B8 path, PNT1 streams, resident control, and current pipeline routing
`main/bioseq/backend/sequences/kmers/minimizers_vulkan.cppm`	1–1438	kernel enumeration, buffer protocol, dispatch, and patch regions
`main/bioseq/backend/gpu/vulkan/phase_d_abi.cppm`	1–584	supported representation admission descriptor layout, replay/span bounds, and Phase C proof obligations
`main/bioseq/backend/gpu/vulkan/shader_compiler.cppm`	866–1268	generated count/emit shader, rolling hash, reset logic, and ownership filter
`main/bench/tool_minimap2/minimap2_sketch.c`	106–171	upstream rolling state, palindrome handling, rightmost ties, duplicate events, ambiguity reset, and final flush
`simd-minimizers-master/src/`	`canonical.rs`, `minimizers.rs`, `sliding_min.rs`	eight-stream canonical orientation and packed prefix/suffix minima
`G3SA-main/minimap2/` and `G3SA-main/g3sa_paper.pdf`	sketch kernel and method	CUDA block/read mapping, 128-base tiling, character input, and resident GPU pipeline comparison

5.4.9. Conclusion

Minimizer extraction began as a familiar rolling-hash loop and became a lesson in systems correctness. The observable object is not merely the minimum hash of a window. It is an ordered stream whose identity depends on canonicalisation, hash width, tie direction, equal-hash K-mer identity, palindromes, ambiguity resets, fragment boundaries, position width, and the final flush. The scalar implementation made those rules explicit. The two-stack work showed how to remove rescans without changing them, and its failures showed that SIMD width alone does not provide a speedup.

The final CPU solution therefore exposes two honest choices: ExactSimd retains the minimap-compatible contract, whereas PNT1 versions a faster paper-derived contract and binds it to the stored index. The final GPU solution adopts G3SA's resident-pipeline insight but reconstructs the sketch around BioSeq's packed representations, exact reset semantics, replay ownership, and deterministic count/scan/emit allocation. In both cases the governing design principle is the same: an optimisation is accepted only when the seed scheme, storage proof, continuation state, and measured downstream result agree.

Chapter 6

Chaining seed anchors

Chapter 7

Chaining: from local seed hits to candidate mappings

Minimizers identify short local agreements between a query and a reference, but they do not by themselves constitute an alignment. A repetitive genome can produce thousands of mutually incompatible hits, while a true mapping is usually represented by a sparse, approximately collinear subset. The purpose of chaining is to select that subset. It is therefore the first stage at which the mapper reasons about the geometry of a complete candidate mapping rather than about individual k-mers.

This chapter presents the chaining implementation that is actually used by BioSeqDB. The distinction is important because the project contains several generations of experiments: a minimap2-compatible scalar dynamic program, an augmented-tree RMQ formulation, two SIMD policies, and a Vulkan compute path inspired by G3SA. Some historical variants were faster but approximate; some apparently attractive SIMD changes regressed because they increased instruction-cache pressure; and the GPU path deliberately changes the candidate set for very long ranges. The final system consequently exposes these implementations as explicit policies instead of pretending that they have identical semantics.

7.1. The geometric problem

After seed lookup, BioSeqDB represents a hit by an anchor

$$a_i = (r_i, q_i, \sigma_i, \ell_i, g_i, \phi_i),$$

where r_i and q_i are the reference and query end coordinates, σ_i is the strand, ℓ_i is the query span contributed by the seed, g_i is the query segment identifier, and ϕ_i contains the remaining flags. The current C++ object occupies 24 bytes: both positions are full 64-bit values, the strand has a dedicated 32-bit field, the span is an 8-bit value, and the segment and flags occupy the remaining fields. The 64-bit storage matters for database-resident references because a concatenated reference collection need not fit in the 32-bit coordinate model assumed by older stand-alone mappers. It does not, however, imply end-to-end 64-bit chaining arithmetic: the current scalar and SIMD pair scorers narrow coordinate differences to signed 32-bit values. This limitation is revisited below.

Figure 7.1 gives the usual dot-plot interpretation. Each anchor is a point. A plausible chain advances in both axes and remains near one diagonal; isolated points and points on an incompatible diagonal are decoys. A deletion relative to the reference increases r faster than

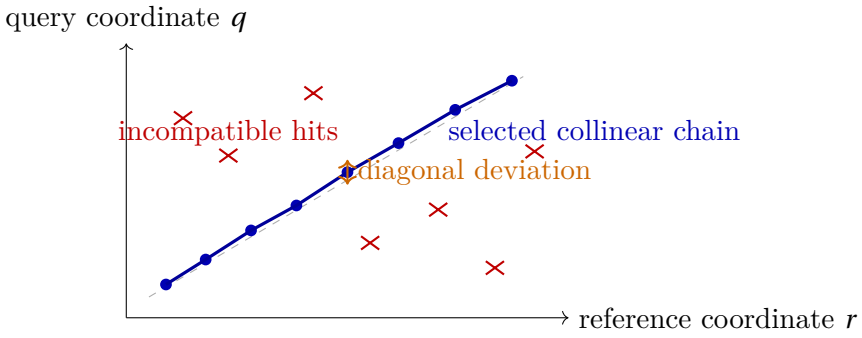


Figure 7.1: Anchor geometry. Chaining selects an increasing, approximately collinear path; the final alignment stage subsequently explains the gaps between its anchors.

q; an insertion does the opposite. Chaining does not resolve either event base by base. It merely pays a gap cost and passes the selected region to the final aligner.

7.1.1. Canonical order is part of the contract

All forward dynamic programs assume anchors are sorted first by strand and reference coordinate, then by query metadata and coordinate. BioSeqDB obtains this order through two packed keys:

$$\begin{aligned} K_r(a_i) &= (\sigma_i \ll 63) \mid r_i, \\ K_q(a_i) &= (g_i \ll 48) \mid (\phi_i \ll 40) \mid (\ell_i \ll 32) \mid (q_i \bmod 2^{32}). \end{aligned} \tag{7.1}$$

The comparison is lexicographic in K_r, K_q, and finally the full q_i. The last comparison is not redundant: the packed query key retains only the low 32 coordinate bits, so it disambiguates anchors only when those packed low words are equal. The resulting order is a compatibility order, not a globally increasing full-q order once coordinates exceed 2^{32}. The present pipeline normally chains one query whose length is far below that boundary; a future database-wide query-coordinate space would require a different key and arithmetic contract.

The packed keys should not be confused with the physical representation of an anchor. They are comparison projections. Earlier project reports describe a 16-byte anchor and sorting networks designed around that historical layout; the present implementation is the 24-byte, full-coordinate representation. This is one example of why the thesis gives precedence to current source and to the most recent design documents over older benchmark notes.

The stored seed positions are inclusive k-mer end coordinates. For a forward chain, the region stage converts the first anchor to $q_{\text{start}} = q_{\text{first}} + 1 - \ell_{\text{first}}$ and the last anchor to $q_{\text{end}} = q_{\text{last}} + 1$. Reverse-strand anchors are normalized before chaining and converted back as $[L - q_{\text{end}}, L - q_{\text{start}})$, where L is query length. External mapping regions are consequently conventional zero-based, half-open intervals even though the DP operates on inclusive seed endpoints.

7.2. The scalar dynamic program

The scalar implementation is the semantic reference for SQL execution. It is derived from minimap2's single-segment, non-cDNA chaining recurrence, while specializing away branches

that BioSeqDB does not need at this stage.[1] Let

$$d_r(i,j) = r_i - r_j, \qquad d_q(i,j) = q_i - q_j, \qquad \delta(i,j) = |d_r(i,j) - d_q(i,j)|,$$

and let

$$h(i,j) = \min\{d_r(i,j), d_q(i,j)\}. \tag{7.2}$$

A predecessor is geometrically admissible only when it is earlier on the same strand, both coordinate differences are positive, it lies within the active reference and query windows, and $\delta(i,j) \le B$, where B is the chaining bandwidth. The outer loop maintains the reference-distance condition; the pair scorer checks the remaining conditions. Writing λ_g and λ_s for the gap and skip coefficients, BioSeqDB assigns the edge contribution

$$\alpha(i,j) = \min\{\ell_j, h(i,j)\} - \begin{cases} 0, & \delta(i,j) = 0 \ \wedge \ h(i,j) \le \ell_j, \\ \left\lfloor \lambda_g \delta(i,j) + \lambda_s h(i,j) + \frac{1}{2}\,\mathrm{fastlog}_2(\delta(i,j)+1) \right\rfloor, & \text{otherwise.} \end{cases} \tag{7.3}$$

All terms inside the floor are non-negative, so the conversion in the C++ implementation is equivalent to truncation toward zero. The logarithmic term is evaluated only for a nonzero diagonal difference. The default coefficients are derived from the minimizer length k: $\lambda_g = 0.01 k s_g$ and $\lambda_s = 0.01 k s_s$, with default scales $s_g = 0.8$ and $s_s = 0$. Other default limits are a reference gap of 5000 bases, bandwidth 500, 25 skip events, and at most 5000 examined predecessors.

The best score ending at anchor i and its predecessor are

$$\begin{aligned} f_i &= \max\left(\ell_i, \max_{j \in W_i}\{f_j + \alpha(i,j)\}\right), \\ p_i &= \arg\max_{j \in W_i}\{f_j + \alpha(i,j)\}, \end{aligned} \tag{7.4}$$

where W_i is the bounded set of admissible earlier anchors. A chain may begin at i, in which case $f_i = \ell_i$ and $p_i = -1$. This is a sparse longest-path problem in a directed acyclic graph: sorting provides the topological order, and every admissible edge points forward.

The effective reference and query search limits in the scalar entry point are both raised to at least the bandwidth, $M = \max(\text{max-gap}, B)$. This avoids the contradictory state in which a configured diagonal band is wider than the coordinate window that is supposed to contain it. The iteration cap then takes the newest max-iterations anchors from that geometric window.

7.2.1. Why the control flow is semantically observable

Equation 7.4 is the conceptual objective, but a faithful implementation must also preserve several details of minimap2's bounded search. Candidates are visited newest-to-oldest, an update requires a strict improvement, and therefore the newest predecessor wins a score tie. The `max_skip` condition is not simply "stop after 25 bad anchors." The algorithm stores generation marks and increments the counter when a non-improving candidate's own predecessor was already encountered in the current generation. A later improvement decrements that counter. Reordering the scan or reducing a vector batch before replaying these events can therefore change both the stopping point and the selected predecessor even when every pair score is numerically exact.

[1] H. Li, "Minimap2: pairwise alignment for nucleotide sequences," Bioinformatics 34(18), 2018. The exact code comparison in this chapter uses the bundled `main/bench/tool_minimap2/minimap2/lchain.c`.

Algorithm 10 BioSeqDB scalar chaining, with control semantics made explicit

Require: anchors sorted by the canonical order; chaining parameters

for $i = 0, \ldots, n-1$ do
  expire predecessors outside the strand/reference window
  $f_i \leftarrow \ell_i,\ p_i \leftarrow -1,\ c \leftarrow 0$
  $j_{\min} \leftarrow \max(\text{window-begin}, i - \text{max-iterations})$
  for $j = i-1, \ldots, j_{\min}$ in descending order do
    if (a_j, a_i) is geometrically admissible then
      $v \leftarrow f_j + \alpha(i, j)$
      if $v > f_i$ then
        $f_i \leftarrow v,\ p_i \leftarrow j,\ c \leftarrow \max(0, c-1)$
      else if $p_j \geq 0$ and predecessor p_j was marked for i then
        $c \leftarrow c + 1$
        if $c >$ max-skip then
          break
    mark j as visited by generation i
  if the best earlier global endpoint was not scanned and improves f_i then
    adopt it as p_i ▷global-best rescue
return f, p

The global-best rescue is a second subtlety. Early stopping can leave an old, high-scoring endpoint outside the scanned suffix. BioSeqDB tracks the best completed endpoint and tests it separately if it was not visited. This does not turn the bounded heuristic into exhaustive dynamic programming, but it protects an important class of long-range paths. Both the exact SIMD implementation and the endpoint-qualification corpus test this behavior explicitly.

The scalar score corresponds to the non-cDNA, single-segment path of the bundled minimap2 `lchain.c`. Upstream minimap2 also carries multi-segment and cDNA-specific branches in the same routine. Those branches are not latent features of BioSeqDB's scorer and should not be inferred from the common ancestry. Conversely, BioSeqDB preserves full 64-bit reference and query coordinates in the anchor object rather than relying on a 32-bit stored position.

There is nevertheless a live large-coordinate boundary. The scalar scorer casts $q_i - q_j$ and $r_i - r_j$ to signed 32-bit integers before testing them, and the SIMD scorer intentionally subtracts low 32-bit words. The full-width reference window prevents ordinary distant reference candidates from reaching the scorer, but a query difference larger than 2^{31}, or one congruent to a small value modulo 2^{32}, can be misinterpreted. The Vulkan shader checks unsigned 64-bit distances before narrowing and does not share this particular failure mode. Thus BioSeqDB has 64-bit anchor storage, but scalar/SIMD chaining is presently proven only for bounded per-query coordinate differences.

7.3. Recovering chains and competing mappings

The forward pass computes an endpoint score for every anchor, not a single mapping. Backtracking ranks endpoints by the pair (f_i, i), walks them in descending score order, and follows p_i until the path terminates or reaches an anchor already claimed by a stronger path. A newly recovered path is kept only when it satisfies both the minimum chain score and minimum seed-count thresholds (defaults 40 and 3). The stored chain record contains its score, number of anchors, and offset into the compacted anchor array.

If a walk merges into an already claimed path, the score attributed to the new prefix is reduced by the score at the merge endpoint. This prevents a weak alternative from inheriting the entire score of a stronger chain merely because the two share a suffix. Rejected traversals remain marked, which also affects later candidate endpoints. The resulting list represents competing candidate regions; later mapping-region logic decides parent/secondary relationships and computes mapping quality.

The shared scalar/RMQ backtracker is intentionally simpler than the complete upstream minimap2 `mg_chain_backtrack` plus `compact_a` sequence. In particular, the current BioSeqDB helper has no explicit `max_drop` parameter and does not reproduce minimap2's final target-position ordering step byte for byte; the forward peak-score array is passed to the helper but currently unused. Thus "minimap2-compatible score and predecessor search" is a more accurate description than "identical minimap2 chaining." This boundary is consequential when interpreting exactness tests: they verify the project-defined contract and selected endpoint coordinates, not every internal array permutation of the upstream program.

7.3.1. From competing chains to primary and secondary regions

After coordinate derivation, the pipeline score-sorts chain regions and treats the highest-scoring region as primary. For each later region it computes the union of query overlap with existing primaries and the query span not explained by that overlap. Let o denote overlap, u uncovered length, and $m_{\min}, m_{\max}$ the smaller and larger spans of the two regions. The candidate becomes secondary when

$$\frac{o}{m_{\min}} - \frac{u}{m_{\max}} > \text{mask-level} \quad \wedge \quad u \le \text{mask-length}.$$

This formulation discounts a candidate that overlaps strongly but also adds a substantial novel query interval. The defaults use a mask level of 0.5 and an effectively unbounded mask length.

Filtering always retains primary regions. A secondary is retained only when its score is sufficiently close to its parent:

$$s_{\text{secondary}} \ge \rho\, s_{\text{parent}} \quad \vee \quad s_{\text{secondary}} + \Delta_{\min} \ge s_{\text{parent}},$$

subject to a global `best_n` limit (default 5); coordinate-identical duplicates are removed. The default score ratio is $\rho = 0.8$. These rules are downstream of the chaining DP, so a change in backtracking order can still change which candidate is presented first and hence influence classification.

7.4. An augmented-tree RMQ formulation

The scalar recurrence can examine up to I predecessors per anchor, giving $O(nI)$ work in a dense range. BioSeqDB also implements the RMQ reformulation used by modern minimap2.

The insight is to separate the part of the edge penalty that is linear in the predecessor coordinates. Candidate j receives the priority

$$\pi_j = -\left[f_j + \frac{\lambda_g}{2}(r_j + q_j) \right]. \tag{7.5}$$

An AVL tree is keyed by (q_j, j), and every node caches the minimum priority in its subtree. This supports insertion, expiry, and a minimum query over a query-coordinate interval in $O(\log n)$. Ties remain deterministic because the anchor index is part of the key.

Anchors sharing a reference position are inserted only after that complete position group has been scored. Otherwise an anchor could become the predecessor of another hit with zero reference distance. Entries expire when they leave the reference window, and an explicit tree-size cap prevents unbounded storage under pathological repetition. The tree proposes an endpoint; BioSeqDB then recomputes the actual pair score rather than treating the RMQ surrogate as the final score.

There are two qualifications. First, the RMQ path calls a simplified scoring routine whose penalty trigger tests whether the query distance exceeds the predecessor span, whereas the scalar routine tests the minimum of reference and query distance. The paths are consequently algorithmic variants, not merely two data structures around one bit-identical kernel. Second, the code contains an optional inner-tree refinement, but the standard trait dispatch sets its inner distance to zero. The common SQL RMQ path therefore uses the primary augmented tree only. Its attraction is asymptotic behavior on dense anchor sets, while its cost is pointer-rich tree maintenance and a semantic surface distinct from the scalar oracle.

7.4.1. Current RMQ correctness boundary

Source inspection found two live differences from the bundled minimap2 RMQ routine that are more serious than the intentional simplified score. The primary BioSeqDB query computes the diagonal width but does not reject a candidate whose width exceeds the configured bandwidth. Its closed query interval can also include an equal-q predecessor, while minimap2's sentinel keys exclude both the lower boundary and equal-query hits. Since the simplified pair scorer does not repeat the full validity checks, such a candidate can be selected.

The disabled inner-tree path has additional differences in its skip accounting and capacity management. No direct RMQ unit test or operation-level RMQ proof record was found in the current tree. For this reason the RMQ implementation is documented here as an important intermediate algorithm and an optimization target, not as an exact replacement for either scalar BioSeqDB or minimap2 RMQ. Existing benchmark rows labelled `minimap2_rmq` cannot close this gap: the wrapper does not set minimap2's `MM_F_RMQ` flag, so that comparison actually executes its scalar dynamic program.

7.5. SIMD chaining on the CPU

Chaining resists naive vectorization. Pair scores for a fixed i are independent, but the decision to stop scanning is an ordered state machine. Backtracking is even less regular: predecessor paths have data-dependent lengths and collide through a shared visited array. BioSeqDB uses Highway to make vector width a compile-time target property while keeping one portable source for NEON, AVX2, AVX-512, and other supported backends.

7.5.1. Structure-of-arrays scoring

The SIMD forward pass projects anchors into 32-bit structure-of-arrays buffers for reference position, query position, and span. Pairwise differences are formed modulo 2^{32}. Because candidates are already constrained to a small same-range distance window, the low-word subtraction gives the correct bounded difference even when the absolute 64-bit coordinates are large. The full anchor representation and ordering nevertheless remain 64-bit; only this local arithmetic projection is narrow.

For each vector of predecessor candidates, Highway evaluates admissibility, minimum gap, diagonal difference, base reward, linear penalties, and the logarithmic correction under lane masks. Tail lanes are masked explicitly. The default skip scale is zero, so a specialized instantiation removes that multiply and its associated loads rather than hoping the compiler discovers the invariant through layers of templates.

7.5.2. `exact_visited`: vector arithmetic, scalar decisions

The exact policy vectorizes score production but preserves the scalar candidate event order. A batch that cannot improve the current best can count its visited-generation hits in bulk. If any lane may improve, the relevant lanes are replayed newest-to-oldest so that strict ties, counter decrement, and early termination match the scalar control path. Since the common predecessor relation is $p_j = j - 1$, contiguous generation marks can be stored as a vector. The global-best rescue is likewise located with a vector scan, then applied under the scalar eligibility rule.

This design illustrates a useful principle: exact SIMD need not mean that every operation is vectorized. It means that vectorization is placed where lanes are independent and ordered effects are retained where they are observable. Historical attempts to replace small reductions with horizontal SIMD operations did not automatically improve this balance. A May 2026 regression report measured a 39.1% slowdown in one very-long-read chaining configuration after such a reducer change; template duplication also increased instruction-cache pressure. The change was reverted. Those measurements are historical and configuration-specific, but they explain why the current code avoids equating "more vector instructions" with "faster chaining."

7.5.3. `bounded_batch_approximate`: a deliberate speed policy

The bounded policy processes candidates newest-to-oldest in Highway batches and maintains lane-resident best values. Its non-improving-batch budget is

$$C_{\max} = \max\left(1, \left\lfloor \frac{\text{max-skip}}{L} \right\rfloor + 1\right), \tag{7.6}$$

where L is the hardware lane count. Any improvement resets the counter; after $C_{\max}$ consecutive non-improving batches, the older unscanned interval is discarded. When different lanes produce an equal maximum, the largest predecessor index wins, recovering the newest-candidate tie rule within the examined interval. A global-best candidate is tested only when it lies in the discarded interval.

This policy does not simulate the scalar visited-generation heuristic. An adversarial old predecessor can be placed beyond the batch cutoff, so no static proof can guarantee the same endpoint for all anchor sets. BioSeqDB therefore names it approximate and qualifies it empirically. The qualification metric compares primary chain endpoints against both the

exact SIMD policy and the bundled minimap2 executable:

$$D = \max\{|q_s - q'_s|,\ |q_e - q'_e|,\ |r_s - r'_s|,\ |r_e - r'_e|\}. \tag{7.7}$$

The fixed corpus accepts a comparison only for $D < 15$ bases and also checks missing mappings, endpoint mismatches, and a dedicated global-rescue fixture. Passing that corpus is evidence about the tested distribution; it is not a mathematical equivalence result.

7.5.4. Vector backtracking

Both SIMD forward policies use a dedicated SIMD backtracker. Primary paths are consumed in vector chunks, with a fast path for contiguous predecessor indices. Remaining ranked starts occupy independent lanes, but lanes are committed in endpoint-rank order because overlap through the visited array is observable. Consequently the implementation vectorizes path discovery while retaining scalar orchestration at collision points. There is no hidden call to the shared scalar backtracker and no scalar fallback if a selected SIMD target cannot provide a valid multi-lane implementation; selection is a contract rather than a best-effort hint.

7.6. From G3SA to the Vulkan implementation

The GPU implementation began from the observation developed by G3SA: chaining is expensive not only because it performs many score evaluations, but because the conventional CUDA organization repeatedly reads source anchors and writes target scores through global memory. G3SA tiles the dynamic program. In a prologue, anchors within one tile depend on earlier anchors in that same tile, producing a triangular dependency pattern. In the tail, completed source values reside in shared memory while target accumulators stay in registers. Threads rotate their source assignment so that a tile-to-tile all-pairs comparison reuses shared data.

Figure 7.2 reconstructs this idea rather than copying the paper artwork. The left panel represents repeated global-memory traffic in the earlier source-at-a-time organization; the right panel separates the within-tile prologue from the register-accumulating tail. It is based on Figure 6 of Han et al.'s G3SA paper.[2]

7.6.1. What the original G3SA implementation did

The copied G3SA CUDA source divides ranges into short (≤ 512 anchors), medium (≤ 5120), and long classes. Its tiled path uses a 1024-anchor shared-memory tile with 512 or 1024 threads, rotates source assignments with a modulo-1024 index, and uses relative 16-bit predecessors inside a tile. The original implementation assumes the usual DNA/minimap2 configuration and hard-codes a query span of 15 in the DP kernel. It then keeps the RMQ and backtracking/compaction stages on the GPU. The paper's claim is not merely that one score calculation is fast, but that a mapping pipeline avoids CPU–GPU round trips.

Those choices are sensible for a CUDA program tuned to a fixed GPU. They are not directly portable to a PostgreSQL extension that may be called for different alphabets, seed lengths, devices, and projection requirements. BioSeqDB therefore adopted the score-generation idea and the demand for an explicit GPU execution contract, but not the complete kernel topology.

[2] D. Han et al., "G3SA: GPU-accelerated sequence alignment," ICS 2025, Figure 6; project copy: `G3SA-main/g3sa_paper.pdf`.

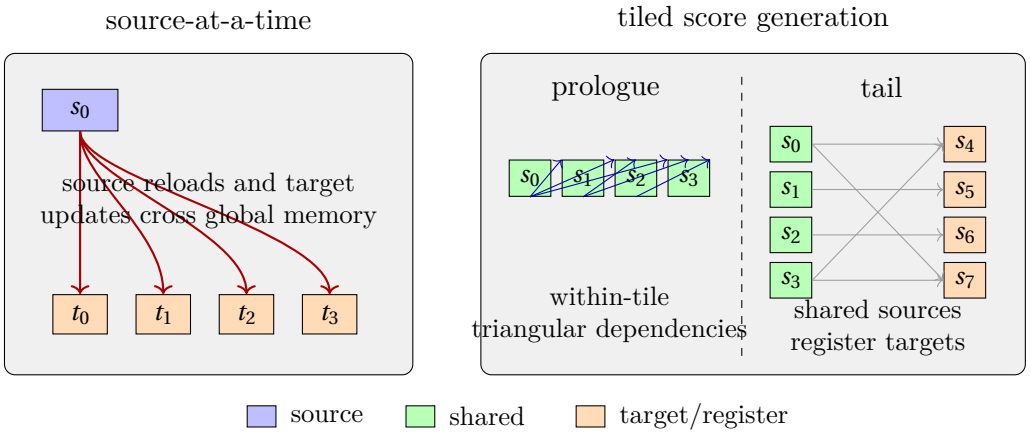


Figure 7.2: Reconstruction of the memory-locality argument in G3SA Figure 6. This explains the research lineage, but it is not a diagram of BioSeqDB's current shader, which uses a different 32-lane subgroup organization.

7.6.2. The live Vulkan kernel

The current shader assigns one 32-lane subgroup to one hard range. Hard ranges are split when orientation changes or when the reference gap between adjacent anchors exceeds the configured maximum. Anchors are advanced sequentially; for each target anchor, the subgroup scores predecessors in newest-first batches of 32 and reduces the batch maximum. Equal scores select the largest predecessor index. A workgroup barrier follows every completed target because the next target can depend on it.

This organization is notably different from the older 384/512/1024-workgroup selector still mentioned in comments and old tests. The live selector returns a 32-thread subgroup key. Compared with G3SA, BioSeqDB's shader improves generality in three concrete ways: it reads the actual per-anchor span instead of assuming 15, checks full 64-bit coordinate differences before scoring, and stores absolute predecessor indices. Chaining itself is alphabet-neutral once anchors have been produced.

It is also deliberately approximate. The device iteration limit is

$$I_{\mathrm{GPU}} = \min(I_{\mathrm{requested}}, 3000),$$

so the standard CPU limit of 5000 is not honored. The shader stops after $\lfloor \text{max-skip}/32 \rfloor + 1$ non-improving predecessor batches instead of replaying scalar visited-generation events, and it has no global-best rescue. A hard range longer than 131072 anchors is thinned toward 65536 anchors by preserving its endpoints and sampling the interior at an adaptive stride. Experiments with targets of 16384, 32768, and 49152 moved endpoints beyond the accepted tolerance; 65536 was the last focused configuration to pass, not a proof that it preserves every input.

7.6.3. Two projection-dependent completion paths

When the SQL projection asks for CIGAR, divergence, or mapping-quality data, BioSeqDB downloads device f and p arrays, validates predecessor and score invariants, and runs the shared CPU backtracker. When only coordinates are needed, a GPU reducer ranks at most 16 endpoints and returns the first qualifying chain. That route avoids the full arrays and emits only one chain, but a valid seventeenth candidate can be missed after sixteen rejected

Table 7.1: G3SA and BioSeqDB Vulkan chaining are related designs, not equivalent implementations.

Aspect	G3SA CUDA source	Current BioSeqDB Vulkan source
Parallel unit	512/1024-thread, 1024-anchor shared tile	One 32-lane subgroup per hard range
Score locality	Explicit prologue and rotated tile-to-tile tail	Sequential targets, predecessor batches of 32
Coordinates	SoA tuned to minimap2 assumptions	Full 64-bit distance checks in shader
Seed span	DP path assumes 15	Reads each anchor's span
Predecessor	Relative 16-bit values in tiled path, global representation later	Absolute index
Search bounds	Runtime iteration limit, short-/medium/long kernels	At most 3000 predecessors plus batch cutoff
Very long ranges	Dedicated long-range CUDA path	Stride compaction toward 65536 anchors
Backtracking	GPU RMQ, backtracking, and compaction pipeline	CPU for full projection; GPU top-16 reducer for coordinates only
Portability	CUDA/NVIDIA-specific tuning	Vulkan device contract, but current subgroup width fixed at 32

starts. The host contains a coordinate-chain helper with equivalent intent, but it has no current caller.

The Vulkan route is fail-closed rather than opportunistic. It rejects software devices, requires host-visible coherent unified buffers, performs an explicit memory-admission check, and does not silently fall back to the CPU. This makes an EXPLAINed GPU choice meaningful, but it also means the caller must opt in with a trusted device. Some trait metadata still says that `max_chain_skip` is unused and that backtracking is always on the CPU; the shader and coordinate-only path show that both statements are stale.

7.7. How the implementation evolved

The current policy split is the result of intermediate failures as well as successful optimizations.

Template and reducer experiments. Early 2026 work attempted wider template dispatch, SIMD sorting, horizontal maximum reducers, and multiple `SIMD_MM2` variants. Some experiments improved isolated operations but harmed the complete chaining stage through extra dispatch, code size, or scratch traffic. The regression investigation superseded the earlier optimistic implementation notes and restored the baseline reducer. A separate `SIMD_MM2_V2` route was removed rather than retained as an ambiguous second "SIMD" mode.

Exact SIMD as an oracle. The next milestone separated pair-score vectorization from ordered decisions. Exact-visited SIMD gained bit-for-bit forward-state comparisons against the BioSeqDB scalar path, dedicated overlap tests for the vector backtracker, and fail-closed target checks. This provided a faster executable oracle for experiments without weakening the contract.

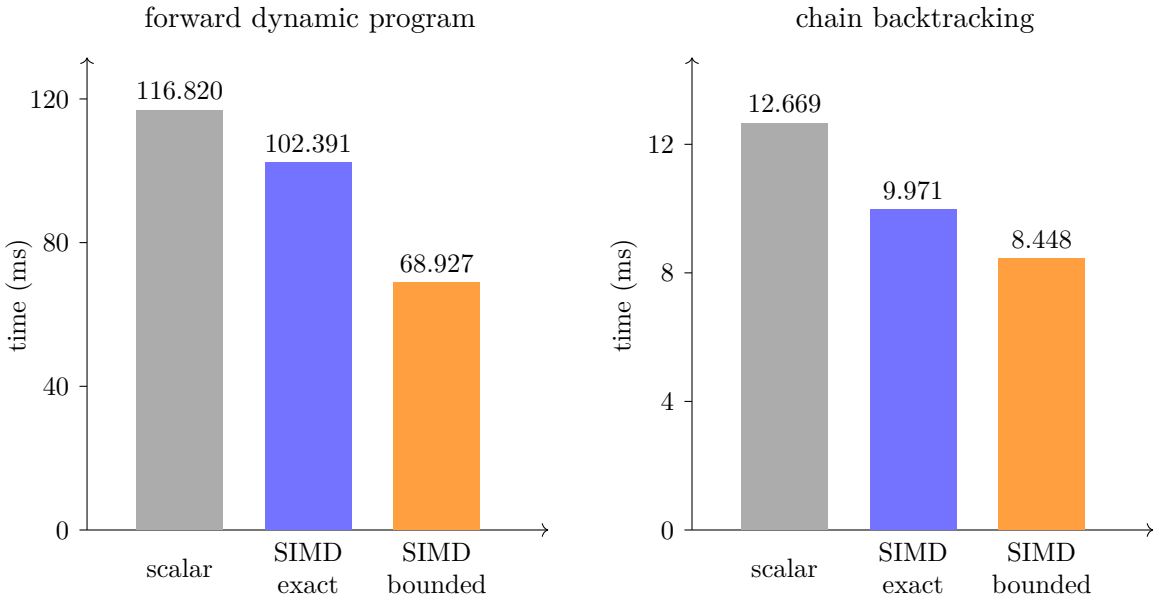


Figure 7.3: Chaining-only development aggregates from report 192. Exact SIMD reduces forward time by 12.4% relative to scalar; bounded SIMD reduces it by 41.0%, while using explicitly approximate stopping semantics.

Approximation as a named policy. Historical very-fast SIMD results came from a batch cutoff that did not preserve minimap2's visited-generation control. Rather than describe the speedup as exact, the final design names the behavior `bounded_batch_approximate`, retains a constructed counterexample, and requires endpoint qualification. The exact and bounded policies share the vector score and vector backtracking infrastructure but differ in their forward stopping semantics.

GPU specialization. The G3SA port first motivated larger tiled workgroups. The live Vulkan code instead converged on a one-subgroup kernel, range compaction, and a projection-dependent coordinate reducer. This is a database-oriented compromise: coordinate-only SQL avoids transferring structures it will never expose, while richer projections retain CPU backtracking. Its accepted 65536-anchor compaction target emerged from failed more-aggressive targets and remains an empirical operating point.

7.8. Measured behavior

Figure 7.3 reports the chaining-focused aggregates from `report_192.json`, created on 27 July 2026 from commit `f9570f4f3da3` with a dirty worktree. The XXL HiFi case used a 0.5-Gbp reference and a 5-Mbp query; each displayed aggregate has two samples. Because the report is marked `history_eligible=false`, it is a reproducible development snapshot, not a promoted release baseline.

On this case, exact SIMD used 87.7% of scalar forward time, while bounded SIMD used 59.0%. Bounded SIMD was 32.7% faster than exact SIMD in the forward stage. All three rows produced one match with zero reported endpoint offset. The bounded row recorded 914453 cutoff stops and skipped a mean of 786467451 candidate evaluations, which explains the size of the speedup more directly than instruction-level vectorization alone.

Table 7.2: Intermediate CPU chaining snapshots. Values are focused-stage aggregates in milliseconds; all cited reports were development worktrees.

Snapshot	Scalar	Exact SIMD	Bounded SIMD	Best/scalar	Interpretation
report 112	107.671	97.373	–	0.904	exact-SIMD milestone
report 118	109.844	95.573	63.263	0.576	first bounded qualification
report 192	116.820	102.391	68.927	0.590	latest focused snapshot

Table 7.3: Fixed-corpus endpoint evidence for bounded SIMD in report 192.

Observation	Result
Fixtures mapped	16/16
Maximum delta versus exact SIMD	0 bp
Maximum delta versus bundled minimap2 DP	0 bp
Missing or mismatched mappings	0
Global-best rescue probes / wins	205/1
Scalar score or shared-backtrack calls	0

The absolute values in Table 7.2 should not be compared as a time series without controlling the host, build, and worktree. Their useful signal is within-report: exact SIMD repeatedly improves on scalar, while bounded stopping removes substantially more work. Whole-row wall time is not a chaining metric because choosing a row can also select a different minimizer implementation.

7.8.1. Endpoint qualification

The endpoint suite in report 192 contained 16 fixtures and produced 16 mappings. Its maximum coordinate difference was zero bases against exact SIMD and zero against the bundled minimap2 dynamic program. It recorded no missing or mismatched mappings. Importantly, the rescue fixture performed 205 global-best probes and produced one winning rescue, so the suite exercises the path rather than merely checking a dormant counter. Across the suite the bounded policy stopped 1110558 times and skipped 942275290 candidates.

These data justify enabling bounded SIMD for an explicitly selected PostgreSQL SIMD request on the qualified workload. They do not erase the counterexample in which a useful old predecessor lies beyond the batch budget. Likewise, the available GPU figure—4.415 seconds for a coordinate-only row versus 4.929 seconds for a SIMD row in one June 2026 accepted sample—is a historical whole-region result with different endpoint offsets (48 versus 3 bases), not a current chaining-only speed claim. More recent source had not received a fresh equivalent GPU comparison when this chapter was written.

7.9. The final execution policy

BioSeqDB's result is not a single universally best chainer. It is a policy boundary that makes semantic and hardware trade-offs visible. The `Minimap2Params` object and the PostgreSQL GUC both default to the scalar minimap2-compatible implementation. A C++ caller that selects SIMD without overriding its policy receives `exact_visited`. PostgreSQL finalization maps an explicitly selected SIMD algorithm to `bounded_batch_approximate`; every non-

Table 7.4: Chaining modes and their intended roles in the final system.

Mode	Contract	Principal advantage	Current boundary
Scalar MM2	Default semantic baseline	Closest forward control to bundled minimap2; simplest oracle	Bounded $O(nI)$ search; BioSeqDB backtracking differs from upstream
SIMD exact-visited	Exact to BioSeqDB scalar f/p contract	Vector score and backtracking with ordered decisions	CPU coordinate deltas remain 32-bit; selected target must support lanes
SIMD bounded-batch	Named approximation, endpoint-qualified	Largest measured CPU reduction by skipping old batches	No universal endpoint equivalence; PostgreSQL SIMD maps here
RMQ	Experimental asymptotic variant	$O(n \log n)$ augmented-tree candidate search	Live bandwidth/equal-q gaps; benchmark reference row miswired
Vulkan	Explicit GPU approximation	Full-width shader checks and coordinate-only projection path	Range thinning, 3000 cap, batch stop, top-16 reducer; no fallback

SIMD backend resets the SIMD policy to exact so stale options cannot leak across calls. Vulkan and RMQ are explicit choices, and selected backends fail instead of falling back silently.

This separation is more than configuration hygiene. It prevents three common category errors: treating a benchmark-qualified approximation as a theorem, treating shared ancestry with minimap2 as byte-identical behavior, and treating a G3SA-inspired Vulkan shader as a direct CUDA port. It also permits the database planner and future cost model to choose based on projection: a coordinate-only query can benefit from GPU reduction, while an explainable, high-sensitivity analysis can retain the scalar or exact SIMD contract.

7.10. Summary

Chaining converts a cloud of minimizer hits into a small set of collinear candidate mappings. BioSeqDB's scalar recurrence rewards non-overlapping seed coverage and penalizes diagonal drift, while bounded predecessor windows, visited-generation stopping, strict ties, and global-best rescue make its control behavior more subtle than the recurrence alone suggests. Backtracking then claims ranked predecessor paths and leaves primary/secondary classification to the region stage.

The implementation history produced three different acceleration strategies. RMQ changes the data structure and asymptotic search, but the current port still needs correctness repairs. Highway SIMD preserves the scalar contract under exact-visited replay or trades it explicitly for batch-bounded speed. Vulkan adopts the GPU locality motivation of G3SA while using a distinct 32-lane kernel, full-width shader checks, range compaction, and projection-dependent completion. Report 192 shows why the distinction matters: exact SIMD reduced forward time by 12.4%, while bounded SIMD reached 41.0% by skipping hundreds of millions of candidates. The latter passed the fixed endpoint corpus with zero observed difference, but remains approximate by construction.

All implementation claims in this chapter were checked against the current `chaining_utils`, `chaining_scalar`, `chaining_simd`, `chaining_rmq`, and `chaining_vulkan` modules; the Vulkan shader compiler and selector; the PostgreSQL custom-scan finalizer; the bundled minimap2

`lchain.c`; and the copied G3SA CUDA source. The documentary audit and conflict-resolution log are recorded separately with the thesis research materials.

Chapter 8

Final alignment and extension

8.1. From a chain of seeds to a base-level alignment

The chaining stage identifies an ordered set of anchors that is plausible at the scale of genomic coordinates. It does not yet say which bases are substituted, inserted, or deleted between those anchors. The final-alignment stage resolves that ambiguity. It refines the ends of a candidate region, assigns a dynamic-programming score, and, when requested, reconstructs a CIGAR edit path. In BioSeqDB this is a conditional stage. If an SQL statement needs only seed-derived coordinates and does not request either CIGAR or exact edit accounting, `AlignStage` is disabled. If `bioseq.cigar(...)` or an exact `bioseq.edit(...)` predicate is present, the stage is enabled and traceback is mandatory.

Calling this operation "final" does not imply that every configuration computes an unconstrained optimum over the whole query and target. KSW2 uses a finite band and may stop on Z-drop. Block Aligner deliberately explores an adaptive subset of the matrix. The dedicated GPU route first obtains a coordinate window from its seeding and chaining stages, then aligns that window when CIGAR is requested. These are controlled approximations around an already selected chain. Within the admitted cells, however, the score recurrences and trace decisions are explicit. When BioSeqDB reports an exact edit count, it counts substitutions and gaps on the emitted path rather than using a divergence estimate.

The current CPU implementation exposes two selectable algorithms:

1. a Highway port of KSW2, retaining minimap2's single- or two-piece-affine recurrence, diagonal-difference representation, banding, Z-drop, endpoint selection, and packed CIGAR convention;

2. a native Highway port of Block Aligner, using a single affine gap model, an adaptive block that shifts, grows, and shrinks, and native global or X-drop termination.

The generic C++ parameter object selects Block Aligner by default. Its current in-process block interval is 16–32 cells. The PostgreSQL configuration layer also selects Block Aligner, but its GUC defaults are still 32–256 and the custom scan copies those values into the parameter object. This distinction matters when interpreting the benchmark in Section 8.9. The 22% CPU improvement attached to 16–32 is not automatically obtained by an unmodified SQL installation using 32–256.

The dedicated all-GPU route adds a third execution contract. Admission requires the configuration tuple to name Block Aligner, but no CPU Block kernel runs inside that route. Vulkan seeding and chaining produce coordinate windows directly. If CIGAR is requested,

a separate Vulkan two-piece-affine CIGAR shader solves those windows. The enum value is thus a route-admission identity, not evidence that the CPU Block recurrence executed. BioSeqDB records execution proofs and fails closed so that these two meanings cannot be silently conflated.

8.2. Affine-gap dynamic programming

8.2.1. Why one gap-open cost is not enough

Let $Q = q_1 \dots q_n$ be a query and $T = t_1 \dots t_m$ a target interval. Let $s(q_i, t_j)$ be a substitution score. A linear gap model charges the same amount for every missing symbol. Biologically, opening a gap and extending an existing gap are different events. A single insertion of length ten is usually more plausible than ten independent single-base insertions. An affine model therefore charges

$$g(k) = q + ke, \qquad k \geq 1,$$

where $q > 0$ is the open penalty and $e > 0$ is the extension penalty. This is the convention used by minimap2 and BioSeqDB. The first gap symbol costs $q + e$.

With target positions on the horizontal axis and query positions on the vertical axis, a global Smith–Waterman–Gotoh recurrence is

$$I(i,j) = \max\{H(i-1,j) - q - e,\ I(i-1,j) - e\}, \tag{8.1}$$

$$D(i,j) = \max\{H(i,j-1) - q - e,\ D(i,j-1) - e\}, \tag{8.2}$$

$$H(i,j) = \max\{H(i-1,j-1) + s(q_i,t_j),\ I(i,j),\ D(i,j)\}. \tag{8.3}$$

The I and D states remember whether the best partial path already ends in a gap. For global alignment the axes are initialised by cumulative gap costs. Local alignment additionally permits zero in the maximum. A full matrix costs $O(nm)$ time and, if every trace decision is stored, $O(nm)$ memory.[1]

BioSeqDB inherits minimap2's default nucleotide scores:

$$a = 2, \quad b = 4, \quad (q_1, e_1) = (4,2), \quad (q_2, e_2) = (24,1).$$

A match scores +2, an ordinary mismatch scores −4, and the two gap families define

$$g(k) = \min\{4 + 2k,\ 24 + k\}.$$

The short family is cheaper through length 20, while the long family becomes cheaper beyond that crossover. Algebraically, KSW2 maintains two insertion states and two deletion states:

$$I_r(i,j) = \max\{H(i-1,j) - q_r - e_r,\ I_r(i-1,j) - e_r\}, \tag{8.4}$$

$$D_r(i,j) = \max\{H(i,j-1) - q_r - e_r,\ D_r(i,j-1) - e_r\}, \qquad r \in \{1,2\}, \tag{8.5}$$

$$H(i,j) = \max\{H(i-1,j-1) + s(q_i,t_j), I_1, I_2, D_1, D_2\}. \tag{8.6}$$

This two-piece concave cost is important for long reads, where a structural variant should not be penalised as a series of unrelated short errors.

Block Aligner uses only (q_1, e_1). Its recurrence is therefore not a drop-in semantic replacement for KSW2 under the default two-piece model. This is an intentional production trade-off. The adapter projects the short pair, the block interval, and the X-drop threshold. It does not pretend to honour the second affine pair, KSW2's band, right-alignment tie flag, end bonus, or approximate-maximum flag.

[1] O. Gotoh, "An improved algorithm for matching biological sequences", Journal of Molecular Biology 162(3), 1982, doi:10.1016/0022-2836(82)90398-9.

8.2.2. Banding and Z-drop

Anchors imply that the desired path should remain near a diagonal. A band of half-width w discards cells satisfying

$$|i - j| > w$$

after accounting for the interval's coordinate offset. This changes work from $O(nm)$ to approximately $O(w(n+m))$. The saving is substantial when $w \ll \min(n, m)$, but the optimum is lost if a true insertion or deletion leaves the band. BioSeqDB derives the ordinary alignment width as

$$w_{\text{align}} = \lfloor 1.5\, w_{\text{chain}} + 1 \rfloor.$$

With the default chaining bandwidth 500, this is 751.

Z-drop supplies a second early-termination rule. Let $S^\star$ be the best score seen at $(i^\star, j^\star)$, and let $S(i, j)$ be the current anti-diagonal maximum. KSW2 may terminate when both coordinates have advanced past the best cell and

$$S^\star - S(i, j) > Z + e_z \left| (i - i^\star) - (j - j^\star) \right|. \tag{8.7}$$

The diagonal-displacement term does not punish a long gap as harshly as an equal score drop along the same diagonal. In the dual-affine kernel BioSeqDB uses the long-gap extension penalty e_2 in this test. The default threshold is $Z = 400$. Block Aligner does not implement Equation 8.7. Its left and right extensions use the original Block algorithm's X-drop rule and require two consecutive below-threshold steps before termination.

When an exact SQL edit predicate requires traceable gap fills, the KSW2 path sets both band and Z-drop to -1 for those mandatory fills. This removes two sources of premature truncation. It does not turn the whole mapper into a full-matrix aligner. Seed selection, chain selection, the outer coordinate window, and any Block configuration remain part of the search contract.

8.3. The minimap2 extension strategy and BioSeqDB's port

8.3.1. Three different DP problems, not one matrix

The current implementation follows the central shape of minimap2's `mm_align1`: extend left of the first seed, fill between seeds, then extend right of the last seed.[2] These phases have different endpoint semantics:

Phase	Input orientation	KSW2 contract	Block contract
left extension	reversed sub-ranges	extension-only, right-aligned, reverse CIGAR, band and Z-drop	native X-drop on directly emitted reversed compact input
seed-to-seed fill	forward	global fill, approximate maximum by default, two-piece affine	native global adaptive block, single affine
right extension	forward	extension-only, band and Z-drop	native X-drop

[2] H. Li, "Minimap2: pairwise alignment for nucleotide sequences", Bioinformatics 34(18), 2018, doi:10.1093/bioinformatics/bty191.

The first and last anchor delimit the inner seeded interval. BioSeqDB expands an outer interval on either side, bounded by `max_gap` and by an additional score-derived allowance. Left extension is solved in reverse so that both end extensions can use a forward DP recurrence. Current compact and lazy sequence views can emit this reversed orientation directly. They do not first materialise an entire reversed query.

The seed-to-seed walk is especially consequential. A cursor begins at the first seed. Intermediate anchors whose accumulated query or target distance is below `min_ksw_len` are skipped without advancing the cursor. Once both accumulated distances reach the threshold, a global fill is emitted and the cursor advances. The last anchor always forces the remaining positive rectangle. With the default threshold 200, a megabase-scale region becomes a sequence of DP problems often near 200×200, not one million-by-million matrix. A source-embedded engineering note attributes roughly a sevenfold total improvement at 903 kbp to this decomposition. That number is historical and configuration-specific, but the mechanism remains in current code.

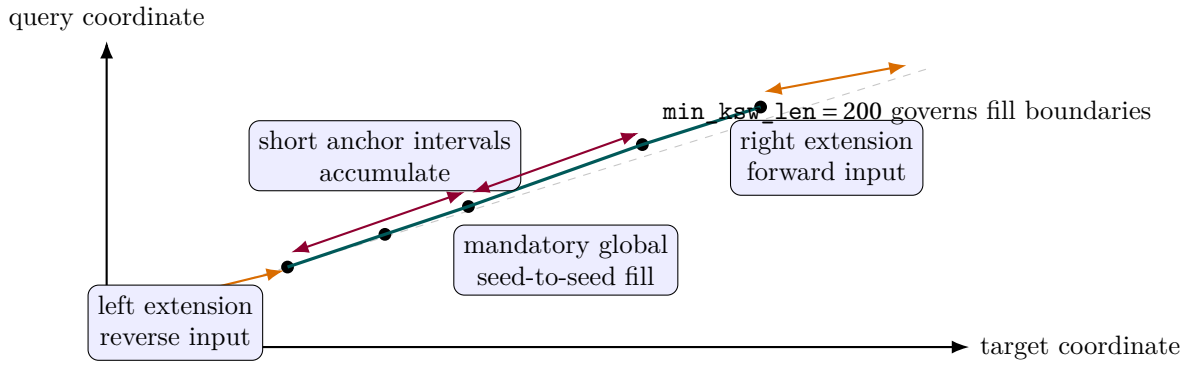


Figure 8.1: BioSeqDB's three-phase region alignment. The drawing reconstructs the implemented control flow. It is not a copy of a minimap2 figure. Exact seeds constrain the path, short intervals accumulate, mandatory global fills resolve gaps, and end extensions refine the outer coordinates.

8.3.2. Where the port intentionally stops matching minimap2

The inspected minimap2 reference contains more policy than the three boxes in Figure 8.1. It handles ignored and tandem seeds, long-join bandwidth expansion, short-read ungapped shortcuts, splice junction scores, RNA-specific paths, an approximate first pass followed by `mm_test_zdrop`, inversion Z-drop, and region splitting after a dropped gap. The current BioSeqDB path implements the ordinary DNA seed-to-seed and end-extension core. It does not contain the splice, long-join, inversion-retry, or second-pass logic in its alignment modules. Consequently, "minimap2-derived" is more accurate than "minimap2-equivalent" for those special modes.

There is also a subtle difference after a dropped phase. KSW2 suppresses the right extension if a prior phase Z-dropped. The Block path still admits a right extension because its gap fills are global and its two end extensions use their own native X-drop domains. This is encoded in the compile-time algorithm branch rather than hidden in a common KSW-shaped adapter.

Algorithm 11 Final alignment of one chained region

Require: query view Q, target view T, ordered anchors A, parameters P
derive inner seed bounds and bounded outer extension bounds
reserve region CIGAR capacity from the outer query and target extents
if both left outer intervals are non-empty then
    run reverse-oriented extension with KSW2 or Block X-drop
    update the left endpoint from the selected maximum or query-end score

$(q_s, t_s) \leftarrow$ start of the first seed
for each later anchor a_i do
    $(q_e, t_e) \leftarrow$ end coordinates of a_i
    if a_i is not last and $q_e - q_s < P$.minLen or $t_e - t_s < P$.minLen then
        continue without advancing (q_s, t_s)

    if both accumulated intervals are non-empty then
        solve one mandatory global fill and append its packed CIGAR
        $(q_s, t_s) \leftarrow (q_e, t_e)$

if right outer intervals are non-empty and the selected contract permits it then
    run forward extension and update the right endpoint

project reverse-strand query coordinates and final score metadata

8.4. KSW2 in BioSeqDB

8.4.1. The Suzuki–Kasahara difference representation

A straightforward SIMD implementation stores absolute H, I, and D scores. Long alignments require wide integers because absolute scores grow with sequence length. Wide lanes reduce the number of cells processed per vector. Suzuki and Kasahara observed that neighbouring-score differences remain bounded by the scoring parameters even when the absolute score is large. KSW2 therefore represents quantities related to

$$U(i,j) = H(i,j) - H(i-1,j), \qquad V(i,j) = H(i,j) - H(i,j-1),$$

together with transformed gap offsets. The hot recurrence operates on signed 8-bit partial values. An absolute 32-bit score row is reconstructed only for exact maximum tracking and endpoints.[3]

BioSeqDB ports that organisation to Highway. A scalable byte tag supplies the partial-score lanes, while a scalable 32-bit tag reduces reconstructed maxima. The source instantiates a single-affine kernel only when $(q_1, e_1) = (q_2, e_2)$. Under the ordinary defaults the pairs differ, so the dual-affine kernel is selected. The band is stored by anti-diagonal, and one direction byte encodes the winning state plus continuation bits for each of the four gap states.

This representation is intra-sequence SIMD: lanes represent cells from one DP problem. It should not be confused with batching independent sequence pairs into lanes. The latter was explored separately and is discussed in Section 8.7.

[3] H. Suzuki and M. Kasahara, "Introducing difference recurrence relations for faster semi-global alignment of long sequences", BMC Bioinformatics 19, 2018, doi:10.1186/s12859-018-2014-8.

8.4.2. Endpoint and CIGAR semantics

KSW2 tracks several endpoint summaries:

- the best score and its query/target coordinates;
- the best score at the query end, with the associated target coordinate;
- the best score at the target end, with the associated query coordinate;
- the bottom-right global score if the band reaches it.

Traceback starts from the bottom-right cell for an undropped global fill. For an extension-only call, reaching the end of the query wins when its score plus the end bonus exceeds the internal maximum. Otherwise traceback starts at the best observed cell. Left-extension operations are emitted in reverse and then reversed back into source order.

The KSW packed representation stores one run as

$$\text{word} = (\text{length} \ll 4) \mid \text{opcode}.$$

The shared opcodes are $0 = M$, $1 = I$, $2 = D$, $3 = N$, $7 = =$, and $8 = X$. Adjacent equal opcodes are merged both within a traceback and when fills are appended to a region. A CIGAR such as `58M1I33M` is therefore three integers, not 92 per-base records.

8.4.3. Port-specific memory and hot-loop changes

The port is not merely the upstream SSE file wrapped in a module. The current implementation makes several changes needed by BioSeqDB's data model and execution engine:

1. It accepts an alphabet-generic lazy sequence view. Only the admitted subrange is encoded, through the view's `ptr_at` slot ring. A fragmented or PostgreSQL-backed sequence need not be flattened in advance.
2. DP state, exact-score rows, encoded scratch, and the backtrack matrix live in a per-alignment arena. The returned CIGAR has independent lifetime.
3. Buffers that are overwritten before first read are not eagerly zero-filled. In the dual-affine state, the short and long gap arrays are initialised directly to their required offsets. The trace matrix likewise skips a full clear.
4. The common nucleotide scoring layout uses a fused equality-based score selection. Generic matrices retain a table path.
5. Exact anti-diagonal maxima are reduced with Highway 32-bit vectors. The same source dispatches to supported x86 and Arm targets.
6. The single- and dual-affine kernels share one compile-time core, removing runtime checks from the inner cell loop.

These changes preserve the KSW2 recurrence and result structure while moving allocation, encoding, and instruction selection into BioSeqDB's common infrastructure. They also explain why a comparison with standalone minimap2 must include more than the arithmetic kernel. BioSeqDB saves conversions when the source is already compact, but pays PostgreSQL, batching, and lazy-view costs that minimap2 does not have.

8.5. Block Aligner: adaptive SIMD dynamic programming

8.5.1. The original algorithm

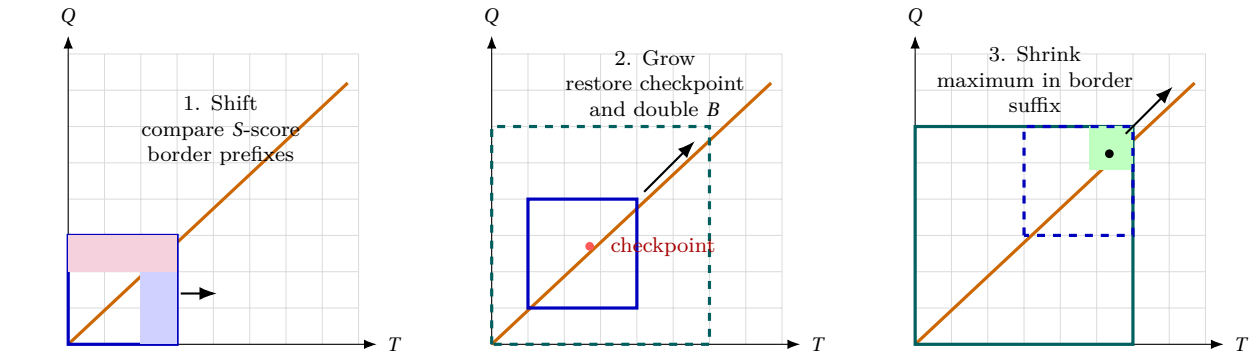
Block Aligner starts from the same single-affine recurrence as Equations 8.1–8.3, but it does not fill a static band. It keeps a square $B \times B$ block positioned over the DP matrix. A step computes only a strip of width S on the right or bottom edge, where the reference implementation uses $S = 8$. The next direction is selected by comparing the first S scores on the two outgoing borders:

$$\text{direction} = \begin{cases} \text{down}, & \max_{0 \le k < S} H(i + B - 1, j + k) > \max_{0 \le k < S} H(i + k, j + B - 1), \\ \text{right}, & \text{otherwise.} \end{cases}$$

The block follows the region in which scores appear most promising. Unlike a static diagonal band, it may drift far from the main diagonal.

Greedy direction selection can be fooled by a long gap or a low-similarity island. The defining contribution of Block Aligner is therefore adaptive geometry. When several steps fail to produce a new maximum, it restores a checkpoint and doubles B. The larger block spans more competing paths. If a later maximum lies near the suffix of both outgoing borders, the block may halve and advance diagonally. This recovers the speed of a narrow block in easy regions while retaining a chance of crossing a difficult region.

The original paper reports global, local, and X-drop modes, nucleotide and protein matrices, and position-specific scoring matrices. Across its chosen datasets it reports more than five- to tenfold speedups over some comparison methods with an error rate below 3% on the evaluated protein and long-read sets. These are paper results on the authors' hardware and workloads. They are not BioSeqDB speedups and are not directly comparable with the SQL E2E measurements below.[4]



(a) right/down decision (b) no new maximum (c) halve and move diagonally

Figure 8.2: Reconstruction of the adaptive-block mechanism described by Liu and Steinegger. A step computes an outgoing strip, growth replays from a saved checkpoint, and shrinking retains the promising suffix. The orange line illustrates a possible optimum path. This TikZ drawing was rebuilt from the algorithm and the paper's Figure 2, not copied from the paper artwork.

[4] D. Liu and M. Steinegger, "Block Aligner: an adaptive SIMD-accelerated aligner for sequences and position-specific scoring matrices", Bioinformatics 39(8), 2023, doi:10.1093/bioinformatics/btad487.

8.5.2. SIMD inside a block

Block Aligner lays vectors vertically or horizontally rather than on anti-diagonals. One gap dependency therefore runs between lanes of the same vector. For a vector $x_0, \ldots, x_{L-1}$, the needed prefix operation has the form

$$y_i = \max_{0 \le k \le i} \{x_k + (i-k)g\}, \tag{8.8}$$

where $g = -e$. It is evaluated by shift-and-maximum stages of distances 1, 2, 4, and, for 16 lanes, 8. This takes $O(\log L)$ vector operations. A correction carries the last value of the preceding vector into the next one. The other gap direction and the diagonal transition have ordinary vector dependencies.

Only four score borders and their checkpoint copies are needed for score computation, giving $O(B)$ working score memory. Traceback increases storage to $O(B_{\max}(n+m))$ because every computed strip must retain decisions that the adaptive path may later traverse. A checkpoint also stores the trace cursor so speculative strips can be discarded when growth replays the region.

8.5.3. The current Highway port

BioSeqDB's implementation is based on `block-aligner-research/src/scan_block.rs`, but it is a native C++26 module rather than a foreign-library call. The production core has the following concrete geometry:

- a fixed step $S = 8$;
- signed 16-bit lanes with saturating arithmetic and a bias of 2^{14}, so local relative scores stay representable;
- a Highway capped tag of at most 16 lanes;
- admitted lane counts of exactly 8 or 16, corresponding to the proven reference geometries;
- power-of-two minimum and maximum block sizes;
- global mode for mandatory seed gaps and two-confirmation X-drop for left and right extension.

At 8 lanes, a shift step is one full vector. At 16 lanes, it is half a vector. Wider Highway targets are deliberately capped rather than allowed to change the algorithm's step semantics. An unsupported lane shape, penalty, coordinate range, matrix layout, or block geometry aborts the selected route. There is no scalar biological implementation hidden beneath this module.

The hot rectangle kernel loads a scoring-matrix row once per target column and reuses it for all query vectors in that column. It computes diagonal, column-gap, and row-gap candidates, using Equation 8.8 for the row state. Border arrays are the only scores written back. Trace masks are compressed with Highway's `BitsFromMask`: two lane masks are interleaved into one two-bit decision word. This replaces repeated vector stores followed by scalar bit-shifting.

The traceback itself is a lookup-table state machine over diagonal, row-gap, and column-gap states. It walks the recorded adaptive strips backward and accumulates run lengths

directly into packed $M/I/D$ operations. If exact edit accounting was requested, a diagonal step compares the already resident padded ranks, while an insertion or deletion contributes one edit. Score, CIGAR, and exact edit distance are thus produced from one trace walk.

8.5.4. BioSeqDB-specific adaptations

Aspect	Original Block Aligner	BioSeqDB port
Implementation	Rust with explicit SSE2, AVX2, Neon, WebAssembly SIMD and C bindings	C++26 module with Highway target dispatch and a capped, proven 8/16-lane geometry
Input	contiguous sequence or PSSM buffers	alphabet-generic lazy views over packed, fragmented, or PostgreSQL-backed storage
Reverse extension	caller-provided reversed input	direct reverse-oriented emission from compact storage, with no whole-buffer reverse copy
Production modes	global, local, and X-drop are library features	global seed-gap fill and X-drop end extension only
Gap model	single affine, including position-specific variants in the paper	one generated score matrix and the short minimap2 affine pair
Scratch lifetime	library allocation objects	per-alignment BioSeqDB arena, with bounded subregion encoding
Trace	four-bit decisions decoded by a LUT	lane masks compressed with `BitsFromMask`, LUT trace, direct packed run construction, optional edit count
Fast exact case	no equivalent resident packed proof	DNA4/u64 or byte-rank diagonal certificate can bypass DP
Route integrity	library call selected by caller	execution proof records phase, lane width, cells, growth, shrinking, trace, certificate, and forbidden fallbacks

The adapter intentionally omits KSW-only fields. A request that depends on the second affine pair or KSW tie and endpoint flags must select KSW2. Reusing the shared `KswAlignResult` structure is only an output-storage convenience. It does not make the Block recurrence KSW-compatible.

8.5.5. A proof that can avoid the DP entirely

Many mandatory fills are equal-length and nearly identical. BioSeqDB tests whether their coordinate diagonal is provably the unique global optimum before constructing Block scratch. Let the common length be n, let

$$D = \sum_{i=1}^{n} s(q_i, t_i)$$

be the exact coordinate-diagonal score, and let M be the largest entry in the admitted generated matrix. Any different global path between equal-length strings must contain the same positive number d of inserted and deleted symbols. It has at most $n-d$ diagonal pairs and at least one gap run in each direction. Its score is bounded above by

$$(n-d)M - 2q - 2ed \le (n-1)M - 2(q+e) = U.$$

Therefore

$$D > U \quad \Longrightarrow \quad \text{the coordinate diagonal is the unique optimum.} \tag{8.9}$$

Strict inequality is essential. Equality could expose a different traceback tie policy.

For resident DNA4/u64 storage, the fast certificate reads packed two-bit symbols directly, reverses packed groups when required, XORs query and target, and collapses each two-bit mismatch group for a population count. It computes the score and edit count without byte expansion. The result is one nM CIGAR run. Its decision is three-valued:

1. not eligible: no packed scan ran, so the ordinary path, including the generic byte certificate, remains available;
2. rejected: exact packed content was scanned but Equation 8.9 did not prove uniqueness, so Block DP runs with the byte certificate disabled to avoid scanning twice;
3. certified: score, endpoint, edit count, and nM CIGAR are committed without constructing padded buffers, scratch, trace, or even an otherwise empty result CIGAR vector before the proof succeeds.

This is an exact fast path inside an otherwise heuristic adaptive algorithm. It does not claim that every Block result is an unconstrained optimum. It proves one common geometry and delegates all other cases to the normal Block search.

8.5.6. Why fail-closed attestation is part of the algorithm

Runtime algorithm selection is resolved to a template registry before the hot loop. For Block Aligner the operation-local proof derives, from immutable region geometry, how many left, gap, and right calls should be eligible. The kernel then records how many actually ran, which lane width was used, how many SIMD vectors and cells were evaluated, whether native X-drop ran, whether the block grew or shrank, and whether a diagonal certificate replaced DP. The proof also counts forbidden KSW2 and scalar fallback attempts. A selected Block route is accepted only if the derived and observed phase counts agree and all forbidden counters are zero.

This is stronger than logging an enum. It makes a common benchmark error observable: a test can request a fast backend, encounter an unsupported shape, and silently measure a slower fallback while still printing the requested label. BioSeqDB instead aborts that operation. The price is a narrower admitted contract, but the reported algorithm identity is falsifiable.

8.6. Traceback, CIGAR, and exact edit accounting

The forward score alone cannot explain an alignment. Variant calling, coordinate projection, and an SQL edit predicate need the path. BioSeqDB uses one common `CigarOp` representation after backend-specific traceback: a 32-bit run length and a 32-bit opcode. KSW2, Block Aligner, and Vulkan all cross their low-level boundary in KSW's packed $(\text{length} \ll 4) \mid \text{opcode}$ form, then append into this common vector.

Before aligning a region, the CPU path estimates CIGAR capacity from the outer query and target extents. The reserve hint is capped at $4\texttt{max_gap} + 64$ operations so a megabase seed span does not preallocate a gigabyte-scale vector. This cap is a performance hint, not a semantic maximum. The vector can still grow if a highly alternating path needs more runs.

There are three cases worth separating:

1. A region with several seeds appends the left trace, every mandatory fill trace, and the right trace. Adjacent runs with the same opcode merge at phase boundaries.

2. A single-seed region has no seed-to-seed DP rectangle. Its implicit seeded interval emits a generic M run over the common length, followed by I or D for any length difference.

3. A certified Block diagonal emits one nM run without allocating trace storage.

Block Aligner internally knows whether a diagonal step was an equality or a mismatch. The common minimap-style ABI deliberately projects M, =, and X from Block traceback to generic M. It then coalesces adjacent runs again so an equality-to-mismatch boundary does not become `32M1M`. Exact edit distance must consequently not be inferred by counting only I and D in the displayed Block CIGAR.

When exact edit accounting is requested, Block counts mismatching diagonal steps during its existing trace. KSW2 reconstructs the same quantity by walking its packed CIGAR over the clipped query and target ranks. The result is the unit-cost edit count of the emitted path:

$$E_{\text{path}} = \#\{\text{unequal diagonal steps}\} + \sum_{\text{I runs}} \text{length} + \sum_{\text{D runs}} \text{length}.$$

This count is exact for that path. It should not be confused with the affine score, where a gap run contributes $q+ke$, or with seed-derived divergence. Chapter 8 explains how an exact SQL predicate first uses a safe prefilter and then applies its strict bound to this traced count.

After alignment, BioSeqDB stores the accumulated DP score as both `dp_score` and the current simplified `dp_max_score`. It derives block length by summing diagonal and gap runs. For a Vulkan CIGAR it replays diagonal runs against the packed symbol arrays to obtain the number of equal symbols, because the shader likewise emits generic M.

8.7. Runtime dispatch and the batched-KSW2 experiment

8.7.1. Enum outside, templates inside

PostgreSQL and the C++ API expose a runtime base-alignment selector. The pipeline registry instantiates a kernel trait for each enum value, dispatches once, then uses `if constexpr` inside the region loop. A KSW2 fill does not test on every cell whether it should have been Block Aligner, and a Block fill cannot quietly call KSW2. The current internal enum contains:

Value	Implemented meaning
`KSW2`	Highway intra-sequence single- or dual-affine KSW2
`BLOCK_ALIGNER`	Highway adaptive-block single-affine alignment
`KSW2_BATCHED`	deferred fill collection, geometry bucketing, lane-by-lane KSW2 execution, and ordered restitching

The generated PostgreSQL enum omits `ksw2_batched`, although the internal registry still instantiates it. Ordinary SQL users choose KSW2 or Block Aligner. This omission reflects the true implementation state.

8.7.2. What was built for inter-sequence SIMD

The batched design first converts each left extension, global gap, and right extension into a descriptor. It records subrange coordinates, strand, phase, bandwidth, owning region, order within the region, scratch CIGAR, score, edit count, and final offsets. Descriptors are stably sorted into length brackets

$$32,\ 64,\ 128,\ 256$$

and band brackets

$$16,\ 32,\ 64.$$

Reverse and forward fills are kept separate. A fill with either dimension above 256 is marked long and routed to ordinary KSW2. After execution, descriptors are sorted back by region and fill order. CIGARs, scores, edit counts, and endpoints are stitched into their original chains.

This is useful scheduling infrastructure. It is not yet an inter-sequence SIMD recurrence. The current function named `ksw2_extz2_batched_inner` loops over occupied lanes and calls `run_single_fill`; that function invokes the ordinary intra-sequence KSW2 kernel once per lane. Execution evidence deliberately calls them bucket lanes rather than SIMD DP lanes. The earlier DOC0254 design and some source comments describe the intended fused-lane endpoint, but current code is authoritative.

The experiment still yielded two lasting design lessons. First, separating fill collection from execution is sufficient to expose batch geometry without changing region semantics. Second, a label such as "batched" does not prove that the arithmetic itself is vectorised across independent sequences. BioSeqDB's final public CPU choice therefore remains native Block Aligner or intra-sequence KSW2.

8.8. GPU extension and gridded traceback

8.8.1. The G3SA idea

The G3SA extending stage addresses a conflict between the forward pass and traceback. GPU forward DP is fastest when scores remain in registers or shared memory. Conventional traceback wants one direction value per cell in global memory. Writing that direction matrix can remove the benefit of the GPU kernel.

G3SA's gridded traceback stores only score states on regularly spaced tile boundaries. Once the final endpoint is known, the backward kernel loads the nearest upper-left boundaries, recomputes one tile in fast memory, traces through that tile, discards its directions, and repeats for the next tile on the path. Forward memory traffic falls from an entire direction matrix to a grid of borders. Traceback performs additional arithmetic only in tiles touched by the final path.[5]

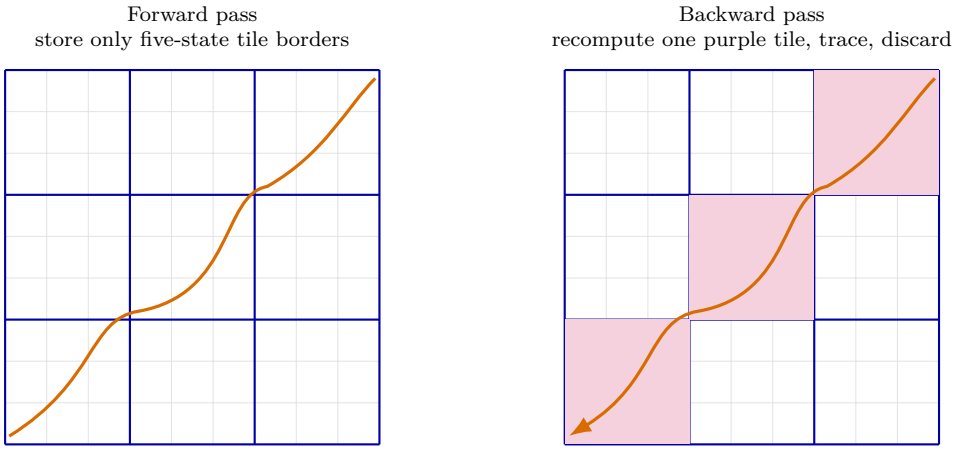
G3SA's CUDA implementation uses subwarps, packed bases, anti-diagonal forward computation, a separate traceback kernel, and a principal grid size of 64 selected by sensitivity analysis. Its paper reports at most 1.80-fold end-to-end speedup with one GPU and 6.21-fold with four GPUs over its CPU baselines. It reports up to 9.73-fold for an individual stage comparison. Those values describe G3SA's datasets, hardware, and baselines. They motivate the memory strategy, but they are not evidence about BioSeqDB.

8.8.2. BioSeqDB's Vulkan implementation

BioSeqDB transfers the border-and-recompute idea to Vulkan and to its PostgreSQL execution boundary. The current dedicated route proceeds as follows:

1. A resident Vulkan minimizer and chaining route emits candidate coordinate windows.

[5] Y. Han, S. Kim, S. Park, and J. Lee, "G3SA: A GPU-Accelerated Gold Standard Genomics Library for End-to-End Sequence Alignment", ICS 2025, doi:10.1145/3721145.3729516.



(a) boundary spill

(b) tile-local replay

Figure 8.3: Reconstruction of gridded traceback from the G3SA mechanism and the current BioSeqDB Vulkan shader. Blue grid boundaries survive the forward pass. Purple tiles are recomputed only while the orange path is traced. The drawing is a new TikZ reconstruction, not copied paper artwork.

2. If CIGAR is not requested, those coordinate results are returned without running the CPU `AlignStage`.

3. If CIGAR is requested, the PostgreSQL scheduler packs only each selected query and target window into 32-bit symbol ranks. It classifies exact, simple-gap, bounded, and gridded jobs.

4. One Vulkan dispatch solves the batch. Results contain score, endpoints, a packed-CIGAR slice, and status. The host verifies every slice, replays generic M against the rank buffers to count equal symbols, and constructs the SQL result.

The selected all-GPU tuple is deliberately narrow. Vulkan must be enabled, GPU seed lookup and the fused frontend must be selected, the target index must be built on the GPU, minimizer extraction must be the Vulkan minimap-integer variant, chaining must be Vulkan tiled DP, and the base-alignment selector must be Block Aligner. Divergence, mapping quality, exact edit filtering, and the prefilter are excluded from this dedicated tuple. CIGAR additionally requires `vulkan_extend_enabled`. If any selected component cannot execute, the statement errors before entering the generic CPU/SIMD pipeline.

The CIGAR shader implements a two-piece-affine global recurrence with five score states H, I_1, I_2, D_1, D_2. Spans up to 512 symbols use a bounded full direction matrix. Longer general spans, up to the current 16,384-symbol admission limit, use a 128-by-128 grid. For a job with tile dimensions b_q, b_t, the planned traceback capacity in 32-bit words is

$$\left\lceil \frac{n}{b_q} \right\rceil \left\lceil \frac{m}{b_t} \right\rceil 5\big[(b_q+1)+(b_t+1)\big]+(b_q+1)(b_t+1). \tag{8.10}$$

The first term stores five states along the bottom and right border of every tile. The second is one reusable tile-local direction scratch matrix.

The forward invocation visits every tile in row-major order and stores only those borders. Traceback begins at the bottom-right global endpoint. It recomputes the containing tile,

records source and gap-continuation bits in the shared scratch slice, walks until it crosses a tile boundary, then recomputes the next tile. Packed $M/I/D$ runs are generated backward and reversed at completion.

Several constrained scripts avoid DP altogether: exact equal-length spans, one- or two-gap scripts, a bounded unit-gap walk, and a greedy anchored gap walk. Exact and recognised unit-gap jobs can reserve zero traceback words. The current caps are 256 symbols for an individual simple gap and 64 gap operations for the simple-script family.

8.8.3. What was inherited and what was not

Aspect	G3SA	BioSeqDB
Programming model	CUDA kernels integrated with modified minimap2	Vulkan compute shaders invoked by a PostgreSQL custom-scan scheduler
Forward parallelism	subwarps cooperate on packed anti-diagonal blocks	current CIGAR shader assigns one scalar DP job to each invocation, relying on batch-level parallelism
Grid size	principal experimental setting 64	fixed traceback tile length 128 in the current ABI
Stored boundary	compact row/column states used by the CUDA kernels	five two-piece-affine states on each bottom and right tile border
Trace endpoint	global maximum for the described lightweight extension	bottom-right of the selected global coordinate window in the CIGAR shader
Gap model	inspected gridded CUDA trace uses one affine family	two affine families with short and long gaps
Input	packed CUDA query/reference batches	alphabet-rank arrays packed from lazy/PostgreSQL views after coordinate selection
Fast paths	GPU extension kernels and G3SA scheduling optimisations	exact, simple-script, unit-gap, bounded full-trace, and gridded routes
Integration proof	standalone end-to-end aligner comparisons	SQL E2E counters include selected branch, jobs, waves, CIGAR jobs, traceback words, declines, and forbidden fallbacks

Two current-source limitations prevent an over-broad equivalence claim. First, invocations within a CIGAR workgroup do not cooperate on one matrix. The batch is parallel, but each admitted job executes scalar row loops. This is unlike G3SA's subwarp-level forward kernel. Second, the CIGAR wire carries band width, Z-drop, and end bonus, but the current bounded and gridded global CIGAR recurrences use neither Z-drop nor end bonus, and their cell loops do not prune by the supplied band. Band width is validated at admission. A separate score-only Vulkan prototype does implement single-affine local banding for spans up to 512, but that runtime surface is not the same as the dedicated SQL CIGAR route.

The result is best described as a G3SA-inspired traceback memory layout, not a Vulkan translation of G3SA. BioSeqDB gains the border/recompute trade-off and batch amortisation, while retaining its own SQL admission rules, alphabet surface, gap model, result ABI, and fail-closed evidence.

8.9. Intermediate measurements and current evidence

8.9.1. CPU block-size sweep

The current `Minimap2Params` source preserves a tuning sweep performed on an Apple M1/Neon long, very-long, and extra-long CCS benchmark grid. It used ntHash minimizers and SIMD chaining, with KSW2 reported at 172 ms for the cited very-long baseline. Table 8.1 reproduces the displayed Block timings. The final column is calculated directly from the two printed millisecond values as $1 - t_{\text{Block}}/172$. It is not copied from the nearby rounded source annotations, which aggregate the broader grid and do not always equal the single displayed ratio.

Table 8.1: Historical Block Aligner interval sweep embedded in current source. Positive reduction means lower time than the displayed 172 ms KSW2 reference. This was not rerun during thesis drafting.

B_{min}	B_{max}	Block time (ms)	Derived reduction
32	256	191	−11.0%
32	128	163	5.2%
32	64	161	6.4%
8	32	139	19.2%
8	16	132	23.3%
8	64	137	20.3%
16	64	135	21.5%
16	32	129	25.0%

The trend explains the implementation decision. Most production rectangles in that workload were short gap fills. A maximum of 256 enlarged eight border and checkpoint buffers plus the trace geometry even though growth rarely passed 32. The in-process parameter default was therefore changed to 16–32. A smaller maximum is not universally superior. High-error legacy Oxford Nanopore reads and long unseeded gaps are exactly the shapes for which Block growth was designed. PostgreSQL's retained 32–256 defaults are conservative for those shapes but forgo the measured short-fill win.

The sweep also does not establish score equivalence with KSW2. KSW2 uses two affine families and a static band. Block uses one affine family and an adaptive heuristic. A faster time can coexist with a different optimum or a different placement of a tied gap. Performance and semantic-distance tests must therefore be reported separately.

8.9.2. Resident GPU evidence

The newest authoritative final-alignment evidence in DOC0356/DOC0357 is the final 2026-06-28 `report_47.json` signoff. Earlier chronological sections in those documents say that general gridded traceback was incomplete. The final addenda explicitly supersede those statements, and the current shader contains the implemented boundary/recompute path.

The 4096-job CIGAR case executed 4096 gridded jobs in one extension dispatch and planned 327,069,696 traceback words. It reduced wall time by 59.891%. The 512-job version was correct and used 40,883,712 traceback words, but GPU and SIMD wall times were effectively equal. This is a useful negative result: gridded storage alone does not make a small

Table 8.2: Selected PostgreSQL E2E evidence from `report_47.json`. "Correct" is the report's case-specific acceptance measure. All three GPU rows recorded zero CPU fallbacks, zero SIMD fallbacks, and zero Vulkan declines.

Case	GPU (s)	SIMD (s)	Speedup	Correctness
`E2E_BATCH_512_MANY_GAP_CIGAR`	1.241472	1.240568	0.999×	512/512 matches and CIGARs
`E2E_BATCH_4096_MANY_GAP_CIGAR`	1.809468	4.511529	2.493×	4096/4096 matches and CIGARs
`E2E_BATCH_16_300M_SEG_COORD`	10.585378	25.746413	2.432×	16/16 expected queries found

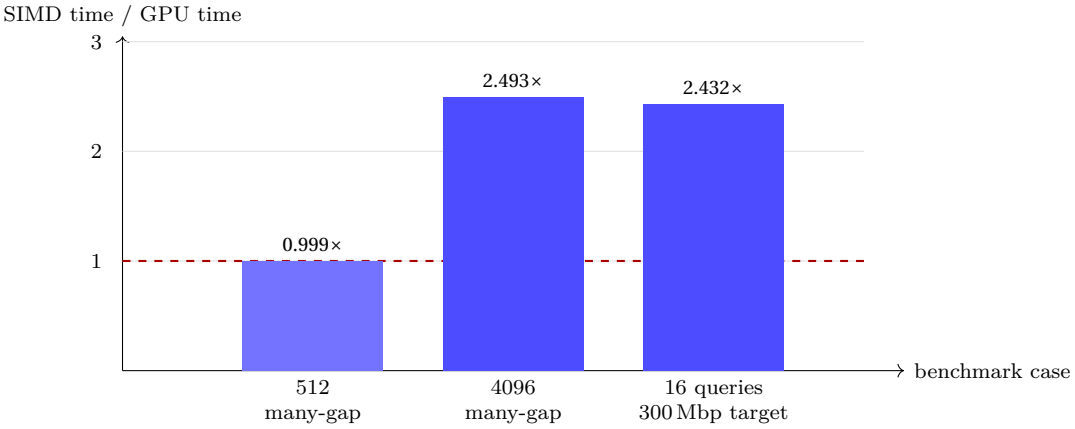


Figure 8.4: Batch-size effect in the selected E2E evidence. The dashed line is break-even. A 512-job gridded CIGAR batch is tied, while larger or target-heavy batches amortise the PostgreSQL and Vulkan fixed costs.

batch fast. Runtime initialisation, rank packing, job classification, allocation, submission, and result construction remain.

The 300 Mbp coordinate case gives a different view of amortisation. Its GPU region time was 3.857821 s versus 25.612293 s for the SIMD control, a 6.639-fold region speedup. Total wall time improved by 2.432-fold because the GPU path also spent approximately 12.082 ms in query sketching, 1547.931 ms in target deduplication, 785.808 ms in index building, and 70.653 ms in seed/chaining runtime. It processed 608 problems in 38 waves, built 38 segment indexes, and reused them 570 times.

Correctness for that coordinate case means at least one in-tolerance hit for each of 16 expected queries. It is not exact row-set parity. The GPU tuple emitted 233 match rows while the SIMD tuple emitted 16. This caveat is material for downstream SQL that expects one row per query. It does not affect the 4096-job CIGAR case's reported 4096/4096 CIGAR comparison, but it prevents using the coordinate benchmark as a blanket equivalence proof.

Other source-config proof rows in the same report covered 64- and 512-job long CIGARs, 512- and 4096-job bounded CIGARs, a 4096-job indel case, and an 8192-job bounded case. They reported correct non-empty CIGARs and zero fallback/decline counters. The final unit and integration gate contained 196 test cases, of which 195 passed and one was skipped, with 3071 of 3071 assertions passing. These tests prove the checked-in acceptance matrix. They do not remove the length cap, scalar-per-job limitation, or row-set caveat.

8.9.3. Why the paper and project numbers should not be merged

Three superficially similar speed claims now appear in this chapter:

- Block Aligner's paper comparison against several CPU aligners;
- BioSeqDB's embedded M1 Block-vs-KSW2 tuning sweep;
- G3SA's CUDA results and BioSeqDB's Vulkan/PostgreSQL E2E results.

They answer different questions. The Block paper isolates a library algorithm and reports a score-error rate. The M1 sweep chooses scratch geometry inside BioSeqDB's CPU pipeline. G3SA measures a modified standalone mapper on CUDA. The BioSeqDB E2E cases include PostgreSQL planning, data preparation, target index handling, batching, GPU submission, and SQL result projection. A single "alignment speedup" column across all four would be misleading. The thesis therefore uses each number only for the design decision it actually supports.

8.10. The final solution and its boundaries

8.10.1. What was retained

The resulting architecture is a portfolio with explicit route contracts, rather than one kernel forced onto every query:

1. The region geometry remains minimap2-derived. Left extension, accumulated seed-gap fills, and right extension preserve the useful decomposition that prevents enormous DP matrices.
2. KSW2 remains the semantic reference option for two-piece-affine scoring, finite bands, Z-drop, and minimap-style endpoint selection. The BioSeqDB port adds lazy subrange encoding, arena allocation, eliminated clears, and portable Highway dispatch.

3. Native Block Aligner is the default CPU selection. It is favoured for the many short mandatory fills exposed by the seed walk. The implementation adds direct compact input, compressed trace masks, fused edit counting, exact diagonal certificates, and fail-closed phase evidence.

4. Batched KSW2 retains descriptor collection, geometry grouping, and deterministic restitching as an internal experiment. It is not exposed as a generated PostgreSQL GUC because its arithmetic remains lane-by-lane.

5. The optional dedicated GPU tuple uses resident Vulkan seeding and chaining. Its CIGAR route applies a G3SA-inspired 128-cell border/recompute scheme with a BioSeqDB-specific two-piece-affine shader and SQL evidence.

Table 8.3: Current final-alignment selection rules. "Default" is stated at the layer where it is actually defined.

Route	How selected	Best reason to select	Main limitation
CPU Block	C++ and PostgreSQL base-alignment default	many short fills, portable SIMD, low scratch at tuned block sizes	single affine and adaptive heuristic; C++ 16–32 and SQL 32–256 defaults differ
CPU KSW2	explicit base-alignment option	two-piece affine, band/Z-drop, closest current path to minimap2's ordinary extension core	missing minimap2 splice, inversion, and second-pass policies in the surrounding port
internal batched KSW2	internal enum only; generated SQL value omitted	experimentation with deferred fill scheduling and restitching	no fused inter-sequence SIMD DP yet
dedicated Vulkan	complete all-GPU tuple, plus explicit extension enable for CIGAR	sufficiently large batches or targets, resident index reuse, gridded CIGAR memory	narrow SQL feature tuple, 16,384-symbol CIGAR span cap, scalar-per-job DP, no band/Z-drop in current CIGAR recurrence
no base DP	neither CIGAR nor exact edit requested	avoid work when seed/chain coordinates satisfy projection needs	no base-level path or exact edit result

8.10.2. How the final design differs from the starting point

The starting point was conceptually simple: port minimap2's KSW2 calls and obtain a CIGAR. The final design differs at every surrounding boundary. Sequences are not eagerly converted whole. A chain is split into accumulated mandatory fills. The kernel is selected once and compiled into a branch-free inner path. Block Aligner is not disguised behind all KSW flags. Traceback can produce the exact SQL edit count. A packed diagonal proof can eliminate DP. Algorithm execution is attested. A GPU route owns its whole selected seed–chain–coordinate–CIGAR path and is forbidden from falling back.

The changes also reveal which optimisations did not become the final solution. The first idea of inter-sequence KSW2 batching stopped at scheduling and stitching. A Block interval of 32–256 was too expensive for the measured short-fill CPU workload, although it remains

the SQL safety-oriented default. The first bounded GPU CIGAR route was insufficient for long many-gap cases, which motivated gridded traceback. Even after gridded traceback, 512 jobs did not beat SIMD because fixed costs dominated. The final system therefore retains multiple routes and reports the break-even behaviour instead of declaring one accelerator universally faster.

8.10.3. Open technical limitations

Four limitations define useful future work:

1. Semantic modes. The BioSeqDB minimap2 port needs explicit splice/RNA, long-join, inversion, and two-pass Z-drop support before those minimap2 modes can be claimed. Block Aligner would need a two-piece-affine extension, or a documented routing rule back to KSW2, for closer default score compatibility.

2. Configuration convergence. The 16–32 C++ tuning choice and 32–256 PostgreSQL GUC defaults should be reconciled through an error-profile-aware policy. A single hard-coded interval is unlikely to be optimal for both high-accuracy CCS and noisy legacy reads.

3. True inter-sequence parallelism. The descriptor buckets in KSW2-batched and the Vulkan CIGAR job batch are scheduling parallelism. A fused CPU lane recurrence or cooperative Vulkan tile would add arithmetic parallelism within one dispatch.

4. GPU parity and endpoint policy. The CIGAR shader should either implement its admitted band, Z-drop, and endpoint fields or remove them from that wire contract. Exact emitted row-set parity should be added to the 300 Mbp E2E gate, not inferred from one correct hit per query.

The central conclusion is therefore narrower and more useful than "GPU DP is fast" or "Block Aligner replaces KSW2". BioSeqDB makes final alignment a query-dependent, inspectable execution stage. It preserves KSW2 where its two-piece-affine and endpoint semantics are required, uses native Block geometry for the measured short-fill CPU regime, and admits a gridded Vulkan route only for a complete, fail-closed GPU statement shape. CIGAR, edit distance, score, route identity, and benchmark scope remain separate pieces of evidence. That separation is what allows a database-resident mapper to evolve without hiding algorithmic changes behind one SQL function name.

Chapter 9

Alignment prefiltering

9.1. Why filter after chaining but before alignment?

The seed and chaining stages do not prove that two sequence intervals are similar. They prove only that a set of exact or hashed seed matches can be arranged into a plausible region. A repetitive target, a short seed, or a noisy query can therefore produce regions whose base-level alignment is hopeless. Sending every such region to affine-gap dynamic programming is correct, but it can be unnecessarily expensive: alignment must fill or traverse a band, maintain several score states, and often retain enough state for traceback. A pre-alignment filter is a cheaper decision procedure inserted between candidate construction and dynamic programming. Its useful answer is not "these strings align", but "this candidate cannot possibly satisfy the requested edit threshold".

This distinction is important in BioSeqDB because filtering is exposed through an exact SQL predicate. A query may ask for

```
WHERE mmsearch(seq, query)
  AND bioseq.edit(seq) < 10
```

The SQL expression means that returned alignments have unit-cost edit distance strictly below ten. It does not mean that the database may return the approximation computed by SneakySnake or GateKeeper. The planner therefore normalises the exclusive bound $B = 10$ to the inclusive prefilter threshold $E = B - 1 = 9$. A cheap stage may remove a candidate only when it can prove that its distance is greater than E. Every survivor still undergoes the selected base aligner, and an exact post-traceback stage applies the original strict bound. In current source this ordering is explicit in `pipe.cppm`: the `PrefilterStage` is followed by `AlignStage`, then `ExactEditFilterStage`, before the final primary/secondary classification and mapping-quality calculation (`pipe.cppm`, lines 8618–8628 in the inspected revision).

Let $D(Q, T)$ denote the global unit-cost Levenshtein distance of a mandatory query gap Q and target gap T. A conventional safe filter computes a quantity $L(Q, T)$ with

$$0 \le L(Q, T) \le D(Q, T).$$

It rejects only if $L(Q, T) > E$. The two possible filter outputs are therefore asymmetric:

Filter output	What is established	Pipeline action
reject	the threshold cannot be met in the safe domain	omit expensive alignment for this region
accept	nothing more than "not disproved"	align, reconstruct the exact edit count, and enforce the SQL predicate

A false accept wastes time but does not change query results. A false reject removes a valid alignment and is a correctness error. This is why the chapter uses conservative accept for unsupported geometry and why benchmark tables report false rejects separately from throughput.

9.1.1. The safe rejection domain in BioSeqDB

The mapper does not globally align an entire chained region in one rectangular problem. Seeds already account for exact anchored pieces, while alignment fills the gaps between successive seeds and may extend locally at the left and right ends. A lower bound for a mandatory seed-to-seed gap is composable: disjoint mandatory gaps require disjoint edits, so their lower bounds may be summed for a region. A bound derived from a local extension window is not composable in the same way. X-drop or local alignment may legitimately omit an expensive prefix or suffix; rejecting because that optional window has a large global distance would delete a valid local result.

BioSeqDB therefore materialises only mandatory global fill windows. The iteration at `pipe.cppm`, lines 7985–8048, converts each window to flat alphabet ranks, applies reverse-query orientation when required, and records which region owns the job. Left and right local extensions are deliberately excluded. Results are accumulated per region with saturating addition, and stable in-place compaction retains survivor order (lines 8105–8162). This is narrower than applying a published filter to the whole candidate interval, but it matches the semantic unit that the aligner is obliged to solve.

Three additional rules close common correctness gaps:

1. For unequal-length windows, both implemented filters use only $||Q| - |T||$, an exact lower bound on the number of insertions and deletions. The equal-length chip-maze or bit-mask approximation is not extrapolated to geometry for which the port has no proof.

2. Invalid, overflowing, or unsupported jobs return status `unsupported_conservative` and are accepted. "Fail open" here means preserving candidates, not ignoring an execution error.

3. If a Vulkan backend was explicitly selected and its runtime or hardware execution fails, the query fails rather than silently changing to CPU. This is a route-integrity rule. It is distinct from the conservative-accept result returned for a representable unsupported job.

The common ABI makes these cases visible. `PrefilterJob` is a 20-byte record containing two offsets, two lengths, and E; `PrefilterResult` is a 16-byte record containing a lower bound, status, accept bit, and diagnostic word (`prefilter_types.cppm`, lines 16–61). The three statuses are `success`, `conservative_unequal`, and `unsupported_conservative`. The same records cross the scalar, Highway, and Vulkan boundaries, avoiding a host-side translation whose semantics could drift.

9.1.2. Difference from the minimap2 baseline

The minimap2 pipeline supplies the surrounding algorithm but not this query-scoped filter. After chaining and region selection, ordinary minimap2 invokes its KSW2 extension machinery for the selected candidate gaps and uses alignment scores, z-drop, and traceback to decide how the region should be reported. It has seed-frequency and chaining heuristics that reduce the candidate set, but these are not a proof of the SQL condition $\mathsf{edit} < B$. The SneakySnake and GateKeeper papers likewise evaluate candidate read/reference pairs before an exact aligner; neither defines how an edit predicate should participate in PostgreSQL planning.

BioSeqDB preserves minimap2-derived seeds, chains, region geometry, and the chosen final aligner. It inserts one additional conditional stage after divergence estimation and before base alignment. The stage is enabled only when the query requests exact edit accounting and provides a representable positive strict bound. Rejection removes KSW2 or Block-Aligner work; acceptance does not change scoring parameters, z-drop, CIGAR construction, primary versus secondary classification, or mapping quality. The exact survivor stage runs before the final classification pass so a candidate that violates the SQL predicate cannot influence another survivor's metadata.

This placement also explains why the implementation filters seed-to-seed gaps rather than the raw pair used by the standalone paper tools. Minimap2 has already fixed exact seed anchors and may use local end extension. BioSeqDB adapts each underlying algorithm to that decomposition instead of changing the decomposition to resemble the paper benchmark. The whole-pipeline evidence in Section 9.5 therefore reports avoided KSW2 calls and input-cell area, which are more informative for this port than a standalone filter throughput number alone.

9.2. SneakySnake: routing through a chip maze

The original paper, SneakySnake: A Fast and Accurate Universal Genome Pre-Alignment Filter for CPUs, GPUs, and FPGAs, reformulates approximate string matching as a single-net-routing problem. For equal-length strings of length m and threshold E, it considers a grid with m columns and $2E+1$ rows. A row represents one relative diagonal $d \in [-E, E]$. At column j, the cell is free when the target symbol matches the query symbol at the shifted coordinate and is an obstacle otherwise:

$$Z_d(j) = \begin{cases} 0, & 0 \le j + d < m \text{ and } Q[j+d] = T[j], \\ 1, & \text{otherwise.} \end{cases}$$

The paper describes rows as horizontal routing tracks and columns as vertical routing tracks. A horizontal run of zero cells followed by at most one obstacle is an escape segment. At a checkpoint, the algorithm examines all $2E+1$ diagonals, takes the longest zero run, crosses its following obstacle if one exists, and advances the checkpoint. Reaching the right boundary with at most E obstacles means that exact alignment is still necessary; observing more than E obstacles permits rejection.

Figure 9.1 reconstructs this mechanism rather than copying the paper artwork. Its visual grammar follows the project figure reconstruction reference: a white background, explicit row labels, black obstacles, and one amber route. The particular bits are illustrative; the invariant is the movement from one checkpoint to the furthest available horizontal segment.

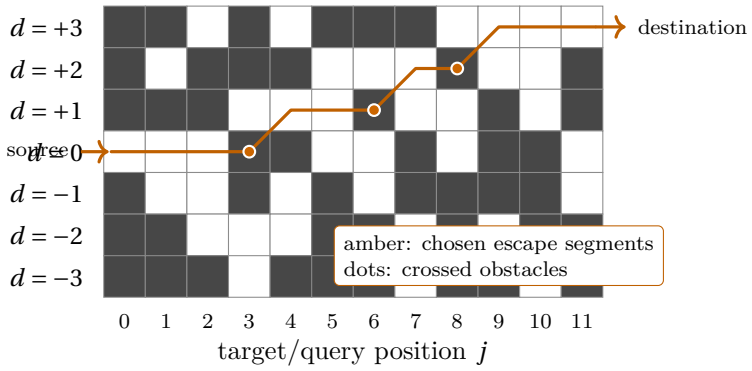


Figure 9.1: Reconstruction of the SneakySnake chip-maze walk. Each row is a query–target shift, a white cell is a match, and a dark cell is an obstacle. From each checkpoint the route takes the longest immediately available horizontal match run. The number of crossed obstacles is used as an edit-distance lower bound; the route does not reconstruct an edit script.

9.2.1. Why the obstacle count can filter

The chip maze is intentionally less expressive than dynamic programming. It does not charge vertical distance between escape segments and it does not record which insertion, deletion, or substitution produced a shift. That loss of information is useful only in one direction: the route may underestimate the true edit distance, but a reliable implementation must not overestimate it. If the route crosses r obstacles and $r > D(Q, T)$, then a valid pair could be rejected. The paper's central argument is that the greedy route minimises obstacles in its routing model and that this number is a lower bound for the corresponding equal-length global matching problem.

In executable form, the core walk is:

```
lower_bound = 0
for each non-overlapping submaze [start,end):
    checkpoint = start
    while checkpoint < end:
        best = longest matching prefix among shifts -E..+E
        if best reaches end:
            break
        lower_bound += 1                    # cross one obstacle
        if lower_bound > E:
            reject
        checkpoint += best + 1
accept for exact alignment
```

The current BioSeqDB scalar and Highway implementations follow this structure in `sneaky_snake.cppm`, lines 350–400. The production submaze width is 512 symbols. Partitioning a long maze makes execution and GPU resource requirements bounded; it can reduce filter strength by producing a smaller lower bound, but that direction creates false accepts rather than false rejects.

9.2.2. From the reference implementation to the BioSeqDB port

The checked-in reference scalar code operates on character arrays, loops over DNA-like strings, partitions by a caller-provided `KmerSize`, and uses labels to leave a diagonal scan at its first mismatch. Its CUDA counterpart uses fixed packing and compile-time dimensions. BioSeqDB retains the algorithmic walk but changes the representation and execution contract:

1. Alphabet-generic ranks. The filter compares normalised rank bytes for equality. It does not encode A, C, G, T inside the algorithm, so DNA4, DNA5, amino-acid, and generated alphabets use the same primitive once the pipeline has materialised the mandatory gaps. Equal numeric ranks are compared only after the surrounding sequence type has established a common alphabet.

2. Checked flat batches. Each job names slices in two flat rank arrays. Offset and length checks occur before a load. Oversized spans and invalid slices conservatively survive.

3. Safe unequal lengths. Instead of importing the archived unequal-maze deduction, the implementation returns the exact length difference and marks the result `conservative_unequal` (`sneaky_snake.cppm`, lines 402–425).

4. Pipeline-safe windows. Only mandatory global fills are evaluated. Reverse-strand gaps are materialised in their oriented rank order. Local extension regions never cause a rejection.

5. Query-scoped threshold. E is derived from the SQL predicate, not from a global feature switch. The same query also enables exact edit accounting after alignment.

The first CPU port performed one rank comparison at a time and the first SIMD design vectorised several independent jobs with gather-heavy state. That implementation was correct but slower than the bundled C reference in most standalone cells. The retained scalar path now scans a matching prefix with unaligned 64-bit loads, XOR, and first-differing-byte detection (`sneaky_snake.cppm`, lines 74–154). The retained Highway path loads contiguous rank bytes, compares complete vectors, and locates the first true mismatch lane (lines 156–214). At a checkpoint, the implementation first measures the unshifted diagonal, freezes the best length, and tests a candidate shift only if the query symbol at that best-offset could improve the result. Highway applies this candidate test to a contiguous window, avoiding the earlier per-lane job-state machinery (lines 263–348).

The optimisation is exact, not heuristic. Suppose the current best run has length b. A candidate diagonal can improve the maximum only if its first b symbols match; in particular, it must match the target at offset b. Testing that symbol discards only candidates that cannot beat the current best. Full prefix scanning is performed for every surviving candidate.

9.2.3. SneakySnake on Vulkan

The Vulkan route uses the same job and result layout as the CPU paths and one hardware-only runtime. The shader domain is bounded by a 512-symbol submaze and $E \le 31$ (`phase_c_abi.cppm`, lines 61–64). Larger thresholds return an unsupported conservative result. Two execution modes were retained after measurement:

- mode 1 assigns an independent invocation to a job and is used for low thresholds, where synchronisation and team setup cost more than they save;

- mode 8 uses an eight-lane team when at least one supported equal-length job has $E \geq 16$, exposing diagonal work in parallel.

The resolver is visible at `runtime.cppm`, lines 14939–14975. Both modes have persistent recorded-command identities. Descriptor bindings, push bytes, group count, and pipeline identity are part of the replay key, so reusing a command buffer cannot replay stale geometry. Rank buffers, descriptor state, memory-admission accounting, fences, and result storage are shared with GateKeeper rather than duplicated. A hardware support probe never turns software Vulkan such as llvmpipe into a claimed GPU result.

9.3. GateKeeper: shifted masks, amendment, and an exact guard

SneakySnake is attractive over a wide range of lengths and thresholds, but its checkpoint loop and diagonal scans leave fixed overhead for very small seed gaps. GateKeeper is a bit-vector filter derived from Shifted Hamming Distance. The original GateKeeper-GPU system packs DNA symbols into two bits, creates one CUDA thread per read/reference pair, and fixes read length and threshold at compile time. Its small, regular boolean data flow is a better match for the 32- and 64-symbol gaps that occur in the BioSeqDB pipeline.

For equal-length Q, T and threshold E, define a mismatch mask on the unshifted diagonal,

$$H_0[j] = [Q[j] \neq T[j]],$$

and two masks for every displacement $d = 1, \ldots, E$:

$$H_{-d}[j] = [j \geq d \ \wedge \ Q[j-d] \neq T[j]],$$
$$H_{+d}[j] = [j + d < m \ \wedge \ Q[j+d] \neq T[j]].$$

Thus the filter constructs $2E+1$ logical masks. A zero in a mask denotes a possible match. Insertions and deletions move a true match run away from the main diagonal, so the shifted rows can expose it.

9.3.1. Why masks are amended

A single accidental zero dominates a bitwise AND even if all other diagonals contain mismatches at that position. GateKeeper therefore amends short internal runs of zeros before combining masks. In the BioSeqDB clean-room implementation, a zero run is filled only when it is bounded by ones and has length one or two. Edge runs and runs of three or more are preserved. For a conceptually unbounded bit vector M, with zeros outside the valid sequence interval, the operation can be expressed as

$$\mathsf{single} = \neg M \wedge (M \ll 1) \wedge (M \gg 1),$$
$$\mathsf{pairStart} = \neg M \wedge (\neg M \gg 1) \wedge (M \ll 1) \wedge (M \gg 2),$$
$$\mathsf{amend}(M) = M \vee \mathsf{single} \vee \mathsf{pairStart} \vee (\mathsf{pairStart} \ll 1).$$

The result is masked back to valid positions. Multiword implementations must carry neighbouring bits across 32-bit boundaries; otherwise a two-zero run straddling a word boundary would be treated differently from the same pattern inside a word.

Shifting introduces vacant edge positions. Leaving them as zero would let the final AND hide a leading or trailing error. Following the GateKeeper-GPU improvement, BioSeqDB amends the shifted mask first and then marks the newly exposed boundary as a mismatch:

$$\mathsf{amend}(H_{-d})[d-1] \leftarrow 1, \qquad \mathsf{amend}(H_{+d})[m-d] \leftarrow 1.$$

The ordering matters. Applying compensation before amendment could cause the new boundary one to fill an adjacent edge run and change the intended mask.

All amended masks are ANDed. The resulting bit vector is divided into four-bit groups. A zero nibble costs zero. The patterns

$$0101,\ 0110,\ 1001,\ 1010,\ 1011,\ 1101$$

cost two, and every other nonzero nibble costs one. Accumulation stops at $E+1$. This reproduces the reference cost semantics without importing its 16-entry CUDA lookup table.

Figure 9.2 reconstructs the data flow from the original paper and kernel. It emphasises a feature that is easy to miss in a prose description: amendment is performed per shifted mask before the mask intersection, and the final count is a grouped estimate rather than a popcount.

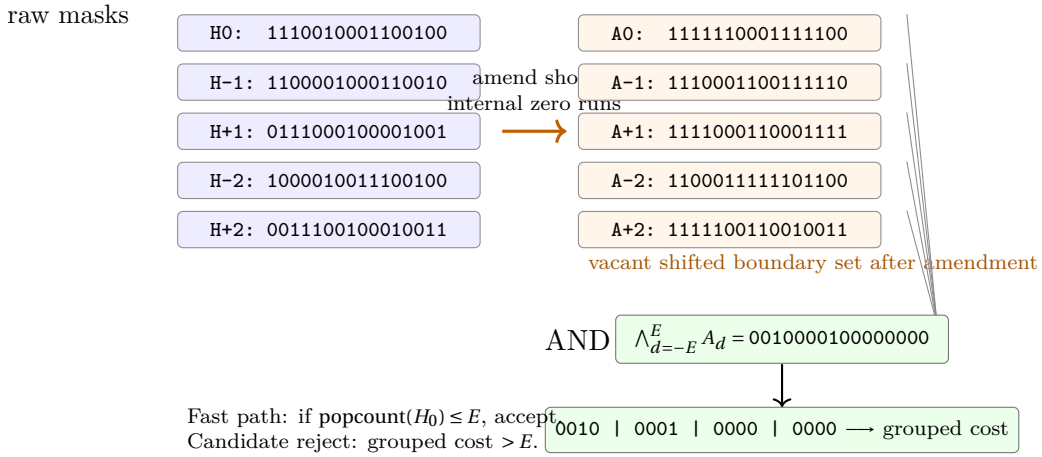


Figure 9.2: Reconstructed GateKeeper mask flow for $E = 2$. Shifted Hamming masks are amended independently, their vacant boundaries are compensated, and all $2E+1$ masks are intersected. Four-bit groups of the final mask produce a threshold estimate. The strings are explanatory masks rather than a reproduced experimental pair.

9.3.2. The exact guard changes the correctness argument

The GateKeeper-GPU paper reports empirical filtering accuracy, including its false-reject measurements, but an empirical zero is not a proof that every possible input is safe. Moreover, mask amendment deliberately changes match bits to mismatch bits. BioSeqDB therefore does not use the mask estimate alone as a rejection proof.

The current scalar algorithm at `gatekeeper.cppm`, lines 178–279, implements the following decision:

```
H = unshifted mismatch mask
if popcount(H) <= E:
    accept
if E == 0:
    reject                          # unequal symbols prove D > 0
A = amend(H)
for d = 1..E:
```

```
    A &= compensate(amend(H[-d]))
    A &= compensate(amend(H[+d]))
estimate = grouped_four_bit_cost(A), capped at E+1
if estimate <= E:
    accept
if exact_banded_Levenshtein(Q,T,E) says D > E:
    reject
accept                              # mask candidate was not exact
```

Only a mask candidate that is confirmed by exact thresholded global Levenshtein distance becomes a rejection. The banded predicate uses $2E+3$ cells per row, saturates values at $E+1$, and for $E \le 8$ stores two fixed 19-cell rows on the stack (`edit_distance_threshold.cppm`, lines 99–159). Its time is $O(mE)$, but it runs only on pairs that the much cheaper mask wants to reject. The four measured fixed Vulkan shaders use an equivalent bounded wavefront guard and reuse mismatch masks for longest-common-prefix extension.

This design gives a stronger guarantee than the published heuristic alone: a mask false reject is converted to an accept by the exact guard. It also changes the meaning of the result fields. A GateKeeper mask acceptance contributes numeric lower bound zero. A confirmed rejection returns $E+1$. The raw mask estimate is diagnostic and must not be summed across region gaps. Four bits in the result word identify a mask-rejection candidate, guard activation, guard-confirmed rejection, or the exact $E = 0$ Hamming rejection (`prefilter_types.cppm`, lines 63–78 and 128–155).

9.3.3. Clean-room, alphabet-generic implementation

The reference CUDA kernel has several properties that are unsuitable as a database extension boundary. It encodes only A, C, G, T in two bits; passes undefined symbols by a separate rule; allocates a compile-time $(2E+1)$-by-word mask matrix per thread; performs explicit carry repair between 32-bit words; uses a constant-memory amendment lookup table; and requires recompilation when length or threshold changes. Host orchestration uses CUDA unified memory and mapper-specific batches.

BioSeqDB re-derived the operations over equality-comparable rank bytes. The CPU oracle uses a 128-bit logical mask, with direct support for equal nonempty lengths up to 128 and $E \le 8$ (`gatekeeper.cppm`, lines 21–26 and 164–172). It preserves short internal zero-run semantics using shifts rather than importing the lookup table. DNA rank zero has no special interpretation, and the same code works for DNA5 or amino-acid ranks because a shifted vacant cell is represented by a mask boundary rule, not by a fake alphabet symbol.

The Vulkan implementation consists of one generic shader plus four fixed production variants:

$$(m, E) \in \{(32,2), (32,4), (64,2), (64,4)\}.$$

All have local size 64 and maximum span 128; the static variant registry is explicit in `shader_variants.cppm`, lines 344–353. A fixed variant is chosen only for a homogeneous batch whose rank offsets are four-byte aligned. It loads the packed rank words once, derives Hamming and shifted masks in registers, retains only the raw diagonals needed by its pruned exact wavefront, and does not reread rank storage during the guard. Any other direct GateKeeper batch uses the generic shader. The production pipeline is even narrower: automatic GateKeeper selection requires every job to have length 32 or 64 and threshold 2 or 4.

Property	GateKeeper-GPU reference	BioSeqDB GateKeeper
Input symbols	two-bit DNA; undefined base handled specially	alphabet-normalised rank equality; no DNA constant in the algorithm
Execution API	CUDA kernel, one pair per thread, compile-time m, E	shared batch ABI; Vulkan generic shader plus four measured fixed variants
Mask storage	$2E+1$ word arrays per thread	streamed/intersected masks; fixed variants keep packed ranks and required diagonals in registers
Amendment	16-entry LUT plus cross-word repair	boolean run definition, explicitly tested at word boundaries
Shift boundary	leading/trailing OR compensation	same semantic compensation, applied after amendment and valid-bit masking
Rejection basis	approximate grouped mask count	mask candidate plus exact bounded global-edit guard
Unsupported input	mapper-specific handling	status-2 conservative accept; selected runtime failure remains an error

Table 9.1: Algorithmic and systems differences between the research GateKeeper-GPU implementation and the BioSeqDB port.

9.4. Dispatch and integration with the query pipeline

Algorithm choice and backend choice are related but not identical. Scalar and Highway production always use SneakySnake. A Vulkan batch is resolved as GateKeeper only when every job belongs to the measured small-window domain; otherwise the whole batch uses SneakySnake. The pure resolver at `pipe.cppm`, lines 7912–7934, is equivalent to:

```
if backend != Vulkan:
    return SneakySnake
if jobs is nonempty and
   every job has equal length in {32,64} and E in {2,4}:
    return GateKeeper
return SneakySnake
```

Batch-wide resolution avoids two uploads, two queue submissions, and a result merge. It also makes the selected algorithm visible in execution proof. Direct GateKeeper tests may exercise lengths through 128 and thresholds through 8, but that larger correctness domain is not automatically promoted to production. Performance evidence, not mere functional support, controls the AUTO domain.

At the PostgreSQL boundary, the planner accepts exactly one top-level `bioseq.edit(sequence_column) < integer_constant` predicate using the built-in operators for 16-, 32-, or 64-bit integer constants. An overloaded operator merely named "<", a nonconstant bound, a wrong target column, ≤, or an ambiguous placeholder does not enter the optimisation (`custom_scan.cppm`, lines 1976–2050). Unsupported forms fail with a feature-not-supported error rather than silently evaluating a stub with different semantics.

When the strict predicate is active and $B > 0$ fits the prefilter ABI, the custom scan sets $E = B - 1$. Backend selection then follows current execution capability:

- an existing Vulkan minimisation, seeding, chaining, or extension route selects Vulkan for the prefilter too;
- a per-call `aligner_options(vulkan_enabled => true)` may explicitly request the standalone Vulkan prefilter;
- otherwise the scan selects Highway when vectorisation is enabled and scalar when it is disabled.

The global Vulkan capability flag alone does not unexpectedly move a scalar query to the GPU. This logic is visible at `custom_scan.cppm`, lines 23383–23414. Negative or zero exclusive bounds still reach the exact post-alignment predicate, but do not manufacture an unsigned prefilter threshold.

The seven persistent command-replay routes make the heterogeneous data plane explicit: SneakySnake mode 1, SneakySnake mode 8, generic GateKeeper, and the four fixed GateKeeper variants. `runtime.cppm`, lines 136–223, maps each shader/push-mode combination to a distinct slot. Both algorithms reuse one admitted high-water workspace and descriptor cache. Result validation checks not just output count but status, threshold bounds, algorithm-specific diagnostic bits, and the relationship between rejection and guard counters.

An `EXPLAIN ANALYZE` proof can consequently distinguish capability from actual work. The final GateKeeper SQL run recorded: `algorithm: batch_resolved_gatekeeper_or_sneaky_snake`, `Prefilter Resolved Algorithm: gatekeeper`, three Vulkan dispatches, three evaluated jobs, one rejected region, one mask candidate, one guard activation, and one guard-confirmed rejection. The following exact stage retained two rows with edit counts zero and four under the strict bound five.

9.5. Intermediate results and the final routing policy

The performance history matters because the final policy was not obvious from the papers. A fast published filter can become slower after generic rank conversion, safe bounds checks, batching, runtime validation, and GPU synchronisation are included. The project benchmark therefore measures `backend_batch_ns`: upload, dispatch, synchronisation, and result readback are inside the timed region. Correctness preparation and the exact truth oracle are outside it.

9.5.1. The first SneakySnake port: correct but usually slower

The final-source intermediate run `main/reporting/bioseq_bench/report_127.json` was produced on 2026-07-16 with `meta.commit=679aa4264853eeaaa8a3bd2c891e8e2da7e8e967`, `dirty=true`, and `fast=false`. The implementation report attributes the dirty flag to local builder configuration and an untracked generated report and states that compiled source matched the pushed commit; the artefact is consequently described as a scoped measurement, not as a clean-tree run. Each standalone cell used a deterministic 1,024-pair prefix from the bundled upstream corpus. Decisions matched the original and false rejects were zero, but Table 9.2 shows that the initial BioSeqDB SIMD path was between 2.39× and 2.85× the original kernel time.

Even a slower filter can improve an end-to-end mapper if it removes enough alignment work. The same report measured the complete candidate pipeline:

The HiFi case is the break-even warning: if a filter rejects nothing, its work is pure overhead and the result is neutral or slightly slower. The ONT fixture is the intended case:

Fixture/threshold	Original	Scalar	Highway	Highway/original
ERR, 100 bp, $E = 2$	0.144 ms	0.137 ms	0.343 ms	2.386
SRR, 250 bp, $E = 8$	0.919 ms	1.255 ms	2.375 ms	2.585
ERR, 100 bp, $E = 40$	9.471 ms	12.731 ms	27.015 ms	2.852
SRR, 250 bp, $E = 100$	58.331 ms	87.035 ms	161.071 ms	2.761

Table 9.2: Intermediate SneakySnake standalone P50 results from report 127. All rows had identical decisions and zero false rejects. The table explains why the first SIMD design was not accepted as the final kernel.

Fixture	Off	Scalar	Highway	Observed downstream work
MEDIUM_HIFI	22.145 ms	22.193 ms	22.309 ms	no region rejected; 175 KSW2 calls and 6,672,027 input cells in every row
MEDIUM_ONT_MODERN	21.864 ms	10.646 ms	10.373 ms	all six candidate regions removed; KSW2 calls 160 → 0, input cells 6,915,415 → 0

Table 9.3: Whole-pipeline P50 results from report 127. Threshold-qualified truth/correct counts matched the no-prefilter row.

avoiding dynamic programming more than repays the filter and produces approximately 2.1× speedup. The ONT exact threshold truth set was empty, so "recall one" there is vacuous; parity is instead established by the independently computed truth/correct counts and unchanged empty match result.

9.5.2. Optimised SneakySnake CPU and Vulkan

The next-day optimisation replaced gather-heavy independent-job SIMD with the contiguous-byte traversal described earlier, added the frozen-best candidate test, introduced the hybrid Vulkan mode, and retained recorded commands. DOC0382 identifies report 138 as the final uncontended reproduction:

Comparison in report 138	Ratio to bundled original CPU	Interpretation
production Highway, equal-weight mean over cells	0.708	29.2% faster across the configured matrix
production Highway, summed P50	0.482	51.8% less summed kernel time
Vulkan, summed P50	0.175	82.5% less summed time; 5.73× throughput

All final Vulkan cells reported an available backend, 1,024 evaluated jobs, valid result count, runtime validation, zero reference-decision mismatches, and zero false rejects. Command replay reduced summed GPU P50 by 9.1% relative to the pre-cache hybrid in report 134, and reports 137 and 138 differed by only 0.45% in summed GPU P50.

This aggregate result must not be read as "GPU wins for every threshold". In report 138 the synchronous Vulkan round trip was still 4.94–11.81× the original CPU at $E = 2$ and 1.90–2.56× at $E = 8$; the $E = 16$ and $E = 31$ team paths supplied the aggregate gain. The report metadata records commit `71fe8366555f682571c87005c3469a8e207307ab`, creation time 2026-07-17T00:54:28Z, and `dirty=true`. The result is therefore useful as the documented optimisation decision but is not described here as a clean commit artefact.

9.5.3. GateKeeper closes the small-window Vulkan gap

GateKeeper was accepted against two complete, unfiltered, clean hardware reports. Report 148 placed all eight GateKeeper/SneakySnake ratios in 0.540…0.838; report 151 independently placed them in 0.492…0.881. Table 9.4 gives the second run. Each cell contains five samples over 4,096 pairs; P50 includes the full synchronous backend batch.

E	Corpus prefix	Length	GateKeeper	SneakySnake	GK/SS
2	ERR E2	32	307,709 ns	349,208 ns	0.881
2	ERR E2	64	278,791 ns	377,625 ns	0.738
2	ERR E40	32	315,000 ns	377,709 ns	0.834
2	ERR E40	64	291,625 ns	387,208 ns	0.753
4	ERR E2	32	315,792 ns	515,042 ns	0.613
4	ERR E2	64	311,875 ns	633,500 ns	0.492
4	ERR E40	32	317,750 ns	537,750 ns	0.591
4	ERR E40	64	316,958 ns	577,167 ns	0.549

Table 9.4: GateKeeper versus SneakySnake Vulkan P50 in report 151. Every cell used a real hardware backend, passed runtime/result validation, and recorded zero false rejects.

Report 151 is `main/reporting/bioseq_bench/report_151.json`, commit `0c2c8fb7cb4570a046bb01f682b` `dirty=false`, `fast=false`, created 2026-07-17T15:43:33Z. Its 13 benchmark blocks and raw evidence were complete. The process exit status was nonzero because unrelated chaining, historical XXL, and paper-ntHash rules failed; no violation or regression referenced `bench_prefilter_small_seeds`. Keeping this distinction prevents a global red status from erasing valid scoped evidence, while also avoiding a false claim that the repository-wide benchmark was green.

The resulting production choice is deliberately piecewise:

- scalar or Highway: optimised SneakySnake;
- Vulkan, homogeneous 32/64-symbol batch with $E = 2$ or $E = 4$: specialised GateKeeper with exact guard;
- other Vulkan geometry through $E = 31$: SneakySnake, using independent invocations at low E and eight-lane teams at high E;
- unsupported geometry: conservative acceptance followed by ordinary exact alignment.

This is the principal difference from choosing one paper as a universal winner. BioSeqDB uses the algorithm whose measured end-to-end backend cost is appropriate to the batch shape, and it keeps exact post-alignment semantics independent of that choice.

9.6. Failure modes, limitations, and what the filter does not prove

Acceptance is not similarity. Both algorithms can accept a pair whose edit distance exceeds E. SneakySnake may underestimate by changing diagonals without charging their vertical distance or by solving bounded submazes. GateKeeper may lose evidence when different masks contribute zeros to the final AND. These false accepts reduce speed but are harmless because alignment and the exact predicate remain authoritative.

GateKeeper's mask estimate is not composable. The grouped four-bit cost is a threshold heuristic. BioSeqDB stores it in the diagnostic word, returns lower bound zero on acceptance, and uses $E+1$ only after the exact guard confirms rejection. Treating the raw estimate like a SneakySnake lower bound and summing it over gaps would be unsound.

Local ends are deliberately unfiltered. A whole-region global distance is not a safe reason to reject a local or X-drop extension. Only mandatory seed-to-seed fills enter the stage. This choice sacrifices some potential filtering work to preserve the mapper's alignment mode.

Unequal lengths take a weaker path. The absolute length difference may reject a gap that cannot fit E, but it often accepts a highly dissimilar unequal pair. No equal-length maze or mask claim is extrapolated. Subsequent exact alignment resolves it.

Threshold and geometry bounds are visible. SneakySnake Vulkan supports the wire threshold through 31 and a 512-symbol submaze. Direct GateKeeper supports equal lengths through 128 and thresholds through 8, while AUTO uses only 32/64 and 2/4. An out-of-range job returns a conservative status. A strict SQL bound that is nonpositive or cannot be represented does not wrap to a large unsigned E.

GPU latency and availability matter. The small-E SneakySnake GPU cells demonstrate that high throughput on larger work does not imply low latency. GateKeeper's specialised variants improve the measured small-window route, but first dispatch can still contain a 25–31 ms pipeline/runtime outlier while steady-state P50 is about 0.28–0.36 ms. The benchmark claim is a five-sample steady-state P50 claim, not a first-query latency claim. Software Vulkan is rejected as unavailable, and a selected hardware route never hides a CPU fallback.

Exact edit accounting must remain exact across aligners. KSW2 may expose explicit equality/mismatch operations, whereas the Block Aligner boundary can collapse both to a generic M. BioSeqDB compares aligned ranks symbol by symbol for a generic M and adds insertion/deletion lengths. Reverse-strand coordinate and CIGAR handling are tested separately. Without this post-traceback accounting, a safe prefilter would still not make `bioseq.edit` an exact SQL function.

Execution proof is part of observability, not the algorithm. Dispatch, evaluated/unsupported jobs, rejected regions, length-difference decisions, mask candidates, guard activations, and guard-confirmed rejections are accumulated only after a complete batch. Their invariants detect routing or result corruption and make `EXPLAIN` useful, but a counter does not replace the threshold proof or exact baseline.

9.7. Conclusion

BioSeqDB's prefiltering stage illustrates how a published approximation must change when it becomes part of a database query contract. SneakySnake contributes a broadly applicable lower-bound walk: search shifted diagonals for the longest escape segment, count unavoidable obstacles, and reject only when the count exceeds the threshold. GateKeeper contributes a compact small-window GPU path: build $2E+1$ shifted mismatch masks, amend short internal matches, compensate exposed boundaries, intersect the masks, and count grouped error patterns. BioSeqDB adds the safety and systems layers that neither paper alone defines for

this pipeline: mandatory-gap scoping, alphabet-generic ranks, checked batch slices, conservative unequal and unsupported results, an exact GateKeeper rejection guard, hardware-only Vulkan routes, stable region compaction, SQL-derived thresholds, and exact post-traceback filtering.

The intermediate measurements also explain the final design. The first SneakySnake port preserved decisions but its SIMD kernel was slower than the bundled original. Contiguous Highway scans, an exact candidate filter, hybrid Vulkan teams, and command replay reversed that aggregate result. The remaining small-threshold GPU latency motivated GateKeeper specialisation, which beat SneakySnake by more than ten percent in all sixteen comparisons across two complete reports. The final solution is therefore not "prefilter everything" and not "always use the GPU". It is a conservative stage whose domain and algorithm are selected from measured batch geometry, and whose survivors are always judged by the exact alignment semantics requested in SQL.

Chapter 10

Sequence representations and lazy evaluation

10.1. One logical sequence, several physical forms

The algorithms in the preceding chapters are easiest to describe over an array of nucleotide codes. A database system rarely has such an array ready. The same logical sequence may be an in-memory bit-packed object, an externally TOASTed PostgreSQL value, a set of columnar fragments, a slice of a provider object, a concatenation of several values, or a reversed view of any of these. Converting every case to a byte-per-symbol vector would make the algorithm interfaces simple, but it would also discard one of the main advantages of bringing alignment into the database: the ability to read and decode only the part selected by the query.

BioSeqDB therefore separates four ideas which are easily conflated:

1. the logical sequence, a length and an ordered stream of alphabet ranks;
2. the packed geometry, which states how ranks occupy physical storage units;
3. the representation, which owns or locates the symbols and determines their lifetime; and
4. the access requirement of a consumer, which decides whether streaming is sufficient or a contiguous packed region must be acquired.

This separation is the common data plane beneath minimizer extraction, indexing, chaining input preparation, alignment, PostgreSQL operators, and GPU uploads. It also gives a precise meaning to "lazy". Laziness does not mean that an expression allocates nothing. A slice or concatenation may allocate tree nodes, retain a source handle, and record provenance. It means that the complete symbol payload is not decoded and repacked until a consumer declares a boundary which requires it.

The final design has two principal C++ interfaces. The `compact_sequence_range` family describes a resident packed range whose alphabet and storage-unit type are known at compile time. The `basic_lazy_sequence_view` facade describes a logical rank stream whose backing representation is selected at run time. PostgreSQL expanded values, source-backed pieces, fragmented scanners, and flat packed values are adapted to the latter. A consumer which truly needs packed resident storage uses an explicit access plan to borrow or materialize a region. Figure 10.1 summarizes these layers.

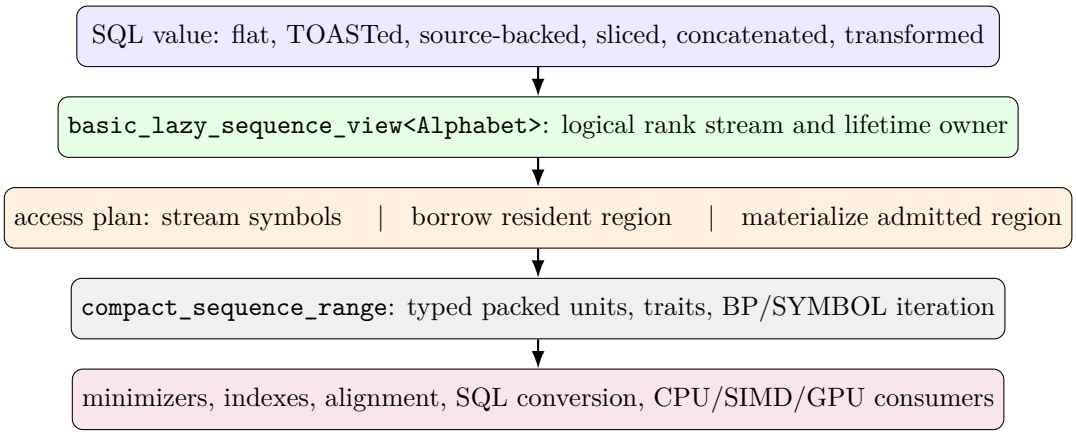


Figure 10.1: The final sequence data plane. A consumer reaches a typed packed range only when its access requirement makes that representation necessary.

10.2. The compile-time compact range

10.2.1. Alphabet, container, and range concepts

The core definitions are in `main/bioseq/backend/sequences/sequences.cppm`. An alphabet type satisfies `sequence_alphabet` when SeqAn can obtain a rank and a compile-time cardinality from it. A `sequence_custom_alphabet` combines that logical alphabet with a physical unit and a chosen bit width. A proper unpacked sequence container is a sized random-access range whose reference is an alphabet value. Finally, `supports_compact_sequence_range` identifies source types from which the packed range family can be formed. Listing 10.1 presents the public shape; function bodies and diagnostic details are omitted.

Listing 10.1: Abridged C++26 concepts and packed-range traits from `sequences.cppm`, symbols at lines 37–113 and 874–929.

```
template<class A>
concept sequence_alphabet = requires (A const a) {
    { seqan3::to_rank(a) };
    { seqan3::alphabet_size<A> };
};

template<sequence_alphabet Alphabet,
         class StorageUnit,
         size_t LetterBitWidth>
struct sequence_custom_alphabet {
    using alphabet_type = Alphabet;
    using storage_unit = StorageUnit;
    static constexpr size_t letter_bit_width = LetterBitWidth;
};

template<class R>
concept sequence_proper_container =
    sequence_alphabet<
        remove_reference_t<ranges::range_reference_t<R>>> &&
    ranges::sized_range<R> &&
    ranges::random_access_range<R>;

template<class R>
concept supports_compact_sequence_range =
```

```
    is_sequence_custom_alphabet<R> ||
    sequence_proper_container<R> ||
    sequence_alphabet<R> ||
    is_compact_sequence_stdc_instance<R>;

template<class Derived, class Alphabet, class StorageUnit,
         size_t LetterBitWidth>
struct compact_sequence_range_base {
    struct traits {
        using alphabet_type = Alphabet;
        using storage_unit = StorageUnit;
        static constexpr size_t bits_per_char = LetterBitWidth;
        static constexpr size_t unit_width = sizeof(StorageUnit) * 8;
        static constexpr size_t letters_per_unit =
            unit_width / bits_per_char;
        static constexpr size_t unit_used_bits =
            letters_per_unit * bits_per_char;
        static constexpr size_t unit_leftover_bits =
            unit_width - unit_used_bits;
        static constexpr StorageUnit char_mask =
            (StorageUnit{1} << bits_per_char) - 1;
        static constexpr size_t alphabet_size =
            seqan3::alphabet_size<Alphabet>;
        static constexpr size_t alphabet_size_n =
            alphabet_size + 1;
    };
};
```

The abstraction deliberately exposes both the logical alphabet and the physical unit. An algorithm such as minimizer extraction can use `traits::alphabet_type` for complement semantics, `traits::bits_per_char` for masks, and `traits::letters_per_unit` for vectorized loads without hard-coding DNA4 or a 64-bit word. The additional `alphabet_size_n = alphabet_size + 1` trait is part of the current algorithm-facing API; it must not be confused with the true cardinality, which remains `alphabet_size`.

There are two important qualifications. First, `supports_compact_sequence_range` is a construction-domain concept, not a promise that its argument already owns packed bytes. A public algorithm which is intended to consume a range should accept the resulting range or the new behavioral lazy facade, rather than repeatedly reconstructing it. Second, storage geometry is not an algorithm's arithmetic state. The K=27 failure documented in DOC0213 arose when a 32-bit physical DNA4 unit—which contains only 16 ranks—was also treated as the rolling-hash state. The sequence was stored correctly, but the k-mer was silently shortened. The final rule is that packing width controls loads and iteration; a hash, k-mer, score, or coordinate type is selected independently from the mathematical range it must represent.

10.2.2. Ranks, symbols, and orientation

The base range exposes two scalar iterator interpretations. BP iteration returns integer ranks, while SYMBOL iteration converts each rank to the alphabet type. Algorithms whose inner loops need only small integers avoid constructing alphabet objects; text output and alphabet-aware validation use the symbolic form. The logical length is

$$N = U_{\text{count}} L - P,$$

where U_{count} is the number of storage units, L is `letters_per_unit`, and P is the number of trailing padding slots. `bp_size()` returns N, whereas `size()` denotes storage units. Keeping these names distinct matters: using unit count as symbol count is correct only when $L = 1$.

The conversion API includes full extraction and range extraction. The full `to_encoded_vector()` method is intentionally eager and is appropriate for a legacy byte-oriented library such as KSW2. The range form and `to_encoded_into<IsReversed,WriteReversed>` decode only $[from, to)$. The two Boolean dimensions are independent: `IsReversed` mirrors logical coordinates and applies complement semantics for the reverse strand, while `WriteReversed` controls the order placed in the destination. The four combinations are required by canonical minimizer cursors. Collapsing the two flags into one "reverse" bit once caused source-backed reverse-write calls to read the wrong coordinates; the current facade preserves the full four-callback table.

10.3. Bit layout and reinterpretation

10.3.1. Least-significant symbol first

Let an alphabet contain σ ranks and let

$$b = \lceil \log_2 \sigma \rceil$$

be the number of bits per symbol. For a physical unit of W bytes, the current layout stores

$$L = \left\lfloor \frac{8W}{b} \right\rfloor \quad \text{symbols and leaves} \quad r = 8W - Lb$$

unused high bits. For logical position i,

$$u(i) = \lfloor i/L \rfloor, \qquad s(i) = b(i \bmod L), \qquad rank(i) = \big(unit[u(i)] \gg s(i)\big) \,\&\, (2^b - 1).$$

The first logical symbol occupies the least significant b bits of the first unit. Figure 10.2 reconstructs a 16-bit DNA4 example. It is a description of BioSeqDB's source invariant, not a generic property of every biological file format.

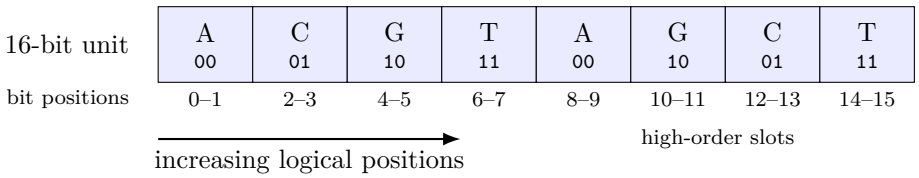


Figure 10.2: DNA4 ranks in one 16-bit unit. Position zero is in bits 0–1, so sequential extraction masks the low bits and shifts right.

Table 10.1 shows why non-power-of-two alphabets require care. DNA4 fills every supported width exactly. DNA5 uses three-bit ranks, so one or two bits per physical unit are not symbol slots. The padding count describes empty slots in the last used unit and is separate from those per-unit unused bits.

For a resident packed range, reading an arbitrary rank is $O(1)$: one unit load, one shift, and one mask. Sequential traversal is $O(N)$, with a constant determined by unit width, iterator form, and vectorization. Packing or extracting an interval of m symbols is $O(m)$. The range extraction path seeks directly to the containing unit; it does not decode the prefix before `from`. These bounds do not imply that a source-backed rank is always $O(1)$ in wall time: a provider may need a fragment lookup or bounded I/O. The facade's guarantee is that only the requested window is asked for.

Table 10.1: Current packing geometry for DNA4 and DNA5. W is the physical unit width in bytes.

Alphabet	b	W	$L = \lfloor 8W/b \rfloor$	unused bits/unit
DNA4	2	1	4	0
DNA4	2	2	8	0
DNA4	2	4	16	0
DNA4	2	8	32	0
DNA5	3	1	2	2
DNA5	3	2	5	1
DNA5	3	4	10	2
DNA5	3	8	21	1

10.3.2. The bit-casted view

`bit_casted_view` in `sequences.cppm`, lines 726–866, changes how a resident byte stream is stepped without changing its content. A DNA4/`uint64_t` range may, for example, be viewed as `uint32_t` units when source and target have the same alphabet and bit width, their unit widths divide one another, and both geometries have no leftover bits. The view owns only a reference to its base. Its `read_cptr` reinterprets the address, while `advance_iter` and `view_distance` scale by the power-of-two width ratio. Logical length and byte length do not change.

This facility is narrower than an arbitrary codec conversion. It does not change alphabet, symbol width, padding semantics, byte order, or lifetime. Nor does it legalize every typed access. The lazy access planner has a separate `byte_upload_compatible` policy for one little-endian 64-to-32-bit case. That policy returns a descriptor whose bytes may be copied to a Vulkan buffer; the source allocation still has an effective `uint64_t` type, so a CPU consumer must not dereference it through a `uint32_t*`. Native typed borrowing and byte-compatible upload are recorded as different receipt kinds.

Table 10.2: Packed-range operations and their data movement.

Operation	Cost	Ownership/copy behavior
construct owning range from symbols	$O(N)$	packs into an owned vector
construct range over standard C object	$O(1)$	stores a reference to the aligned payload; zero copy
read one resident rank	$O(1)$	one unit load and bit extraction
iterate ranks or symbols	$O(N)$	no payload copy; symbols may be constructed on demand
extract $[a, b)$ to ranks	$O(b-a)$	writes exactly the selected ranks
unit-aligned `split_units`	$O(k)$ metadata	records unit offsets; only aligned boundaries are zero-copy
construct `bit_casted_view`	$O(1)$	reference-only reinterpretation under the strict geometry contract

10.4. A C-compatible packed object

10.4.1. Standard layout and a zero-copy adapter

An owning C++ range is unsuitable as a PostgreSQL datum because `std::vector` is neither a C ABI nor a varlena payload. The `compact_sequence_stdc` family, introduced in the DOC0258 work and subsequently revised, provides the shared flat representation. Its current declaration is in `sequences.cppm`, lines 2076–2255. Listing 10.2 shows the representation-level API.

Listing 10.2: Abridged standard-layout sequence API. Methods are declarations; implementations and factories are omitted.

```
struct compact_sequence_stdc_type_params {
    uint8_t alphabet_code;
    uint8_t storage_unit_byte_width;
    static constexpr int32_t encode(...);
    static constexpr compact_sequence_stdc_type_params decode(int32_t);
};

template<class SupportedAlphabets,
         class BpSize = uint32_t,
         class VarlenaHeader = int32_t,
         class SupportedWidths = all_storage_unit_byte_widths>
struct compact_sequence_stdc {
    VarlenaHeader vl_len_;
    typename SupportedAlphabets::alphabet_code_t alphabet_code;
    uint8_t storage_unit_byte_width;
    uint16_t padding_letters;
    BpSize bp_size;
    [[no_unique_address]] alignment_reserved_holder<...> _reserved;
    uint8_t data[1];

    static constexpr size_t header_bytes();
    template<class Handler>
    static constexpr void dispatch_instance(
        alphabet_code_t, uint8_t byte_width, Handler&&);
    template<class Handler> void convert_to(Handler&&) const;
    size_t unit_count() const noexcept;
    size_t payload_bytes() const noexcept;
};

template<class Stdc, sequence_alphabet Alphabet, size_t WidthBytes>
struct compact_sequence_stdc_instance {
    using compact_sequence_stdc_type = Stdc;
    using alphabet_type = Alphabet;
    using storage_unit = /* uint8_t, uint16_t, uint32_t, or uint64_t */;
    static constexpr size_t byte_width = WidthBytes;
    static constexpr size_t letter_bit_width =
        seqan3::detail::ceil_log2(seqan3::alphabet_size<Alphabet>);
};

template<class Instance>
requires is_compact_sequence_stdc_instance<Instance>
struct compact_sequence_range<Instance> {
    using storage_unit = typename Instance::storage_unit;
    explicit compact_sequence_range(
        typename Instance::compact_sequence_stdc_type const&) noexcept;
    size_t padding() const noexcept;
    size_t size() const noexcept;       // physical units
```

```
    size_t bp_size() const noexcept;    // logical symbols
    storage_unit const* cbegin() const noexcept;
};
```

The header records run-time type parameters, logical length, and last-unit padding; `data[]` contains the packed payload. The reserved field makes the payload offset a multiple of eight, allowing every enabled 1-, 2-, 4-, or 8-byte instance to form naturally aligned typed loads. The PostgreSQL specialization uses a 32-bit base-pair count and a 32-bit varlena header. On the target little-endian PostgreSQL ABI the stored header uses the four-byte uncompressed varlena encoding, rather than a raw byte count. A raw count would make PostgreSQL interpret only one quarter of the object and truncate detoasted copies.

`compact_sequence_stdc_instance` carries no run-time state. It pins the two header bytes—alphabet code and physical width—as C++ template parameters. Once dispatched, the `compact_sequence_range<Instance>` specialization holds a const reference to the datum and points directly at `data[]`; it does not create a vector. Construction validates that header width, logical length, unit count, and padding agree. This is the main zero-copy boundary between a flat PostgreSQL value and the resident algorithm interface.

10.4.2. Contrast with minimap2's split representation

The copied minimap2 reference makes a different, internally consistent trade-off. Its input record `mm_bseq1_t` stores an integer length and `char*` fields for name, sequence, quality, and comment (`main/bench/tool_minimap2/minimap2/bseq.h`, lines 14–17). Mapping entry points such as `mm_map` receive the query as `const char*`. The reference index uses another representation: `mm_idx_t::S` is a `uint32_t*` containing a four-bit packed reference (`minimap.h`, lines 80–100). Reverse complementation of an input record mutates the byte string in place.

This split is well suited to a command-line mapper which reads batches from FASTA/-FASTQ, constructs its own index, and releases batch-owned memory after mapping. It is not an SQL value model. A PostgreSQL slice cannot safely be represented as a borrowed `char*`; a concat cannot mutate its operands; and an expression may outlive the scan node which discovered the source fragment.

The BioSeqDB benchmark adapter preserves the reference contract rather than pretending minimap2 accepts the lazy facade. The helper `compact_sequence_to_acgt` in `main/bench/tool_minimap2/` lines 58–87, performs one $O(N)$ rank-to-character pass. The reference wrapper then calls `mm_idx_str` and `mm_map` with those strings (lines 218–310). BioSeqDB-native routes avoid that boundary when their kernels support compact or lazy ranks. This is a systems difference rather than a change to minimap2's seed–chain–extend mathematics.

10.4.3. Runtime metadata becomes compile-time specialization

Alphabet registration is itself type-level. An `alphabet_entry<A,Name>` pairs an alphabet with a compile-time string. `supported_alphabets<...>` verifies unique types and names, assigns a small integer code, and supplies code-to-name, name-to-code, and dispatch operations. A typical dispatch has the following form:

Listing 10.3: Runtime header dispatch into a compile-time alphabet/width specialization.

```
#if BIOSEQ_OPTTIMIZATION_LEVEL == 0
using pgseq_supported_alphabets =
    supported_alphabets<
        alphabet_entry<seq::dna4, "dna4">>;
```

Table 10.3: Representation boundary in the minimap2 reference and BioSeqDB.

Aspect	minimap2 reference	BioSeqDB
mapping query	ASCII-compatible `char*` and integer length	alphabet rank stream; bytes are produced only for a byte-oriented boundary
reference index	separate four-bit packed `uint32_t` array	packed geometry selected by alphabet/width policy and exposed through typed ranges or lazy access
reverse operation	may mutate a batch sequence buffer	logical edge transform; operands remain immutable
slice/concat lifetime	caller/batch responsibility	expanded object, arena dependencies, retained source handle, and declared flattening boundary
alphabet dispatch	nucleotide lookup tables and reference-specific encoding	compile-time alphabet pack selected once from run-time datum metadata

```
#else
using pgseq_supported_alphabets =
    supported_alphabets<
        alphabet_entry<seq::dna4, "dna4">,
        alphabet_entry<seq::dna5, "dna5">>;
#endif

seq::dispatch_instance(runtime_alphabet_code, runtime_width,
    [&]<class Instance>() {
        using range_type = compact_sequence_range<Instance>;
        using alphabet_type = typename Instance::alphabet_type;
        range_type packed{datum};
        consume<alphabet_type>(packed);
    });
```

The exact PostgreSQL profile is in `main/bioseq/backend/pg/types/compact_sequence/flat_abi.cppm`. At optimization profile 0, `pgseq_supported_alphabets` contains only DNA4; higher profiles add DNA5. Profiles 0 and 1 enable only an eight-byte physical unit, while profile 2 enables $\{1,2,4,8\}$. SQL width requests are validated and normalized through this policy. Consequently the symbol named in some older design notes as `pg_supported_sequences<>` is not the implemented abstraction. The current aliases are `pgseq_supported_alphabets` and `pgseq_supported_storage_unit_byte_widths`.

Dispatch is paid at the boundary, not in the rank loop. After the nested alphabet/width selection, masks, letters per unit, complement operations, and lane types are compile-time constants. This is how one stored SQL base type can remain alphabet-generic without placing a switch statement inside every symbol operation.

10.4.4. Dispatch is not SQL code generation

A second compile-time-looking mechanism has a different purpose. The PostgreSQL type alias is annotated in `flat_abi.cppm`:

Listing 10.4: The C++ declaration that owns the generated SQL type metadata.

```
using seq [[bioseq_sql::type({
    .schema = bioseq::sql::fixed_schema<"bioseq">,
    .internal_length = bioseq::sql::variable_length,
    .alignment = bioseq::sql::type_alignment::double_,
    .storage = bioseq::sql::type_storage::extended,
})]] = compact_sequence_stdc<
    pgseq_supported_alphabets,
    uint32_t,
    int32_t,
    pgseq_supported_storage_unit_byte_widths>;
```

The `bioseq_sql` attributes are consumed by `main/codegen/sql`: the annotation model describes entities and options, the collector reads annotated declarations, the graph layer resolves dependencies and fmgr facets, and renderers produce installation SQL, C/fmgr boundary code, and manifests. Function, result, parameter, operator, and subscript attributes in `concat_reverse.cppm`, `subscript.cppm`, `letter.cppm`, and related partitions keep the SQL signature next to its C++ authority.

This generation path does not select a nucleotide alphabet at query run time. Conversely, `supported_alphabets` does not generate SQL. The former removes duplicated boundary declarations at build time; the latter specializes data processing at run time. Keeping the two mechanisms separate is important both for understanding the code and for assigning failures to the correct layer.

10.5. From fragmented storage to one lazy facade

10.5.1. Why a packed pointer is not a sufficient interface

The flat adapter solves only the case in which the entire payload is already resident in one datum. Columnar storage and external providers return bounded pieces. A range may cross fragment boundaries, and a reverse-strand cursor may request those pieces in the opposite logical order. Requiring a single pointer would force all fragments to be joined before a consumer could inspect the first rank.

The first BioSeqDB lazy design, still visible in `main/bioseq/backend/storage/lazy_compact_sequence` was parameterized by a concrete range and scanner. Its no-fragment specialization was close to a pointer-sized wrapper. The fragmented specialization held a small fragment cache and stitched a requested window into scratch storage when it crossed a boundary. This was useful because existing minimizer iterators could ask for a small contiguous run without knowing how fragments were loaded. It was nevertheless representation-specific: an algorithm's type mentioned the scanner, the underlying range, and the fragment machinery.

The final facade moves the representation choice behind an operation table. It is specialized by logical alphabet, but not by PostgreSQL width, source provider, tree shape, or scanner class. The interface emits rank bytes for sequential consumers and can optionally describe an already-resident packed region. Listing 10.5 shows the essential API from `lazy_compact_sequence.cppm`, lines 169–568.

Listing 10.5: Abridged declaration-only form of the canonical lazy facade.

```
template<sequence_alphabet Alphabet>
struct lazy_rank_stream_traits {
    using alphabet_type = Alphabet;
    using storage_unit = uint8_t; // one emitted rank, not packed storage
    static constexpr size_t logical_bits_per_symbol =
        seqan3::detail::ceil_log2(
```

```
            seqan3::alphabet_size<Alphabet>);
};

template<sequence_alphabet Alphabet>
class basic_lazy_sequence_view {
public:
    using ptr_at_fn = uint8_t const* (*)(
        void const* state, sequence_size_t from,
        size_t count, span<uint8_t> scratch);

    enum class packed_bit_order : uint8_t {
        least_significant_symbol_first
    };

    struct resident_packed_region {
        void const* units;
        size_t unit_count;
        sequence_size_t bp_length;
        uint8_t storage_unit_byte_width;
        uint8_t bits_per_symbol;
        uint16_t leading_symbol_offset;
        uint16_t trailing_padding_letters;
        packed_bit_order bit_order;
    };

    struct backend_ops {
        sequence_size_t (*bp_size)(void const*) noexcept;
        array<ptr_at_fn, 4> ptr_at;
        optional<resident_packed_region> (*try_resident_packed)(
            void const*, sequence_size_t, sequence_size_t,
            uint8_t) noexcept;
        bool (*copy_full_packed)(
            void const*, void*, size_t, uint8_t);
        bool (*worker_shareable)(void const*) noexcept;
    };

    basic_lazy_sequence_view(
        void const* state, backend_ops const* ops,
        sequence_payload_allocation_observer const* = nullptr) noexcept;

    sequence_size_t bp_size() const noexcept;
    bool worker_shareable() const noexcept;
    optional<resident_packed_region> try_resident_packed(
        sequence_size_t from, sequence_size_t to,
        uint8_t requested_width) const noexcept;

    template<bool IsReversed, bool WriteReversed = false>
    void to_encoded_into(
        span<uint8_t> output, size_t from, size_t to) const;
};
```

The `uint8_t` called `storage_unit` in `lazy_rank_stream_traits` is a rank-stream element, not the physical packed unit of the backing object. DNA4 still contains two logical bits per symbol. A facade callback writes one rank per scratch byte only for the requested window. This distinction avoids implying that the lazy interface expands the whole value to bytes.

The state pointer and static operation table form a small representation-erased object. Backends provide four `ptr_at` callbacks for the Cartesian product of logical reverse orientation and destination write order. A callback may return a pointer into backing storage when

its representation permits it, or return the supplied scratch buffer after filling the request. The PostgreSQL facade currently emits ranks into scratch for both flat and tree traversal; its separate resident probe is what permits a zero-copy packed borrow.

10.5.2. Ownership is part of the abstraction

A borrowed view without a lifetime is an unsafe pointer with a better name. `lazy_sequence_call_owner<Al` is therefore a move-only RAII object containing the facade and the release action for its state. PostgreSQL constructs these owners in a read-session memory context. The helper `with_pg_sequence_view<Result>` in `view_session.cppm`, lines 995–1064, enforces at compile time that a callback result is not a pointer, reference, range view, or iterator and is trivially destructible. The view cannot accidentally escape after a detoast copy, source domain, or memory context has been released.

This restriction also addresses PostgreSQL's non-local error model. Source reads may raise a PostgreSQL error while C++ owners are live. The read-session boundary converts the error long enough to unwind registered owners, then rethrows the original PostgreSQL error. An ordinary destructor alone would be insufficient because PostgreSQL `ERROR` performs a `longjmp` across C++ frames.

Worker sharing is explicit as well. A flat resident value is shareable. A tree is shareable only if every source leaf's provider says that its state is shareable. Current source-backed PostgreSQL views remain on the backend thread. A CPU task or Vulkan operation which outlives that call first obtains an admitted resident owner; both the allocation and its memory permit remain alive until task completion or the GPU fence.

10.6. Persistent SQL expressions as immutable piece trees

10.6.1. Leaves, branches, and transforms

PostgreSQL needs lazy values to survive ordinary expression evaluation, concatenation, and nested slices. A raw tuple pointer cannot provide that lifetime: its memory can be detoasted, copied, or reset, and an arbitrary slice may begin in the middle of a packed word. The final representation is a PostgreSQL expanded object whose payload is an immutable balanced piece tree. The public POD shapes are declared in `main/bioseq/backend/pg/types/compact_sequence/tr` lines 31–222, and are abridged in Listing 10.6.

Listing 10.6: Persistent sequence-piece API. Transforms are edge metadata; there is no payload-bearing reverse node.

```
struct piece_edge {
    piece_tree_arena* arena;
    piece_node const* node;
    sequence_transform transform;
};

struct source_piece {
    pg_sequence_source_handle source;
    seq_locator locator;
    compact_sequence_stdc_type_params stored_type_params;
    uint16_t last_padding_letters;
    uint64_t from;
    uint64_t to;
};
```

```
struct literal_piece {
    piece_tree_arena* storage_arena;
    byte const* bytes;
    uint64_t symbol_from;
    uint64_t symbol_to;
    compact_sequence_stdc_type_params stored_type_params;
    literal_encoding encoding;
    literal_validation validation;
};

struct branch_piece { piece_edge left; piece_edge right; };

struct piece_node {
    enum class kind : uint8_t { branch, source, literal } node_kind;
    uint64_t bp_length;
    uint32_t piece_count;
    uint16_t height;
    union {
        branch_piece branch;
        source_piece source;
        literal_piece literal;
    } payload;
};

struct pg_sequence_expanded {
    ExpandedObjectHeader hdr;
    MemoryContext object_context;
    MemoryContextCallback reset_callback;
    compact_sequence_stdc_type_params type_params;
    bool alphabet_bound;
    uint64_t bp_length;
    piece_edge root;
    slice_trace_attachment* trace;
    uint32_t producer_value_id;
};
```

A source leaf stores a retained provider handle and stable logical coordinates, not an address into a tuple. A literal leaf stores owned packed or unbound bytes and its validated type. Concatenation creates a branch. Each node caches logical length, piece count, and AVL height. Existing subtrees are immutable and retained through arena dependencies, so a new expression shares them rather than copying their symbols.

Reverse and complement state is attached to an edge. Composing a transform changes metadata and preserves the subtree. For a branch reached through a reversed edge, logical child order is reversed during traversal. For a literal or source leaf, coordinate mirroring, output order, and complementation are applied at the facade callback. This design makes a reverse or reverse-complement operation $O(1)$ in payload size.

The current node kinds deserve emphasis. Design documents sometimes use “slice piece” and “transform piece” as conceptual terms, but the source does not store separate payload nodes with those enum values. A slice narrows leaf ranges by splitting the tree and shares unaffected nodes; a transform is carried by `piece_edge`. Describing the conceptual expression DAG is useful in SQL `EXPLAIN`, but the physical piece-tree enum is {`branch`, `source`, `literal`}.

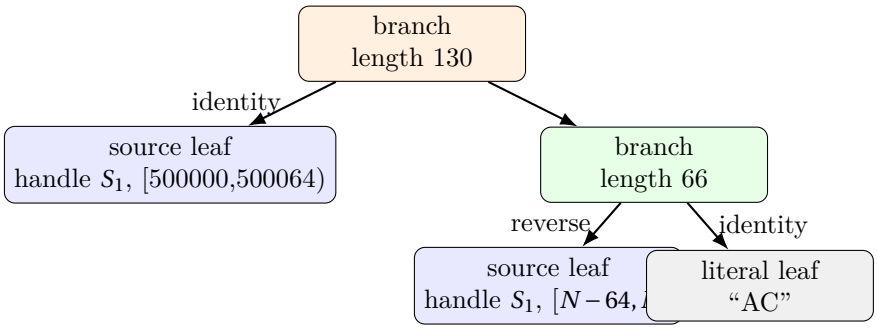


Figure 10.3: An illustrative persistent composition. Slices narrow logical coordinates, concatenation adds balanced branch metadata, and reversal is an edge transform. The source payload is not copied into the tree.

10.6.2. Composition and complexity

`concat_pg_sequences` in `packed_output.cppm`, lines 3306–3361, acquires one writable expanded value, resolves bound/unbound type adoption, checks the summed logical length, and joins the two immutable roots. The join in `tree_ops.cppm`, lines 1665–1765, descends according to subtree height and performs an AVL single or double rotation when needed. Work for already validated roots is bounded by the height-guided join path, not by the total number of pieces. Adjacent compatible leaves may be coalesced.

`slice_tree` at lines 1887–1911 splits first at the upper and then at the lower coordinate, releases the two discarded sides, and returns the middle. In a balanced tree, each split follows a boundary path; payload outside those paths is shared. The cost depends on tree height and boundary leaf construction, not on the number of symbols in the selected interval. Reading the selected symbols later remains $O(m)$ for an interval of length m, plus provider/fragment overhead.

The arenas make persistent sharing compatible with PostgreSQL memory contexts. They reference older arenas and source dependencies, keep generation ordering, and release dependencies iteratively. Construction uses a fixed cleanup ledger because a PostgreSQL error may bypass C++ stack destructors. This bookkeeping is not merely defensive infrastructure: it is what allows an expanded sequence returned by one SQL operator to remain valid for the next operator without copying the original payload.

10.6.3. Bound, unbound, and letter values

The expanded header carries both physical type parameters and an `alphabet_bound` flag. A bound value has an alphabet and packing width. An unbound SQL literal contains canonical ASCII but has not yet adopted an alphabet. Alphabet-independent operations—length, slicing, reversal, comparison, and unbound concatenation—remain possible. Combining one unbound and one bound value validates the unbound symbols and adopts the bound alphabet. Two different bound alphabets are never silently recoded.

Element access returns `bioseq.letter`, a fixed two-byte value containing $(alphabet_code, rank)$. This is more robust than returning a character: the alphabet identity survives arrays, prepared statements, and places where PostgreSQL does not retain expression typmods. An in-range element of an unbound sequence is an error because no physical alphabet is known; an out-of-range element is still SQL `NULL`.

Equality and hashing follow logical content rather than storage. For unbound values the key is a state tag and canonical ASCII bytes. For bound values it is a different state tag, unsigned alphabet code, and the logical rank stream. Storage width, padding, flat versus expanded form, source backing, and piece boundaries are excluded. Hence a flat value and a many-piece lazy value compare equal and hash identically when their logical state, alphabet, and ranks agree. A bound value is intentionally not equal to an unbound value with the same printed text.

10.7. SQL slicing and logical transforms

The production subscript handler is implemented with PostgreSQL 17 `SubscriptRoutines` in `subscript.cppm`. Its semantics are Python-like: indices are zero-based, slices are half-open, negative values are relative to the logical length, omitted bounds select the corresponding end, and bounds are clamped. If the normalized upper bound precedes the lower bound, it is set equal to the lower bound and the result is a typed empty sequence. A missing element produces `NULL`. A SQL `NULL` bound propagates SQL `NULL`. Assignment, steps, and multiple dimensions are rejected.

Listing 10.7: Current SQL coordinate and composition semantics.

```
WITH v(s) AS (
  VALUES ('ACGTA'::bioseq.seq('dna4', 32))
)
SELECT s[0]::text,         -- A
       s[-1]::text,        -- A
       s[:2]::text,        -- AC
       s[1:-1]::text,      -- CGT
       bioseq.len(s[4:2]);-- 0

SELECT ('AC'::bioseq.seq('dna4',32) || 'TG')::text;
SELECT bioseq.reverse('AAGC'::bioseq.seq('dna4',32))::text;
SELECT bioseq.reverse_complement(
         'AAGC'::bioseq.seq('dna4',32))::text;
```

Concatenation is symmetric for sequences, `text`, `varchar`, one-dimensional `text[]` and `bioseq.letter[]` operands. Empty arrays act as identities; null elements and multidimensional arrays fail explicitly. The overloads are strict, so a null operand yields null without inspecting the other operand. `bioseq.reverse` is valid for bound and unbound values because it only changes order. Reverse-complement requires a bound alphabet and a complete complement mapping.

Same-type typmod casts are logical operations. An identical alphabet/width is metadata-only; a same-alphabet width change records the target geometry and delays repacking until a flat consumer requests it. Binding an unbound literal validates every symbol. Relabeling a bound value as another alphabet fails. These rules apply equally to flat, TOASTed, expanded, and source-backed data.

The coordinate convention is a concrete example of why date precedence matters. DOC0313 proved that PostgreSQL generic subscripting could be attached to the custom type, but its proof of concept used PostgreSQL array-style one-based inclusive bounds and eagerly repacked each slice. The later DOC0387 reference and the current `subscript.cppm` supersede those choices with zero-based half-open logical views. The older report's safety finding remains correct: a user-visible value must not hold a raw pointer into another varlena. The final tree and source-owner design satisfies that constraint without forcing eager payload materialization.

10.8. Access planning and materialization boundaries

10.8.1. Consumers state what they require

The abstraction cannot make every algorithm streaming. A scalar rank scan needs only sequential symbols. A block-oriented kernel may need bounded random windows. A GPU upload or a SIMD routine which forms typed unit pointers may require one contiguous packed region. BioSeqDB makes this difference explicit through `sequence_access_requirement` in `sequence_scanner.cppm`, lines 217–221, and the planner in `lazy_compact_sequence.cppm`, lines 2148–2405.

Listing 10.8: Declaration-only form of lazy access requirements and plans.

```
enum class sequence_access_requirement : uint8_t {
    sequential_symbols,
    bounded_random_window,
    contiguous_resident
};

enum class sequence_access_route : uint8_t {
    stream_symbols,
    borrow_resident,
    materialize_admitted
};

enum class sequence_resident_borrow_kind : uint8_t {
    none,
    native_typed_storage,
    reinterpreted_bytes
};

template<sequence_alphabet Alphabet>
struct sequence_access_plan {
    sequence_access_requirement requirement;
    sequence_access_route route;
    optional<resident_packed_region> resident;
    sequence_resident_borrow_kind resident_borrow_kind;
    uint64_t admission_bytes;
    sequence_size_t from;
    sequence_size_t to;
    uint8_t requested_width;
};

template<sequence_alphabet Alphabet>
sequence_access_plan<Alphabet> plan_sequence_access(
    basic_lazy_sequence_view<Alphabet> view,
    sequence_access_requirement requirement,
    sequence_size_t from, sequence_size_t to,
    uint8_t requested_width = 0,
    sequence_resident_borrow_policy =
        sequence_resident_borrow_policy::native_typed_storage_only);
```

For `sequential_symbols`, the current planner immediately selects `stream_symbols`. A caller uses the facade's chunk iterator or `to_encoded_into` and supplies bounded scratch space. For `contiguous_resident`, the planner first probes the representation at the requested physical width. Borrowing is accepted only when the descriptor is internally consistent and its `leading_symbol_offset` is zero. A nonzero offset means that the requested interval begins inside a packed unit; the ordinary `compact_sequence_range` interface has no field for that

offset and typed consumers would read the wrong first rank. The current generic planner sends `bounded_random_window` to admitted materialization as well; a consumer which can formulate its computation as forward bounded chunks should request the sequential route instead.

If no valid borrow exists, the plan reports the exact admission charge for a packed owner over $[from, to)$. The caller acquires a `MemoryPermit`, and `materialize_packed_region` packs only that interval at the requested width. The returned owner contains the stable allocation, descriptor, and permit. Moving ownership into an asynchronous task or Vulkan submission also moves the permit, preventing accounting from ending before the last load or completion fence.

Table 10.4: The three current access routes.

Route	Typical producer/consumer	Copy and lifetime rule
`stream_symbols`	fragment/source tree to a forward minimizer or conversion loop	decodes bounded rank chunks into caller scratch; no whole-sequence packed allocation
`borrow_resident`	aligned flat payload to a typed CPU kernel, or strict byte-compatible GPU upload	no payload copy; owner of the flat datum must remain live; typed and byte-only receipts are distinct
`materialize_admitted`	unaligned flat range, source/tree range, or resident-only CPU/SIMD/GPU kernel	packs exactly the selected interval; allocation and memory permit live through the consumer/fence

10.8.2. What "zero copy" does and does not mean

BioSeqDB has several useful zero-copy cases, but no universal zero-copy promise:

- constructing `compact_sequence_range<Instance>` over an already resident, uncompressed flat datum is reference-only;
- constructing a slice, concat, or transform can be payload-free because it creates or shares metadata;
- a flat aligned interval at its native physical width can be borrowed; and
- a fragmented or source-backed interval can be streamed without forming a complete packed result, although requested chunks are decoded into scratch.

There are corresponding boundaries where copying is required. PostgreSQL may first detoast a compressed or external flat value; the zero-copy C++ adapter does not bypass PostgreSQL's varlena ownership rules. A typed resident kernel cannot borrow an interval with an in-unit start. Text, binary send, and `bioseq.letter[]` output require materialized destination forms. A heap tuple or index tuple requires a durable representation. A provider view is transaction-scoped, so it must be flattened before it crosses a durable or cross-transaction boundary. Finally, a logical expanded value may be larger than the flat

varlena representation can encode; metadata operations can continue to work, but a later flat boundary raises SQLSTATE `54000` rather than truncating it.

Figure 10.4 follows one expression through these decisions. The important observation is that SQL evaluation and payload materialization are separate events.

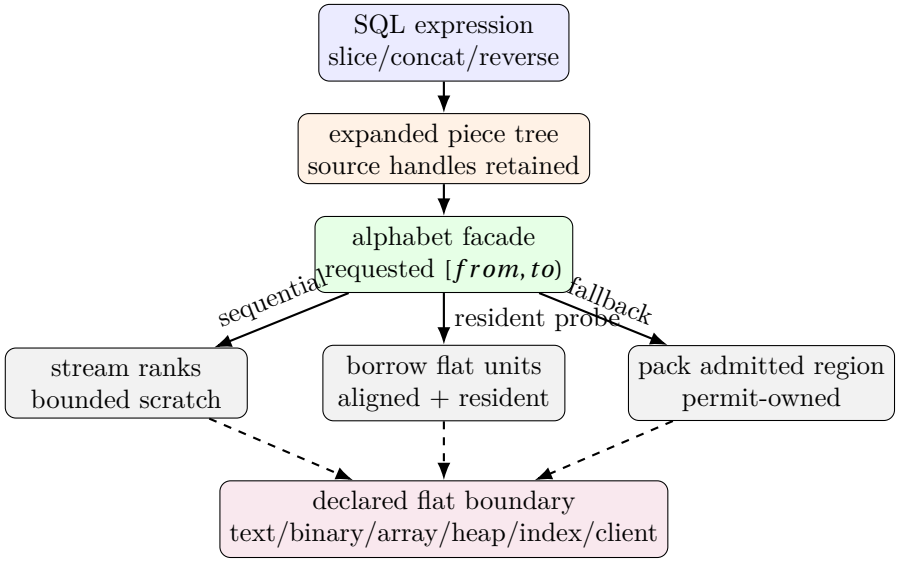


Figure 10.4: Lifecycle of a lazy SQL value. The lower boundary is reached only when the selected consumer requires a durable or alternate representation; an algorithm may finish on any of the three access routes without reaching it.

10.8.3. Bounded source reads

The PostgreSQL adapter state in `view_session.cppm`, lines 430–738, contains a read session, a logical descriptor, and either a flat pointer or a tree edge. Its tree-range emitter descends only through branches intersecting the requested coordinates. Literal ranks are extracted from the relevant packed range; a source leaf invokes the provider's bounded `read_ranks` operation. Reversal maps the logical interval to base coordinates before traversal and only then changes destination order and complements ranks. A small terminal slice therefore does not require a prefix scan of the source.

The resident probe is intentionally stricter. It succeeds only for a flat value at the requested native width. Tree and source states return no resident descriptor, even if a provider internally has a cache page, because the generic facade does not own a stable typed allocation for that page. This honesty prevents an optimization from extending a cache pin or PostgreSQL memory-context pointer beyond its contract.

10.9. Evolution of the design

The sequence layer did not emerge as a single rewrite. Table 10.5 orders the principal stages by date. The sequence is important because several documents marked "active" describe approaches superseded by later implementation.

Three engineering lessons follow from this history.

First, the safety problem in DOC0313 was real but did not imply that all slices must be eager. A flat varlena has neither an arbitrary bit-start field nor an owner pointer, so merely

Table 10.5: Evolution from eager resident data to the final lazy SQL value.

Date/evidence	Design stage	Outcome
07-05, DOC0215	owning packed inputs plus reference-tool ASCII conversion	exposed the cost and benchmark bias of decoding a 500 Mbp target at the wrong boundary
22-05, DOC0258	standard C-compatible packed object	introduced `compact_sequence_stdc` and an in-place typed range rather than duplicating PostgreSQL and standalone layouts
24-05, DOC0261	type-level alphabet names and packs	replaced hand-written string ladders with one runtime-to-template dispatch authority
25-05, DOC0263	scanner-parameterized lazy fragmented sequence	added fragment-aware cursors, direct reads within one fragment, and scratch stitching across fragments
12-06, DOC0313	native-subscripting feasibility report	proved custom subscripts, correctly rejected persistent raw pointers, but recommended eager materialization and one-based semantics
14-07, DOC0376	lazy slicing goal	required nested half-open slices, concatenation, reversal, letters, and explainable materialization
19–23-07, DOC0387 and DOC0388 report	final expanded tree and facade	delivered zero-based logical SQL views, source ownership, one alphabet-generic facade, admitted resident routes, tracing, tests, and bounded benchmarks

incrementing `data[]` is unsound. The final solution changed the persistent representation instead of weakening the invariant: it stores stable source coordinates or arena-owned literal ranges and retains the source domain.

Second, the older fragmented iterator and the final facade solve different levels of the problem. Fragment stitching is a useful backend mechanism. It should not be the type named by every algorithm. Representation erasure allows the same logical cursor to traverse a flat value, a piece tree, a provider, or an admitted packed owner.

Third, materialization cannot be hidden in a convenience conversion. Once the consumer declares its requirement, the system can estimate bytes, obtain a permit, report the reason in `EXPLAIN`, and keep ownership through asynchronous execution. Without that declaration, a seemingly harmless `to_vector()` can expand an entire target in the scheduler's timing window or on every row of a query.

10.10. Measured consequences

10.10.1. An early boundary-placement failure

DOC0215 investigated a benchmark containing one 5 Mbp fragment and a 500 Mbp reference. The DNA4 target occupied approximately 125 MB in packed form, but the minimap2 reference wrapper converted it to a 500 MB ASCII string inside the timed call. The miti-

gation moved conversion into input preparation and serialized benchmark cells sharing the huge cached input. The resulting one-iteration figures are reproduced in Table 10.6.

Table 10.6: Historical XXL_HIFI wall times from DOC0215. Conversion placement and test-case concurrency changed together, so deltas are diagnostic rather than isolated codec speedups.

Route	old	mitigated	change
BioSeqDB Block Aligner SIMD	62.27 s	7.69 s	−88%
BioSeqDB KSW2 scalar	9.89 s	9.13 s	−8%
BioSeqDB KSW2 SIMD	62.40 s	8.49 s	−86%
minimap2 reference	36.99 s	11.76 s	−68%

The experiment does not show that the compact abstraction alone made the algorithms eight times faster. It shows two measurement failures that a database implementation must avoid: unrelated cells were contending for the same memory bandwidth, and only the reference route paid a full packed-to-text conversion inside its measured region. After the mitigation all routes reported the same correct mapping, but the single repeat remains too small for a general performance claim. Its role in this chapter is historical: it motivated explicit conversion boundaries and comparable input ownership.

10.10.2. Small slices perform bounded work

The final DOC0388 implementation report gives a more direct PostgreSQL test. One row contained 8,388,608 DNA4 symbols. Seven alternating measurements converted either a 64-symbol middle slice or the whole source to text. The database had JIT, column cache, and vectorization disabled for this focused comparison. Results appear in Table 10.7.

Table 10.7: PostgreSQL source-slice timing reported by DOC0388.

Expression	symbols	minimum	median	maximum
64-symbol slice	64	404 μs	613 μs	778 μs
whole source	8,388,608	38,177 μs	38,964 μs	40,590 μs

The median ratio was 0.0157. Timing alone could still be explained by caching or a special text path, so the materialization trace is the stronger evidence: the slice expression reported one SQL conversion with 64 input symbols, 64 output symbols, and 68 bytes, and did not report a whole-source packed boundary. This establishes bounded source access for that expression. It does not imply that every arbitrary provider has constant latency or that a resident-only alignment kernel can process a lazy tree without packing its selected region.

10.10.3. Deferred composition versus eager intermediates

The same closure report records a focused lazy-I/O benchmark with twelve schedule-matched repeats and seven blocking paired rules. The deferred route and an eager comparison produced identical 6,291,456-symbol output and exact matching hashes. Table 10.8 shows the allocation and time differences.

The main result is not merely a lower median. The counters match the intended lifecycle: five eager intermediate payloads disappear, while one final payload boundary remains because the output itself must exist. This is the operational definition of successful lazy composition in BioSeqDB.

Table 10.8: Focused deferred/eager lazy-I/O evidence from `bioseq_io_bench/report_7.json`.

Metric	deferred	eager	deferred/eager
wall-time median	22,503,667 ns	69,166,708 ns	0.32435
payload materializations	1	6	0.16667
intermediate materializations	0	5	0
payload bytes	1,572,864	6,815,744	0.23077
admitted bytes	1,572,952	6,816,272	0.23076

10.11. Practical contract for algorithm authors

The final representation is broad, but its safe use can be summarized in a small set of rules:

1. Write alphabet-generic code against `compact_sequence_range` traits or the behavioral lazy facade. Deduce masks, ranks, and alphabet cardinality from the selected type. Do not infer a hash-state width from the physical storage unit.

2. Request the weakest access requirement that implements the algorithm. A forward scan should remain a scan. A resident-only routine must request a region and accept the admission/lifetime owner.

3. Convert only the selected $[from, to)$ region at the point where a byte-oriented external API actually requires it. Hoist invariant conversions out of repeated or timed calls.

4. Treat a resident descriptor as a capability with a precise type. A byte-upload receipt is not permission to form a differently typed CPU pointer, and an unaligned first symbol is not a flat compact range.

5. Never persist a pointer into a tuple, TOAST value, cache page, or provider buffer. Persistent laziness uses retained handles, logical coordinates, immutable arenas, and declared flattening boundaries.

6. Keep SQL generation and alphabet dispatch conceptually separate. C++ attributes generate boundary artifacts; the supported-alphabet pack specializes execution after a datum is read.

These rules explain how the final solution differs from the original resident-vector model. The original model made the algorithm's preferred layout the mandatory representation of every input. The final model makes logical ranks and ownership the mandatory contract, while packed contiguity is a requested capability. This inversion is what permits an alignment pipeline to consume database-resident slices without pretending that TOAST, fragments, source providers, GPU lifetimes, and SQL persistence are all the same kind of memory.

Chapter 11

PostgreSQL API and custom scans

11.1. Alignment as a PostgreSQL execution primitive

The alignment pipeline described in the preceding chapters is not naturally a scalar SQL function. Its useful unit of work is a batch of sequence pairs, its result for one pair is a set of matches, and its implementation may need to retain an index, submit work to a scheduler, or materialize a query collection before it emits the first row. PostgreSQL, in contrast, initially sees an SQL predicate as an expression that is evaluated once for each input tuple and returns one value. The main purpose of the BioSeqDB PostgreSQL API is to reconcile these two execution models.

The public expression is deliberately ordinary:

```
SELECT t.pk,
       bioseq.score(t.pk),
       bioseq.target_start(t.pk),
       bioseq.target_end(t.pk),
       bioseq.cigar(t.pk)
FROM target_sequences AS t
WHERE mmsearch(
        t.sequence,
        'ACGTACGT...'::bioseq.seq('dna4', 32));
```

The parser can resolve the types of this statement without learning a new SQL grammar. It produces a normal `FuncExpr` for `mmsearch` and normal function expressions for the projected match fields. Nevertheless, the executor must not call those expressions in the ordinary way. `mmsearch` is a rendezvous expression: it marks the semantic location of sequence search so that a custom execution node can claim it. The projected helpers name fields of the match currently being emitted by that owner. They do not each run another alignment.

This distinction is visible in cardinality. Let input row i produce m_i accepted alignments. The custom scan emits

$$N_{\text{out}} = \sum_{i=1}^{N_{\text{in}}} m_i, \qquad m_i \in \{0, 1, 2, \ldots\}.$$

Thus a source row with no match vanishes, one with one match behaves like an ordinary filter result, and one with several matches is repeated with different match-field values. A Boolean function can say whether $m_i > 0$, but it cannot supply the m_i rows or ensure that score, coordinates, CIGAR, strand, and mapping quality all refer to the same member of that set.

Table 11.1: The semantic mismatch between an ordinary predicate and a BioSeqDB-owned search.

Property	Ordinary scalar expression	BioSeqDB executor owner
Input granularity	One expression evaluation for one tuple	Several retained tuples or sequence pairs per dispatch
Output cardinality	Exactly one Boolean or nullable Boolean	Zero, one, or many projected rows per input
Expensive work	Repeated at every function call site	Performed once by the alignment owner; fields are projected afterwards
Accelerator use	Submission overhead occurs at tuple granularity	The scheduler can choose SIMD, CPU-task, or accelerator routes for a batch
Result consistency	Separate helpers have no shared match identity	All placeholders are populated from one `MatchResult` before projection
Planner safety	A surviving function copy may execute anywhere it is placed	The exact restriction has one owner and no executable descendant copy

The final design therefore treats SQL syntax, planner ownership, and executor cardinality as one contract. A supported `mmsearch` must be consumed by exactly one BioSeqDB node. A query shape that cannot satisfy this contract fails closed. It is not permitted to fall through to a permissive Boolean implementation, because doing so would turn an unsupported alignment query into a query that silently accepts every row.

11.2. The SQL surface is a rendezvous, not a hidden mode switch

11.2.1. Two typed entry points

The current extension exposes a two-argument search with session defaults and a three-argument form with a typed options value. In schematic form their signatures are

$$\texttt{mmsearch} : (\texttt{bioseq.seq}, \texttt{bioseq.seq}) \to \texttt{boolean},$$
$$\texttt{mmsearch} : (\texttt{bioseq.seq}, \texttt{bioseq.seq}, \texttt{bioseq_aligner_options}) \to \texttt{boolean}.$$

Both are declared immutable and parallel safe at the SQL expression boundary and both name `mmsearch_support` as their PostgreSQL support function. These declarations allow PostgreSQL to reason about typing and expression placement. They are not a promise that the scalar C++ body implements the alignment. The body raises `ERRCODE_FEATURE_NOT_SUPPORTED` with the message "mmsearch qualifier has no Bioseq executor owner". The three-argument entry point preserves SQL null behavior for null operands before reaching that error boundary.

This fail-closed body is important for two reasons. First, PostgreSQL can copy or move expression trees during planning. If an unexpected copy survives as a filter, it produces a visible error rather than false-positive biological results. Second, behavior does not depend on a thread-local flag set by the scan. A hidden flag would make an immutable function return different values for the same arguments, would interact badly with constant folding, and could remain stale after PostgreSQL's non-local error unwinding. The owner is instead represented explicitly in the plan tree.

11.2.2. Match projections

The same SQL module defines typed match projections. The current set includes the alignment and chain scores, query and target lengths and coordinates, number of matches, block length, edit distance, mapping quality, seed count, strand, primary status, divergence, CIGAR availability, and the CIGAR text itself. A user can request only the fields needed by the query:

```
SELECT t.pk,
       bioseq.mapq(t.pk),
       bioseq.is_reverse(t.pk),
       bioseq.edit_distance(t.pk)
FROM targets AS t
WHERE mmsearch(t.seq, $1::bioseq.seq('dna5', 64))
  AND bioseq.edit_distance(t.pk) < 20;
```

The apparent argument `t.pk` is an ownership anchor. It lets the planner and custom scan associate the helper with the row and target lineage that produced the match. It is not the storage location of the score. The executor later replaces an attributable helper expression with a typed placeholder node. A helper that refers to the wrong target recipe is not silently populated from whichever alignment happened to run.

The design also avoids computing optional data unnecessarily. During `BeginCustomScan`, the owner examines which result fields survived planning. It enables CIGAR, edit-distance, divergence, or mapping-quality work only when the selected expressions require those fields. Per-call options are overlaid on a snapshot of the session alignment settings, after which the dispatcher records a structured description of the actual pipeline selected for this scan. This connects SQL projection pruning to algorithmic work without changing the meaning of any requested field.

11.3. Planner support, costs, and exact ownership

11.3.1. Why the ordinary expression has a high cost

PostgreSQL calls `mmsearch_support` with a `SupportRequestCost`. The function reports a startup cost of 100000 and a per-tuple cost of zero for the ordinary expression. This large value tells the planner that independently invoking the SQL expression is not comparable to a cheap Boolean predicate. It also gives the custom path hook an unmistakable rendezvous in the normal PostgreSQL cost model.

The meaning of this number changed during development. In the first proof of concept the large ordinary cost was paired with a custom path cost near -100000. The desired path won an artificial race. That experiment proved that a custom scan could be selected, but it did not prove that a different retained plan was semantically valid.[1] The final implementation no longer assigns a negative sentinel. The support cost describes only the non-owning scalar alternative; it is explicitly excluded from the cost tree of a real owner.

For an owning base scan, the conceptual cost is

$$C_{\text{owner}} = C_{\text{storage}} + C_{\text{operands}} + C_{\text{alignment}}(L_t, L_q, B, I) + C_{\text{residual quals}} + C_{\text{projection}},$$

[1] DOC0052, archived, records the true-returning stub, mutable score constant, and negative custom-path cost.

where L_t and L_q are estimated target and query symbols, B is the dispatch capacity, and I records whether a compatible persistent index may be read. Join ownership additionally accounts for complete query child materialization, target-child scanning, residual join expressions, and estimated result rows. Every component is clamped to a finite nonnegative PostgreSQL cost. An incumbent path's cost may be retained as EXPLAIN evidence, but it is not used as a magic threshold.

11.3.2. A semantic operation cannot be selected by cost alone

A cost comparison is appropriate when two paths implement the same semantics. It is not appropriate when one path owns an alignment and the other would call a forbidden rendezvous function. The current planner therefore uses a two-step rule:

1. construct and cost a path that proves it can own one exact `RestrictInfo` containing the supported `mmsearch`;

2. after that owner exists, remove alternatives that would execute the same restriction without owning it.

The pruning is narrow. It does not clear the relation's entire path list and does not declare every BioSeqDB path cheap. It follows pointer identity and the exact restriction carried in base restrictions, parameterized path information, or join restrictions. Ordinary paths unrelated to the claimed expression remain planner choices. This is the essential difference between cost preference and semantic ownership.

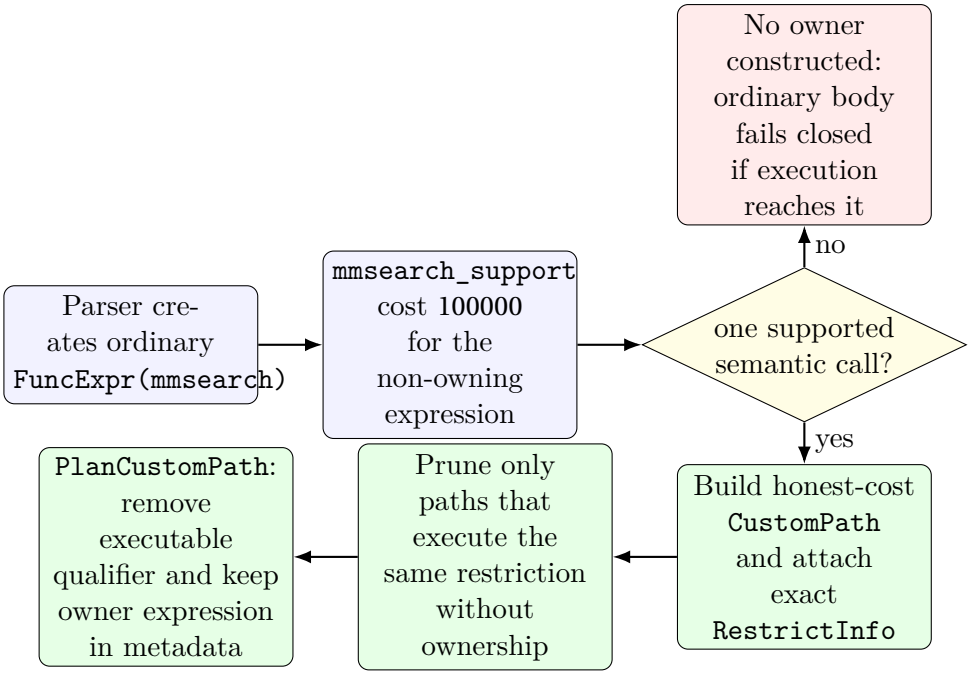


Figure 11.1: Current `mmsearch` planning. The high support cost makes the scalar alternative visible as expensive, but correctness comes from exact ownership and structural pruning, not from a negative cost.

11.3.3. Relation and join hooks

BioSeqDB registers both a relation-path hook and a join-path hook. For a `USING bioseq` relation, `BioseqColumnSetRelPathlistHook` first allows normal PostgreSQL and the columnar table access method to construct their paths. `AddBioseqColumnScanPath` then builds a `CustomPath`, copies the relation's `PathTarget`, gathers the base and parameterized restrictions available at that path, and searches for an exactly ownable `mmsearch`. The recursive path builder explores a bounded subset of parameterizations because join clauses may make an outer relation's value available as a `PARAM_EXEC` input.

For an INNER join whose restriction itself contains one semantic `mmsearch`, `BioseqBatchJoinSetJoinPat` can construct a join-level owner. It either retains a proven parameterized base-scan owner or constructs a `BioseqBatchJoin` path with target and query children. Lazy slice or other evaluated operands may require a `BioseqSliceProject` provider and a per-row recipe. These shapes are discussed in Section 11.7.

The hooks are deliberately conservative. They check extension presence, expression arity and OIDs, target/query lineage, parameterization, row security, volatile sibling restrictions, option foldability, typmods, parallel properties, and provider capabilities. More than one semantic join-level call is rejected with guidance to split the predicates into separate query levels. Failing an eligibility check means "do not claim this expression", not "assume that the Boolean stub is close enough".

11.4. From CustomPath to CustomScan

PostgreSQL separates planning data from executable state. BioSeqDB follows that separation rather than putting C++ objects or process-local pointers in serialized plan metadata. Table 11.2 summarizes the principal callbacks and their responsibilities.

11.4.1. What crosses the planning boundary

PostgreSQL's `CustomPath` and `CustomScan` offer several lists for extension data. BioSeqDB assigns them narrow roles:

- `custom_restrictinfo` identifies the exact restriction owned by the path. It supports planning proofs; the expression is not left in an executor qualifier.
- `custom_paths` carries child paths, such as the locator-side BioSeqDB scan and the query child of a batch join.
- `custom_private` stores versioned integer tags, relation and attribute identities, typmod/layout data, source recipes, and diagnostic costs. It contains PostgreSQL nodes that can be copied and serialized, not live C++ objects.
- `custom_exprs` stores expressions that must pass through PostgreSQL's setrefs machinery. In the base scan, the fourth entry carries the owned `mmsearch` expression as executor-only state. An outer `Var` can therefore become the correct `PARAM_EXEC` without making the expression an executable qualification.

`PlanCustomPath` removes the owned expression from `scan.plan.qual` and asserts that no `mmsearch` remains there. It also prevents the vectorized qualification list and the storage pushdown list from retaining a copy. This is more than a cosmetic EXPLAIN cleanup: each

Table 11.2: Planner and executor lifecycle of a base `BioseqScan`.

Phase	Current symbol	Responsibility
Relation path hook	`BioseqColumnSetRelPathlistHook`	Observe table paths, add supported BioSeqDB paths, and leave unrelated relations untouched.
Path construction	`AddBioseqColumnScanPath`	Choose the exact owned restriction, establish target/-query provenance, carry needed expressions, and assign an honest cost.
Path-set finalization	`AddBioseqColumnScanPaths`	After an owner exists, prune alternatives that would execute the same `RestrictInfo` without ownership.
Plan construction	`BioseqColumnScanPath_PlanCustomPath`	Build `BioseqCustomScan`, remove the owned predicate from executable, vector, and storage qualifiers, and place setrefs-safe expressions and private tags in PostgreSQL node lists.
State creation	`BioseqColumnScan_CreateCustomScanState`	Allocate the PostgreSQL state shell and bind method tables; defer runtime resources so plain EXPLAIN does not execute the pipeline.
Initialization	`BioseqColumnScan_BeginCustomScan`	Create per-scan runtime, initialize expression states and source providers, bind constants or parameters, snapshot options, configure the dispatcher, and register cleanup.
First execution	`CustomExecScan`	Allocate typed placeholder constants and rewrite attributable projection helpers in the finalized target list.
Iteration	`Inner_CustomExecScan`	Collect qualified inputs, dispatch full or terminal batches, drain match cursors, populate placeholders, and project one output row per match.
Rescan	`BioseqColumnScan_ReScanCustomScan`	Clear cursors and retained rows, reset source state, re-evaluate `PARAM_EXEC` operands, and rescan the access method.
Normal teardown	`BioseqColumnScan_EndCustomScan`	Clear parent-visible slots, close providers while resource owners are live, and mark runtime cleanup as complete.
Diagnostics	`BioseqColumnScan_ExplainCustomScan`	Report owner ownership, storage, pipeline, batching, fallback, and runtime counters in PostgreSQL EXPLAIN formats.

of those lists corresponds to an execution site that otherwise could call the fail-closed SQL body.

Hidden child outputs are installed in the `PathTarget` before plan construction. This timing matters because PostgreSQL's setrefs pass can only replace expressions with child-output `Var` nodes when the child already promises to publish them. Adding a target-list expression later in the executor would be too late for upper joins, results, aggregates, or `RETURNING` projections to bind to it reliably.

11.4.2. Plain EXPLAIN does not allocate the aligner

`CreateCustomScanState` constructs only the executor-state shell. Pipeline objects, query copies, sequence providers, buffer arenas, and accelerator resources are initialized in `BeginCustomScan`. A plain `EXPLAIN` can therefore inspect the plan without opening an alignment session. `EXPLAIN ANALYZE` follows the normal executor lifecycle and adds observed counters after the node runs.

This separation also keeps plan serialization meaningful. A prepared statement may outlive the memory in which it was planned; a parallel or cached plan cannot safely contain a pointer to a C++ scheduler object. Versioned PostgreSQL node metadata survives copying, while the runtime is rebuilt under the query's memory and resource owners.

11.5. Batch execution and zero–one–many output

11.5.1. The write–process–read state machine

The base scan uses a two-phase state machine implemented by `Inner_CustomExecScan` and the dispatch buffer. "Two phase" does not mean that the entire relation must be retained. The current buffer has a capacity of 8192 submissions. Reaching capacity closes and processes the current batch, drains its cursors, releases its copied slots, and resumes the still-live access-method scan. End of input performs the same transition for the terminal partial batch.

During the write phase PostgreSQL owns the active scan slot and is free to reuse it on the next access-method call. The buffer therefore copies the tuple representation it needs and retains stable sequence-source descriptors. During the process phase the selected mapper consumes the captured target/query inputs and builds an output map. During the read phase each cursor entry pairs one copied input row with one `MatchResult`. The scan installs that input as `ecxt_scantuple`, updates the placeholders, calls `ExecProject`, and returns one virtual tuple to its parent.

An input with zero matches has no cursor and produces no row. An input with three matches is visited three times by the read phase, with the same stable input attributes and three different sets of placeholder values. This provides set-valued behavior through PostgreSQL's pull executor without exposing a set-returning function in the SQL surface.

11.5.2. Scalar and vectorized access converge before alignment

The table access method can yield a scalar slot or a vector slot. In the vectorized path, pushable ordinary qualifications produce a keep mask, after which the scan extracts each retained row into its scalar slot. Both paths then reset the per-tuple expression context, establish `ecxt_scantuple`, evaluate the residual frontier and any authoritative target/query `ExprState`, and call the same submission interface. Alignment itself is therefore not skipped merely because table filtering was vectorized.

Algorithm 12 Conceptual execution of a scan-owned `mmsearch`

```
initialize an empty dispatch buffer and per-scan runtime
while the source is not exhausted or buffered output remains do
    if the buffer is in read mode then
        o ← next match cursor entry
        if o exists then
            populate all requested placeholder constants from o.match
            temporarily promote lazy sources needed by the projection
            emit Project(o.input_row, placeholders)
            restore the stable locator-backed input row
        else if the source is exhausted then
            return end of scan
        else
            release the completed physical batch and return to write mode
    else
        r ← next scalar row or next kept row from a vector batch
        if r does not exist then
            mark source exhausted and process the terminal batch
        else
            evaluate residual qualifications and owned operand expressions
            if the residual frontier passes and operands are non-null then
                copy stable row/source references into the dispatch arena
                if the buffer reaches capacity then
                    process the batch and construct per-input match cursors
```

This convergence repairs an early defect in which the vectorized scan advanced through input without invoking the buffer's submission callback. The typed integration report later recorded byte-identical output with vectorization enabled and disabled.[2] Current integration tests preserve the stronger invariant for base scans and joins: toggling `bioseq.enable_vectorization` may change how ordinary qualifications are evaluated, but not the match set or its projected fields.

11.5.3. Lazy sequence sources and tuple lifetimes

Chapter 9 showed that a sequence value may be a compact inline value, a fragment locator, or a lazy recipe. The custom scan is the point at which PostgreSQL tuple lifetime and sequence-source lifetime meet. A deferred table scan can leave a stable locator in the buffered row instead of eagerly assembling every sequence. The alignment provider bulk-acquires the fragments required for the current dispatch. Output-only sequence expressions are promoted just before projection and then restored to the locator form.

The restoration step is necessary when one row has several matches. PostgreSQL resets the per-tuple memory context between returned rows. If the buffered slot were overwritten with a detoasted or assembled pointer owned by that context, the second match would dereference released memory. Current code captures the source cells before promotion and restores them on normal return, C++ exception, and PostgreSQL `longjmp`. Match objects and copied minimal tuples live in the batch arena; ephemeral expanded objects remain confined to the current projection.

11.5.4. A lower-bound prefilter does not become the SQL predicate

When a prefilter such as SneakySnake is enabled, it is permitted to reject a candidate only according to its proven lower-bound contract. Every survivor still reaches exact alignment. If SQL additionally requests a strict edit-distance condition, the read phase evaluates that exact boundary against the resulting `MatchResult`. This ordering prevents an accelerator gate from becoming an approximate implementation of the user's SQL predicate.

11.6. Injecting match fields into planned expressions

11.6.1. Mutable typed Const nodes

PostgreSQL evaluates a projection from an expression tree. Rebuilding that tree for every match would be expensive, while calling each SQL helper would have no access to the scan's current match. BioSeqDB instead rewrites each owned helper to a typed `Const`. The node address remains stable for the executor lifetime and only its `constvalue` and `constisnull` fields change.

On the first call to `CustomExecScan`, the scan allocates the placeholder family and runs `var_injector_mutator` over the finalized target list. The mutator recognizes function OIDs, verifies that the helper belongs to this match-owner recipe, and substitutes the corresponding `Const`. Unrelated functions are traversed normally. A projected `bioseq.edit` whose target argument does not match the `mmsearch` target raises a diagnostic instead of receiving fields from the wrong sequence.

Before each `ExecProject`, `populate_placeholder_consts` copies one `MatchResult` into the requested nodes. Coordinates and lengths use PostgreSQL `BIGINT`, because the C++

[2] DOC0058 records the intermediate five-input/nine-output run and the vectorization parity check.

source values are unsigned 64-bit sequence sizes. Scores, map quality, and seed counters remain `INT4` because their source fields are 32-bit. Strand and status use `BOOLEAN`, divergence uses a floating type, and CIGAR is formatted as PostgreSQL text only when a CIGAR placeholder exists.

Table 11.3: Examples from the match projection ABI.

Projection family	Placeholder type	Source and update rule
`score`, `dp_score`, `mapq`, `seed_count`	`INT4`	Copied from 32-bit `MatchResult` score/-counter fields.
`query_start`, `query_end`, `target_start`, `target_end`, lengths	`BIGINT`	Value-preserving conversion from unsigned sequence-size fields, with the project's supported size ceiling below `INT64_MAX`.
`is_reverse`, `is_primary`, `has_cigar`	`BOOLEAN`	Updated for every emitted match from the same result object.
`divergence`	floating value	Written only when the query projects or otherwise requires divergence.
`cigar`	`TEXT`	Formatted from traceback data; traceback generation can be disabled when no dependent field is required.
`symbol_count`	typed numeric placeholder	Not a `MatchResult` field; resolved from the row's metadata sidecar using relation and row identity.

The mutable constant is an executor-local data conduit, not a SQL constant in the semantic sense. It is installed only after planning and setrefs have finished. PostgreSQL never has an opportunity to fold its changing value during planning. The target list is compiled against the stable node, and the scan mutates it immediately before projection.

11.6.2. Why joins need PlaceHolderVar

A bare helper such as `bioseq.cigar(t.pk)` can be copied or pushed into a join child's target list. In an outer join, the result also carries null-extension semantics: the helper must become null when the originating side is null-extended. Replacing every visible copy with an unqualified `Const` would lose that placement and nulling information.

The support function `bioseq_placeholder_support` handles `SupportRequestSimplify`. When the query contains joins or aggregates, it finds the originating `Var`, copies the helper, and wraps it with PostgreSQL's `PlaceHolderVar`. The wrapper preserves `phlevelsup`, `phrels`, and `phnullingrels`. PostgreSQL can then carry the value upward from the correct evaluation level without re-evaluating the helper in a sibling scan. At executor rewrite time, the wrapper remains intact while its contained owned function is replaced.

This mechanism was introduced after a LEFT JOIN plus CIGAR projection exposed a crash: the planner had pushed a helper into a child that had no current `MatchResult`. It also solves aggregate placement, where a projected match field must be computed below the aggregate rather than invoked after the alignment owner has been drained. The current helper support is deliberately a placement mechanism; exact ownership is still checked by the custom scan.

11.6.3. Stable identity is more precise than function OID

Function OID alone is insufficient when a query contains multiple sequence relations, aliases, slices, or nested owners. Two `bioseq.target_start(...)` calls have the same OID but may refer to different alignments. Current owner descriptors therefore carry the target recipe, base lineage, relation/attribute identities, placeholder identity or `phid`, and stable match-site information. A descendant owner may publish an already-computed helper to an upper result; the upper owner must pass it through rather than relabeling it as its own.

This is the database analogue of keeping fields from one mapper record together. The stable identity guarantees that coordinates, CIGAR, and score are not assembled from different match rows merely because their function names coincide.

11.7. Join ownership and expression-tree surgery

11.7.1. Four plan shapes

The phrase "mmsearch in a join" covers several different data flows. BioSeqDB distinguishes them rather than applying one global rewrite.

Table 11.4: Principal current ownership shapes involving joins.

Shape	Owner	Operand supply	Key condition
Base predicate below a join	`BioseqScan`	Target from its relation; query constant, same-row expression, or parameter	The base restriction is exactly attributable to the scan. An upper LEFT JOIN is possible because the alignment itself remains base-owned.
Parameterized nested execution	Inner `BioseqScan`	Outer value arrives as `PARAM_EXEC` and is rebound on rescan	A retained or reconstructed nested path proves one exact inner owner.
Raw two-child INNER search	`BioseqBatchJoin`	Query child is materialized; target child streams locator-backed rows	Exactly one secure semantic join restriction with supported base operands and no unsafe residual shape.
Transformed/lazy INNER search	`BioseqBatchJoin` with `BioseqSliceProject` and, where required, a per-row recipe	Child providers publish evaluated slice/recipe outputs before ownership	The recipe and its base lineage are explicit; the join EXPLAIN identifies per-row ownership and contains no executable join filter.

The join-level hook currently claims only `JOIN_INNER`. This boundary does not mean that every query containing a LEFT JOIN is unsupported. A literal search predicate on one base relation can be owned below a LEFT JOIN, with `PlaceHolderVar` preserving the projected helper's nulling rules. What is intentionally narrow is transformation of the join-level `mmsearch` restriction itself.

11.7.2. Raw BioseqBatchJoin

For the raw two-child case, the query child is fully materialized before the first output. The target child is a BioSeqDB locator scan and is read once. The batch join can then combine target rows with retained query references, dispatch the alignment work, and publish each match together with the corresponding child columns. Its startup cost includes the entire query child and materialization work; its total cost adds the complete target scan, operand evaluation, alignment, residual qualifiers, and output rows.

This plan is not equivalent to merely placing a custom node above an existing NestLoop, HashJoin, or MergeJoin. The child join path may already contain the `mmsearch` as a join filter. BioSeqDB reconstructs a private child path with the exact `RestrictInfo` removed, or declines the transformation. It never wraps a descendant that would still call the SQL rendezvous function.

11.7.3. Parameterized scans and rescans

When PostgreSQL chooses a nested parameterized plan, the query operand may be a `PARAM_EXEC` supplied by the outer row. `BeginCustomScan` records the exact post-setrefs argument cell and initializes `MmsearchParamRebind`. On `ReScanCustomScan`, the scan clears match cursors, retained slots, provider state, and source exhaustion, then re-evaluates the parameter from the owned expression before rescanning the table access method.

This explicit rebind replaced an early approach that inspected only constants at initialization. Without it, the first outer query could be reused for later outer rows, producing plausible but wrong join results. ReScan is therefore not only a performance callback; it is part of the SQL correlation semantics.

11.7.4. Removing every executable copy

PostgreSQL may represent the same semantic restriction in several planner/executor locations. The current EXPLAIN audit names five classes:

1. owned `RestrictInfo` path copies;
2. executable `mmsearch` qualification copies;
3. vector-qualification copies;
4. storage-pushdown copies; and
5. descendant owner copies.

For a join-level owner, each counter must be zero below the authoritative node and `Bioseq Residual Mmsearch Qual` must be `absent`. The regression helper also searches the textual plan for `Filter`, `Join Filter`, `One-Time Filter`, `Index Cond`, and `Recheck Cond` whose expression begins with `mmsearch`. This audit is stronger than checking that a `BioseqBatchJoin` appears somewhere in the plan: it proves that the same semantic call has not survived at a second execution site.

11.7.5. Security and evaluation order

Expression-tree surgery must respect PostgreSQL security and volatility rules. The first join-level implementation declines row-security shapes, pseudoconstant or non-promotable restrictions, and volatile sibling qualifiers whose relative evaluation order it cannot preserve. The optional third argument must fold to the expected typed constant form before the hook claims it. Parallel child plans are rejected where the provider lifecycle cannot be transferred safely.

These checks intentionally sacrifice some plan shapes. A database extension cannot justify moving a volatile expression, bypassing a security barrier, or reusing a value across rows merely because doing so would produce a larger alignment batch. Unsupported shapes remain visible through the fail-closed SQL boundary.

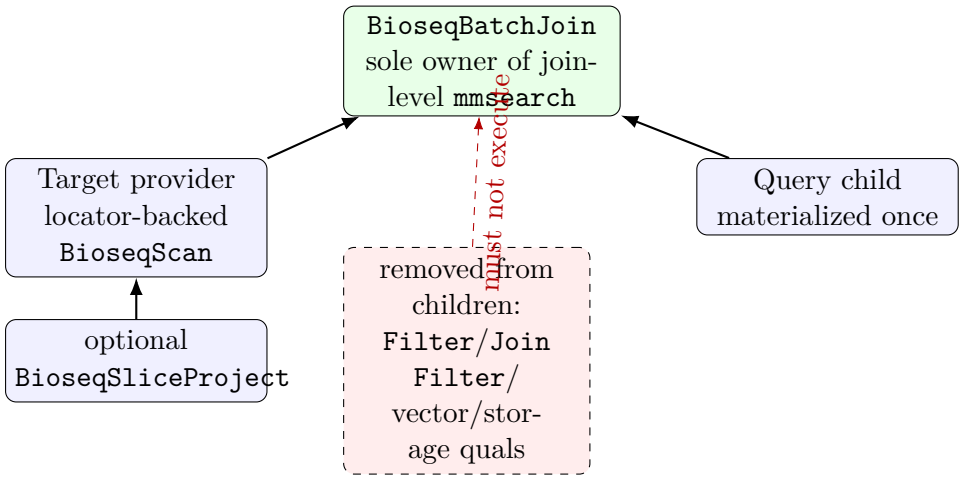


Figure 11.2: Join-level ownership. Child providers publish operands and ordinary columns; only the parent alignment owner consumes the semantic `mmsearch` restriction.

11.8. What BioSeqDB borrowed from ParadeDB

The copied `paradedb-experiments` repository provided a concrete PostgreSQL extension example in which planned projection helpers are substituted by executor-owned values. Its PDB scan recognizes score and snippet projections, injects typed placeholder constants, and updates `const_score_node` with the score of the current search hit. Its `placeholder_support` also wraps projection expressions in `PlaceHolderVar` for joins and aggregates, preserving `phlevelsup` and `phnullingrels`.

These mechanisms answered an important feasibility question: an extension does not need custom SQL syntax for every result field. It can allow PostgreSQL to parse an ordinary function call, preserve the expression's placement through planning, and replace the call with an executor-local value at the custom scan boundary. The early BioSeqDB proof of concept copied this pattern for one synthetic score.

The complete BioSeqDB problem is nevertheless different. Text search commonly associates one score with one hit already enumerated by the search node. Sequence search must itself create a variable number of match rows after batch processing. Its target value may be a large lazy sequence locator rather than a value that can be held in a conventional tuple datum, and each match has a coordinated record of scores, coordinates, strand, traceback, and mapping-quality fields.

The resulting design should therefore be described as inspired by ParadeDB's projection-node technique, not as a port of its custom scan. The mutable constant is the common mechanism. Batch cardinality, alignment ownership, deferred sequence access, parameter rebinding, and the match-row ABI are BioSeqDB-specific extensions.

11.9. Evolution from proof of concept to the final owner protocol

The PostgreSQL integration changed repeatedly because each larger SQL shape exposed a different hidden assumption. Table 11.6 orders the important stages. The rows are not simultaneous alternatives; later rows supersede earlier behavior.

Table 11.5: Relationship between the copied ParadeDB technique and the final BioSeqDB design.

Concern	ParadeDB experiment	BioSeqDB adaptation
SQL projection	Score/snippet helper recognized in a PDB custom scan	Family of match and metadata helpers bound to an exact alignment owner
Injected node	Typed score/snippet `Const`; executor updates `const_score_node`	Typed constants for a multi-field `MatchResult`; coordinates are widened and CIGAR is produced conditionally
Expression placement	`PlaceHolderVar` carries a helper across joins/aggregates	Same PostgreSQL mechanism, plus recipe/base-lineage checks and stable placeholder identities
Cardinality	Projection decorates the current text-search hit	Owner creates zero, one, or many SQL rows for each sequence input
Input lifetime	Search fields managed by the PDB scan	Copied tuple slots, fragment locators, lazy providers, per-output promotion, and restoration across memory-context resets
Path forcing	Copied experiments include clearing paths or assigning extreme fallback costs	Final design uses honest nonnegative costs, then removes only paths that would execute the exact owned restriction
Failure boundary	Specific to the PDB search interface	`mmsearch` is a fail-closed rendezvous; no unowned plan may execute it

Table 11.6: Design evolution of `mmsearch` planning and execution.

Evidence	Intermediate design	What it demonstrated	Why it changed
DOC0052	True-returning Boolean stub, one synthetic score `Const`, high scalar cost, negative custom cost	PostgreSQL could select a custom scan and an executor could mutate a planned projection node	No real alignment, no filtering, unsafe fallback semantics, artificial cost race, and only one output value
DOC0054	Write/batch/read buffer, full placeholder list, proposed multirow draining	A pull executor can expose post-batch zero/one/many cardinality	The first implementation still had global state, incomplete vector handling, and incomplete resource/rescan behavior
DOC0058	Typed `bioseq.seq` operands, real `minimap_port`, detoast/-copy ownership, vector parity	Real matches and CIGARs could flow through the same SQL projection interface; five inputs emitted nine rows	Join correlation and expression placement were not yet safe
DOC0083	Per-scan runtime, memory-context callback, parameter rebinding, ParadeDB-inspired PHV support	Repeated scans, cross joins, LEFT JOIN projections, and correlated inner execution exposed and repaired global-state assumptions	Join selection still used cost forcing and did not prove exact single ownership for every copied restriction
DOC0321–DOC0334	Planner-support/custom-scan contract, structured EXPLAIN, expanding integration coverage	Made plan shape, batch callbacks, and vector/scalar parity explicit workflow requirements	Complex slices, join reconstruction, and hidden target-list dependencies still required a stricter ABI
DOC0377, DOC0388, current source	Base, parameterized, batch-join, and slice providers; fail-closed SQL; versioned owner descriptors; exact restriction removal; honest costs	One semantic operation has one executor owner, one stable match ABI, and auditable descendant state	This is the current first-release boundary; unsupported join/security shapes remain explicit limitations

11.9.1. Why the true-returning stub was especially dangerous

Returning true looked convenient during the first experiment because it kept the predicate in the planned tree while the custom scan supplied synthetic scores. It also meant that any planner path which did not perform the custom rewrite returned every row. A cost estimate then became a correctness mechanism: if the custom path ever lost, SQL results changed. The final erroring body reverses this default. A new PostgreSQL transformation, a missed expression shape, or a disabled custom scan becomes an observable unsupported-query error and cannot masquerade as a successful search.

11.9.2. Why per-scan state was not optional

The original file-static buffer appeared to work for one scan node. A cross join creates two scan nodes in the same backend, and repeated execution reuses the process. One node could move the global state to read-mode EOF while the other still expected write mode. An early EXPLAIN ANALYZE symptom was an outer scan reporting one row where the fixture contained five. Moving the buffer, feature flags, output cursor, parameter state, placeholder nodes, and pipeline description into `BioseqScanRuntime` gave each `CustomScanState` an independent lifecycle.

The cleanup design also changed. PostgreSQL errors use non-local jumps, so ordinary C++ stack unwinding cannot be assumed at every boundary. The current normal path clears slots and closes providers in `EndCustomScan`, while the registered `MemoryContextCallback` deletes the runtime on context teardown and covers abort paths. The callback object itself lives inside the runtime whose lifetime it governs, so it is registered once and not left pointing at a moved object.

11.9.3. Why an owner needs a versioned ABI

As join and slice support expanded, a simple target relation number was no longer enough to identify the source of a helper. Plan construction may introduce resjunk columns, `INDEX_VAR` references, subquery outputs, or `PARAM_EXEC` operands. Current private tags carry a version and fixed field count, while expression-bearing metadata travels through `custom_exprs`. Plan finalization validates that the encoded descriptor, target list, helper sites, and child outputs agree. A mismatch is reported as corrupted or unsupported plan state rather than decoded using an older layout.

11.10. SQL and EXPLAIN evidence

11.10.1. Intermediate cardinality evidence

The first typed real-aligner report supplied five stored target rows and observed nine emitted alignment rows. The distribution was `3,1,1,1,3` matches for inputs one through five. The first and fifth rows therefore demonstrated the critical multirow property rather than merely a Boolean filter. The report included CIGARs such as `29M` and a gapped `9M3I13M`, and recorded identical output when vectorized table access was toggled.[3]

A later PostgreSQL/Vulkan report inserted two target rows and captured 12 output alignment rows. Its EXPLAIN fragment was:

[3]DOC0058, archived integration report. Its short-query parameter policy is historical; only the observed cardinality and parity are used here.

```
Custom Scan (BioseqScan) on doc0351_sql_dna5
  (actual rows=12 loops=1)
  Filter: mmsearch(target, query)
  Bioseq Recomputed: minimizers 100%
    (read_sequence_index disabled)
  ChainStage: Vulkan tiled DP
    vulkan_enabled: true
    [accelerator: vulkan]
  * Submit results
    [targets=2 prebuilt_index=0 recomputed=2]
```

This is valuable evidence that the custom scan expanded two stored rows into 12 match rows and surfaced the selected pipeline. It is also visibly an intermediate plan: the displayed `Filter: mmsearch` is forbidden by the current owner protocol. Presenting the fragment without its date and status would confuse an old EXPLAIN rendering with the final executor invariant.[4]

Table 11.7: Recorded intermediate SQL cardinality evidence. These are integration receipts, not comparative throughput benchmarks.

Evidence	Input target rows	Emitted match rows	Conclusion supported
DOC0058	5	9	The typed CPU path emitted several rows for inputs one and five; output was identical with vectorization on and off.
DOC0352	2	12	The PostgreSQL custom scan expanded both targets and reported the Vulkan chaining route and recomputed-index counters.

Neither row count is a biological sensitivity result by itself. The fixtures, parameters, and sequence lengths differ, and a larger number of alignments is not automatically better. Their purpose in this chapter is more precise: they prove that the SQL executor exposed a mapper's set-valued output rather than collapsing it to one Boolean row.

11.10.2. Current plan invariants

The current regression suite does not freeze one complete textual plan, because costs, formatting, and harmless plan details can change across PostgreSQL builds. It freezes semantic properties:

- a base search contains `Custom Scan (BioseqScan)`;
- the base node does not contain `Filter: mmsearch`;
- `EXPLAIN ANALYZE` reports actual execution for that custom scan, and JSON EXPLAIN names both `Custom Scan` and `BioseqScan`;

[4] DOC0352, current report documenting an earlier implementation stage.

- a transformed slice INNER join contains `Custom Scan (BioseqBatchJoin)`, reports `Bioseq Mmsearch Ownership: per-row recipe`, contains `BioseqSliceProject`, and has no `Join Filter: mmsearch`;
- the five descendant-copy audit counters are zero and the residual qualifier is reported as absent.

The latest implementation log adds a second current data point: an ordinary two-child ABI-v3 `BioseqBatchJoin` is covered by a regression that expects exactly one upper `BioseqSliceProject`, no per-row owner, and no residual `Join Filter: mmsearch`. The same log records an `Aggregate + Custom Scan (BioseqScan)` path with no vector aggregate owner and invariant results under the vectorization toggle.[5]

11.10.3. Structured pipeline descriptions

The EXPLAIN output is not assembled from a parallel set of hard-coded names. The templated pipeline work introduced a runtime `PipelineDescription` composed of stage descriptions. The same dispatch decision that selects minimizer, chaining, alignment, prefilter, and accelerator policies supplies their metadata. A generic walker renders this structure to TEXT and the structured PostgreSQL formats.

This design addresses a common observability failure: an execution path changes, but EXPLAIN continues to print the old algorithm name. The dispatcher tests require runtime selection and description selection to agree. The custom scan augments that algorithmic description with database facts such as owned-qual audit counters, projected columns, index read/recompute decisions, storage hydration, submission counts, and fallback reasons.

11.11. GUCs, prepared plans, and resource safety

11.11.1. GUC behavior

BioSeqDB has settings that change storage/index policy, vectorized qualification, column caching, mapper parameters, and custom-scan availability. They do not all have the same semantic status.

- `bioseq.enable_vectorization` changes the table-qualification route. Current tests require identical match rows and require the custom scan to remain the owner in both states.
- `bioseq.enable_column_cache` changes a storage optimization, not alignment semantics. The custom owner persists with the cache on or off.
- `bioseq.enable_bioseq_index_scan` changes index-use policy. The owner may read a compatible index or recompute, but it remains a `BioseqScan`.
- Per-call aligner options override a snapshot of the session mapper settings for one owned expression. This snapshot prevents a batch from observing a half-changed configuration.
- The master custom-scan setting controls whether the extension offers these owners. Disabling the only supported owner does not authorize the Boolean body to return approximate or universally true results; an attempted execution reaches the fail-closed boundary.

[5] Latest successful entries in DOC0386; DOC0388 summarizes the final ownership implementation.

The tests that keep `BioseqScan` selected across the first three toggles are specifically a regression against the old cost-race design. Changing a cache or vectorization setting may legitimately change the honest cost estimate, but it must not resurrect an unowned executable copy of `mmsearch`.

11.11.2. Prepared statements and once-only evaluation

Prepared and parameterized statements separate planning time from the values seen at execution. BioSeqDB keeps expression trees, source recipes, and parameter coordinates in the plan, then evaluates operand `ExprState` objects in the scan's expression context. A constant query can be prepared once, while a `PARAM_EXEC` query is rebound for each rescan. For a join-level batch owner, query-child expressions are materialized once into the query context before targets are streamed.

The exact operand is evaluated once at its authoritative boundary. A lazy slice is not recomputed separately for `mmsearch`, CIGAR, and coordinate helpers. Hidden child outputs and recipe identifiers let upper nodes reuse the published value. This is both a performance property and an SQL semantics property for expressions whose evaluation can fail or whose source lifetime is short.

11.11.3. Memory contexts and ResourceOwners

PostgreSQL uses memory contexts for bulk lifetime management and `ResourceOwner` objects for relations, snapshots, buffers, and similar resources. A C++ destructor alone is insufficient because `ereport(ERROR)` leaves a protected region through `longjmp`. Conversely, deleting every C++ object only from a memory callback is too late for resources that should be closed while their owner is still active.

The current scan uses a split protocol:

1. `BeginCustomScan` constructs per-node runtime and registers one callback in the query memory context.
2. `EndCustomScan` clears result and scan slots, removes parent-visible expanded-object references, closes sequence providers and access-method resources, and marks normal cleanup complete.
3. The memory-context callback deletes the C++ runtime after normal end or performs abort-path deletion when execution did not reach `EndCustomScan`.

The batch join applies the same principle to materialized query slots, admission permits, locator providers, and both child plans. It releases tuple-visible references before resetting the arena that owns them. These details are necessary for repeated queries in one backend and for `EXPLAIN ANALYZE`, which executes and then formats the same plan.

11.11.4. Parallelism

SQL declarations can mark an expression parallel safe while a particular custom owner remains parallel unsafe. The declaration says that the function's semantics do not depend on backend-local side effects; it does not make a stateful scheduler, provider, GPU queue, or materialized query child transferable between parallel workers. BioSeqDB disables parallel ownership for plan shapes whose runtime state cannot be divided safely and rejects incompatible parallel children in the batch join. This conservative boundary avoids sharing mutable placeholders or provider state between workers.

11.12. Limitations and engineering trade-offs

The final owner protocol is deliberately narrower than arbitrary SQL.

1. Join-level transformation currently targets INNER joins. Base-owned searches can occur below other joins, but arbitrary outer-join `ON mmsearch(...)` semantics are not claimed.

2. One semantic join-level `mmsearch` is accepted. Stacked calls must be separated into query levels so that each has an unambiguous owner and match-row ABI.

3. Row-security, security-barrier, pseudoconstant, volatile-sibling, and unsupported option-expression shapes are declined when the implementation cannot prove equivalent evaluation placement.

4. Batch execution adds latency before the first row. The base scan bounds that latency and memory with an 8192-submission rollover. A raw batch join intentionally materializes its query child, so its cost and admission policy must account for that complete collection.

5. Mutable placeholders are tightly coupled to PostgreSQL expression planning. Every new result helper needs an OID mapping, a correctly typed placeholder, stable ownership metadata, population logic, EXPLAIN coverage, and join/aggregate tests.

6. A custom scan is tied to BioSeqDB's table access and source-provider contracts. An ordinary heap value or arbitrary function wrapper cannot automatically provide deferred fragment hydration, persistent-index compatibility, or batch-source lifetimes.

These restrictions are preferable to implicit fallbacks. A standalone mapper can reject an unsupported command-line combination before processing a file. A database extension must additionally consider planner copies, rescan, null extension, row security, volatility, and resource unwinding. Failing closed makes that boundary explicit to the SQL user.

11.13. Conclusion

BioSeqDB's PostgreSQL API presents sequence search with familiar typed SQL, but its semantics are implemented by a plan node rather than by repeated scalar calls. `mmsearch_support` marks the ordinary expression as expensive; relation and join hooks prove that one custom path can own the exact restriction; plan construction removes every executable descendant copy; and the executor batches inputs and emits one projected row for each alignment.

The result-injection mechanism began with ParadeDB's mutable score constant and grew into a stable, recipe-aware match ABI. Typed placeholders let one alignment populate score, coordinates, strand, mapping quality, divergence, edit distance, and CIGAR without repeated mapper calls. `PlaceHolderVar` preserves placement and null-extension semantics across joins and aggregates, while hidden child outputs and setrefs-safe metadata support slices and parameterized execution.

The most important difference from the original proof of concept is not a smaller cost number. It is the removal of cost as a correctness mechanism. The current implementation uses finite nonnegative estimates and then makes ownership structural: if a supported owner exists, a path that would execute the same rendezvous restriction is not an alternative

implementation of the query. If no owner can be proven, the SQL body reports the unsupported shape instead of inventing alignment results. This owner protocol is what turns the seed–chain–extend pipeline from a library called by PostgreSQL into a database execution primitive.

Chapter 12

Physical sequence storage in PostgreSQL

12.1. Storage is part of the execution model

Storing a biological sequence in PostgreSQL appears at first to be a type implementation problem. A compact value can be placed in a variable-length attribute, and PostgreSQL can then store, retrieve, and compare it like any other datum. This is sufficient for short values, but it does not describe the requirements of BioSeqDB. A target sequence may be larger than one PostgreSQL allocation. An alignment may need only a few bounded windows from that target. A metadata query may project an identifier and a length without reading any sequence bytes. A long-running search may be requested concurrently by several sessions. Consequently, the physical representation must cooperate with the reader, the planner, memory accounting, and concurrency control.

The current implementation separates four concerns which earlier prototypes partly conflated:

1. Ordinary table columns are stored by a PostgreSQL table access method in stripes, chunk groups, and per-column streams.

2. A sequence value in the parent row is normally a small locator. Its packed fragments reside in length-bucketed shard relations.

3. Persistent minimizer indexes use their own relations and generation metadata. Their format is discussed in Chapter 12.

4. Reuse of a complete query result is implemented by the separate `bioseq_cache` extension. It is not the same object as either a sequence fragment cache or a persistent minimizer index.

This separation gives a useful projection property. A query such as `SELECT sample_id, length FROM targets` can read narrow scalar streams without materializing sequence payloads. A search operator can instead follow the locator and acquire fragments lazily. A repeated whole query can be served from a sealed result generation only when its execution identity and dependencies still match. Each mechanism has its own lifetime and correctness contract.

The design was reached through the Phase A, B, and C storage series (`DOC0260`, `DOC0263`, and `DOC0264`) and several corrective reports. Those documents are useful for understanding the rejected alternatives, but they do not define the current byte layout. In particular, the Phase C specification described a version-2 locator. The current source defines version 3 and

adds an immutable source epoch. The account below follows the source in `backend/storage/bucket_math.cppm` and `backend/pg/storage_pg/sequence_storage_manager.cppm`.

12.2. The Citus-derived column store

12.2.1. Lineage and retained ideas

The base table access method has a concrete lineage in Citus Columnar. The correspondence is visible in both organization and operations. Citus `columnar_storage.c` maps logical offsets onto PostgreSQL blocks, keeps a metapage, and reserves stripe, row, and byte ranges. Its writer buffers rows into stripes and chunk groups, serializes a validity stream and a value stream for every column chunk, and writes skip metadata. Its reader selects chunk groups using metadata before reconstructing rows. BioSeqDB retains this family of mechanisms in the C++ modules `pg/column/storage_physical.cppm`, `pg/table/io/writer.cppm`, and `pg/table/io/reader.cppm`.

This is more specific than saying that both systems are columnar. The physical page convention, the reservation counters, the stripe and chunk-group vocabulary, the per-column validity/value streams, compression metadata, and min/max pruning all have direct counterparts. BioSeqDB then changes the surrounding architecture. The code was moved behind a PostgreSQL-specific module boundary, renamed from generic "columnar" terminology to the BioSeq column store, and coupled to sequence locators, generated catalog rows, vector-batch slots, alignment custom scans, and sequence-specific pruning rules. These changes were consolidated by the refactor described in `DOC0322` and its implementation report `DOC0323`.

Table 12.1: Citus Columnar mechanisms retained and extended by BioSeqDB.

Concern	Citus-derived mechanism	BioSeqDB-specific extension
Physical bytes	Metapage, logical offsets, PostgreSQL pages, generic WAL	C++ module boundary and typed physical/storage APIs
Row organization	Stripe, chunk group, one value stream and one existence stream per column chunk	Sequence values may be locators whose payloads live in separate fragment shards
Metadata	Stripe and chunk catalogs, stream offsets, compression, min/max	Annotated C++ catalog structs, generated SQL/tuple codecs, shard registry
Reading	Projection mask, chunk pruning, decompression, row reconstruction	Sequence extrema excluded from ordinary pruning; scalar and vector slots; lazy sequence scanner
Execution	PostgreSQL TableAM and custom scan support	Alignment-aware custom scans, vector aggregates, query-result cache provider

12.2.2. Pages, logical offsets, and reservations

The main relation fork begins with two non-data blocks. Block 0 contains `BioseqColumnMetapage`. Block 1 is reserved and empty. Logical data starts after the PostgreSQL page header in block 2. If

$$P = \mathrm{BLCKSZ} - \mathrm{SizeOfPageHeaderData}$$

is the number of usable bytes in one page, then the first valid logical data offset is $2P$. For a logical offset x, the storage layer computes

$$\text{block}(x) = \left\lfloor \frac{x}{P} \right\rfloor, \qquad \text{inpage}(x) = \text{SizeOfPageHeaderData} + (x \bmod P).$$

Higher layers can therefore reserve and address one continuous byte space without manually splitting every stream at page boundaries.

The metapage records the format version, a storage identifier, the first unused stripe identifier, the first unused logical row number, the first unused logical byte offset, and an unlogged-reset flag. Reservations modify these counters while holding the relation extension lock. Initialization and truncation require an access-exclusive relation lock. Metapage changes use PostgreSQL generic WAL through `GenericXLogStart`, `GenericXLogRegisterBuffer`, and `GenericXLogFinish`. Thus the column format does not replace PostgreSQL's page, buffer, lock, or recovery machinery. It builds a logical append space on top of it.

The distinction between a reservation and a completed stripe matters for concurrent writers. A writer first obtains non-overlapping row, stripe, and byte ranges. It can then serialize the stripe without racing another writer for identifiers or offsets. Completion publishes the actual data length and row count in the metadata catalogs. A reader considers the published metadata, not an arbitrary byte suffix, to determine which stripes are visible.

12.2.3. Stripes, chunk groups, and column streams

The physical hierarchy is shown in Figure 12.1. A stripe is the large append and metadata unit. It is divided into chunk groups, each containing the same row interval for every column. For every pair of column and chunk group, the writer produces an existence bitmap and a serialized value stream. The current defaults are 150,000 rows per stripe and 10,000 rows per chunk group. Both are per-table options. The allowed ranges begin at 1,000 rows and extend to 100,000,000 rows.

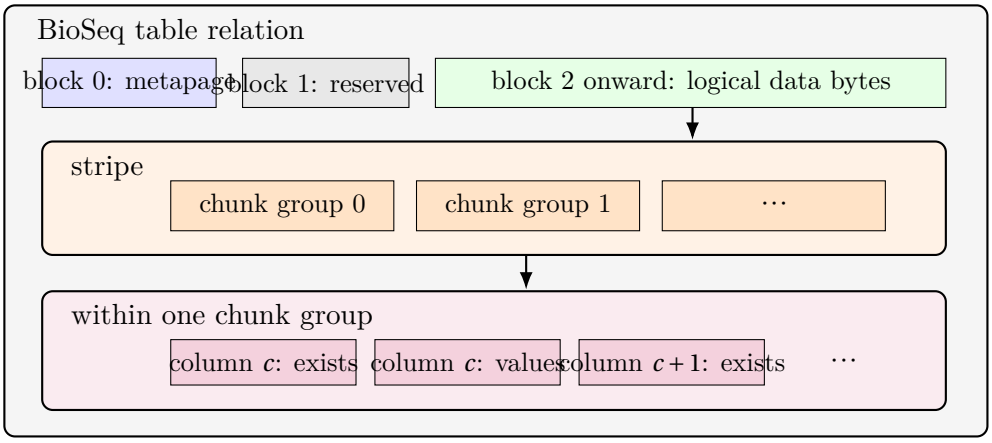


Figure 12.1: The column-store hierarchy. Metadata catalogs map stripe and chunk identifiers to offsets in the logical byte space.

Rows are accumulated in a `ChunkData` object. Its arrays distinguish NULL presence from Datum values and keep one growing serialized buffer per column. Serialization is performed by the zero-cost `pg_datum` wrapper from `pg/pg_utils/datums.cppm`. The wrapper computes PostgreSQL's aligned datum length, zeroes alignment padding, copies pass-by-value or pass-by-reference representations correctly, and advances the buffer. It also provides an explicit

`copy` operation for by-reference min/max values. These small ownership distinctions prevent a temporary PostgreSQL Datum from being kept after its memory context has been reset.

When a chunk group is complete, its Boolean presence array is packed at one bit per row. The requested compression algorithm is applied to the serialized value stream, but the compressed result is retained only when it is smaller than the original. The metadata therefore records the actual compression algorithm, which may be `none`. The compile-time feature set chooses the default in this order: Zstandard when available, otherwise LZ4 when available, otherwise PostgreSQL PGLZ. The default level is 3. This policy avoids paying decompression cost for an incompressible chunk merely because the table requested compression.

During stripe flush, offsets are computed before bytes are written. For each column the physical order is all of its existence buffers, followed by all of its value buffers. The writer then completes the stripe reservation, writes the streams into the reserved logical range, and records stripe, chunk-group, chunk, and row-mask metadata. A single value whose aligned Datum representation reaches 256 MiB is rejected. The writer also flushes a stripe before a value buffer can approach the roughly 1.024 GiB buffer bound. These checks concern one column chunk buffer; they are separate from the fragment layout used for very large sequence values.

12.2.4. Skip metadata and generated catalogs

For ordinary columns, the writer finds the B-tree comparison procedure and tracks the minimum and maximum non-NULL Datum in each chunk. This information is stored together with stream offsets, compressed and decompressed lengths, row counts, and compression identifiers. The reader can construct a PostgreSQL predicate over the extrema and ask whether the predicate is refuted for that chunk.

Sequence columns are deliberately excluded from this optimization. Their serialized values may be locator stubs, and the physical rank-stream ordering does not define a useful biological ordering. Treating an arbitrary minimum and maximum packed sequence as a valid range bound would be at best unselective and at worst misleading. Scalar metadata columns next to the sequence, such as a numeric length, date, or sample category, remain eligible.

The catalog definitions are not maintained independently in C++ and SQL. Structures in `pg/column/metadata/sql_rows.cppm` carry `bioseq_sql` annotations. The code generator produces the installation SQL and tuple codecs used by `pg/column/metadata/core.cppm`. Chapter 15 describes this generation system. Table 12.2 summarizes the storage-specific result.

Table 12.2: Principal internal catalogs of the BioSeq column store.

Catalog	Role
`options`	Per-relation stripe size, chunk-group size, compression type, and compression level.
`stripe`	Storage identifier, stripe number, logical offset, byte length, column and chunk counts, row count, and first row number.
`chunk_group`	Row count and deletion count for one row interval of a stripe.
`chunk`	Per-column and per-chunk extrema, stream offsets and lengths, compression information, decompressed length, and value count.
`row_mask`	A byte mask and row interval describing deleted rows.
`seq_sub_table`	Canonical registry that maps a parent storage identity, sequence column, length bucket, and shard to its fragment and sidecar relations, capacity state, and byte counters.

The shard registry key is deliberately wider than a relation OID or relfilenode. It contains database OID, tablespace OID, parent relfilenode, column attribute number, bucket index, and shard index. This is the correction to an early design in which relfilenode alone could collide across databases or tablespaces. Partial and prefix indexes support active-shard lookup and parent cleanup. Check constraints prevent negative byte counters.

12.2.5. Projection, pruning, and reconstruction

A scan begins with a projection mask. Only requested columns have their streams loaded. For every stripe, the reader obtains the skip list, tests eligible predicates against chunk extrema, and builds a selected-chunk mask. For a selected projected chunk it reads the existence stream and value stream, decompresses the latter if necessary, and reconstructs aligned Datums. A column added after an old stripe was written is supplied from its PostgreSQL default or as NULL. The row mask excludes deleted row numbers.

This ordering matters for a table containing a sequence, a sample identifier, and many metadata fields. A predicate on a date may discard most chunk groups before the sequence locator column is read. A projection of only the sample identifier need not touch the locator at all. Columnar storage therefore reduces both memory traffic and the number of times the sequence storage manager is entered.

The same read state can publish scalar tuple slots or vector slots. The vector slot capacity is 10,000 lanes, which matches the default chunk-group row count. Each projected column owns a `VectorColumn`; the slot also contains a keep mask and logical row numbers. Deleted rows and predicates clear lanes. Residual scalar expressions can force extraction and repacking, but a supported vector path keeps the chunk group as a batch.

12.3. Sequence payload indirection

12.3.1. Why a locator is necessary

A PostgreSQL varlena is a convenient envelope for a compact sequence, but a single ordinary allocation is bounded by `MaxAllocSize`, approximately one GiB. Eager reconstruction also defeats the purpose of a lazy alignment algorithm: an operation that requires a few windows would first copy the whole chromosome or assembly. BioSeqDB therefore treats the parent sequence Datum as a typed address when the payload is externalized.

The storage unit remains alphabet-generic. For a storage-unit width B bytes and an alphabet-specific number L of letters per unit, one fragment contains

$$U_f = 1024 \text{ storage units}, \qquad S_f = 1024B \text{ bytes}, \qquad N_f = 1024L \text{ letters}.$$

For the common DNA4/`uint64` representation this is 8 KiB and 32,768 letters. For another alphabet or unit width, the formula changes without introducing a DNA-only storage path.

12.3.2. The version-3 locator

The current locator payload is a frozen, standard-layout, trivially copyable 40-byte structure. It is copied into the data area of a normal 16-byte `bioseq.seq` header, making the parent stub 56 bytes. The high bit of the header's storage-width byte marks the value as a locator; the low nibble retains the logical storage-unit width. The alphabet code and last-fragment padding remain available in the header.

Table 12.3: Current `seq_locator` version-3 payload.

Offset	Bytes	Field	Meaning
0	1	`version`	Format version, currently 3.
1	1	`bucket_idx`	Length-bucket index.
2	2	padding	Must be zero.
4	4	`shard_idx`	Shard inside the bucket.
8	4	`num_fragments`	Number of fragment rows.
12	4	padding	Must be zero.
16	8	`total_bp_size`	Full logical sequence length.
24	8	`first_sub_row_no`	Row number of fragment zero.
32	8	`source_epoch`	Immutable PostgreSQL-owned content version.

Version 2 already addressed the important geometric problem: the parent row could locate a contiguous run of fragments and report a 64-bit length. Version 3 adds `source_epoch`. An address alone says where bytes were once written. The epoch says which immutable logical source those bytes belong to. Current validation requires a non-zero epoch and checks it against the source catalog before a lazy binding or persistent index is accepted. This prevents a locator or index generation from silently referring to a different content version after lifecycle changes.

The two padding fields are also checked. The expected fragment count and final padding are recomputed from the logical length, unit width, and letters-per-unit. A malformed locator is a data-corruption error, not an invitation to read whatever rows follow its address.

12.3.3. Length buckets, shards, and fragment rows

The production storage manager is instantiated as `sequence_storage_manager<uint64_t,8,42>`. It generates 35 length buckets. The first contains bit widths up to 8. Bit widths 9 through 41 each receive one bucket. The final bucket covers the remaining widths through 64. The bucket is selected from the bit width of the requested size using a compile-time lookup table.

Length bucketing is not a biological classification. It is an operational device. Similar-sized payloads have similar fragment counts and access behavior, while very small and extremely large sequences do not compete for one active shard. A bucket can contain several shards. The whole sequence is placed in one shard and its fragments receive contiguous row numbers, so the address of fragment i is

$$r_i = \texttt{first_sub_row_no} + i, \qquad 0 \le i < \texttt{num_fragments}.$$

The fragment relation is created lazily with one `bioseq.seq(alphabet,width)` column and uses the BioSeq access method. A matching heap sidecar relation stores a bytea blob keyed by the first fragment row number. The sidecar is a storage attachment point; persistent minimizer-index generations and their validation are covered in Chapter 12. The canonical `seq_sub_table` catalog records both relation OIDs.

12.3.4. Concurrency, shard rollover, and lifecycle

Two kinds of transaction advisory lock protect the fragment layout. A shard allocation lock serializes creation or selection of a new shard for a parent-column and length bucket. A writer lock keyed by database, parent storage identity, column, bucket, and shard serializes

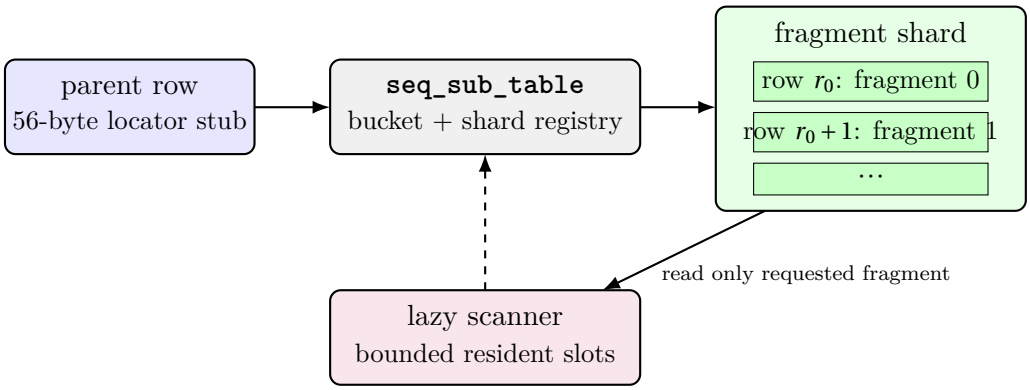


Figure 12.2: Sequence indirection. The parent locator resolves through a canonical registry to contiguous fragment rows; the algorithm sees a lazy scanner rather than an eagerly concatenated value.

fragment row-number assignment. This is a suitable use of advisory locking because the locked identity is an internal storage resource and the lock is held by the transaction that writes it.

Before writing a sequence, the manager computes its projected payload size. If the active shard lacks enough remaining configured capacity, it selects a fresh shard first. A sequence is not split across shards. If the sequence itself exceeds the configured per-shard cap `bioseq.storage.max_seq_sub_table_gbs`, insertion fails with a diagnostic rather than producing an ambiguous multi-shard locator. Byte deltas are accumulated in the per-execution cache and committed to the registry when write states close.

Dynamic fragment relations are deliberately not installed as extension members. An early approach attempted to alter extension membership from nested SPI and resource-owner contexts and could provoke severe lifecycle failures. The current design instead treats `seq_sub_table` as the canonical registry. Parent-table drop hooks remove associated relations, and an orphan reaper handles mismatches in either direction. This makes cleanup an explicit storage-manager responsibility.

12.3.5. Lazy fragment acquisition

The per-execution `seq_caches` object is a sorted small map keyed by column attribute number, bucket, and shard. Each entry retains a relation and the appropriate BioSeq read or write state. It is not thread-local and it is not embedded by value in a `palloc0`-allocated C-shaped structure. The latter was a critical defect found in the Phase A/B/C review: raw zeroed memory does not construct a C++ container. Current scan and write structures hold an opaque pointer to a correctly constructed cache.

On the read side, `sequence_scanner_pg` resolves a fragment row through that cache and copies its packed storage units into a scanner-owned PostgreSQL memory context. It has 16 fixed slots. For DNA4 with 64-bit storage units, the maximum packed fragment payload resident in those slots is approximately

$$16 \times 1024 \times 8 = 131{,}072 \text{ bytes.}$$

The lazy sequence layer normally evicts more aggressively, but the scanner capacity supplies a hard local bound. A request contained in one resident fragment can return its pointer directly.

A request crossing a boundary is stitched into caller-supplied scratch storage. Algorithm-specific encoded views use a similarly bounded slot discipline. The result is that logical sequence length and resident fragment memory are decoupled.

This does not prove that an entire alignment is constant-memory. Anchor collections, indexes, and dynamic-programming workspaces need their own budgets. `DOC0276` is important precisely because it found a target index whose linear memory growth could force a 16 GiB machine into swap even though its input was fragmented. The later segmented path bounded that consumer. The storage claim is narrower and testable: acquiring sequence bytes does not require constructing one full payload.

12.4. Caches with different scopes

The word "cache" appears in three layers of BioSeqDB. Treating them as one feature obscures both correctness and performance.

Table 12.4: Cache-like mechanisms in and around sequence storage.

Mechanism	Scope	Key	Stored object
Lazy fragment and `seq_caches`	One execution/backend context	Column, bucket, shard, fragment	Open relations/read states and bounded fragment bytes
Column decompression cache	One PostgreSQL backend	Relation, stripe, chunk group, column	Decompressed column value buffer
`bioseq_cache`	Shared across backends in one postmaster	Exact execution identity or reusable coverage identity	Sealed columnar query results and coverage fragments

12.4.1. Backend-local decompression cache

The reader's decompression cache is implemented in `pg/table/io/cache.cppm`. Its key contains relation, stripe, chunk, and column identifiers. Only compressed chunks benefit, because an uncompressed chunk can already use its raw buffer. The cache lives below `TopMemoryContext` in one PostgreSQL backend; another backend cannot see it. The default is disabled, with a configured size of 200 MiB when enabled.

Entries are maintained in an intrusive list and account for their allocated bytes. On reaching the configured ceiling, eviction removes enough least-read data to return to approximately 90 percent occupancy. Chunk groups currently marked in use are not evicted. A cache hit returns a borrowed buffer, not a fresh copy, so the caller must respect the entry lifetime. This cache is useful when one backend repeatedly scans compressed metadata, but it does not coordinate duplicate expensive SQL requests.

12.4.2. The separate query-result cache extension

`bioseq_cache` addresses the latter problem. It is an independently buildable PostgreSQL extension and DSO under `main/bioseq_caching`. It does not require the BioSeq alphabet, minimizer, or alignment modules. Optional provider interfaces add BioSeq-specific reusable boundaries without making the two DSOs link to each other.

The user-visible optimization GUC has three modes:

`disabled` No cache path is created, no request claim is made, and execution follows the ordinary plan. This is the default.

`same` The first executor of one exact execution identity is the producer. Concurrent identical executors wait, and later identical executors replay the completed ordered multiset while it remains valid.

`reuse` Exact coalescing remains available, and compatible coverage at an expensive operator boundary may be reused with residual predicates and hidden lineage. If that proof cannot be made, execution falls back to exact behavior.

An exact key is stronger than normalized SQL text. It includes the analyzed query shape, runtime parameter values, the execution and security environment, and a dependency version vector. A final result and reusable coverage are different products. The former can answer only the same execution identity and preserves result order and duplicates. The latter carries enough source and occurrence lineage to re-evaluate a new predicate without confusing SQL bag semantics. A final projection containing only alignment coordinates is not sufficient to evaluate a later predicate on sample metadata.

12.4.3. Claim state and request locking

Request locking is implemented as a shared registry state machine, not as a long-running SQL advisory lock. The legal main path is

$$\text{ABSENT} \rightarrow \text{CLAIMED} \rightarrow \text{BUILDING} \rightarrow \text{COMPLETE}.$$

Claimed or building generations can become failed; building or complete generations can become invalidated; terminal generations pass through evicting before becoming absent. A complete generation is never mutated back into a building generation. A retry receives a new generation identity.

Every entry identity contains a digest, a postmaster boot nonce, a generation, and a monotonically assigned claim sequence. Waiters use PostgreSQL condition variables and check interrupts. They copy state and producer identity under the cache lock, release the lock, check liveness and dependency epochs, and then sleep with a bounded timeout. Cancellation removes the waiter reference without cancelling the producer. Producer cancellation marks its generation failed, wakes waiters, and makes a controlled takeover possible.

Nested cached execution creates a potential deadlock: producer A may execute SQL that requests B, while producer B requests A. The implementation prevents this by allowing a backend to wait only for a claim sequence strictly smaller than every generation it currently owns. Wait edges therefore form a descending directed acyclic graph. No registry or DSA lock is held while sleeping.

The distinction from sequence shard locking is important. Fragment insertion uses a transaction advisory lock for a small internal resource with a 64-bit identity. Query coalescing needs generation state, waiters, readers, dependencies, manifests, failure codes, and ABA-safe identities. The shared registry supplies that information; the PostgreSQL lock table does not.

12.4.4. TTL, publication, and hybrid storage

The second GUC, `bioseq.query_cache_ttl`, defaults to one hour. Age starts when a generation becomes complete, not when its potentially long computation begins. A building request therefore cannot expire before it finishes. TTL zero still permits an executor that

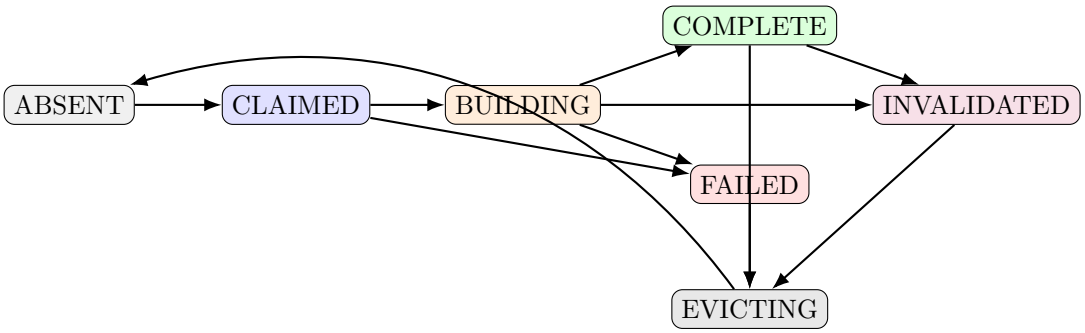


Figure 12.3: Simplified query-cache generation state machine. Publication of `COMPLETE` occurs only after all blocks and the manifest have been sealed and validated.

already registered as a waiter to receive a one-time just-completed entitlement. A session that first discovers the already complete generation receives an immediate miss. The consumer applies the smaller of the producer's captured TTL and its own current TTL.

Publication is ordered. The producer seals every block, seals and validates the manifest and dependency snapshot, stores counts and handles, performs a release barrier, changes state to complete, and broadcasts. A reader performs an acquire load before following the manifest. Reader pins and waiter references prevent eviction while a generation is in use. Generation plus claim sequence prevents an old token from referring to a newly reused slot.

Cached tuples are stored in row groups which are internally columnar. A row group contains a row ordinal interval, one chunk descriptor per column, validity information, offsets for variable-width values, encoded values, and checksums. The current wire layer uses a fixed magic and format version, little-endian metadata, schema identity, PostgreSQL-major and boot identity, block CRC32C, and a manifest digest. Direct encodings are used only for representations proven safe. Varlena values are flattened; external TOAST and expanded-object pointers are never persisted as if they were ordinary bytes.

The backing store is hybrid. A pinned dynamic shared area is the hot tier. Larger results can use cache-owned private files, and a generation manifest names the tier of every sealed block. Admission is reserved with shared atomic counters before allocation. Memory admission observes a cache ceiling and system free-memory floor; file admission observes both a capacity fraction and a free-space floor. If neither tier can accept the next row group, caching fails open: the producer continues the source SQL but publishes no reusable result. A background janitor handles expiration, invalidation, dead owners, unpinned eviction, partial files, and startup purge. Version 1 does not claim that cache data survive a postmaster restart.

12.5. Vectorized processing of scalar metadata

Columnar storage reduces the cost of reading narrow metadata, but PostgreSQL's ordinary executor still evaluates most expressions and aggregate transitions one scalar tuple at a time. BioSeqDB adds a batch representation and a guarded aggregate rewrite for a useful subset of metadata queries. The term "metadata aggregate" here means an aggregate over ordinary scalar columns stored beside a sequence, not an aggregate over packed sequence bytes.

12.5.1. From chunk group to vector slot

`ReadChunkGroupNextVector` fills a vector slot directly from the selected chunk group. Each projected `VectorColumn` contains up to 10,000 values and a NULL mask. The slot carries a keep mask and source row numbers. Vector comparison and Boolean nodes update lanes, and the vector aggregate node invokes a transition function once per vector input rather than once per PostgreSQL scalar tuple.

The current transition implementations are C++ lane loops. It would therefore be inaccurate to claim that every aggregate uses a hardware SIMD instruction. The gain comes from batch transport, fewer executor calls, compact NULL handling, and improved locality. Highway vector kernels exist elsewhere in the system, but "vectorized aggregate" describes the executor interface rather than promising one machine instruction per operation.

The generated aggregate registry recognizes 16 scalar-to-vector associations:

- `count(*)` and `count(any)`;
- `sum`, `avg`, `min`, and `max` for `smallint`, `integer`, and `bigint`;
- `min` and `max` for `date`.

Integer averages retain PostgreSQL-compatible numeric finalization. `bigint` sum and average use PostgreSQL's 128-bit aggregate state rather than silently narrowing the sum. The registry validates the scalar aggregate, vector aggregate, transition and final functions, and input, state, and result types. It does not infer compatibility merely from a `v` prefix in a SQL name.

12.5.2. Planner eligibility and safe fallback

The planner hook is intentionally conservative. It considers a SELECT with exactly one range-table entry and requires vectorization to be enabled. It replaces only a plain PostgreSQL `Agg` whose child can publish a BioSeq vector tuple slot. DISTINCT aggregates and aggregate FILTER clauses are not rewritten. Unsupported arguments, functions, or plan shapes retain the scalar aggregate.

Two exclusions prevent a type-confusion failure. `BioseqSliceProject` publishes ordinary scalar Datums, so its output cannot be interpreted as a `VectorColumn`. A BioSeq scan that owns an `mmsearch` expression publishes one scalar result row per alignment occurrence. Aggregates depending on that projection or executor-owned search also remain scalar. The planner can support joins for alignment through other custom-scan machinery, but this particular vector-aggregate rewrite currently rejects a statement whose range table has more than one entry.

The vectorized aggregate is therefore an optimization, not a second SQL semantics. Unsupported cases fall back to PostgreSQL's ordinary plan. Chunk-level min/max pruning remains independent: it can reduce the batch set even when aggregation itself is scalar, and a vector aggregate can still process a chunk for which no useful extrema exist.

12.6. What the final storage design changes

The final design differs from the earliest storage prototype in several load-bearing ways.

1. A biological sequence is not assumed to fit in one PostgreSQL Datum. The parent row carries a validated version-3 locator, while fixed-size fragments live in a shard chosen by length bucket.

2. Lazy reading is an algorithm contract, not a special case hidden in one SQL function. The PostgreSQL scanner satisfies the same sequence abstraction used by in-memory and synthetic fragmented inputs.

3. Storage identity includes database, tablespace, parent relfilenode, column, bucket, shard, and immutable source epoch. A raw relfilenode or row address is insufficient.

4. C++ objects with constructors are not placed directly in raw zero-initialized PostgreSQL memory. Cache ownership is explicit, snapshots are registered, and relation/resource-owner discipline differs between readers and writers.

5. The Citus-derived column engine remains responsible for ordinary scalar streams, stripes, pruning, compression, deletion masks, and TableAM behavior. Sequence fragment indirection is an added layer, not a wholesale replacement.

6. Cache names no longer imply identical scope. Fragment slots, decompressed column chunks, persistent indexes, and shared query results have different keys, invalidation rules, and lifetimes.

7. Vectorized metadata execution is guarded by exact plan and type eligibility. Unsupported queries retain PostgreSQL semantics through fallback rather than forcing a batch representation.

These properties make the storage engine useful to the rest of the system. The minimizer and alignment stages can request bounded sequence windows. Metadata filters can prune chunks before those requests. Persistent indexes can bind their generations to immutable source epochs. Concurrent identical queries can coalesce without treating a half-written result as readable. Most importantly, each layer states what it bounds. Fragmentation bounds resident sequence bytes, columnar projection bounds unnecessary stream reads, cache admission bounds retained results, and vector eligibility bounds the set of executor plans that may bypass scalar transitions. The resulting system is more complex than a single sequence blob, but its complexity follows the actual lifetime and concurrency boundaries of PostgreSQL.

12.7. Implementation reference map

Table 12.5 records the principal source locations used for this chapter. These paths are preferable to historical line numbers, because the implementation is still actively refactored.

Table 12.5: Current implementation anchors for the physical storage chapter.

Topic	Current source
Page layout and WAL	backend/pg/column/storage_physical.cppm
In-memory storage structures	backend/pg/column/metadata/types.cppm
Generated storage catalogs	backend/pg/column/metadata/sql_rows.cppm
Stripe writer and reader	backend/pg/table/io/writer.cppm, reader.cppm, reader_chunk.cppm
Chunk pruning and decompression cache	backend/pg/table/io/reader_pruning.cppm, cache.cppm
Locator and bucket mathematics	backend/storage/bucket_math.cppm
Fragment shard manager and scanner	backend/pg/storage_pg/sequence_storage_manager.cppm, sequence_scanner_pg.cppm
Datum serialization	backend/pg/pg_utils/datums.cppm
Vector aggregate rewrite	backend/pg/column/planner_hook.cppm, backend/pg/vectorization/types/aggregates.cppm
Shared query cache	bioseq_caching/core/state_machine.cppm, core/column_format.cppm, pg/shared_registry.cppm, pg/storage.cppm
Citus comparison baseline	citus-main/src/backend/columnar/columnar_storage.c, columnar_writer.c, columnar_reader.c

Chapter 13

Persistent sequence indexes

13.1. An index is derived database state

A standalone mapper can treat its index as a file generated from another file. The user decides when to rebuild it, gives the resulting pathname to the next process, and accepts that the mapper's command-line parameters are part of the file's meaning. Inside PostgreSQL this model is insufficient. A BioSeqDB index belongs to a particular physical relation and sequence column. It was produced from one source epoch, alphabet, minimizer scheme, value width, k, w, occurrence threshold, and backend ABI. A transaction may replace it while another transaction is still reading the previous version. A failed build must not leave an apparently complete object. A query that requests a strictly compatible persisted index must not silently obtain a recomputed approximation.

For these reasons BioSeqDB treats a persistent sequence index as derived database state with an explicit identity. The data plane may be a PostgreSQL relation or, for the newer MPF path, an immutable mapped file, but the control plane remains inside PostgreSQL. Catalog state decides which generation is current; a snapshot determines which descriptor a reader can see; compatibility metadata decides whether the generation can be consumed; and memory permits bound the pieces admitted into a backend. This is more machinery than a conventional `.mmi` file, but it solves a different problem: making indexed alignment participate safely in transactional SQL.

The index subsystem is deliberately separate from the base sequence storage described in Chapter 11. The base relation owns rows and sequence locators. An index sidecar owns a reproducible transformation of those sequence bytes. Deleting or replacing the index does not change the source sequence; changing the source epoch makes an old index ineligible. It is also separate from `bioseq_cache`. The cache remembers a completed SQL request under an execution identity, whereas a minimizer index remembers a reusable search data structure under a source and algorithm identity.

13.1.1. The compile-time registry and SQL policy

The public abstraction begins with `SequenceIndexType`. It describes physical granularity rather than a specific algorithm:

1. `CHUNKED_PER_FRAGMENT` associates one or more chunks with a source fragment and an application-defined sub-key;
2. `INLINE_PER_SEQUENCE` stores one value for a sequence; and

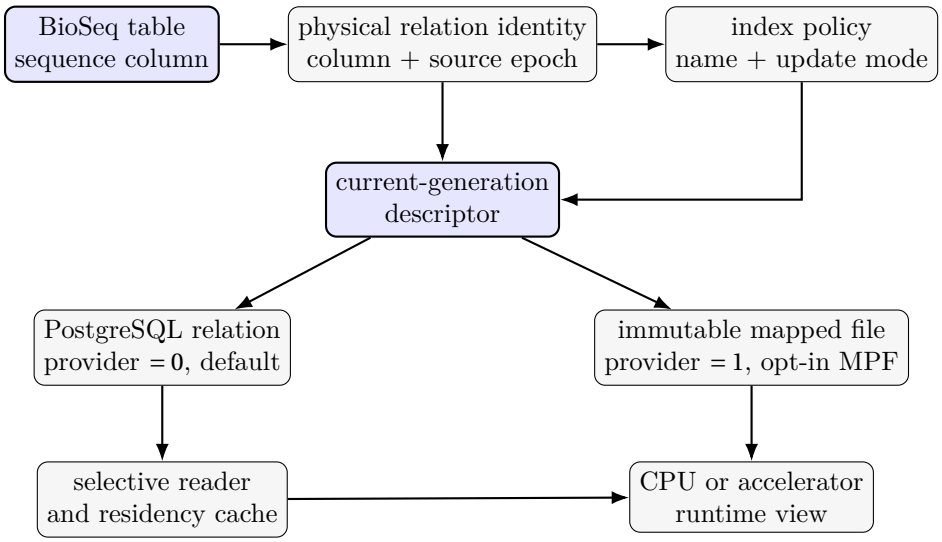


Figure 13.1: Persistent-index control and data planes. The descriptor visible through PostgreSQL selects a provider and binds the payload to source and algorithm identities; the current provider setting is never consulted to reinterpret an existing generation.

3. `CHUNKED_PER_TABLE` stores chunks belonging to a whole table.

Index traits bind an algorithm to one of these layouts at compile time. The PostgreSQL registry currently includes `MinimizersPerFragmentIndex` (MPF), `MinimizerSequenceMergedIndex` (MPS), and `SymbolCountsIndex`; a table-level count index exists only in test builds. Both minimizer implementations use the fragment-chunk storage adapter, while symbol counts use the per-sequence adapter. Static assertions verify that each registered algorithm and storage class agree about their layout. This prevents a name added to SQL from accidentally selecting a class whose row key has different semantics.

Table 13.1: Current registered indexes and default policy. "Automatic" means that the policy is enabled on insertion; it does not mean that every read is forced to use the resulting representation.

Index name	Layout	Default update	Role
`minimizers_per_fragment`	chunked per fragment	on insert	Default active read index; preserves fragment-native construction and permits direct assembly of the target lookup.
`minimizers_per_sequence`	chunked per fragment	manual	Merged sequence-wide, hash-partitioned buckets with a directory for query-selective loading.
`symbol_counts`	inline per sequence	on insert	Compact metadata used by sequence-aware planning and filtering rather than anchor lookup.

Update policy and read policy are intentionally distinct. An index may be `UPDATE_ON_INSERT` or `UPDATE_MANUALLY`. At read time the query contract can allow a persisted index, require its

absence, or require a compatible persisted index. The default active minimizer read name is MPF. The setting that permits stored-index reads controls whether persistence is consulted; it does not dictate the transient in-memory shape used after loading. This separation permits experiments with MPS or accelerator views without changing the SQL-visible index name, and it supplies a strict mode for experiments that are intended to measure cold persistence rather than hidden recomputation.

13.2. Two logical minimizer organizations

13.2.1. Minimizers per fragment

MPF is closest to the way sequence data enters the pipeline. Each source fragment is sketched independently, with the overlap rules described in Chapter 5, and serialized as a `MinimizerHashmapFragmentIndex`. Its in-memory representation has two contiguous components:

- a hash table from minimizer hash to a packed (*offset, count*) pair; and
- a positions array in which an occurrence is encoded as $(\mathit{globalPosition} \ll 1) \mid \mathit{strand}$.

The version-2 legacy serialization begins with a 32-byte header whose magic is `0xB1000001`. Every occupied logical bucket is a 16-byte record {`u64 hash,u32 offset,u32 count`}, followed by the occurrence vector. This representation is attractive at build time: fragments can be produced without one global sort, and each unit has an independent source key and checksum boundary. At query time, however, multiple fragments must be assembled or adapted into the lookup shape expected by the pipeline. The current fragment-native path reduces that conversion and can derive either a CPU lookup or a GPU-resident view without changing the source sequence.

13.2.2. Minimizers per sequence

MPS was introduced to remove the cost of repeatedly merging all fragment indexes for a long logical sequence. Instead of one serialized hash table per fragment, it stores hash-ordered buckets for the whole sequence. A compact directory maps hash intervals to storage sub-keys:

$$D_i = (h_i^{\mathrm{lo}}, h_i^{\mathrm{hi}}, s_i),$$

where each current version-2 entry occupies 24 bytes {`u64 lo,u64 hi,i64 sub_key`}. The current directory header is 40 bytes and uses magic `0xB1000002`. Directory and bucket sub-keys occupy a reserved namespace so that temporary runs, position extents, dense seals, generation heads, and the final directory cannot collide.

MPS construction scans 32,768-base sequence ranges with the required halo, extracts globally positioned minimizer pairs, sorts them into runs, and performs a k-way merge. Buckets target approximately 8 MiB. An unusually frequent hash is allowed to cross a physical chunk boundary so that one heavy hitter cannot force unbounded memory. Most importantly, the directory is written last. Its presence is a completeness witness: readers do not infer a valid MPS index from a prefix of buckets left by a failed build.

The directory makes the cold query path selective. The reader first deduplicates the query's minimizer hashes. Binary search over the directory identifies only the intersecting bucket sub-keys. Those buckets are loaded, global occurrence limits are applied, positions are

clipped to the requested target interval, and a transient lookup is assembled. In a DOC0301 experiment the query selected 4,227,179 of 523,428,324 target minimizers, or 0.8078 percent. That result demonstrated the value of query selection, but the specific A16 path was transient and persisted nothing; it is not evidence for a durable MPS build.

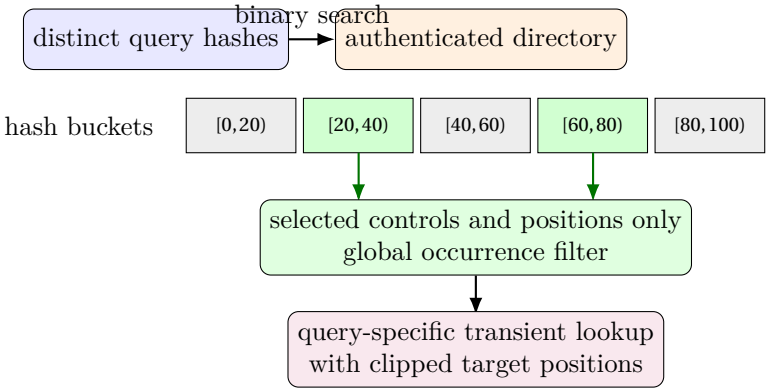


Figure 13.2: Query-selective MPS loading. The directory is small enough to authenticate and search as a whole; only buckets intersecting the query hash set are admitted. Format 6 generalizes the same principle to separate directory, control, and postings streams.

13.2.3. Why both organizations remain useful

MPF and MPS are not a simple old/new pair. MPF aligns with the fragment-native storage and ingestion pipeline, avoids a sequence-wide external sort, and is the current default. MPS moves more work to construction so a query can avoid touching unrelated hash ranges. The appropriate choice depends on source fragmentation, query selectivity, update frequency, and whether the dominant cost is construction, cold admission, or runtime conversion. This is why the registry keeps both names and why persistence preference is explicit: if both deferred-persistence policies are enabled, current dispatch chooses MPF first rather than racing two incompatible writes.

13.3. Bounded construction and publication

13.3.1. The failure of an apparently simple global sort

The first sequence-wide builder accumulated a large vector of $(\mathit{hash}, \mathit{position})$ pairs, sorted it, and emitted buckets. That algorithm is easy to state and performs well while the vector fits in memory. It is not a safe PostgreSQL operator for multi-gigabase targets. The early 4 Gbp external-sort experiments recorded approximately 80 GB of temporary storage for dna4 and 93 GB for dna5; a 6 Gbp run exceeded 120 GB. The lesson was not merely to increase a buffer. A database operator needs a resource envelope that does not depend on the target being friendly.

DOC0299 replaced the unbounded phase with sorted runs backed by PostgreSQL `BufFile`. The wire pair is exactly 16 bytes:

$$P = (\texttt{u64 kmer}, \texttt{u64 encodedPositionAndStrand}).$$

Pairs first occupy a memory-bounded run. A full run is sorted and may remain resident only while the run-residency budget permits; otherwise it is spilled. The merge opens a bounded

number of readers and combines runs in passes. The current sequence builder also spills the accumulator for a single extremely frequent k-mer, so neither the number of runs nor a heavy hitter can bypass the memory contract.

Algorithm 13 Bounded MPS construction and directory-last publication

```
Require: source sequence identity I, parameters k, w, memory budget
reserve a new build identity and snapshot the selected provider
for each 32,768-base source range with minimizer halo do
    sketch the range and convert local occurrences to global positions
    append 16-byte (hash, encodedPosition) pairs to the active run
    if the run budget is full then
        sort the run; retain it only if a residency permit is available
        otherwise spill it to a PostgreSQL BufFile

finish and sort the last run
while more runs exist than can be opened simultaneously do
    merge one bounded fan-in group into a new run

merge remaining runs in hash order
form approximately 8 MiB buckets; spill a heavy-hash accumulator when necessary
write every bucket with length, checksum, source, and compatibility identity
write the ordered directory last
validate the complete unpublished generation
atomically publish its descriptor as the current generation
```

Bounded does not automatically mean fast. DOC0300's first implementation used 15 sorted runs, of which 14 were resident and one was file-backed. It retained 3.14 GiB and spilled 4.976 GiB, yet the measured region took 455.452 s (456.715 s wall time). PostgreSQL accounted 33.148 GiB of temporary traffic, 440,117 temporary files, and six merge passes. This result was valuable precisely because it separated semantic success from a viable build: the algorithm completed under a bound, but its file granularity and merge amplification were unacceptable.

13.3.2. Synchronous and identity-only asynchronous work

An index build performs two qualitatively different kinds of work. Sketching, sorting, and serializing are ordinary C++ computation. Opening relations, registering snapshots, reading tuples, and publishing catalog state are PostgreSQL operations that must run in a valid backend or background worker. Earlier scheduler proposals risked obscuring this boundary by treating the build as a generic task payload.

The current asynchronous MPS route uses an identity-only job. The submitting backend transfers stable identifiers and parameters, not live PostgreSQL pointers, relation handles, snapshots, or memory-context objects. The PostgreSQL worker rechecks relation identity and policy, registers its own snapshot, reopens the source, and rebuilds the MPS representation. The mapped provider is rejected for this MPS asynchronous route. Current settings also separate `index_async_persist` from `recompute_async_persist`; both must permit the deferred write, and the production gate is restricted to the supported $k = 15, w = 10$ profile.

This model has an intentional cost: the worker reads and sketches the source again instead of consuming a large staged payload. In exchange, cancellation, transaction abort, and backend exit cannot leave a detached task holding backend-local state. The design follows

a general rule used throughout BioSeqDB: transfer immutable values across the scheduler boundary and reacquire PostgreSQL capabilities on the thread that owns them.

13.4. Physical persistence in PostgreSQL

13.4.1. Legacy sidecar relations

The original storage adapters materialize ordinary PostgreSQL relations in the `bioseq_internal` schema. Their names are derived from the physical identity of the parent relation together with the index name, bucket, and shard. This is more robust than deriving a name only from the user-visible table name: rename does not change the relation identity, and drop/recreate does not make a new table inherit the previous table's index accidentally.

The three generic layouts have deliberately small schemas:

Table 13.2: Legacy physical adapters. The minimizer implementations use the first row; the remaining rows show why the abstraction is not minimizer-specific.

Layout	Columns and primary key	Storage properties
Chunked per fragment	`query_key BIGINT`, `sub_key BIGINT`, `chunk_no INT`, `chunk_bytes INT`, `blob BYTEA`; primary key (*query_key, sub_key, chunk_no*)	Heap relation; `blob` uses external storage. Supports MPF and MPS directories/buckets.
Inline per sequence	`query_key BIGINT`, `chunk_bytes INT`, `blob BYTEA`; primary key (*query_key*)	One logical object per sequence, with external storage for the byte value.
Chunked per table	`chunk_no INT`, `chunk_bytes INT`, `blob BYTEA`; primary key (*chunk_no*)	Whole-table abstraction retained and tested, but no production index currently uses it.

Chunking has two purposes. First, a serialization can exceed PostgreSQL's practical datum limits. Second, a reader can allocate, checksum, and release bounded units. The legacy maximum is one GiB minus a 64 MiB safety margin, but newer generation formats deliberately choose much smaller physical payloads. Lengths in the row are checked against the actual bytea payload before a chunk is accepted. The application-level sub-key is not a physical tuple number: for MPS it identifies directory or hash-range material, while MPF can use it to identify logical sections of a fragment image.

The first update path used replace-in-place semantics. It cleared an old sidecar and wrote the replacement. Directory-last publication prevented a partial MPS build from looking complete, but readers and writers still shared one mutable physical namespace. This model remains useful for the generic legacy adapters; it is not the strongest lifecycle now available.

13.4.2. Immutable generations

The generation design changes the unit of publication. A writer allocates a positive generation identifier and appends all of its rows without modifying the generation visible to current readers. It validates the finished object and only then atomically replaces a distinguished generation-zero head. The old descriptor becomes a predecessor or retired generation rather than being overwritten under a reader.

The PostgreSQL-relation provider uses an eight-column row:

(`query_key`, `generation_id`, `position_segment`, `sub_key`,
`chunk_no`, `chunk_bytes`, `checksum`, `blob`).

The relation is `UNLOGGED`, has heap fill factor 100, and stores `blob` as `PLAIN`. Its single unlogged B-tree primary index has key

(`generation_id`, `query_key`, `sub_key`, `position_segment`, `chunk_no`)

and fill factor 90. Creation uses the source relation's tablespace. The reader inspects the relation descriptor and rejects a sidecar whose number, order, types, nullability, storage modes, access method, or primary-key definition do not match the expected schema.

The choice of an unlogged relation is deliberate. A minimizer index is reproducible from durable sequence data, and logging every derived payload byte would duplicate write amplification. Unlogged is not the same as unchecked: generation identity, manifest data, lengths, and checksums still detect partial or misaddressed content. After an unlogged reset, the generation must be rebuilt rather than treated as valid. The 8064-byte Format-5 row payload was selected so a PLAIN bytea and its tuple header fit on one ordinary PostgreSQL page, avoiding a TOAST fetch for every small cold-read unit.

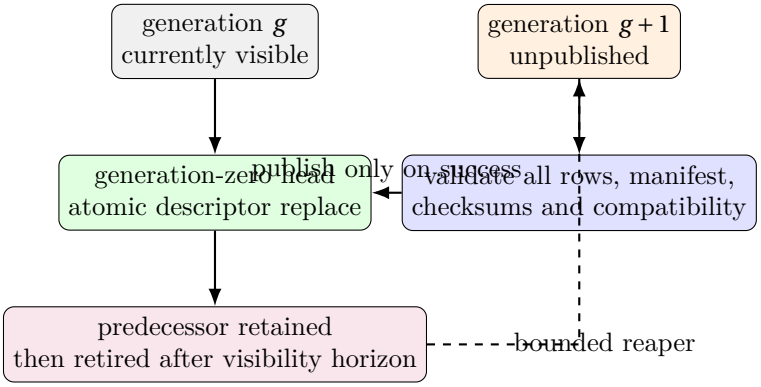


Figure 13.3: Copy-on-publish generation lifecycle. A failed build can leave removable unpublished material, but it cannot make a partial generation current. Readers that resolved the predecessor under an older snapshot may finish before reclamation.

13.4.3. The optional mapped-file provider

The storage history contains an important reversal. DOC0381 rejected an mmap fallback under a strict interpretation that every index byte had to reside in a PostgreSQL relation. Its diagnosis was nevertheless decisive: a cold `just_index` run spent 103.839560417 s in index-and-map work versus 17.168589792 s for minimap2, a 6.048x region disadvantage. Of the BioSeqDB time, 62.519740125 s read 4,463,431,732 payload bytes from 562,075 rows and 41.319820292 s remained outside raw payload fetch. The old accounting loop made 103,003 passes over the extent set—about 1.768 billion visits—and a second checksum pass reread the same 4.46 GB. The primary-index route then performed roughly 562,000 heap fetches. PostgreSQL ownership alone did not make this tuple shape an efficient large immutable image.

DOC0383 reframed ownership more precisely. PostgreSQL 17 does not expose a supported extension hook for a custom storage manager, so forcing a tuple-per-page emulation was not the only safe option. BioSeqDB could keep the lifecycle and authority in PostgreSQL while placing the immutable MPF data plane in an extension-owned file. DOC0384 records that implementation. `postgres_relation` remains provider value 0 and the default; `mapped_file` is provider value 1 and opt-in. A generation descriptor stores its provider, and every read

obeys that stored value rather than the current configuration setting. Changing a GUC can affect a future build but cannot reinterpret an existing descriptor.

The mapped MPF image has an explicit little-endian MPF5 version-2 layout and a 4096-byte header. Unlike a dump of native C++ objects, its offsets, lengths, alignment, reserved bytes, and integer widths are format contracts. The open path validates the header and identity without performing an eager CRC over several gigabytes. When a selected extent is used, its address-bound checksum is verified. Thus a cold reader pays integrity cost for the material it actually admits rather than hashing the entire index twice before the first lookup.

Publication follows a filesystem protocol designed to make the descriptor truthful:

1. create a private temporary file with mode 0600 in the extension-owned database/tablespace directory;
2. write the complete image, finalize its header, and synchronize it;
3. publish with a no-replace operation so an existing generation cannot be silently overwritten;
4. synchronize the containing directory, record device/inode identity, and make the immutable artifact mode 0400; and
5. publish the PostgreSQL descriptor transactionally.

Mappings are registered with a PostgreSQL `ResourceOwner`, so error unwinding closes them even when normal C++ control flow is bypassed. Transaction callbacks use a fixed-capacity action queue rather than allocating or throwing inside a callback; prepared transactions are rejected while such actions are armed. Replacement records a full transaction identifier. The reaper waits until PostgreSQL's global visibility test says that identifier is removable, then reclaims the retired descriptor and artifact. Its work is bounded per pass: at most 64 descriptors, 4096 relation rows, 64 MiB of relation payload, 64 files, and 64 orphan candidates. Readers remain lock-free after resolving the descriptor under one registered snapshot; writers, replacement, clear, and reaping are serialized by an advisory transaction lock.

This design should not be described as "an arbitrary external cache file." The file is immutable, privately named, identity checked, transactionally described, visibility-delayed on deletion, and unable to select itself. It is an extension-owned data plane governed by PostgreSQL. Conversely, it should not be described as PostgreSQL heap storage: backups, replication, crash handling, and administration must account for the provider distinction.

13.5. Formats 5 and 6: from complete images to selective streams

13.5.1. Shape-aware Format 5

The generation codec evolved through several shapes. Format 3 used sorted dense controls with global 64-bit positions. Format 4 grouped 32-bit hashes and local 32-bit positions. Format 5 stores a grouped, structure-of-arrays shape suitable for direct CPU or accelerator assembly. Its preferred logical segment is approximately 8 MiB, while the PostgreSQL relation provider splits that segment into 8064-byte physical rows. A 64-byte generation head points to manifests; Format-5 manifests and directory entries carry the additional section sizes and shape metadata required to authenticate the image.

The essential change is that the persistent shape is no longer merely "some bytes that deserialize into a C++ hash map." It is a specified arrangement of controls and positions that can be range checked, checksummed, and adapted without rebuilding the original container. The same logical Format-5 generation can be backed by PostgreSQL rows; the MPF mapped image carries a closely related direct layout with a provider-specific outer header.

13.5.2. Hash-major Format 6

Format 6 makes query selectivity a physical property. It separates three storage-neutral streams:

1. an authenticated directory over hash ranges;
2. control blocks containing up to 4096 hashes and global occurrence counts; and
3. postings extents containing positions for hashes that survived the occurrence policy.

The fixed head is 224 bytes and the manifest is 336 bytes; a directory entry is 80 bytes. PostgreSQL payload rows remain 8064 bytes and the reader coalesces authorized ranges into read slabs of at most 8 MiB. It first validates the complete directory, intersects query hashes with directory ranges, reads only the corresponding control blocks, rejects hashes above `max_occ`, and authorizes only the resulting postings intervals. Exact-count exception data preserves correctness where a compact count field would otherwise saturate.

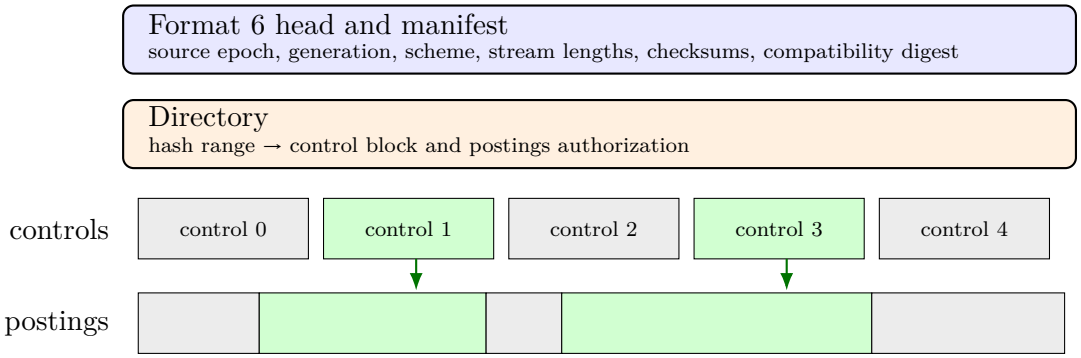


Figure 13.4: Format 6 separates navigation, hash controls, and variable-length postings. Green regions illustrate the blocks authorized by one query; unselected postings need not enter the backend.

The current implementation routes the supported MPS dna4 $k = 15, w = 10$ profile to this format only when its compatibility registration and grouped generation path are enabled. MPF remains on Format 5. The grouped persisted generation setting defaults to false in the current source, so Format 6 is an implemented specialized route, not the unconditional default. Unsupported or unregistered profiles fall back to the compatible Format-5 path rather than guessing a wire identity.

13.5.3. A rejected per-query occurrence merge

Selective loading introduces another temptation: compute target occurrence counts separately for every query and merge them on demand. DOC0385 compared that candidate with the compact global query-occurrence radix used by the current path. On its canonical

case $(N, R, U) = (7{,}440{,}026, 7{,}258{,}660, 7{,}258{,}562)$, the compact global method took 267,837,208 ns and 206,618,624 bytes peak RSS, while the per-query merge took 516,441,416 ns and 294,830,080 bytes. The candidate was slower and larger on every measured workload and was rejected. This is an example of a local-looking optimization that duplicates work across the SQL batch; the global radix amortizes it while retaining exact occurrence policy.

13.6. Compatibility is part of correctness

Two index images can have identical SQL schemas and still be mutually unreadable. A 32-bit position image cannot represent a target that required 64-bit positions. A hash produced by the ntHash SIMD scheme does not match the legacy minimap-style scheme. A GPU kernel compiled for one control layout cannot safely interpret another merely because both contain $k = 15$ minimizers. BioSeqDB therefore authenticates meaning as well as bytes.

The compatibility contract pins at least:

- alphabet identifier and storage width;
- minimizer k and window w;
- scheme identifier, scheme version, and producer backend;
- position width and stored shape/format;
- occurrence-limit semantics; and
- accelerator ABI when the persistent shape is consumed directly by a device path.

These fields contribute to a compatibility digest carried by the generation metadata. The reader also binds the generation to the expected relation, column/query key, source epoch, and generation identifier. It checks manifest ordering, section lengths, address arithmetic, per-section or per-extent CRCs, and the content identity appropriate to the format. Reserved bytes must have their specified value. Integer addition and multiplication are checked before forming a span, so a corrupt count cannot wrap into an apparently valid address.

The consequence for SQL semantics is deliberate. A read policy of `require_compatible_persisted` fails closed on absence or mismatch. It may not rebuild an index and continue, because that would invalidate a cold load benchmark and can violate a caller's operational requirement. An `allow` policy may fall through to recomputation. A `require_absent` policy supplies the converse experimental contract: the query must exercise the no-index route.

This strictness also protects accelerator dispatch. A GPU-resident view is derived only after host-side format and compatibility validation. The Vulkan worker receives plain spans and immutable descriptors, not PostgreSQL tuple objects. If a device view is cached, its key includes the persisted assembly identity or the exact in-memory shape. A superficially similar generation cannot alias the old device allocation.

13.7. Selective cold loading and memory accounting

13.7.1. The useful numerator is admitted memory

Reporting the serialized index size is not enough to establish that a database query is bounded. During a cold read the backend can simultaneously own a directory, control rows, postings, decoded occurrence arrays, a transient hash lookup, query-occurrence tables, I/O slabs, and

Table 13.3: Examples of mismatches and the safe response.

Observed mismatch	Why bytes cannot be reused	Response
Different source epoch	The same logical row key can now name different sequence bytes	Reject generation; rebuild only when policy permits.
Same k, w, different hash scheme	Query and target keys inhabit different hash languages	Reject; no conversion can recover omitted minimizers.
Unsupported position width	A decoded occurrence can truncate or wrap	Reject before allocation or device upload.
Current GUC names a different provider	The generation's data plane does not move when configuration changes	Use the provider stored in its descriptor.
Checksum or section-bound failure	Payload is corrupt, misaddressed, or incomplete	Reject the selected extent/generation; never continue with partial postings.
Strict persisted requirement, no generation	Recompute would change the experiment and operational contract	Raise an error rather than falling back.

accelerator staging. Some allocations are visible to PostgreSQL memory contexts, some belong to C++ containers, and some are device memory. A correct budget must cover the high-water overlap, not just the largest final vector.

BioSeqDB uses explicit memory permits for the significant owners. Format 6 reports separate evidence for directory, control, and posting admission. Readers authorize an extent before allocating its destination. Merge and construction code similarly accounts for resident sorted runs and spills the next run when a permit cannot be obtained. Accelerator-resident objects own their permit for the complete lifetime of the device allocation rather than returning it immediately after upload.

The DOC0381 diagnosis illustrates the difference between selective logical content and inefficient physical access. Its workload selected 17,166 extents and approximately 251.2 million controls from a generation containing 14,171,377 query hashes. After the occurrence filter it retained 47,948,924 positions and reported 964,465,532 bytes of admitted/owned memory. These figures show that the selective algorithm was already avoiding a full decoded position set. Nevertheless, the relation provider performed hundreds of thousands of small index and heap operations and repeated whole-generation accounting. Logical selectivity did not compensate for an unsuitable physical read path.

The corrective design therefore attacks both axes:

1. directory and control structures make the authorized postings set smaller;

2. accounting walks each extent a constant number of times instead of rescanning the whole set for every batch;

3. adjacent physical ranges are coalesced into bounded slabs;

4. checksums are computed at the same granularity as authorization; and

5. the optional mapped MPF image removes tuple and heap-fetch overhead where a complete directly consumable image is the better representation.

13.7.2. Backend-local residency caches

Cold persistence does not imply that every query must reread every selected chunk. The PostgreSQL backend maintains two bounded LRU caches:

- a chunk cache of at most 256 entries and 2 GiB; and
- an assembled-index cache of at most 32 entries and 4 GiB.

A chunk key includes the full relation and index identity, bucket, shard, query key, sub-key, and chunk number. An assembled key additionally includes backend/format shape, scheme, query signature, and clip interval. Values hold memory permits for their resident bytes. The key breadth is intentional: a cache entry must not survive a source generation or algorithm change merely because a short SQL name remains equal.

Transaction, subtransaction, and relcache callbacks maintain coherence. A write recorded in an aborted transaction is evicted; commit marks eligible entries stable. Relation invalidation and explicit index-identity invalidation remove affected entries. This is a backend-local acceleration, not a cross-backend consistency mechanism: PostgreSQL snapshots and generation descriptors remain authoritative.

GPU residency follows the same idea at a different boundary. The Phase-B implementation assembled a GPU view from existing serialized buckets and kept it in an LRU keyed by the in-memory shape or persisted assembly identity. That first implementation was 5–8x slower on small cases and 1.8x slower on its XXL case, demonstrating that residency alone does not erase conversion and launch overhead. Phase C replaced a problematic open-addressed device table that could trigger device loss with sorted dense buckets and binary search. On its later XXL measurement the GPU route took 7.558 s versus 9.501 s for SIMD, a 1.26x improvement, with 93,309,652 target positions and no fallback. Those numbers characterize an accelerator front-end, not index persistence, but they explain why persistent metadata includes enough shape identity to make safe direct residency possible.

13.8. Runtime selection

At execution time the scheduler does not simply ask "does a table named `mpf` exist?" It resolves an effective index policy for each sequence column and combines it with the query's read requirement, algorithm compatibility, source identity, memory admission, and backend availability. The main decision shape is:

An absent index is thus not always an error, and a present index is not always usable. This distinction is essential to benchmarks. The `no_index` case should prove that absence was required; the `just_index` case should prove that a compatible persisted generation was used and that construction was outside the measured region. A fast row without these attestations can be a recomputation result, a warm assembled cache, or a different algorithm.

The default active read name is MPF. Stored MPF is assembled from fragment images or consumed through the supported direct provider. Stored MPS uses its directory to form a query-specific index. If policy allows recomputation, both routes can fall back to a transient target index. Deferred persistence scans the enabled preference list in MPF-then-MPS order; MPF wins if both are eligible. This deterministic priority prevents two background products from being presented as one implicit "best index."

Algorithm 14 Persisted-index selection in the alignment dispatcher

```
Require: source identity I, query sketch Q, active index policy P
resolve the active read name for the source column
inspect a visible persisted generation only if stored reads are enabled
if a generation is present then
    validate source epoch, algorithm digest, format, lengths and checksums
    if the active name is MPF then
        assemble or map the fragment-native lookup under memory permits
    else if the active name is MPS then
        load the directory, selected buckets/controls, and authorized postings
        build the query-specific clipped lookup
    optionally obtain a compatible cached CPU or accelerator view
else if P requires a compatible persisted index then
    fail closed
else
    sketch the target and build a transient index
    optionally schedule identity-only persistence when all gates permit it
execute seed lookup; retain the selected generation/view until the batch ends
```

13.9. Comparison with minimap2's `.mmi` index

The bundled minimap2 source provides a useful algorithmic baseline because the BioSeqDB seed lookup derives from the same mapping family. Its `mm_idx_t` records bucket bits b, window w, k, flags, sequence metadata, an optional packed reference S, and an array B of buckets. A bucket contains an array of minimizer/position pairs during construction, an open-addressing hash table, and a positions array.

Lookup partitions a minimizer hash with the low b bits selecting the bucket. The remaining high key is found in that bucket's hash table. A low marker bit in the stored key distinguishes a singleton: its one position is stored directly in the hash value. A repeated minimizer stores a packed $(offset, count)$ value whose occurrences occupy the bucket's `p` array. Each occurrence encodes reference identifier, position, and strand. Construction radix-sorts minimizer/position pairs and sorts the multi-hit position lists.

The `.mmi` writer serializes a magic and five 32-bit parameters, sequence names and lengths, every bucket's position array and hash entries, and optionally the packed reference. The loader in the bundled implementation uses `fread`, allocates all buckets and position arrays, and reconstructs the complete in-memory index; it is not a selective mmap reader. Index parameters stored in the file can override command-line values, with a warning. This format is compact and practical for a command-line mapper whose unit of reuse is a reference file.

BioSeqDB's legacy MPF layout resembles minimap2 at the conceptual level: a hash maps to an offset/count and a separate array stores positions. The important differences are not cosmetic serialization choices. BioSeqDB's position has a global sequence coordinate rather than minimap2's packed reference identifier, and the index may begin fragmented because the database stores long sequences in independently addressable pieces. It must also authenticate a minimizer scheme shared with Chapter 5's SIMD and Vulkan implementations. A minimap2 index assumes its mapper controls the surrounding process; BioSeqDB must make the object safe for SQL transactions, lazy source reads, memory admission, and multiple execution backends.

Table 13.4: Persistent-index responsibilities in bundled minimap2 and BioSeqDB. The comparison concerns architecture, not a claim that one wire format is a drop-in replacement for the other.

Concern	minimap2 `.mmi`	BioSeqDB
Ownership	Standalone file named by the caller	Derived object bound to a PostgreSQL physical relation, column, source epoch, and generation descriptor.
Logical shape	Bucket-local hash table; singleton inline, repeated occurrences in `p`	MPF fragment images or MPS hash-range directory; generation Formats 5/6 provide specified controls and positions streams.
Load granularity	Reconstruct complete bucket arrays and optional reference with `fread`	Full MPF image when appropriate, or query-selected MPS/Format-6 controls and postings; bounded admission and backend LRU reuse.
Update	User rebuilds/replaces a file	Manual or insert policy; validate unpublished generation, atomic head replacement, snapshot-safe predecessor retirement.
Compatibility	File parameters and flags; mapper-specific load contract	Alphabet, widths, k, w, scheme/version, producer, occurrence policy, source epoch, and optional accelerator ABI digest.
Transactions	Outside database transaction semantics	Descriptor publication, abort cleanup, registered snapshots, `ResourceOwner` mappings, and visibility-delayed reaping.
Physical providers	Ordinary file	Default unlogged PostgreSQL relation; optional PostgreSQL-governed immutable mapped file for supported MPF generations.
SQL batch sharing	One process consumes one loaded index	Query batch shares occurrence analysis and assembled views; planner and custom scan choose the route.

13.10. Design evolution

The implemented architecture is easier to understand as a sequence of constraints discovered by measurement. The chronology in Table 13.5 does not imply that every mechanism remains enabled. It records what each stage taught the next one.

13.10.1. What the current solution actually selects

The phrase "final index format" would be misleading because the current system deliberately preserves more than one verified route. The default policy is nevertheless concrete:

1. MPF is the default active minimizer index and is enabled for update-on-insert. It matches the fragment-native sequence store and avoids a global MPS build.

2. The default generation data provider is an unlogged PostgreSQL relation. The immutable mapped-file provider is implemented and opt-in for the supported MPF path.

3. Format 5 is the generally applicable shape-aware generation format. The specialized MPS Format-6 hash-major path is available for its registered dna4 $k = 15, w = 10$ profile, but the grouped persisted-generation setting is disabled by default.

4. MPS remains a manual index option when query-selective cold loading is worth its construction and merge cost.

5. A strict persisted-read request fails closed. An ordinary request can recompute, and may schedule an identity-only persistence job when the explicit asynchronous gates allow it.

Table 13.5: Evolution of BioSeqDB persistent minimizer storage. Newer documents and current source supersede older status claims.

Evidence	Design at that point	Observed limitation or result	Retained consequence
DOC0279, DOC0282	Typed registry; MPF migration; MPS introduced	One generic "index blob" API could not express fragment, sequence, and table granularity safely	Compile-time algorithm/layout registry and distinct SQL index names.
DOC0288	PostgreSQL ingestion followed by automatic index update	Early persisted reuse was competitive on several 2/4 Gbp cases, but the old large build could consume 80–93 GB of temporary space	End-to-end correctness is necessary; build resource evidence is a separate acceptance condition.
DOC0298	Sequence-wide hash directory and query-selected MPS buckets	Building and merging all minimizers could retain excessive memory; partial output needed a completeness rule	Reserved sub-key namespace, fixed directory records, and directory-last publication.
DOC0299–DOC0300	Memory-bounded sorted runs and PostgreSQL `BufFile` spill	First measured builder produced 440,117 temporary files and six merge passes despite bounded run memory	Bounded fan-in, coarser physical units, spillable heavy-hash accumulator, and explicit run residency.
DOC0301	Durable, lazy, adaptive, and query-filtered variants	Durable A6 completed correctly but took 303.399 s wall; transient A16 took about 26.3 s by retaining 0.8078 percent and persisted nothing	Persistence attestation is mandatory; query selectivity was retained as a design goal, not confused with a completed durable build.
DOC0307–DOC0311	GPU-resident derived view, then dense sorted GPU buckets	First residency path was slower; open-addressing caused device loss; sorted binary search later beat SIMD on its XXL case	Persistent compatibility includes shape/ABI; device representation is derived only after host validation and owns a residency permit.
DOC0371, DOC0373	Immutable PostgreSQL generations and shape-aware compact layouts	Replace-in-place and generic blob deserialization could not give snapshot-safe publication or bounded selective cold reads	Generation head, manifests, source epoch, compatibility digest, selective controls/postings, unlogged one-page payload rows.
DOC0381	Instrumented cold PostgreSQL relation reader	103.840 s index/map region, quadratic accounting, duplicate CRC, and about 562,000 heap fetches	Linear accounting, coalesced ranges, address-bound CRC; reconsider physical provider rather than optimizing only logical selectivity.
DOC0383–DOC0384	Fragment-native MPF plus PostgreSQL-governed mapped generation	Implemented six-run P50 5.368746 s versus minimap2 16.487527 s, narrowly satisfying a 3x gate under documented caveats	Default PostgreSQL provider plus opt-in immutable mapped MPF provider, transactional descriptor, snapshot-safe retirement, bounded reaper.
DOC0385	Per-query occurrence-merge prototype	Slower and higher RSS on every tested workload	Keep compact global query-occurrence radix for a batch.

This is the point at which the BioSeqDB design differs most clearly from its original minimap2 inspiration. Minimap2 chooses one self-contained reference index file whose internal buckets are rebuilt into process memory. BioSeqDB chooses an index policy tied to relational identity. It can preserve fragment-native construction, publish immutable database generations, authorize only query-relevant postings, account memory across a SQL batch, and derive a device view. The cost is a larger compatibility and lifecycle surface.

13.11. Performance evidence

13.11.1. Early evidence: persistence could help, but did not solve building

DOC0288 measured a PostgreSQL E2E workflow after the loader's flush/finish states triggered `bioseq.update_index` through SPI and reset the backend cache. Table 13.6 reproduces its `just_index` results. They are historical results from that implementation, not current-provider claims.

Table 13.6: DOC0288 historical `just_index` results. Speedup is minimap2 time divided by BioSeqDB time. Values are mean ± reported variation.

Target	Alphabet	BioSeqDB (s)	minimap2 (s)	Ratio
2 Gbp	dna4	14.29 ± 0.22	21.53 ± 0.30	1.51x
2 Gbp	dna5	15.97 ± 0.51	18.66 ± 1.05	1.17x
4 Gbp	dna4	23.20 ± 0.65	32.22 ± 0.89	1.39x
4 Gbp	dna5	25.65 ± 0.94	25.48 ± 0.55	parity

All checked hits were correct and reported target offsets differed by 2–8 bp. The same report's `no_index` rows were mixed: BioSeqDB was slower at 2 Gbp, but at 4 Gbp it took 171.34 s versus 276.41 s for dna4 and 190.40 s versus 227.36 s for dna5. The report observed approximately 15 GB of swap for minimap2 versus about 2.6 GB for BioSeqDB on the 4 Gbp no-index case. These numbers show the interaction between construction and memory pressure; they do not isolate a wire format.

The apparent success of the `just_index` rows coexisted with the 80–93 GB temporary peaks described earlier. This is why the later work did not stop at "persisted queries are fast." It pursued bounded construction, query-selective reading, immutable publication, and finally a physical provider that did not turn a multi-gigabyte image into hundreds of thousands of heap fetches.

13.11.2. Intermediate results must retain their persistence label

The distinction is central to the thesis's narrative. The fastest intermediate solution deliberately stopped solving storage. It proved that the query did not need most target minimizers. The final storage work then had to preserve that advantage without making construction, integrity verification, or tuple access dominate.

13.11.3. Current mapped-provider evidence

DOC0384 reports six retained BioSeqDB `just_index` samples:

6.389712833, 5.401551458, 5.343038250, 5.368746000, 5.610243750, 5.321436000 s.

Table 13.7: Selected DOC0300–DOC0301 MPS experiments. The final column prevents fast transient experiments from being presented as durable index results.

Run	Intent	Wall / region	Persistence	Interpretation
DOC0300 E1f	First bounded `BufFile` build	456.715 / 455.452 s	durable build path	Bounded resident runs, but 33.148 GiB temp accounting and 440,117 files made the implementation non-viable.
DOC0301 A6	Valid foreground MPS plus wait	303.399 / 91.316 s	650 persisted rows	Correct 2 bp offset; about 210 s was outside the foreground region while persistence completed.
DOC0301 A12	transient no-persist recompute	about 89 s	none	Useful recomputation baseline; not a cold persisted read.
DOC0301 A14	adaptive waves	33.797 s wall	none	Faster scheduling/build shape, but no durable artifact.
DOC0301 A16	query-filtered transient target index	26.312–26.454 / 25.071–25.113 s	none; audit rows = 0	Kept 0.8078 percent of target minimizers and returned the correct 2 bp offset. Demonstrates selectivity only.

Their median is 5.368746 s. The report's minimap2 median is 16.487527 s, giving a 3.071020x ratio. Figure 13.5 displays the medians on one scale.

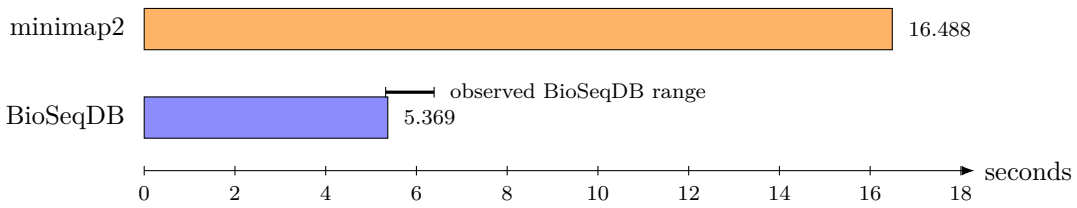


Figure 13.5: DOC0384 six-run median comparison for the implemented mapped MPF path. The thin line shows the range of the six retained BioSeqDB samples; the report supplies a minimap2 median rather than a paired range.

Correctness was exact for all 16 checked cases in match position, strand, and CIGAR. The speed result must retain its caveats. The median was only 0.127096 s below the time corresponding to the report's 3x acceptance limit. The run used a manually assembled, dirty worktree, process-cold operating-system cache state was not fully specified, and not every individual run passed the 3x boundary. A later fresh E2E row in the same report measured 6.080941750 s versus 43.118770584 s (7.0908x in that same run), but it also recorded a wrapper/configuration artifact. It is corroboration that the new path is viable, not a statistically interchangeable replacement for the six-run distribution.

These qualifications are not incidental. DOC0381 had shown a 6.220x outer wall-clock disadvantage for the prior cold relation path; DOC0384 shows that a different physical data plane can reverse the result. Neither number is a universal property of "indexes in PostgreSQL." They measure particular source states, provider paths, and attested execution regions.

13.12. Operational consequences and limitations

Persistent derived state introduces administration that a benchmark process does not face:

- An unlogged relation can be reset after a crash and must be rebuilt from the durable source. The system must not advertise its pre-crash descriptor as reusable.
- The mapped provider needs backup, restore, replication, and tablespace procedures that preserve the private artifact together with its descriptor, or else deliberately discard and rebuild it.
- Source updates need a new epoch and generation. Incremental mutation of an image in place would defeat snapshot safety and compatibility checks.
- The mapped provider currently solves the supported MPF path; MPS asynchronous persistence and specialized Format 6 have narrower gates. A benchmark of one route should not be generalized to every index policy.
- Backend and device caches reduce warm cost but cannot be used as evidence for cold performance unless their state is measured and reported.
- Reaping is intentionally bounded and may take several passes. This prevents cleanup from becoming an unbounded backend pause, at the price of temporary retired generations.

There is also a semantic limitation inherited from deriving a minimizer index. It accelerates seed lookup; it does not turn BioSeqDB into a byte-identical minimap2 execution. Chapter 5 documents minimizer-scheme differences, Chapter 6 documents chaining differences, and Chapter 7 documents extension and CIGAR boundaries. The index compatibility contract ensures that BioSeqDB's own query and target paths agree. It does not claim that any BioSeqDB generation can be handed to an arbitrary minimap2 release as a `.mmi` file.

13.13. Conclusions

BioSeqDB's persistent indexing work began with a conventional idea: serialize the target's minimizer hash table so the next query can skip rebuilding it. The database setting exposed the missing parts of that idea. A useful index must identify the source and algorithm that produced it, publish atomically, coexist with readers of an older version, survive errors without appearing complete, load under a memory bound, select only query-relevant postings when possible, and convert safely into CPU and GPU views.

The final architecture combines several retained decisions:

1. typed MPF, MPS, and metadata indexes rather than one opaque blob API;
2. fragment-native MPF as the default, with MPS available for sequence-wide selective loading;
3. immutable generation descriptors and fail-closed compatibility;
4. Format 5 as the general shape-aware format and a gated hash-major Format 6 route;
5. a default unlogged PostgreSQL-relation provider and an implemented opt-in mapped MPF provider whose lifecycle is governed by PostgreSQL;
6. explicit memory permits, bounded construction spill, and backend/device residency caches; and

7. deterministic runtime policy and identity-only background persistence.

The intermediate results explain why each piece exists. A global build could be fast yet consume tens of gigabytes of temporary storage. A bounded builder could complete yet create hundreds of thousands of files. A query-filtered transient index could be fast yet solve no persistence problem. A logically selective PostgreSQL representation could still lose to hundreds of thousands of heap fetches. The current solution is therefore not simply "store the minimap2 index in a table." It separates logical organization, physical provider, compatibility, publication, and runtime view so each can be measured and changed without weakening the others.

Chapter 14

Reliable sequence ingestion

14.1. Ingestion is part of the database contract

The alignment kernels described in the preceding chapters begin with compact sequences, but public biological archives normally deliver text. A FASTA record contains a header and one or more sequence lines; a FASTQ record adds a separator and a quality string whose length must agree with the sequence. Either format may be compressed, individual records may be much larger than a PostgreSQL allocation, and a header may itself encode several typed metadata fields. Loading is consequently more than calling a file parser before an `INSERT`. It is the boundary at which byte-stream framing, alphabet interpretation, compact storage, relational typing, and transaction semantics must agree.

BioSeqDB treats that boundary as a pipeline with explicit contracts. The non-PostgreSQL I/O modules acquire and parse bytes. They know nothing about `Relation`, `Datum`, memory contexts, or SQL errors. The PostgreSQL loader selects the compile-time alphabet and storage width from the destination column, consumes already packed chunks, creates the out-of-line sequence source, and only then publishes the parent row containing its locator. Header interpretation is also divided at this seam: format recognition and field extraction are ordinary C++ code, whereas conversion to an SQL type, default evaluation, constraint checking, and row insertion remain PostgreSQL operations.

In this chapter, reliable loading has five concrete meanings:

1. a finite input is framed into exactly the intended FASTA or FASTQ records, including multiline records and compressed input;
2. memory use is independent of the length of the largest sequence on the path used by `bioseq.load`;
3. symbols are packed directly into the alphabet and storage-unit layout declared by the destination `bioseq.seq` typmod;
4. every diagnostic identifies the earliest terminal failure and, where possible, the record and destination column; and
5. a failed SQL call does not expose a successfully committed prefix as though it were a completed load.

These requirements differ from those of a standalone mapper. Minimap2's ordinary input route can parse a file into process-owned sequences and build an in-memory index. BioSeqDB

must additionally preserve relational defaults and constraints, stream sequence fragments into database-owned relations, publish locators in a safe order, and cooperate with PostgreSQL's transaction and resource-owner machinery. Conversely, the reusable BioSeqDB batch reader and standalone FASTA/FASTQ writer do not automatically acquire database transactionality merely because the PostgreSQL loader uses them. Keeping those guarantees separate is a recurring theme of the implementation.

14.2. Layered reader architecture

The small protocol at the bottom of the reader is `bioseq.io.source_core`. Its `io_source` concept requires a `read_into(dst,capacity)` operation returning the byte count, end-of-file state, and error state, together with `last_error_message()`. There is no virtual base class. Each parser instantiation is monomorphic in its source, so a POSIX read, an mmap view, zlib, and the custom parallel gzip source share one loop without paying a virtual call at every refill. This lightweight contract also lets generated external-provider clients, discussed in the next chapter, supply bytes without importing the sequence-alphabet or PostgreSQL graphs.

The complete path used by the SQL loader is reconstructed in Figure 14.1. Solid boxes belong to the default path. The dashed queue is the optional overlap experiment retained behind a setting. The diagram deliberately separates file-format selection from compression selection: FASTA versus FASTQ comes from the filename extension, while gzip is decided from the file's magic bytes.

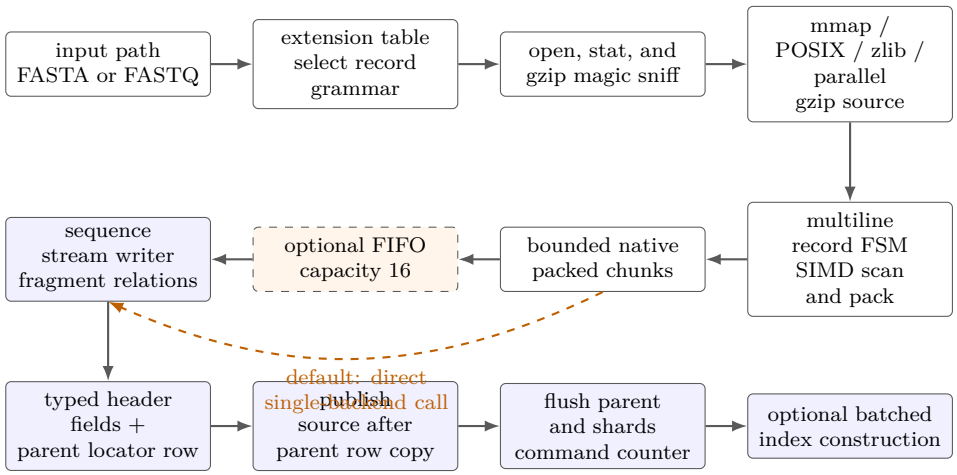


Figure 14.1: The BioSeqDB loading path. The reusable I/O layer ends at native packed chunks. PostgreSQL writes remain on the backend thread. The bounded capacity-16 queue is an optional overlap path; the default loader calls the sequence writer directly.

The principal reader layers and their ownership are summarised in Table 14.1. This decomposition arose from several earlier designs that coupled reading, record objects, and consumers too tightly. In the current version the same source and FSM can feed a compact batch for general computation, a bounded packed stream for PostgreSQL, or a small decompressed prefix for header-format sampling.

14.2.1. Source selection and format selection

`infer_sequence_file_kind` is a PostgreSQL-independent table lookup. It accepts `.fasta`, `.fa`, and `.fna` for FASTA and `.fastq`, `.fq`, and `.fnq` for FASTQ, case insensitively. The op-

Layer	Contract	Current implementation anchor
byte source	fill a caller-owned span; report EOF or first error	`io/source_core.cppm` and `io/sources.cppm`
buffer	retain an incomplete record prefix, compact or grow within a hard cap	`io/buffer.cppm`: `buffer_slab`
record grammar	incremental FASTA/FASTQ state machine over arbitrary refill boundaries	`io/fasta.cppm` and `io/fastq.cppm`
encoding	map symbols directly to a selected `compact_sequence_range` layout	alphabet-templated `encode_body_chunk` kernels
delivery	compact batches or bounded `packed_sequence_chunk<Unit>` events	`io/public_api.cppm`
database sink	append packed bytes, create locator, coerce metadata, and write one tuple	`backend/pg/storage_pg/io/load.cppm`

Table 14.1: Ownership and contracts along the ingestion path. PostgreSQL types appear only in the final layer.

tional suffixes `.gz`, `.bgz`, and `.bgzf` are stripped before that lookup. The resulting gzip-suffix bit is informational. The byte source independently checks for the RFC 1952 magic bytes `1f 8b`. Thus a gzip-compressed file with a misleading plain suffix can still be decompressed, while a plain file named with a gzip suffix does not force the parallel decoder.

The record grammar is not inferred from file contents. This is intentional: the parser is instantiated once for either FASTA or FASTQ and does not branch between grammars per record. It also means that an incorrectly named file produces a precise grammar error rather than being heuristically reclassified. Compression and grammar therefore have different trust boundaries: compression is safely magic-sniffed, but the caller is responsible for a recognised format extension.

With the default options, the dispatcher first attempts `mmap_source::try_open_with_magic`. A plain mapped input can be presented directly to the parser, eliminating slab refills and copies. If mapping is unavailable or outside its configured size limit, a `posix_read_source` retries `EINTR` and supplies sequential access hints where the platform supports them. All sources strip one leading UTF-8 byte-order mark. A gzip input smaller than 16 MiB compressed, or one for which the parallel source cannot be constructed, uses zlib's `gzread`-based source.

A significant reliability rule concerns fallback. Failure while constructing a parallel gzip source may fall back to zlib because no record has yet been emitted. A failure after decompression has begun is terminal. Restarting the file through zlib would emit the already processed prefix a second time unless the entire consumer state were rolled back. The dispatcher therefore never performs a mid-stream restart.

14.2.2. Parallel gzip and the librapidarchive experiment

The current parallel source maps compressed bytes and divides DEFLATE work among workers. Its `chunk_fetcher` owns eight reusable arena slots, delivers chunks to the consumer

in logical order, carries a 32 KiB trailing window between dependent pieces, and rejects a chunk whose decompressed output would exceed 256 MiB. The public option uses a 4 MiB chunk target when no override is supplied; the fetcher's internal default is 32 MiB, a later Apple-Silicon tuning. Automatic worker selection reserves two hardware threads and caps the common path at eight, while the fetcher itself enforces a hard maximum of twelve.

This is the surviving result of several compression experiments. An archived design proposed direct integration with librapidarchive/rapidgzip and its marker-based independent-block technique. The early implementation report showed that simply overlapping parsing with a single dependency chain could not escape the serial DEFLATE ceiling. The current core I/O compression path no longer depends on that library; it contains BioSeqDB-owned gzip header, Huffman, DEFLATE, block-finding, window, and chunk modules. The external-provider subsystem uses libarchive separately for archive containers, but that is not the FASTA/FASTQ gzip backend described here. The conceptual inheritance remains: find independently decodable work, represent unresolved back-references when necessary, resolve them from preceding windows, and drain results in order.

An archived June benchmark on a roughly 6 GiB uncompressed FASTQ corpus reported 69.4 MiB/s of compressed input, approximately 222 MiB/s of decompressed output, for the then-current parallel path, compared with about 174 MiB/s decompressed for zlib. It therefore measured a 22% improvement over the zlib route, not the multi-gigabyte rate originally targeted. A later forensic note is important: a notebook chart displayed approximately 4315 MiB/s under a "parallel gzip" label because the same dispatcher had been run on a plain corpus and fell through to mmap; the chart omitted the actual gzip bucket. This is why Table 14.4 retains the 222 MiB/s decompressed value and does not repeat the mislabeled bar.

There is also a current integrity limitation. The compression tree contains a gzip-footer parser and state fields for expected CRC32 and ISIZE, and `parallel_gzip_verify_crc` defaults to true. In the inspected `parallel_gzip_decoder`, however, the emitted bytes only update a running CRC before the state is reset at a member boundary. No current path compares that value or the observed size with the footer's expected CRC32 and ISIZE. The option therefore pays for checksum accumulation but does not yet establish end-to-end footer verification. The zlib fallback has zlib's normal stream validation, but the custom parallel route should not be described as fully integrity-checking until the comparison is wired and corrupted-member tests prove it.

14.3. Incremental parsing and direct compact encoding

The parser never constructs a temporary ASCII sequence and then repacks it for the database. Its state machines append headers to byte storage and send body spans to alphabet-specific packing kernels. A `buffer_slab` starts at 1 MiB. If a record crosses a refill boundary, `make_room` moves only the unread suffix; if the unread record fills the slab, capacity grows geometrically, with smaller increments after 8 MiB, up to the configured 1 GiB hard limit. This path supports general batch reading. The PostgreSQL streaming path drains complete packed units during a record and therefore does not need the slab to grow to the record's full size.

14.3.1. FASTA framing

The FASTA state machine moves through header sigil, header, body, and finished states. It accepts CRLF and LF line endings and arbitrary body line wrapping. For the common

DNA4/`uint64_t` case, a fused SIMD loop both finds the next record sigil and converts body bytes into two-bit words, avoiding a separate boundary scan over the same memory. Other alphabets use the generic line-oriented encoder. At record completion, the encoder flushes its partial storage unit and records the final padding, which is necessary to distinguish real symbols from unused high-order slots in the final word.

14.3.2. FASTQ framing

FASTQ cannot be parsed by searching every line that begins with `@`. A quality line may legally begin with `@`, `+`, or `>`, and both the sequence and quality may span multiple physical lines. BioSeqDB therefore:

1. consumes one `@`-prefixed header;
2. accumulates sequence lines until a line beginning with `+`;
3. skips the remainder of that separator line; and
4. consumes quality bytes until their newline-stripped byte count equals the raw sequence byte count.

The final count is intentionally not the number of accepted packed letters. A byte may be absent from the selected alphabet and be dropped by the encoder, but it still occupies one quality position in the source file. `fastq_parser_state` therefore tracks `sequence_byte_count` separately from the encoder's accepted-letter count. A truncated quality section and a quality section that overshoots this count are terminal data errors. This distinction prevents alphabet normalisation from shifting the parser onto a later record boundary.

14.3.3. Alphabet and storage-width dispatch

The destination table, rather than the input filename, defines the alphabet. `resolve_load_columns` requires exactly one `bioseq.seq` column, decodes the alphabet code and byte width from its typmod, and rejects an unparameterised sequence type. The loader then performs one runtime-to-compile-time fan-out:

(alphabet code, storage width, FASTA/FASTQ) ⟶ `stream_sequences_file<A,Unit,Format>`.

The same parser can consequently produce DNA, RNA, amino-acid, or other supported compact sequences and can target one-, two-, four-, or eight-byte storage units. Packed chunks already have the column's native layout; the PostgreSQL sink computes their byte count from the alphabet's letters-per-unit value and appends them without a second representation conversion. This is the ingestion counterpart of the generic `compact_sequence_range` abstraction developed in the sequence-representation chapter: representation genericity begins before the data reaches the table.

The current policy for unknown symbols is permissive. Both FASTA and FASTQ invoke the encoder with `unknown_byte_policy::DROP`. Although `reader_options` exposes `strict_alphabet`, the inspected parser does not consult it. Thus strict FASTQ framing is implemented, but strict alphabet rejection is not. This should be understood as a current data-cleaning policy, not as validation that every input symbol belongs to the declared alphabet.

14.4. Bounded delivery and backpressure

The project contains three related delivery shapes, whose memory and ordering properties must not be conflated.

Path	Payload	Bound	Ordering/concurrency
general synchronous or mmap reader	`sequence_batch` with shared header, word, and quality vectors	batch threshold; an exceptionally large record may enlarge storage	single parser, or record-boundary mmap shards for very large files
general asynchronous reader	moved `sequence_batch` tasks	slab and batch targets; no currently wired ring-capacity backpressure	producer/parser on caller, handlers in the global task pool
PostgreSQL stream	header plus native packed windows	one slab, one header, one bounded packed window; no whole-sequence materialisation	single parser by contract; writes on backend thread
optional load overlap	owned begin/chunk/end events	FIFO capacity 16, producers block when full	PG-free producer task; PostgreSQL-only consumer, FIFO order

Table 14.2: The four delivery shapes. Only the last row uses explicit bounded queue backpressure, and it is disabled by default.

14.4.1. General batches

`read_sequences_file` accumulates many records into a `sequence_batch`. Records share backing vectors and are addressed by prefix offsets, avoiding one heap allocation per header, body, and quality string. For plain mmap inputs of at least 1 GiB, the reader may find valid record boundaries and divide the map among at most eight parser shards. Each shard owns its batch storage; handler calls are serialised where required, and cross-shard record delivery is not guaranteed to preserve file order. This is acceptable for consumers whose semantics are set-like, but it is not used by the PostgreSQL loader.

When `worker_count` is nonzero, the general asynchronous path parses on the caller and submits completed batches to the project-wide task pool. Its module comment still describes per-worker SPSC rings, and `concurrency_options` still has a `ring_capacity` field, but the current body dispatches task-group closures directly. The field does not bound outstanding batches. The thesis therefore treats the ring design as history rather than a current guarantee.

14.4.2. The PostgreSQL streaming contract

`stream_sequences_file` forcibly sets the worker count to zero and the parser count to one. Its `run_sync_stream` loop emits:

```
begin_record(header)
chunk(packed_words, represented_base_count, end_of_record=false) ...
chunk(final_words, represented_base_count, end_of_record=true)
end_record()
```

Only full storage units are drained mid-record. The last chunk subtracts the padding captured when the parser finalises the record. The word span remains valid for the duration of the callback, so the default sink copies it straight into `sequence_stream_writer`. If a sink implements `chunk_owned`, ownership can instead cross an asynchronous boundary. This protocol is what permits a multi-gigabase FASTA record to be stored without allocating a multi-gigabyte contiguous value.

The PostgreSQL stream sink calls `writer.begin` once per record, `append_packed_bytes` for every chunk, and `writer.finish` to obtain a `PendingSequenceSource`. It then builds the small parent-row locator and metadata tuple. The call to `publish_after_parent_row` occurs only after `BioseqColumnWriteRow` has copied that tuple. This order links the out-of-line sequence source and its relational locator without first publishing a source that no parent row references.

14.4.3. Optional parser/writer overlap

The only currently explicit reader-to-writer backpressure is the optional load overlap path. A PG-free producer task runs the same single-parser stream and places owned begin, chunk, and end events in a mutex/condition-variable FIFO of capacity 16. The PostgreSQL backend thread drains the queue and performs every relation, memory-context, constraint, and write-state operation. A full queue blocks the producer. A consumer exception closes the queue so the producer can stop rather than deadlock; after joining, the original consumer exception remains authoritative.

This design obeys PostgreSQL's thread-affinity boundary, but it was not a performance win on the measured single-record 2 Gbp load. The June investigation reported 9.701 s for its optimistic split baseline, 12.691 s after adding an overlapped copying queue, and 13.023 s with owned chunk events. A later run with the setting defaulted off measured 13.744 s of task time and 14.460 s wall time; the report also warns that post-restart non-overlapped loads settled near 13–14 s, making the 9.701 s baseline optimistic. The defensible conclusion is qualitative: queue synchronisation and ownership traffic did not accelerate that workload. The option remains available for other source/write ratios, but defaults off.

14.5. Header dialects and typed relational mapping

A FASTA header is nominally free text, but established repositories encode structured identifiers inside it. For example, a UniProtKB header begins with a pipe-delimited database, accession, and entry name, followed by a description and fields such as `OS=`, `OX=`, and `GN=`. Treating the whole header as one text column preserves bytes but loses the opportunity to index accession, organism, taxonomy identifier, dates, ranges, and numeric fields using PostgreSQL types.

BioSeqDB's parser recognises AUTO and PLAIN modes plus UniProtKB, UniRef, UniParc, UniMes, SPD, NCBI generic, NCBI GI, NCBI PDB, RefSeq, GenBank, GISAID, and IMGT dialects. Format definitions are constexpr tables in `io/identifier_parser/identifier_formats.cp` They state whether a field is positional, free description text, a `KEY=VALUE` tag, or a bracket tag, together with canonical names and aliases. Column-name comparison lowercases ASCII and removes underscores and hyphens, so a schema can use conventional SQL spelling without teaching the parser a second set of extraction rules.

AUTO detection is a bounded, once-per-file operation. The loader reads at most 256 KiB of decompressed prefix and samples at most 16 headers using the same source-selection helper as the main stream. FASTQ sampling follows the logical sequence-plus-quality lengths

instead of assuming four physical lines; it therefore does not mistake an `@` in quality data for the next header. Known formats vote across all sampled headers, with deterministic enum-order tie breaking. An unstructured sample becomes PLAIN, while unrecognised structured metadata remains in generic AUTO mode.

14.5.1. Mapping is resolved once

Before streaming records, `resolve_identifier_mapping` operates on a PostgreSQL-free array of `column_descriptor` values. Its priority is:

1. explicit user mappings from `bioseq.column_mapping[]`;
2. implicit matches between fixed-format fields and destination column names;
3. selection of an identifier text column, preferring a unique primary key or a conventional identifier alias;
4. generic metadata routes for AUTO; and
5. a decision whether mapped metadata should be stripped from the identifier value.

The result is a compact routing table from positions or normalised keys to destination slots. Per-record parsing performs one SIMD-assisted scan for `|`, `=`, `[`, and `]`, stores common delimiter tokens in an inline small-vector, and reuses parse scratch across records. The parser writes raw byte ranges through the `is_identifier_dest_writer` concept. A small type-erased `identifier_writer_ref` prevents every header extractor from being instantiated again for every alphabet, storage width, and file format.

The database implementation, `pg_identifier_writer`, caches each destination type's input function and typmod. Plain unconstrained text can use a direct text construction; constrained character types and arbitrary scalar, range, or multirange types use `OidInputFunctionCall`. A convenient `lo..hi` header value is rewritten to PostgreSQL's inclusive range syntax before the type input function is invoked. Date and timestamp interpretation consequently follows the session's PostgreSQL `DateStyle`, just as it does for other SQL input.

Unmapped defaults are compiled as executor expressions once per load and evaluated per row. Volatile defaults are therefore not accidentally frozen at load start. The completed virtual tuple passes through `ExecConstraints` before the columnar row write. Per-tuple expression and write memory contexts are reset only after by-reference datums have been copied. This final design addresses issues found during review of the first header-parser specification: metadata columns must be claimed before choosing an identifier column, AUTO sampling must precede mapping, multiline FASTQ must remain valid, direct column-store writes must explicitly restore constraints, and per-row memory cannot accumulate until end of file.

The measured parser cost is small relative to sequence ingestion. On a 40,000-record, approximately 11.9 MiB corpus, the implementation report measured 4.331 ms for a dumb header path and 33.375 ms for UniProt parsing and routing, or approximately 108 ns versus 834 ns per record. The structured path was about 7.7 times the header-only micro-operation, but still below one microsecond per record while extracting six fields. This is a parser microbenchmark, not an end-to-end SQL throughput claim.

14.6. SQL integration, failure propagation, and transaction semantics

The generated SQL surface is conceptually:

```
bioseq.load(
    target_table  regclass,
    file_path     text,
    header_format text DEFAULT 'auto',
    columns       bioseq.column_mapping[] DEFAULT NULL
) RETURNS bigint
```

The return value is the number of loaded records. Existing two-argument calls remain valid through defaults. The target relation is opened with `RowExclusiveLock`. Schema discovery rejects zero or multiple sequence columns, identity/generated destinations, ambiguous or unknown mappings, and an unmapped `NOT NULL` column without a default before the first record is written.

The steady-state algorithm can be expressed as follows:

```
kind      = infer FASTA/FASTQ from extension
columns   = inspect target; decode alphabet and storage width
format    = sample headers if AUTO
routing   = resolve typed destinations once
dispatch  = instantiate <alphabet, unit-width, record-format> once

for each streamed record:
    sequence_writer.begin()
    for each bounded packed chunk:
        sequence_writer.append_packed_bytes(chunk)
    pending_source = sequence_writer.finish()

    tuple = all NULL
    tuple.seq = locator(pending_source)
    parse header and route raw fields into cached PostgreSQL input functions
    evaluate still-unset defaults
    enforce relation constraints
    write parent tuple
    pending_source.publish_after_parent_row()

flush parent write state and every fragment-shard write state
advance command counter
optionally build configured sequence indexes in one post-load pass
return record count
```

14.6.1. Error domains

Non-PostgreSQL operations use `std::expected` or C++ exceptions carrying owned strings. The PostgreSQL boundary maps them to an SQLSTATE and raises an ERROR through a checked C++ seam. The reader captures only the first terminal diagnostic; later cleanup must not hide it. PostgreSQL type input, default, or constraint errors retain their native message and receive an `ErrorContextCallback` suffix containing the record number, optional column

name, and at most 240 bytes of the header. The bound prevents a corrupt giant header from producing an unbounded error message.

Cleanup is explicit even though the surrounding transaction remains the final atomicity authority. The loader tears down its executor, slot, relation, and write-state resources at the appropriate boundary; successful execution flushes both parent and fragment write states, performs `CommandCounterIncrement`, and only then permits an optional index scan to observe the new locator-backed rows. If an exception escapes, PostgreSQL aborts the statement's transaction scope, so partial rows are not a separately committed successful prefix. A caller that invokes `bioseq.load` inside a larger explicit transaction retains the usual PostgreSQL choice to roll back the transaction or recover at a savepoint.

This is stronger than "delete the output file on parser error". Sequence data spans fragment relations and a parent column-store row; PostgreSQL's write states, resource owners, locks, snapshots, command visibility, and WAL semantics are the mechanisms that define visibility and rollback. The loader therefore avoids inventing a second transaction protocol inside the I/O module.

14.6.2. Comparison with a conventional mapper input path

Concern	Conventional minimap2-style file input	BioSeqDB load path
consumer	process-owned reads or reference index	transactional PostgreSQL relation plus fragment shards
header	name/comment retained for mapper output	optional repository-specific parsing into several SQL-typed columns
sequence representation	mapper's internal encoded buffers	destination alphabet and storage width selected from SQL typmod
large record	bounded by process/index construction strategy	explicit packed-window stream; no whole-record database value
parallelism	reader/index threads may own process memory	all PostgreSQL calls remain on backend thread; optional PG-free producer only
failure	command exits or returns an error	SQLSTATE, record/column context, and PostgreSQL transaction rollback
post-load work	build/use mapper index	flush locators, advance visibility, optionally build persistent table indexes

Table 14.3: Why loading into BioSeqDB requires more than embedding a standalone FASTA/FASTQ parser.

14.7. The standalone writer is a different contract

The I/O module also exports a FASTA/FASTQ `sequence_writer`. It is useful for corpus generation, benchmark round trips, and exporting compact sequences, but it is not the implementation of PostgreSQL atomicity. Each producer thread owns a 64 MiB output buffer by default. At a flush point the sink reserves a disjoint file range through an atomic `fetch_add`; `pwrite` then fills that range. Before starting a new record, the writer flushes a threshold-crossing buffer so chunks from different threads cannot splice a header from one record onto the body of another. Concurrent producers may nevertheless change record order.

The default `posix_pwrite_sink` opens with `O_TRUNC`, retries interrupted writes, and reports the first failure. An optional mmap sink requires an upper-bound size hint and truncates the file to the number of reserved bytes on close. A bounded streamed-record method decodes compact words into ASCII slices rather than materialising a multi-gigabase body. FASTA defaults to no line wrapping for throughput. FASTQ quality-length checking is configurable; in its permissive mode the writer truncates an overlong quality or pads a short one with Phred-zero `!` bytes.

Two limitations matter when interpreting the word "reliable". First, the standalone writer deliberately omits `fsync`; successful close means the kernel accepted and closed the file, not that stable storage has acknowledged it. The mmap close path similarly unmaps and truncates but does not call the `msync` named in its stale comment. Second, opening truncates an existing file immediately and an error can leave a valid prefix or partial file. There is no temporary-file-and-rename transaction. Callers that require durable atomic replacement must add that policy outside `sequence_writer`. None of these limitations weakens `bioseq.load`, whose sink is PostgreSQL storage rather than this file writer.

The archived writer benchmark illustrates why these choices were made. On the measured Apple-Silicon host, the best BioSeqDB configurations wrote about 2.65 GiB/s for a 234 MiB FASTA cell and 1.50 GiB/s for a 6 GiB FASTQ cell, versus 311 and 491 MiB/s for a byte-oriented `std::ofstream` baseline. The mmap sink was not selected as default: it regressed the same 234 MiB cell from approximately 2.54 to 1.75 GiB/s and the 6 GiB cell from 1.27 to 0.43 GiB/s because demand-zero page faults outweighed the avoided kernel copy on that machine. These are historical page-cache throughput measurements with no `fsync`, not durability benchmarks.

14.8. Measured evolution and current limits

Table 14.4 collects only measurements whose interpretation could be established from the reports. The reader and writer rows are archived subsystem measurements on Apple Silicon. The header row is a microbenchmark. The overlap rows are load-only task times from one 2 Gbp experiment. They should not be combined into a single speedup because the corpora and measured boundaries differ.

Experiment	Baseline	BioSeqDB/result	Interpretation
plain mmap FASTA reader, final archived v3 buckets	–	2694–4586 MiB/s	range across synthetic/real bu page-cache reader throughput
6 GiB gzip FASTQ, decompressed basis	zlib 174 MiB/s	parallel 222 MiB/s	22% over zlib; actual gzip bucket mislabeled plain mmap
234 MiB FASTA writer	`ofstream` 311 MiB/s	best 2649 MiB/s	page-cache output, no `fsync`
6 GiB FASTQ writer	`ofstream` 491 MiB/s	best 1502 MiB/s	large-cell writeback-limited result
40k header records	dumb 108 ns/record	UniProt 834 ns/record	six-field parse/routing microbenchr
2 Gbp load overlap, optimistic split	direct 9.701 s	copying queue 12.691 s; owned 13.023 s	overlap did not win; baseline later j optimistic
overlap setting default off, later run	task 13.744 s	wall 14.460 s	supports keeping the optional pat abled by default

Table 14.4: Selected I/O evidence. Units and measurement boundaries are retained from the named project reports; rows are not directly comparable across corpora.

The development sequence behind these results is itself informative:

1. The first reader established an `io_source` abstraction, incremental FSMs, a recyclable slab, and shared batch storage.

2. mmap direct parsing and large-file record-boundary sharding removed copies and exposed parallel parsing where file size amortised shard setup.

3. The writer introduced per-thread chunks and disjoint `pwrite` ranges, then retained pwrite after mmap output regressed on the measured host.

4. gzip work progressed from zlib and overlap experiments toward the current BioSeqDB-owned block/chunk decoder; the originally specified external rapidgzip integration was not retained.

5. Database integration replaced whole-record batches with the storage-unit-parametric stream, so `bioseq.load` could ingest sequences beyond PostgreSQL's ordinary allocation limit.

6. Typed header parsing was placed outside the alphabet/file-format template cross product and reviewed against defaults, constraints, multiline FASTQ, memory-context lifetime, and error context.

7. A bounded producer/consumer queue tested parsing/write overlap. Its measured regression led to a conservative default-off policy instead of making parallelism an article of faith.

The current implementation is therefore not simply the fastest intermediate prototype. It selects the paths whose guarantees compose at the SQL boundary: single-backend database writes, bounded packed streaming, deterministic mapping setup, explicit publication order, and PostgreSQL-managed rollback. At the same time, four open limitations should remain visible in the thesis:

1. file content does not auto-detect FASTA versus FASTQ; the extension selects the grammar;

2. `strict_alphabet` is not wired and unknown alphabet bytes are dropped;

3. the custom parallel gzip path accumulates but does not currently compare CRC32/I-SIZE with the member footer; and

4. the generic asynchronous batch reader does not currently realise the ring-capacity backpressure described by its older comments.

These gaps are local and testable. Content-based format probing can be added before template dispatch; strict-symbol mode can select a rejecting encoder policy; gzip member finalisation can parse and compare both footer fields; and the general asynchronous reader can either wire its SPSC queues or remove the unused option. None requires changing the central SQL loading contract.

14.9. Conclusion

BioSeqDB's ingestion path turns a textual archive record into two coordinated database objects: compact sequence fragments and a typed parent row carrying their locator. It does so without assembling a whole large sequence, without letting worker threads enter PostgreSQL, and without hard-coding DNA4 or a single storage width. The reader's reusable concepts isolate byte sources, incremental grammars, and compact packing; the identifier

parser isolates repository dialects from SQL type conversion; and the loader binds both to PostgreSQL locks, write states, constraints, visibility, and rollback.

The intermediate results explain the final design. mmap and fused packing were retained because they removed data movement. Parallel gzip was retained as a distinct backend but not presented as having met its original multi-gigabyte target. mmap output and parser/writer overlap remained optional after measurement showed regressions. The bounded streaming API replaced a batch-only database path because correctness on a multi-gigabase record matters more than an attractive small-record abstraction. Reliable loading here is therefore not one defensive check; it is the composition of bounded state, precise framing, generic representation, typed relational materialisation, and transactional publication.

Chapter 15

External sequence providers

15.1. Remote biological databases as relations

Public sequence databases are indispensable, but they do not expose one common data model. NCBI separates taxonomy, gene, assembly and sequence retrieval among several services. UCSC organises sequence by genome assembly and genomic interval. UniProt centres its interface on protein accessions and protein-specific annotations, while the EBI Search API provides discovery over several domains. Their identifiers, paging rules, response schemas, rate limits and failure modes differ. A conventional analysis script therefore contains a substantial amount of code that is not part of the biological question: URL construction, authentication, JSON decoding, retries, FASTA parsing, identifier translation and conversion into local records.

BioSeqDB treats this integration problem as a database access problem. It exposes six logical relations in the `bioseq_external` schema: `taxonomy`, `catalog`, `record_genes`, `record_locations`, `sequences` and `sequence_chunks`. A query against one of these relations does not read a local heap. PostgreSQL constructs a `BioseqExternalScan`; the scan translates supported SQL predicates into a provider-neutral query, contacts the selected service, and returns ordinary typed tuples. Consequently, a remote record can participate in a join, projection or residual PostgreSQL filter without first being downloaded by an external script.

This design is not equivalent to wrapping an HTTP call in a scalar SQL function. A scalar function has no natural place to represent a stream of records, cannot conveniently expose provider capabilities to planning, and encourages one remote request per evaluated row. It also makes cancellation and memory bounds accidental properties of the function implementation. The external table instead gives the remote operation an executor lifecycle: `BeginCustomScan`, repeated `ExecCustomScan`, `ReScanCustomScan`, `EndCustomScan`, and `ExplainCustomScan`. The scan can defer network access until the first tuple is requested, retain state between calls, apply backpressure, and cancel its producer during rescan or query teardown.

The six relation declarations are not handwritten SQL. The annotated C++ row types in `main/bioseq/backend/pg/external/sql_catalog.cppm`, namespace `bioseq::external::sql_rows`, are consumed by the SQL generation machinery described in Chapter 15. Every type specifies `access_method = "bioseq_external"`. The sequence-bearing columns use the real `bioseq.seq` PostgreSQL type, while provenance and provider attributes are typed as `jsonb`. The schema revokes public access by default, which is important because a SELECT can cause outbound network traffic.

15.1.1. What each relation means

The schema separates discovery from retrieval. `taxonomy` normalises taxon identifiers, names, ranks and lineage. `catalog` is the cross-provider record inventory: it carries the original identifier, normalised accession, molecule kind, title, sequence length, assembly information and a `retrievable` flag. `record_genes` and `record_locations` explode nested provider metadata into relational rows with stable per-record indices. `sequences` returns a complete record as one `bioseq.seq` datum. `sequence_chunks` returns an ordered sequence of datums with `chunk_index`, `base_offset` and `end_of_record`.

The distinction between the last two relations is a resource contract, not only a presentational choice. A complete `sequences` row must materialise one record or assembly segment. In contrast, `sequence_chunks` can keep its working set independent of a multi-megabase record's length. The current NCBI chunk size is 1048572 bases. It is deliberately just below one MiB and is divisible by the internal block geometry used by the implementation. A one-chunk look-ahead keeps the final non-empty chunk pending, so a record whose length is an exact multiple of the chunk size receives no spurious empty row and exactly one row has `end_of_record=true`.

Table 15.1: Provider capabilities in the configured provider pack. This table describes current behaviour, not every operation present in an upstream API.

Provider	Implemented use	Current boundary
NCBI	taxonomy, catalog, gene locations, nucleotide sequences and chunks	Generated Oat++ client for JSON metadata; direct libcurl FASTA stream for nucleotide accessions; bounded NCBI Datasets ZIP path for `GCF_`/`GCA_` assemblies.
UCSC	genome catalog and interval DNA	Generated client calls the JSON endpoints, including `getDnaSequence`. The result is currently collected before tuple production.
UniProt	protein catalog and gene metadata	Generated client normalises protein metadata. Protein sequence materialisation deliberately raises SQLSTATE `0A000`.
EBI	taxonomy/catalog discovery	Generated EBI Search client; no sequence retrieval capability.

The configured pack is defined by `configured_external_api_provider` in `main/bioseq/external/providers/configured.cppm`. It contains NCBI, UCSC, UniProt and EBI in that order. A SWISS-MODEL API descriptor and generated client also exist, but there is no SWISS-MODEL adapter in the configured runtime pack. This illustrates a useful separation: generating a safe transport client is necessary, but it does not by itself define BioSeqDB's canonical records or SQL semantics.

15.2. Evolution from generated calls to a streaming table provider

The external subsystem was developed in four identifiable steps. Reconstructing those steps matters because the broad design specification and the current implementation are not identical.

The first implementation therefore solved the control-plane problem: turn a supported SQL predicate into a provider request, validate and execute that request, parse the response, and normalise it into rows. It did not yet solve the data-plane problem for large nucleotide as-

Table 15.2: Chronology of the external API subsystem. Higher-numbered and more recent implementation reports override the earlier intentions.

Document	Role	Result
DOC0370	Initial prompt	Proposed deriving typed GET clients from embedded OpenAPI descriptions rather than maintaining provider-specific URL code by hand.
DOC0371	Generated-client specification	Defined cppast/libclang annotation discovery, a validated OpenAPI subset, build-tree-only C++ module generation, Oat++ transport, manifests and live integration fixtures.
DOC0372	Common-provider specification	Defined the provider concepts, canonical query and message model, logical PostgreSQL relations, a separate streaming data plane, resource admission, locator security and request policy.
DOC0373	First implementation proof	Proved all six relations through a real PostgreSQL custom scan and all four configured providers, but sequence retrieval still used whole-response collection.
DOC0374	Current implementation proof	Added the NCBI libcurl pull stream, backpressured channel, archive path, multi-record FASTA handling, process-local rate-policy implementation and live PostgreSQL evidence. It also records the remaining deferrals explicitly.

sets. A generated client returned a heap-owned `std::string` response body, and the provider adapter parsed it after the complete response arrived. This is appropriate for bounded metadata pages, but makes peak memory proportional to the size of a genome.

The final architecture keeps generated clients for metadata and adds a separate byte-streaming route for large assets. The two routes converge only after provider-specific parsing, at the `owning_provider_message` model. Figure 15.1 shows the implemented paths and, crucially, where whole-response ownership still exists.

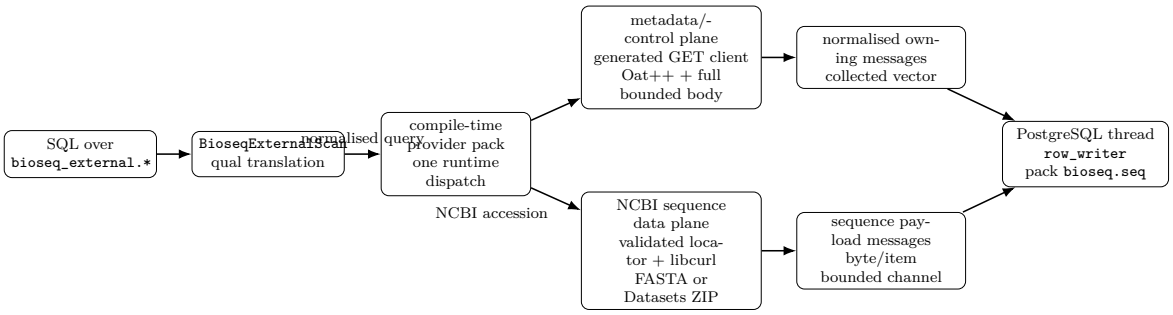


Figure 15.1: Implemented external-provider data flow. Metadata and non-streaming retrieval are collected before tuple production. NCBI accession retrieval can instead publish sequence payloads through a backpressured channel.

This split is more than a transport optimisation. Metadata requests and asset transfers have different correctness criteria. Metadata is usually a bounded JSON document whose value lies in its structure. A sequence asset may be hundreds of megabytes, and its useful representation is a stream of symbols. The control plane needs typed parameter validation and provider-specific JSON normalisation. The data plane needs byte caps, incremental decoding, archive safety, record boundaries, backpressure and an emission rule that determines whether a failed request may be retried.

15.3. Generated clients and handwritten adapters

Provider integration is divided into three layers. The descriptor layer states what an upstream API accepts. The generated client layer implements safe, typed HTTP mechanics for the supported OpenAPI subset. The handwritten adapter layer assigns BioSeqDB meaning to the response.

Source descriptors are the C++ modules `main/bioseq/external/ncbi.cppm`, `main/bioseq/external/ucsc.cppm`, `main/bioseq/external/ebi.cppm`, `main/bioseq/external/uniprot.cppm` and `main/bioseq/external/swiss_model.cppm`. They contain the `bioseq::api_client` annotation and embedded OpenAPI input. They should not be confused with the runtime provider adapters under `main/bioseq/external/providers/`.

The current generator is implemented under `main/codegen/external/`. The annotation parser uses cppast/libclang on a normalised shadow source. The OpenAPI parser accepts GET operations, typed path/query/header parameters, HTTPS server metadata and a restricted API-key security model. It rejects unsupported or ambiguous constructs at build time. The client renderer emits a C++ module named `bioseq.external.api.<provider>`. The artifact handler writes `client/index.cppm`, a generated integration test, a CMake fragment, `manifest.tsv` and `provenance.txt`. Artifacts live in the build tree and are updated with `write_if_different`; generated source is not treated as a manually maintained copy in the repository.

The generated public API is intentionally small. A request consists of an operation identifier and a span of typed `parameter_value` objects. The client validates required parameters, multiplicity and location before percent-encoding the path and query. It then executes `get`, `get_with_accept`, or `get_batch`. Batch transport work is distributed through `GlobalPool`; completed responses are materialised serially. A response owns its body as `std::string`, with a default 64 MiB limit. This heap ownership is deliberate: an earlier attempt to place a large transient HTTP body into the small-block result arena could exceed the arena's allocation geometry. Provider adapters parse the body once into bounded owning messages, after which the transient string can be released.

The following simplified C++26 model captures the actual provider contract:

```
template<class S>
concept external_api_session_implementation =
  requires(S& s, provider_message_sink_ref out, std::stop_token stop) {
    { s.run(out, stop) }
      -> std::same_as<std::expected<provider_statistics, external_error>>;
    { s.request_stop() } noexcept -> std::same_as<void>;
  };

template<class P>
concept external_api_provider_implementation =
  requires(normalized_external_query const& q,
           typename P::plan_type const& plan,
           external_slot_budget const& budget) {
    typename P::plan_type;
    typename P::session_type;
    requires external_api_session_implementation<typename P::session_type>;
    { P::plan(q, provider_capability_snapshot{}, budget) };
    { plan.execution_fingerprint() };
```

```
    { P::open(/* memory, pool, runtime, credentials, */ plan, budget) };
  };
```

The complete definitions are in `main/bioseq/external/provider/concepts.cppm`. There is no virtual provider base class. The template `external_api_provider<Providers...>` in `main/bioseq/external/provider/pack.cppm` checks type and stable-ID uniqueness at compile time. At run time it performs one dispatch from the provider string to the concrete adapter type. Dispatch does not occur for each row or chunk. Adding a provider therefore means implementing one type that satisfies the concept and adding it to the configured pack; the hot row path does not gain a virtual call.

Each adapter lowers a provider-neutral query into an immutable plan, opens a session, calls the generated client, and converts provider JSON into the canonical model. NCBI reads `NCBI_API_KEY` for generated metadata calls and passes it as an owning generated `credentials` value. UCSC maps BioSeqDB's zero-based half-open interval directly to its sequence endpoint. UniProt normalises protein catalog and gene fields but rejects sequence targets because the PostgreSQL sequence materialiser is currently nucleotide-oriented. EBI is discovery-only. The generator thus owns syntactic HTTP correctness; the adapter owns biological and relational semantics.

15.4. Planning and executing a remote table scan

15.4.1. A table access method with no local tuples

The `bioseq_external` table access method is defined in `main/bioseq/backend/pg/external/relation.cppm`. It does not create a physical storage fork and does not contact a provider. Its ordinary physical scan callbacks cannot produce tuples; the custom scan is the legal read path. The path builder in `main/bioseq/backend/pg/external/scan.cppm`, `bioseq_external_try_add_paths`, recognises the access method, removes core-generated paths and inserts one non-parallel `CustomPath`. Its current cost is a fixed 1.0, and its row estimate falls back to 1000. This differs from BioSeqDB's alignment-function custom scan: there is no competing scalar remote-table implementation whose high cost must be beaten. A physical sequential scan would be semantically invalid, so the external custom path is exclusive.

`bioseq_external_plan_custom_path` preserves the real relation OID in `scanrelid`. PostgreSQL therefore provisions a scan slot with the declared table descriptor and applies the plan's projection and residual filters normally. `BioseqExternalScanState` places `CustomScanState` at offset zero, as required by PostgreSQL's node ABI. The C++ runtime object is separately allocated and registered with the query memory context's reset callback. This prevents a C++ container from being silently abandoned when PostgreSQL exits through its error long jump.

Network execution is postponed from `BeginCustomScan` until the first call that requests a tuple. Consequently, ordinary `EXPLAIN` without `ANALYZE` reports the selected provider and entity without issuing a remote request. `EXPLAIN ANALYZE` exercises the provider, because the executor must consume rows.

15.4.2. Predicate lowering and provider selection

Every relation has a `provider` column. An equality predicate selects one provider, while `provider = ANY(array)` selects several. Without an explicit predicate, the current defaults are NCBI for taxonomy, locations and sequence retrieval, UniProt for gene rows, and UCSC

for the catalog. Column names are mapped back to the canonical `external_field` enumeration. The scan currently pushes equality predicates over text, integer and boolean constants, plus `= ANY(...)`. Unsupported expression shapes and types remain PostgreSQL residual filters.

The canonical query model in `main/bioseq/external/provider/query.cppm` is broader than this first lowering implementation. It represents targets, typed scalar values, flat post-order predicate nodes and pushdown quality. The executor currently sets a provider request limit of 1000; an SQL `LIMIT` is still enforced by PostgreSQL rather than being derived into that value. Likewise, inequalities such as `start0 < end0` remain local even though the provider-neutral model can represent richer operators.

An array predicate is expanded into equality-only subqueries because the current adapters consume equality accessions. The Cartesian expansion is capped at 64 combinations. Hitting the cap produces a warning rather than silently pretending that every value was queried. Results are unioned, and stable keys remove duplicates from list-style providers that ignore a particular expanded predicate. Errors from one provider/query pair do not erase successful rows from another; the first error is surfaced only when the union has no rows.

Algorithm 15 Current external-scan execution, simplified from `bioseq_external_scan_next`.

```
Require: relation entity, SQL quals, provider set
lower supported quals to a normalised external query
if first tuple request then
    if entity is sequence or sequence chunk, provider is NCBI, and accession exists then
        start the NCBI producer and a bounded message channel
    else
        expand ANY predicates and run provider plans to completion
        collect and de-duplicate owning messages

if streaming then
    repeat
        timed-pop one charged message lease
        on timeout, call CHECK_FOR_INTERRUPTS
    until item or terminal state
    pack one payload into the PostgreSQL slot and release its charge
else
    translate the next collected message/sub-row into the scan slot

let ExecScan apply residual quals and projection
```

15.5. The NCBI streaming data plane

The streaming path is selected by `is_streaming_query` in `main/bioseq/backend/pg/external/scan.cppm`: the target must be `sequences` or `sequence_chunks`, the provider set must contain only NCBI, and an equality-constrained accession must be recoverable from the normalised query. Other provider/target combinations use the synchronous `run_external_query` bridge in `main/bioseq/external/providers/run.cppm`, which collects messages in a vector.

15.5.1. Pulling bytes without buffering the body

For an ordinary nucleotide accession, the producer constructs an NCBI E-utilities FASTA request. Before opening a socket, `validate_candidate` in `main/bioseq/external/transport/locator.cppm` checks the HTTPS scheme, allowed host suffix, URL length, absence of user information and public address scope. It resolves the host and requires every returned address to be permitted. The numeric addresses are attached to an otherwise non-constructible `validated_transfer_request`; libcurl receives them through `CURLOPT_RESOLVE`. Redirects, proxies and automatic decoding are disabled. This closes the common gap in which a URL passes a textual allowlist but DNS rebinding or a redirect reaches a private service.

`remote_byte_source` in `main/bioseq/external/transport/remote_source.cppm` satisfies the shared `bioseq::io::io_source` concept. It cooperatively drives a libcurl multi handle on the producer thread. The write callback appends into a carry buffer whose default capacity is 256 KiB. If the next callback would exceed that capacity, it returns `CURL_WRITEFUNC_PAUSE`; draining `read_into` resumes the transfer. Hence the network side retains at most the configured carry, rather than a complete FASTA response. The source also applies a 15-second connect timeout, a low-speed threshold of one byte per second over 120 seconds, an optional total timeout, TLS peer and host verification, a wire-byte cap, and a one-second maximum poll interval for cooperative stop checks.

The incremental `fasta_chunk_emitter` in `main/bioseq/external/io/sequence_stream.cppm` discards headers and whitespace while retaining the first header token as the record identifier. For `sequence_chunks`, it flushes fixed-size base chunks and holds one chunk until it knows whether it is final. For `sequences`, it intentionally accumulates a complete record before publication; this is necessary to return one SQL datum and means that the whole-record relation is not record-size-independent. Multi-record archive members reset `base_offset` for each record while keeping a global `chunk_index`, so the pair (queried accession,chunk index) remains unique.

NCBI assembly accessions beginning with `GCF_` or `GCA_` take a different route through `main/bioseq/external/io/datasets_stream.cppm`. The Datasets API returns a ZIP package. The current implementation downloads the compressed ZIP into a bounded in-memory vector with a 64 MiB default wire cap, then uses libarchive to enforce entry and total decoded limits, verify the manifest/MD5 data, and feed FASTA members to the same emitter. It is therefore bounded, archive-safe and incremental after extraction begins, but it is not yet a fully streaming or spooled package transport. A seekable temporary spool for packages above the in-memory cap and the dehydrated `fetch.txt` re-fetch strategy remain explicitly deferred.

15.5.2. Backpressure and PostgreSQL cancellation

The producer must never call PostgreSQL APIs. `start_streaming` creates a `std::jthread`; that thread runs either `run_ncbi_sequence_stream` or `run_ncbi_datasets_stream` and communicates only by owning messages. This is the current, PostgreSQL-proven implementation even though the broader project guideline normally prefers work submitted through the TBB pool and DOC0372 proposed fused provider tasks. The chapter records the executed design rather than silently substituting the preferred tasking model. The current channel is bounded to 8 MiB and four items. Its implementation in `main/bioseq/external/provider/bounded_channel.cppm` charges both bytes and items. A popped item remains charged until its move-only lease is released, preventing the consumer from nominally emptying the queue while retaining unaccounted payload memory. One oversized item may enter an otherwise

empty channel; the admission equation accounts for that staging item and thereby avoids producer deadlock.

The PostgreSQL thread calls `pop_for` with a 100 ms timeout. A timeout releases the channel mutex before `CHECK_FOR_INTERRUPTS` is invoked. This ordering is essential: PostgreSQL cancellation may long-jump, and jumping while holding a C++ mutex would permanently block the producer. Teardown first cancels the channel, waking a producer blocked in `push`, and then lets the `jthread` join before destroying the channel. Rescan follows the same cancel–join–reset order.

Retries obey an emission barrier. The direct FASTA path retries transport errors, HTTP 408, 429 and selected 5xx statuses with bounded exponential backoff and jitter, but only while the emitter has published zero rows. Once a chunk is visible to the executor, restarting the transfer could duplicate rows; a subsequent failure is terminal. The archive path similarly retries the download before extraction, but never restarts after extraction has begun to emit.

15.6. From remote letters to a compact PostgreSQL sequence

Provider messages remain PostgreSQL-independent. The current `owning_sequence_payload` in `main/bioseq/external/provider/model.cppm` owns raw base letters in a `std::string`, together with accession, interval, chunk and audit metadata. It is not itself a `compact_sequence_range`. This is a deliberate boundary: network parsing can be built and tested without PostgreSQL headers or memory contexts.

Packing occurs on the PostgreSQL thread in `set_seq` within `main/bioseq/backend/pg/external/row_writer.cppm`. The function calls the active SIMD target's `stdc_convert_from_cstr<bioseq::` with `pg_alloc`. It normalises nucleotide input according to the configured PostgreSQL sequence alphabet and creates a varlena `bioseq.seq` datum in the current memory context. A chunk lease is released only after this datum has been copied into the virtual tuple slot. Downstream operators therefore receive exactly the same packed type as locally loaded sequences; alignment code does not need a provider-specific representation.

This conversion currently involves one raw-letter payload and one packed PostgreSQL allocation per emitted row. DOC0372 proposed a richer `sink_control` and packing-parser path that could write directly into a reserved sink. That refactor is not yet part of the proven implementation. The existing boundary trades one bounded conversion for a much simpler ownership rule: worker code owns ordinary C++ values, while only the backend thread owns PostgreSQL datums.

15.7. Authentication, transport policy and errors

15.7.1. Authentication and configuration

Generated clients support an OpenAPI API-key scheme in either a header or query parameter. Credentials are currently owning `std::string` values. The NCBI adapter reads `NCBI_API_KEY` when it creates the generated metadata client; the direct nucleotide FASTA path is unauthenticated and does not place the key in its URL. The broader DOC0372 design proposed non-owning/redacted credential views and richer per-provider policy profiles. The concepts expose `request_policy_profiles()` and `locator_policy_profiles()`, but all four current provider adapters return empty spans. Locator enforcement for the NCBI stream is instead provided directly by `ncbi_eutils_policy`.

The metadata transport in `main/bioseq/external/https.cppm` uses blocking Oat++ I/O over a custom OpenSSL stream. It requires TLS 1.2 or newer, system trust roots, SNI, certificate-chain verification and hostname verification. Socket reads and writes have 30-second timeouts. A per-client connection pool is configured for up to 32 connections, a 30-second idle lifetime and a 10-second acquisition timeout. Request targets, header names and header values are validated before transmission, and decoded response bodies are bounded.

The streaming transport has the stricter locator firewall described above. Its current DNS resolution uses the system `getaddrinfo` call before pinning the result. The proposed cancellable c-ares/OpenSSL endpoint connector for the Oat++ control plane remains deferred because it would replace a proven TLS implementation. Thus cancellation of a running libcurl stream is cooperative and bounded mainly by its poll/timeout settings, rather than by an immediate cross-thread socket close.

15.7.2. Rate limits and failure representation

The provider core defines a structured, bounded `external_error` in `main/bioseq/external/provider/errors.cppm`. Its kinds distinguish invalid queries, unsupported capabilities, transport and timeout failures, HTTP errors, rate limits, incomplete responses, size limits, parsing, integrity, cancellation and provider-contract violations. Diagnostics, response excerpts, operation names and identifiers have compile-time length caps, so an untrusted upstream body cannot become an unbounded error object.

`gcra_request_policy` in `main/bioseq/external/provider/rate_limit.cppm` implements a process-local generic cell-rate algorithm with burst tolerance, in-flight slots, `Retry-After` penalties and a credential/origin/tier bucket key. Its atomic rollback and concurrency behaviour are tested. It is not yet wired into the provider request path, and it is not cluster-wide across PostgreSQL backends. The current NCBI stream instead uses local per-request retry loops. Consequently, several simultaneous backends can still exceed a shared public quota; DOC0374 records a shared-memory GCRA variant and multi-backend proof as future work.

The intended PostgreSQL mapping assigns `0A000` to unsupported capabilities, `08000` to connection-class failures, `22000` to data/parse/integrity failures, `54000` to size limits and `57014` to cancellation. The current custom scan implements only the most important distinction: unsupported or unimplemented operations become `0A000`; other provider errors are presently collapsed to `38000` (external routine exception). C++ exceptions are caught at every PostgreSQL callback boundary, and message text is copied into PostgreSQL-managed storage before `ereport(ERROR)` performs a long jump. The richer mapping documented in `main/bioseq/backend/pg/external/errors.cppm` is therefore a design contract that has not yet been fully connected to the scan.

15.8. Measured behaviour and limits of the evidence

The newest implementation evidence is DOC0374, `main/DOCS/CURRENT/REPORTS/0374_DOC0372_NCBI_STREAMING_DATA_PLANE_IMPLEMENTATION_AND_POSTGRES_PROOF.md`. It records a real PostgreSQL proof in which all six external relations planned as `Custom Scan (BioseqExternalScan)` with no `Seq Scan`. The proof covered NCBI, UCSC, UniProt and EBI and checked both results and plan shape. The UniProt protein metadata path returned TP53 accession `P04637`; protein sequence materialisation returned the intended `0A000`. The previously used accession `P62988` had become inactive/demerged upstream, an example of why live-provider fixtures must be reviewed rather than treated as immutable constants.

Table 15.3: Selected current external-provider evidence from DOC0374.

Case	Observed result	Interpretation
Human mitochondrial accession `NC_012920.1`	16,569 bases; one final chunk	Correct short-record boundary handling.
E. coli K-12 `NC_000913.3`	$4 \times 1\,048\,572 + 447\,364 = 4\,641\,652$ bases	Five ordered chunks; offsets monotonic; only the tail is final.
SARS-CoV-2 assembly `GCF_009858895.2`	one 29,903-base final chunk	Live Datasets ZIP, manifest/archive and PostgreSQL path.
Influenza A assembly `GCF_000865725.1`	eight segments, 13,588 bases	Multi-record archive; distinct original identifiers and globally unique chunk indices. The `sequences` relation also returned eight whole-segment rows.
External automated suite	24 cases, 108 assertions; 949 assertions in 53 whole-suite cases	Transfer, archive, rate-policy and provider integration coverage reported by the isolated external target.

A live source test streamed the 4.64-Mbase E. coli record with a measured carry-buffer peak no greater than 256 KiB, approximately 18 times smaller than the response body. A separate deterministic benchmark fed a 16,777,216-base local FASTA through the production `fasta_chunk_emitter`. The streaming path reached 206.3 MiB/s with a 1.06 MiB peak rechunk buffer; the whole-body comparator reached 274.6 MiB/s with 32.00 MiB of peak live buffering. The result is a 30.2-fold measured memory reduction, not a throughput victory. The comparator only counted letters while the streaming path constructed payloads, so the speed difference should not be interpreted as a general network or SQL slowdown.

The benchmark is a standalone Catch2 measurement, not a case in the formal `bench-io` matrix. The PostgreSQL transcript gives an end-to-end functional proof but not a statistically repeated latency study; its recorded 202.533 ms mitochondrial `EXPLAIN ANALYZE` run includes public-network conditions and cannot serve as a stable performance baseline. The defensible conclusions are narrower: plan selection was correct, emitted sequence lengths matched source truth, record boundaries were preserved, and the direct FASTA working set was bounded independently of record size.

15.9. Final architecture and remaining work

The retained solution is a two-plane system. Generated Oat++ clients provide a reproducible, validated control plane for small JSON responses. Concept- checked provider adapters translate these responses into one canonical query and message model. A storage-free PostgreSQL access method forces reads through a custom scan. For the large NCBI nucleotide path, a validated libcurl source, incremental FASTA emitter and byte/item bounded channel form a separate data plane. The PostgreSQL thread alone packs letters into `bioseq.seq` and interacts with executor slots.

Compared with the original generated-client-only implementation, this final path changes the asymptotic memory behaviour of `sequence_chunks`: the record length no longer determines the complete response buffer. Compared with the full DOC0372 proposal, however, the implementation is intentionally smaller. Metadata and non-NCBI sequence responses are still collected; NCBI Datasets ZIP bytes are held in a bounded vector; whole-record `sequences` rows still require one record-sized payload; the process-local GCRA is not wired or shared between backends; the provider policy spans are empty; the scan uses hard-coded channel

capacity rather than the complete prepared-admission path; the push-based transfer service and direct packing parser are partial; and most structured failures still map to a generic SQLSTATE.

These limits do not invalidate the delivered abstraction. They identify the next engineering work without presenting an aspirational design as current behaviour. The essential result is already useful: remote biological databases appear as typed, joinable PostgreSQL relations; provider-specific HTTP details do not leak into SQL; nucleotide chunks enter the same compact sequence type as local data; and the one path most exposed to genome-sized responses has measured, explicit backpressure and memory bounds.

15.10. Implementation reference map

Table 15.4: Authoritative current source locations for the external-provider chapter.

File	Principal symbols or responsibility
`main/bioseq/external/provider/concepts.cppm`	`external_api_session_implementation`, `external_api_provider_implementation`.
`main/bioseq/external/provider/pack.cppm`	`external_api_provider<...>` and one-time runtime dispatch.
`main/bioseq/external/providers/configured.cppm`	`configured_external_api_provider`.
`main/bioseq/external/providers/run.cppm`	`run_external_query`, the current collecting bridge.
`main/bioseq/external/transport/locator.cppm`	`validate_candidate`, `validated_transfer_request`.
`main/bioseq/external/transport/remote_source.cppm`	`remote_byte_source`, bounded libcurl pull source.
`main/bioseq/external/io/sequence_stream.cppm`	`fasta_chunk_emitter`, `run_ncbi_sequence_stream`.
`main/bioseq/external/io/datasets_stream.cppm`	`run_ncbi_datasets_stream`.
`main/bioseq/backend/pg/external/scan.cppm`	`BioseqExternalScanState`, path/plan/executor callbacks, producer and channel drain.
`main/bioseq/backend/pg/external/row_writer.cppm`	`bioseq_external_fill_slot`, `set_seq`.
`main/bioseq/backend/pg/external/sql_catalog.cppm`	Annotated definitions of the six SQL relations.
`main/codegen/external/client_renderer.cppm`	Generated request/response API and Oat++ client implementation.

Chapter 16

C-driven SQL code generation

16.1. Why the SQL boundary became generated code

The preceding chapters describe algorithms, sequence representations, storage, and PostgreSQL execution as parts of one system. They meet at a comparatively small but unusually consequential interface: the extension installation script and the C function-manager boundary. PostgreSQL must know the SQL name and signature of every routine, while the shared library must export a C symbol with a matching `pg_finfo_*` record. The C++ implementation must decode the same argument types, implement the same null policy, and materialize the declared result shape. Tables, sequences, operators, operator classes, aggregates, access methods, configuration variables, grants, comments, and extension dependencies add further copies of the same facts.

In the original arrangement these facts were distributed between a handwritten installation script, PostgreSQL macros, C++ conversion code, catalog constants, GUC registration calls, tuple deformation code, and documentation. Each copy could be locally reasonable and still disagree with another. A function rename, for example, could change the SQL name without changing `prosrc`. A new nullable parameter could be added to SQL while the wrapper continued to call `PG_GETARG_*` unconditionally. A table column could move while an `Anum_*` constant retained the former ordinal. These failures are difficult to detect by reading one file because the defect is a relationship between files.

The final BioSeqDB design treats the C++ declaration as the semantic owner and generates the redundant PostgreSQL surface from compiler-observed facts. This is more than replacing string templates with another templating language. A generator that merely copies handwritten SQL fragments would preserve the same duplication in a less visible form. The implemented system instead performs structural C++ analysis, evaluates a closed annotation model, builds a typed dependency graph, renders SQL and C++ module facets, and publishes manifests that can be joined back to the installed catalog.

Table 16.1 summarizes the principal drift classes. The rightmost column is important: generation does not make an invalid declaration correct. It makes the declaration's obligations explicit and rejects an incomplete or contradictory program before it becomes an extension package.

The source tree reflects this separation of responsibilities. The public annotation vocabulary lives in `main/codegen/sql/annotation_model`. The compiler-facing representation is in `main/codegen/clang_structural`. SQL collection, graph normalization, and rendering occupy the corresponding `collector`, `graph`, and `renderer` module directories. PostgreSQL error analysis is a separate subsystem under `main/codegen/safety`. The reviewed installa-

Duplicated fact	Handwritten failure mode	Generated invariant
Routine identity	SQL name, `prosrc`, exported C symbol, and `pg_finfo_*` diverge	The declaration leaf derives all three names. The manifest and DSO ledgers require one owner for every installed identity.
Argument contract	SQL type, nullability, default, or name differs from the C++ conversion path	The collector combines C++ type facts with typed parameter annotations and the renderer selects a closed converter.
Result shape	`RETURNS TABLE`, composite layout, and tuple materializer disagree	Generated result identities and row codecs are checked against field order, OID, ownership, and nullability.
Catalog ordering	An operator, cast, view, or opclass is emitted before its prerequisite	Explicit and inferred edges form a dependency graph. Compatibility order is recorded separately in a checked physical ledger.
Configuration	GUC name, default, bound, enum value, storage cell, and help text drift	An annotated static member and its initializer own the registration descriptor. Profile parity is checked.
Error handling	PostgreSQL `ERROR` crosses a live C++ object or an exception escapes an fmgr callback	Generated wrappers use one audited boundary, while a CFG and lifetime ledger must classify every reachable recoverable error site.

Table 16.1: Failure modes addressed by C-driven SQL generation.

tion artifact remains `main/bioseq/sql/bioseq--1.0.sql`, but it is a golden output rather than a second authoring surface.

16.2. Architecture and authority flow

Figure 16.1 shows the complete generation and qualification pipeline. Solid arrows carry semantic data. The dashed arrows are verification joins. The generator does not scan an arbitrary directory and guess which files matter. CMake registers the source and output ownership of each stage. A PREPARE step selects commands from the configured compilation database, normalizes them for the linked Clang toolchain, builds the required named module closure, and emits an owned compilation database plus a PCM manifest. Only then may the structural pass visit declarations. The exhaustive post-FINAL compiler-facts and PostgreSQL-ERROR closure is an explicit qualification target. Ordinary builds retain cheaper ledger, determinism, static-contract, and runtime checks rather than rerunning that entire analysis on every edit.

Three generation stages break the apparent bootstrap cycle. Support collects dependency-only row and GUC authorities that are forbidden from importing generated modules. Core emits profile-invariant catalog handles and proves the descriptor hash common to the production and `BIOSEQ_TEST_INDEXES` profiles. Final scans the remaining owners after their support modules are available and emits the complete SQL, manifest, and entrypoint set. The configured profiles have different active ASTs, so conditional test declarations cannot leak into production merely because both appear in the source text.

Generated production C++ is always a named-module `.cppm` artifact. It is partitioned by stable owner instead of accumulated in one translation unit. An fmgr owner module imports the smallest implementation closure needed to call its C++ declarations. A row-codec module

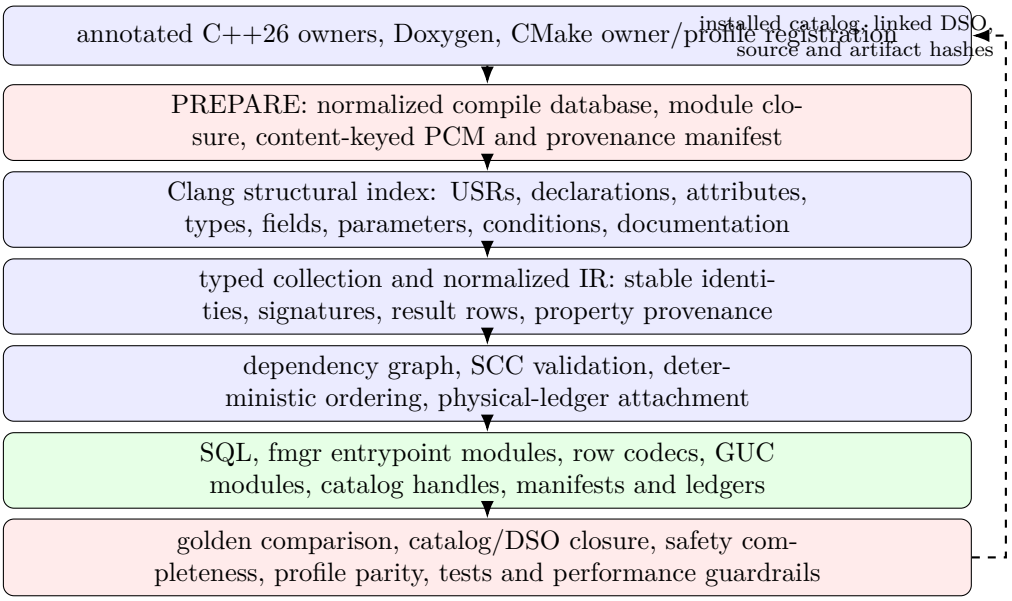


Figure 16.1: The SQL-generation authority flow. Generation and verification use the same registered source, profile, toolchain, and module provenance.

imports the dependency-only `sql_rows` declaration module. Catalog handles form profile-neutral structural descriptors. The GUC registry imports the generated owner facets and invokes their idempotent registration functions. This topology controls both compile cost and module cycles.

The same structural infrastructure is also used by the external-client and website components, but their collectors remain distinct. In particular, `bioseq::api_client` is not reinterpreted as an SQL annotation. This separation allowed the former Clang shim and duplicate AST traversal to be replaced without changing the external API client's output contract.

16.3. The C++26 annotation model

16.3.1. A typed vendor namespace

All SQL annotations use the flat vendor namespace `bioseq_sql`. Thus `[[bioseq_sql::function(...)]]` is valid, whereas a nested spelling such as `[[bioseq::sql::function(...)]]` is rejected. The distinction is not cosmetic. Standard C++ attribute syntax provides one vendor namespace, and a compiler may otherwise ignore a misspelled vendor attribute. The structural collector therefore inventories every `bioseq_sql` token and fails if one is not collected, recognized, placed on a permitted declaration kind, and evaluated by the closed model.

The public model uses structural fixed-capacity text values, typed SQL type identifiers, concrete option structures, and `consteval` builders. A builder such as `function(function_options)` returns an `annotation_spec<function_options>`. Because the overload accepts a concrete aggregate, short forms such as `column({.sql_type = pg::bigint})` are checked by C++ rather than by an ad-hoc dictionary parser. The model also provides `type_of<T>`, fixed and extension-schema bindings, typed declaration references, and `depends(...)` for dependencies which cannot be inferred from the declaration alone.

Listing 16.1 combines representative forms from `main/codegen/sql/tests/annotation_probe.cppm` and the production GUC style in `main/bioseq/backend/config/sql_guc_settings.cppm`. The operator is attached directly to its backing function. The table derives column identity from fields. The GUC derives its name from the qualified static member, its default from the initializer, and its short description from `@brief`.

Listing 16.1: Representative C++26 SQL annotations.

```
struct [[bioseq_sql::type({
    .internal_length = 2,
    .passed_by_value = true,
    .alignment = bioseq::sql::type_alignment::int2,
    .storage = bioseq::sql::type_storage::plain,
})]] letter {
    [[bioseq_sql::column({
        .sql_type = bioseq::sql::pg::smallint,
    })]]
    std::int16_t rank{};

    [[bioseq_sql::function({
        .volatility = bioseq::sql::volatility::immutable,
        .parallel = bioseq::sql::parallel_safety::safe,
    })]]
    [[bioseq_sql::operator_({.symbol = "="})]]
    static bool letter_eq(letter left, letter right);
};

struct [[bioseq_sql::table({
    .schema = bioseq::sql::fixed_schema<"bioseq_internal">,
    .create = bioseq::sql::create_mode::if_not_exists,
})]] row_probe {
    [[bioseq_sql::column({})]]
    std::int64_t storage_id;
};

struct settings final {
    /** @brief Enables extension diagnostic logging. */
    [[bioseq_sql::guc({
        .context = bioseq::sql::guc_context::superuser,
    })]]
    static inline bool logging = false;
};
```

Routine names and public C symbols are intentionally absent from `function_options`. They are derived from the unqualified C++ function name. Parameters similarly acquire their ordinary names and defaults from the function declaration. A parameter annotation is necessary only when the SQL type cannot be inferred strongly enough, for example a raw PostgreSQL `Datum` representing `bioseq.seq` or `regclass`. Redundantly restating an inferred property is rejected. A conflicting value is rejected with a distinct diagnostic. This policy prevents annotations from becoming another complete copy of the declaration.

16.3.2. Documentation and checked SQL

Documentation is also an owned input. A Doxygen `@brief` supplies the short description used for `COMMENT ON` or a GUC description. `@remark` supplies retained installation-script prose. `@ingroup bioseq_sql_public_api` controls website visibility. These facts are associated with a declaration by the structural AST pass, rather than recovered by searching nearby

comments. This matters for linkage blocks and multi-declaration source regions where lexical proximity is ambiguous.

Some extension routines are naturally expressed in SQL. They are still authored as typed C++ declarations returning `sql_function_body<...>`. Their body is a `checked_sql` value, as illustrated by `main/bioseq/backend/pg/sql/functions.cppm`. The declared return type and parameter list remain compiler-visible. The SQL parser can derive dependencies from the body, while the renderer controls the limited location at which an SQL fragment is permitted. This is different from a generic "run arbitrary SQL" action, which the production model does not expose.

The `mmsearch` declarations in `main/bioseq/backend/pg/mod.cppm` demonstrate how the generated catalog contract and custom-scan design meet. Two typed C++ overloads, of arity two and three, share the derived SQL symbol and declare `mmsearch_support`. Their direct bodies fail closed when no BioSeq executor owns the predicate. Placeholder result routines use the same annotation model for their declared support function and, where required, a high ordinary cost. The generator therefore does not decide custom-scan semantics. It preserves the exact catalog metadata which lets the planner code described in Chapter 10 implement those semantics.

16.4. Structural parsing from the real build

16.4.1. Why text scanning is insufficient

C++ attributes may span lines, contain nested aggregate initializers, refer to constants or types, and appear on functions, parameters, fields, aliases, enumerators, or static members. Comments and raw strings may contain text which resembles an attribute. Conditional compilation can make the same source path denote different declarations in the production and test profiles. A line-oriented parser cannot resolve these cases reliably.

The module `main/codegen/clang_structural/model.cppm` defines a compiler-independent wire model. Its `attribute_record` carries the qualified name, argument text, target USR, placement, and source ranges. A `declaration_record` adds the stable USR, declaration kind, qualified and enclosing names, signature, return type, initializer, active condition, owning module, parameter records, documentation, and export state. Aggregate records carry ordered field facts and structural errors. A translation unit also records its source SHA-256, compiler profile, Clang version, and tokens. No Clang AST pointer or `APValue` crosses this boundary.

The public functions in `main/codegen/clang_structural/index.cppm` make the authority distinction explicit. Source-only scans exist for contract tests. Production must call `scan_translation_units_from_database` and `scan_module_graph_from_database`. Both consume the owned compile database and PCM root. This prevents a successful toy parse with include paths, definitions, or modules that differ from the extension target.

16.4.2. PREPARE, module closure, and PCM provenance

The PREPARE operation begins with CMake's raw compilation database. It selects the registered module units, normalizes their commands to the linked Clang toolchain, and invokes dependency scanning to recover the named-module closure. The selected `clang`, `clang-scan-deps`, `llvm-config`, and compiler-module inventory are themselves hash-pinned. The supported authorities include the Support, Core, Final, and safety passes, with separate production and test-index profiles.

Each precompiled module receives a content key derived from the source, normalized arguments, dependencies, and imported-module keys. Its path includes a relocatable projection of that key. A sibling provenance file records the key, source and argument material, dependency hash, and PCM hash. Reuse is refused if the PCM lacks provenance or if either file is stale or corrupt. The resulting `prepared_compilation_environment` contains:

- the owned compilation database and PCM manifest;
- an incremental-build depfile;
- absolute and relocatable compilation signatures;
- relocatable hashes of the database and manifest; and
- the complete dependency path set.

Before any structural scan, the generator calls `validate_prepared_compilation_environment`. Validation recomputes the toolchain, profile, authority, source, dependency, provenance, and PCM joins. A cached BMI is therefore a performance optimization only after it has been proven to represent the requested source state. This design was especially important during the large migration because several CMake build trees and both profiles could coexist.

The structural implementation is isolated in one module implementation unit whose global module fragment includes Clang and LLVM headers. The exported model contains only standard-library and project value types. Consequently a GCC-built generator interface does not serialize Clang implementation types, while the Clang structural pass still has the AST and token information needed for exact collection. The current build supports the prescribed GCC and Clang host configurations, while SQL structural collection uses the configured, pinned Clang path.

16.5. Collection and normalized identities

The SQL collector maps recognized attributes and compiler facts into `collected_entity` records. It first records all declarations, then resolves relationships. This two-phase policy is necessary for references to objects declared later and for reciprocal operator relationships. Each entity carries source module, path, range and hash, typed options with provenance, documentation, dependency references, routine parameters, result form, and physical compatibility bindings. Result-row records separately describe named fields used by set-returning routines and composite materializers.

The type registry is closed. It distinguishes an SQL type from the physical fmgr or executor representation used to carry it. For example, PostgreSQL `character` is `BPCHAROID`, not the internal one-byte type `"char"`. Scalar input uses a move-only `bpchar_view`, while an owning result uses `bpchar_value`. Vector comparison owners use a private vector operand whose storage cells may be `Datum` even though the logical lane is a character value. The same principle covers nullable values, arrays, composites, injected `sql_call_context`, table-AM handlers, and set-returning results. An unregistered C++/SQL pair is a generation error, not an opaque `Datum` fallback.

16.5.1. Stable and installed identities

Every graph node receives a stable textual identity. The current `routine_signature`, `stable_identity`, and `installed_identity` functions are defined in `main/codegen/sql/graph/index.cppm`.

Routine identities include the schema, name, and canonical SQL input types, for example

`sql:function:bioseq.mmsearch(bioseq.seq,bioseq.seq).`

The signature uses a dedicated `stable_identity_sql_type` when a compatibility spelling differs from the rendered spelling. GUCs use a normalized C++ qualified-member path. Schemas and typed install owners have kind-specific forms. This prevents source order and compiler-internal numeric identifiers from becoming external identity.

Extension-owned schema bindings retain the token `$extension` in the stable identity. The installed identity replaces that token with the schema selected by `CREATE EXTENSION`. SQL rendering deliberately leaves extension-owned object names unqualified where PostgreSQL relocation requires it, while the manifest contains both stable and install-resolved views. This distinction lets documentation and parity tools reason about a relocatable extension without pretending that one installation schema is universal.

16.5.2. Dependencies and deterministic ordering

Edges arise from explicit `depends` references, declaration-reference options, `type_of<T>`, enclosing companions, schema ownership, operator and opclass relationships, access-method roles, and parsed checked SQL bodies. Ambiguous references are rejected. Dependencies are stored as stable identities and sorted, making the graph independent of input discovery order.

The graph may contain legitimate strongly connected components, such as catalog objects that require shell and definition phases. Other cycles are errors. The implementation in `main/codegen/sql/graph/index.cppm` uses Tarjan's algorithm to identify components and then performs a lexically stable topological sort of the condensation graph. Algorithm 16 gives the essential procedure.

Algorithm 16 Deterministic ordering of collected SQL entities

Require: entities E with resolved dependency sets $D(e)$
for all $e \in E$ do
    assign StableIdentity(e) from kind, schema, name, signature

$C \leftarrow$ TarjanSCC(E, D)
for all components $c \in C$ do
    if c is cyclic and contains a forbidden entity kind then
        reject the graph with the first stable identity

build condensation DAG G_C
$Q \leftarrow$ zero-indegree components ordered by their smallest stable key
while Q is not empty do
    remove the smallest component c
    emit members of c by entity-kind order, then stable key
    decrement consumers and insert newly ready components into Q

reject unless every entity was emitted exactly once

There are two different orders in the final design. The graph order is semantic evidence that prerequisites are understood and cycles are controlled. The physical order is a compatibility commitment for the emitted installation script. Conflating them would either make

legacy order an accidental semantic rule or let a valid graph reorder a reviewed script unnecessarily.

16.6. Rendering SQL and PostgreSQL C++ boundaries

16.6.1. SQL and module facets

The SQL renderer projects the normalized graph into schemas, tables, sequences, functions, procedures, types, casts, operators, aggregates, operator classes, access methods, views, privileges, comments, configuration dumps, and the bounded typed install actions. Clause spelling and property presence remain explicit where PostgreSQL distinguishes omission from a specified default. Checked SQL bodies are parsed in the permitted entity contexts. There is no general raw statement escape hatch.

One source owner can produce several facets. A routine may produce an SQL statement, a C entrypoint, documentation, and a manifest row. A table may produce SQL, a catalog comment, a row codec, and a catalog handle. The renderer records the relationship rather than making one artifact the authority for another. Table 16.2 lists the main outputs.

Artifact	Format and ownership	Role
Installation script	Build-tree SQL plus checked source golden	Creates the extension catalog surface in reviewed physical order.
Entity manifest	Profile-specific JSON and optional C++ access facet	Carries stable and installed identities, source provenance, dependencies, visibility, converters, and generated ownership.
Fmgr entrypoints	Owner-partitioned generated `.cppm`	Exports C symbols and finfo records, converts arguments, calls typed owners, and materializes results.
Row codecs	Generated `.cppm` beside dependency-only `sql_rows`	Centralizes tuple ordinals, datum conversion, null policy, deformation, and result construction.
GUC facets	Generated owner modules and registry module	Registers authored storage cells, enum tables, bounds, flags, hooks, and descriptions for the selected profile.
Catalog handles	Profile-invariant generated `.cppm`	Resolves declaration-derived relation identities with expected relkind and fresh catalog validation.
Audit ledgers	Checked JSON, TSV, and generated evidence	Joins logical owners to physical statements, C symbols, result rows, GUCs, safety sites, and installed catalog objects.

Table 16.2: Principal products of the SQL generator.

16.6.2. Generated fmgr entrypoints

For an ordinary routine, the fmgr renderer selects a converter for each SQL parameter, injects a call context when declared, calls the typed C++ function, and passes the result to `sql_result_to_datum`. Named and set-valued rows use a generated encoder derived from the result-row record. Void, nullable, scalar, array, composite, polymorphic, record, and SRF forms have separate closed paths.

The public wrapper has global C linkage and exactly one `PG_FUNCTION_INFO_V1` declaration. Its body invokes the common SQL boundary with a probe descriptor and a trivially destructible lambda. For the two audited overload families, the public symbol dispatches on `PG_NARGS()`. The renderer rejects symbol sharing for every name except `from_array` and `mmsearch`, and those two must expose exactly arities two and three. This turns a former naming accident into an explicit, testable exception.

Listing 16.2 shows the essential generated shape. The per-arity helper, converters, and probe identifier are generated from the normalized entity and omitted here for clarity.

Listing 16.2: Abridged generated fmgr wrapper shape.

```
extern "C" {
PG_FUNCTION_INFO_V1(symbol);
}

extern "C" PGDLLEXPORT Datum symbol(PG_FUNCTION_ARGS) {
    return bioseq::pg_utils::invoke_sql_boundary(
        bioseq::pg_utils::sql_boundary_probe_descriptor{
            .site_id = "stable␣generated␣site",
            .fault_key = "generated_boundary_tail.std_exception",
        },
        [fcinfo] -> Datum {
            return invoke_symbol_arity_n(fcinfo);
        });
}
```

The normal generated-routine predicate has one reviewed exception: `bioseq_external_handler`, the external table access-method handler, retains its special handwritten boundary. The exception is encoded by exact function identity, source module, schema, and result type in `main/codegen/sql/renderer/fmgr.cppm`. It is therefore visible and bounded, rather than a convention that any inconvenient routine may bypass generation.

16.6.3. GUC registration

Primitive GUCs are generated into calls to `DefineCustomBoolVariable`, `DefineCustomIntVariable`, `DefineCustomRealVariable`, or `DefineCustomStringVariable`. The address passed to PostgreSQL is the annotated static storage cell. Defaults and bounds are authored C++ expressions. Context, flags, and optional hooks are typed annotation properties. The generated registration closure is statically required to be trivially destructible and is executed through `invoke_pg_as_cpp`.

Enum GUCs use `postgres_enum_registration<Enum>` and generated value tables. Enumerator annotations may omit sentinel values or provide hidden aliases. The generated registry calls every owner registration once and publishes a profile count. Current collector and renderer contract tests require 76 production authorities and 77 test-index authorities. Thirteen production settings carry `no_show_all`, yielding the 63 visible `bioseq.%` settings observed by the final PostgreSQL proof. These numbers describe different projections: generated authority inventory versus ordinary runtime visibility. The distinction is checked rather than resolved by discarding either count.

16.7. Physical order, manifests, and provenance

16.7.1. A compatibility ledger which owns no SQL

The checked physical ledger is a three-column TSV. Its `parse_physical_ledger` and `apply_physical_ledger` implementations are in `main/codegen/sql/physical_ledger.cppm`:

⟨`PNNN`, facet, natural owner stable identity⟩.

The permitted facets are `primary`, `catalog_comment`, `base_type_shell`, and `base_type_definition`. Parsing requires exactly three tab-separated fields and a contiguous sequence from `P001`. Duplicate or missing owners fail generation. Crucially, a ledger row contains no SQL text and no alternate descriptor. It can preserve reviewed physical order without becoming a disguised second implementation.

The core file `main/bioseq/sql/bioseq-physical-ledger.tsv` contains `P001` through `P487`. The optional external overlay continues with `P488` through `P500`. Applying a ledger attaches natural projection bindings to the actual C++ owners. During migration the same mechanism checked the transitional graph against reconciled legacy statements. After transition owners were removed, only the natural owners and their compatibility ordinals remained.

This split solves a subtle problem. A stable dependency graph may legitimately choose a different order after adding a new independent object. A reviewed installation script may nevertheless need minimal textual change for auditing and upgrade analysis. The graph proves that the requested order is legal. The physical ledger states which legal order is canonical. Neither is asked to perform the other's task.

16.7.2. Manifests as joinable evidence

The manifest renderer emits repository-normalized paths, source hashes, compiler profile and signature, entity options, stable and installed identities, dependencies, generated artifacts, fmgr owner and converter keys, result rows, and GUC descriptors. Timestamps, process addresses, and machine-specific build paths are excluded from canonical content.

These records support checks which a textual SQL diff cannot express:

- every `LANGUAGE C` catalog identity joins to one generated or explicitly reviewed C owner;
- every generated public symbol joins to at least one SQL identity, with only the two reviewed overload groups sharing;
- every row result joins to a generated codec with the same ordered types and nullability;
- every catalog handle joins stable and installation-schema identities;
- every GUC storage descriptor joins the selected profile and live PostgreSQL registration; and
- website-visible entities form an explicit subset rather than a source directory heuristic.

The migration ledger in `main/codegen/safety/fmgr_ledger.cppm` fixes the closed routine inventory at 270 C-language SQL identities and 268 public C symbols. The difference is exactly the two same-name overload families. Disposition values record unchanged, retired, shared, BPCHAR-repair, and native-C++ cases. The corresponding manifest policy requires complete generated ownership and rejects a missing, duplicate, or unexpected symbol.

16.8. PostgreSQL errors across a C++ boundary

16.8.1. Why a wrapper catch is not sufficient

PostgreSQL's recoverable `ERROR` mechanism unwinds its own error stack with a non-local jump. It does not behave like a C++ exception and therefore does not automatically execute C++ destructors in frames it crosses. A `std::string`, detoast guard, tuple owner, lock guard, or lambda capture which remains live at such a site can leak resources or leave state inconsistent. Conversely, a C++ exception must never cross a PostgreSQL C callback ABI.

The common `invoke_sql_boundary` handles the outer translation. Within the typed lambda, PostgreSQL errors intended to cross C++ are represented by `PostgresErrorAsCpp`. The boundary catches that type, a standard exception, or an unknown exception. It moves or copies the diagnostic into POD-only state. Only after the catch handler and all owner locals have ended does it call `ReThrowError` or a final `ereport(ERROR)`. The generated probe descriptor identifies the boundary tail for runtime fault-injection evidence.

This wrapper is necessary but not sufficient. Owner functions can call PostgreSQL indirectly through helpers, function pointers, SPI, catalog lookups, allocation, or conversion code before control reaches the outer catch. Safety therefore has to be established over the reachable call graph and the lifetime state at each error site.

16.8.2. Closed-world CFG and lifetime analysis

The safety model records compilation profile, PostgreSQL severity, target resolution, bridge kind, lifetime crossings, and provenance. Recoverable sites must receive one of four classifications:

`bridged` the call executes through the general PostgreSQL-to-C++ error bridge;

`specialized_bridge` an audited mechanism such as an SQL subtransaction or call context owns the transition;

`lifetime_free_direct` no nontrivial C++ lifetime crosses the direct site; or

`terminal_after_unwind` the terminal PostgreSQL raise occurs only after relevant C++ state has been unwound.

`unclassified` is always a failure. Unresolved indirect targets, unavailable bodies, incomplete control-flow assembly, or a call-graph cycle without a fixed point are also failures. The analyzer builds Clang control flow and lifetime facts for both production and test-index profiles, expands the fmgr owner closure, and computes a fixed point over direct and implicit call edges. The final ledger must have complete templates, macros, function bodies, CFGs, lifetime inventories, and call paths. A wrapper cannot be declared safe merely because no dangerous site was found in an incomplete graph.

Severity authority is versioned. The current policy pins PostgreSQL 17 at tag `REL_17_10` and revision `25c49f3a4a742ba283f5cc43cc7f1d361552e917`. It records hashes and source spans for recoverable-error, terminal-severity, bridge, and no-longjmp seeds. Reachable `FATAL`, `PANIC`, unresolved severity, or an unclassified recoverable site rejects the generated boundary. Runtime matrices then join injected and natural probes to the static classification and high-risk owner families. This is a completeness argument, not merely a collection of passing exception tests.

16.9. Deterministic generation and the golden workflow

Normal builds write SQL and modules to the build tree and compare them with checked artifacts. They never silently rewrite the source tree. The `bioseq_sql_check_golden` target regenerates with the current PREPARE environment, validates the manifests and physical ledgers, and checks byte equality. An explicit maintainer target, `bioseq_sql_update_golden`, is the only source-update path. The update is disabled when the optional external package is active, which avoids accidentally replacing the core golden with a profile overlay.

A reproducible update follows these steps:

1. configure exact owners, profiles, toolchain fingerprints, and expected generated paths;
2. PREPARE the owned compile database and content-keyed PCM closure;
3. generate support, core, and final artifacts into isolated build-tree roots;
4. validate SQL parsing, graph completeness, physical order, manifests, fmgr ownership, row codecs, and GUC parity, then run the exhaustive safety ledger when performing the full qualification path;
5. compare the proposed SQL, JSON, and `.cppm` outputs with their checked counterparts and print the bounded delta; and
6. only under the explicit update target, replace the approved checked artifacts and rerun the ordinary check path.

Generation signatures include registered source hashes, normalized compiler arguments, profile definitions, module and Doxygen inputs, toolchain hashes, and the generator executable. Depfiles make an ordinary incremental build sensitive to the same closure. Contract tests permute input discovery and run generation twice, requiring byte-identical output. A clean-tree gate removes generated outputs and BMIs before rebuilding, so determinism is not inferred solely from a warm cache.

The distinction between calibration and qualification is also deliberate. The SQL-codegen benchmark framework can measure cold and no-op generator time, peak RSS, generated SQL/JSON/C++ sizes, affected-target compile times, and database runtime paths. Runtime cases include both `from_array` arities, both `mmsearch` arities, a generated typed wrapper compared with an implementation-equivalent handwritten test-only wrapper, and generated row codecs compared with retained handwritten test-only codecs. Each case checks correctness and path attestation before timing.

A calibration report is visibly non-qualifying. It keeps build qualification and baseline kind at zero. A blocking qualification requires a sealed reviewed baseline, explicit numeric limits, an exact environment and source join, and live publication provenance. The repository records that no historical Stage-0 performance artifact was captured before cutover. Post-cutover recovery data cannot silently claim to be that baseline. This is an honest limitation: the system has a strong performance-regression protocol, but it does not manufacture a pre-migration speedup claim from later data.

16.10. Migration outcome and evidence

16.10.1. What changed from the original boundary

The migration began from a 3,502-line handwritten SQL baseline. The final checked script contains 3,504 lines. Line count is not the success measure. The important change is authority: the final SQL is a deterministic projection of typed owners, while its physical order and approved compatibility choices remain reviewable. Table 16.3 summarizes the transition.

The immediately preceding lazy-sequence implementation report records a centralized but still handwritten set of 64 SQL wrappers and a retained non-module C ABI exception bridge. The corrective SQL-generation inventory then enumerated 270 target C-language routine identities. It required all 494 input parameters to use their C++ identifier, including 309 inputs which had been anonymous in the baseline. Thus the migration was not simply a change in where wrapper text was printed. It converted previously implicit signature and ownership facts into checked graph data.

Concern	Original arrangement	Final arrangement
Installation SQL	Hand-maintained statements and comments	Generated graph projection with a checked 500-entry core-plus-overlay physical ledger.
C entrypoints	Macros and wrappers distributed among owner modules	Owner-partitioned generated `.cppm` entrypoints with closed converter and result policies.
Routine names	Legacy SQL/C-symbol mismatches and anonymous inputs	Declaration-derived leaves. The audited migration normalized 118 historical symbol mismatches and six additional BPCHAR routine identities.
PL/pgSQL	Procedural helpers and anonymous blocks mixed into install SQL	No PL/pgSQL and no anonymous `DO`. Required public behavior is native C++ or typed SQL-function ownership.
Character vector ABI	Six legacy `vchar*` spellings obscured BPCHAR semantics	Exactly six `vbpchar*` routines, `BPCHAROID`, owning vector-slot payloads, and scalar/vector parity tests.
Rows and catalog names	Manual ordinals, deformation, and name constants	Generated codecs, result-identity ledgers, and typed catalog handles.
GUCs	Separate storage declarations and `DefineCustom*` calls	Annotated storage members, initializer-derived defaults, enum tables, owner modules, and a generated registry.
Safety	Wrapper conventions and local tests	270-owner static closure, 268 symbol/finfo closure, versioned PostgreSQL provenance, and runtime classification matrices.

Table 16.3: Migration from the handwritten PostgreSQL boundary to the final C-driven design.

The six BPCHAR corrections illustrate why a generator migration can expose runtime defects rather than merely reproduce text. PostgreSQL SQL `char` in the legacy script meant `character`/BPCHAR, not the internal one-byte `"char"` type. The closed type registry forced

the logical OID and the custom vector storage representation to be reconciled. The final implementation owns detoasted scalar views and vector payloads for the correct lifetimes, preserves trailing-space and collation semantics, and removes the ambiguous legacy symbol names.

The migration also converted former procedural installation behavior. Public alter-set/reset functions, vacuum, and orphan reaping retained their SQL contracts through native C++ implementations. Unused compatibility helpers were removed rather than copied into a generic SQL-action mechanism. Access-method dependencies remain a typed fixed set. SQL-language helper functions use checked bodies and compiler-visible result types. These choices reduce the amount of executable behavior that exists only as installation text.

16.10.2. Checked artifact and test evidence

The final `bioseq--1.0.sql` has SHA-256 `c0a26b78c00cb7dbec4ed4f5a8ccbd148cf01f1edda1475cf08975d5b7` Source, generated, and installed copies were byte-identical in the recorded qualification. The audit found no PL/pgSQL, no anonymous block, exactly six standardized `vbpchar*` definitions, no legacy `vchar*` definitions, and the required nullable `age interval` output in `bioseq.list_metadata()`.

The focused regression selection passed four cases and 121 assertions. The final unfiltered unit and integration qualification executed 1,326 cases: 1,323 passed, none failed, and three were skipped. All 4,449,081 executed assertions passed. A disposable PostgreSQL 17 database then proved extension creation and removal, absence of PL/pgSQL, the 63 visible GUCs, standardized BPCHAR names, scalar/vector comparison parity, selection of `Custom Scan (BioseqScan)`, metadata and shard results, vacuum preservation, orphan reaping, and generated routine shapes.

The ordinary algorithm benchmark produced 13 groups, 79 execution tuples, and 392 populated cells in `report_192.json`. Its process status was nonzero because the framework reported eight historical performance regressions and three relative rules. The preceding full `report_187.json` already contained nine historical regressions and three relative rules. The code-generation migration changed neither the algorithm benchmark configuration nor the implicated alignment and prefilter kernels. Consequently this result is evidence that generation and report production completed, not evidence of a codegen speed regression or of a universal performance improvement.

The ordinary end-to-end path passed generation gates, extension installation, and PostgreSQL restart, then stopped because strict Vulkan was unavailable on the host. A CPU configuration completed its one case with four populated cells, and every correctness check and rule passed. These results support the claim that the generated boundary preserved the exercised runtime paths. They do not replace the dedicated sealed-baseline qualification described above. The implementation report also records that independent reviewer execution was skipped at explicit user direction. Its results are therefore strong implementation evidence, not an independent final audit.

16.11. Lessons and remaining limitations

The central lesson is that code generation is useful here because the boundary contains many relations between artifacts. A source generator which only prints SQL would remove typing from those relations. BioSeqDB instead uses the compiler to identify declarations, a closed IR to represent PostgreSQL meaning, a graph to resolve dependencies, generated modules to own the ABI, and manifests to make completeness queryable.

Several limitations remain explicit:

- the annotation model is intentionally closed. A genuinely new PostgreSQL feature requires a modeled entity, validation, rendering, and tests rather than an arbitrary SQL escape;
- Clang structural collection and named-module preparation are substantial build-time dependencies. Owner partitioning, depfiles, and content-keyed PCM reuse control their cost but do not make it zero;
- the physical ledger preserves the current install order. A future extension-upgrade system still needs version-to-version migration semantics, not only a reproducible version-1 installation;
- safety analysis is pinned to exact PostgreSQL source provenance. Moving to another major or upstream revision requires refreshing and reviewing that evidence; and
- no reviewed pre-cutover Stage-0 performance baseline exists. Current calibration and generated-versus-handwritten comparators can qualify future regressions once an approved baseline is established, but they cannot retrospectively prove a migration speedup.

Within those limits, the final system changes the PostgreSQL extension boundary from a collection of synchronized conventions into an executable contract. A C++ declaration, its SQL identity, its fmgr wrapper, its result materializer, its documentation, and its safety evidence can be traced through stable identities. An ordinary build checks artifact ownership, deterministic bytes, ledgers, and the reviewed SQL golden. The explicit qualification path additionally proves the exhaustive compiler-facts and PostgreSQL-ERROR closure. Together these gates let the algorithmic and storage components described throughout the thesis remain a single maintainable PostgreSQL extension rather than a set of separately versioned interfaces.

Chapter 17

Conclusions

17.1. What was built

This thesis began with a practical question. Could sequence alignment become a database operation without reducing either side of the combination to a thin wrapper? The resulting BioSeqDB system shows that the answer is qualified but affirmative. PostgreSQL can own compact biological sequences, relational metadata, persistent sequence indexes, and the lifecycle of an alignment query. The alignment engine can retain a recognisable seed–chain–extend structure while operating on database-backed and lazily composed sequence values. The planner can replace a scalar-looking SQL predicate with a batch-oriented custom scan. CPU vector units and Vulkan devices can be used where their data-shape and setup requirements are satisfied.

The qualification matters. BioSeqDB is not one algorithm and it is not one unconditionally faster execution path. It is a collection of explicit contracts between representations, algorithms, storage, planning, and runtime selection. A compact sequence range states its alphabet and packing geometry. A minimizer representation states the widths and coordinates that remain recoverable. A chaining implementation states which predecessor set and score semantics it evaluates. A prefilter states the domain in which it may reject a candidate. A GPU route states its admissible batch geometry and whether failure is allowed to fall back. A generated SQL boundary states the source entity, dependency, public symbol, and ownership policy from which an installed object was derived.

The system built around these contracts has five main layers:

1. The sequence layer represents packed, fragmented, PostgreSQL-backed, and lazy values through a common alphabet-aware access model. Slicing, concatenation, reversal, letter extraction, and explicit materialisation are expressed without requiring every consumer to accept a byte string.

2. The alignment layer implements a minimap2-derived pipeline. It extracts minimizers, finds seed hits, chains anchors, constructs regions, optionally prefilters mandatory gaps, extends selected regions, reconstructs CIGAR operations, and calculates final classifications and mapping qualities.

3. The heterogeneous layer supplies Highway SIMD and Vulkan implementations for selected sketching, chaining, filtering, and extension work. Admission, batching, resident state, and result proof are part of the route rather than hidden behind the same name.

4. The database layer stores sequence fragments and index generations, loads input through bounded streaming paths, exposes remote providers, and turns alignment expressions into custom paths, plans, and scans.

5. The boundary-generation layer derives SQL installation objects, fmgr entrypoints, row codecs, GUC bindings, catalog handles, and validation ledgers from structurally analysed C++ sources.

Together these layers are the principal contribution. Any one kernel could be called from PostgreSQL with much less work. The difficult part is preserving coordinate, lifetime, failure, and resource semantics across the complete query.

17.2. From the original pipeline to the final architecture

The starting point was minimap2's mature long-read mapper. Its core is a highly tuned imperative C pipeline over ASCII query strings, packed minimizer records, an in-memory or `.mmi` reference index, scalar chaining, and KSW2 extension. Work is distributed over CPU workers. Allocators and option branches are organised around a standalone process.

BioSeqDB retained the underlying algorithmic decomposition because it is a good decomposition. It did not retain the assumption that all data starts as an ASCII buffer or that one process owns the whole reference index. It also did not reproduce every minimap2 operating mode. The implemented focus is the sequence-search path used by the extension, not a byte-identical clone of pairing, spliced alignment, rescue, and every command-line preset.

Several intermediate designs were valuable precisely because they failed. Naive SIMD work exposed long dependency chains and data-rearrangement costs. Per-query GPU submissions lost to command, fence, and allocation overhead. GPU chaining without a resident upstream and downstream path moved too little work to amortise the boundary. A gather-heavy SneakySnake port preserved the decision procedure but not its performance. Loader overlap added queueing and copy costs to a workload with too little independent work. Persisting every fragment independently increased first-query work and write pressure. Cross-product template dispatch produced thousands of pipeline bodies before the parameter space was collapsed around valid representation cells.

The retained designs share a pattern. They move decisions to a boundary where enough information is available. Runtime configuration is converted to a fixed pipeline specialization before the hot stages. Accelerator work is grouped by compatible batch geometry. Target identity is used to reuse resident state. Mandatory and optional alignment regions are distinguished before prefiltering. Persistent index generations are accepted only after format and compatibility validation. PostgreSQL expression replacement is performed during planning, while per-row values are installed during custom scan execution. Generated SQL is accepted only when the dependency and physical ledgers are complete.

17.3. Answers to the research questions

Can the pipeline operate on a common compact sequence model? Yes, for the implemented paths, but the answer requires more than a range concept at a public function boundary. Iterators need precise symbol and bit geometry. Fragments need stable ownership and cross-boundary context. Algorithms that require contiguous ranks need an access plan that says when a bounded materialisation is unavoidable. PostgreSQL slices and concatenations need

Concern	Starting point	Final BioSeqDB approach
Sequence input	ASCII query strings and a mapper-owned reference	Alphabet-typed packed ranges, including database fragments and lazy compositions
Pipeline control	Runtime option branches inside an imperative mapper	Stage composition selected by bounded runtime-to-template dispatch
Minimizers	Scalar circular-window sketching over encoded DNA	Scalar parity path, Highway variants, rolling ntHash work, and bounded Vulkan sketch routes with explicit tie and invalid-symbol semantics
Chaining	Scalar dynamic programming and minimap2 RMQ variants	Scalar reference path plus bounded SIMD, RMQ, and tiled Vulkan options, each described with its actual predecessor and coordinate rules
Extension	KSW2 in the CPU mapper	Selectable KSW2, Block Aligner, batched CPU work, and Vulkan CIGAR or gridded traceback routes for supported jobs
Candidate filtering	Pipeline heuristics and alignment-stage rejection	Optional SneakySnake and GateKeeper processing of mandatory global gaps, followed by exact post-traceback predicate enforcement
Index ownership	Mapper process and `.mmi` files	Versioned PostgreSQL generations, fragment-native formats, metadata compatibility checks, selective cold reads, and accelerator residency
Execution interface	Command-line invocation and mapper output records	Planner support, custom paths and scans, batched multi-row results, and injected PostgreSQL result nodes
Boundary definition	Handwritten C and SQL declarations	Structural C++ annotations, generated SQL and fmgr modules, stable identities, physical ledgers, and fail-closed completeness checks

Table 17.1: The transformation from the minimap2-derived starting point to the final BioSeqDB architecture. The right column describes implemented system structure, not a claim that every route is selected for every query.

lifetimes that survive planner and executor transitions. The final abstraction makes these choices visible instead of converting every value eagerly.

Which work benefits from SIMD or GPU execution? There is no stage-only answer. Benefit depends on length, error profile, alphabet, intermediate representation, batch size, residency, submission count, and the amount of exact post-processing that remains on the host. SIMD is most effective when loads are contiguous and lane work is uniform. GPU speedups appear when many compatible jobs reuse resident inputs and keep intermediate state on the device. Small jobs and cold first submissions can remain slower. The final system therefore makes CPU, SIMD, and Vulkan routes policy choices whose execution is observable.

Can a batch mapper retain an ordinary SQL surface? Yes. The ordinary function definition provides PostgreSQL types and a fail-closed fallback contract. Its cost encourages planner support to choose a custom path. The custom plan carries the information required by `BioseqScan`. Execution batches input rows and may produce several alignment rows for one input. Injected `Const` and `Var` nodes allow filters, projections, and joins to observe alignment fields without calling an expensive scalar function independently for each row.

Can the surrounding system remain reliable and inspectable? The implementation provides strong mechanisms, but the result is not uniformly complete. Versioned storage, explicit memory admission, bounded PostgreSQL loading, transaction-aware persistence, custom-scan proof, hardware-only accelerator qualification, and generated physical ledgers make important failures visible. The chapter audits also identify places where an option exists but is not wired, a footer is accumulated but not validated, a provider path is partial, or a benchmark is a one-hit correctness gate rather than exact row-set parity. Inspectability requires reporting these facts, not smoothing them into a single success claim.

17.4. Limitations

The first limitation is scope. BioSeqDB is not a replacement for every minimap2 command-line mode or every graph mapper. Graph and de Bruijn approaches are included to explain alternative representations, not because the extension currently performs full variation-graph alignment.

The second limitation is heterogeneous completeness. Implemented Vulkan sketching, seed processing, chaining, filtering, and CIGAR work are real. The strictly attested persisted-format route in which no host biological planning remains is not complete for every storage and extension shape. Unsupported geometry must stay on a CPU-selected route or be accepted conservatively where the relevant algorithm permits it.

The third limitation is evidence comparability. Historical reports were produced on different code revisions and configurations. Some reports are dirty snapshots. Some GPU tests prove that every query has a correct hit but do not prove equality of all emitted rows. First-dispatch latency and steady-state median latency are not interchangeable. This thesis preserves those qualifications and treats the newest clean configuration-matched receipts as the strongest performance evidence.

The fourth limitation is operational hardening. The IO chapter identifies strict-alphabet configuration that is exposed but not connected to the active reader, gzip paths that do not yet compare every accumulated integrity value with the footer, and standalone output paths without atomic replacement and durability guarantees. External providers have a useful

PostgreSQL data plane, but generation, authentication, cancellation, and structured error coverage are not equally mature for every service.

Finally, database integration increases the amount of correctness surface. PostgreSQL can abort through longjmp, the planner may rewrite expressions, a custom scan may duplicate rows, and cached or persisted state may outlive the statement that created it. Generated fmgr and SQL ownership removes a major source of drift, but it does not remove the need for SQL, lifecycle, and failure-path testing.

17.5. Future work

The most valuable future work follows directly from the limitations:

1. complete the authenticated persisted-format accelerator path so that eligible queries can keep minimizers, occurrences, anchors, chaining, windows, extension, and traceback on the device under one attested route;
2. strengthen algorithm parity tests for RMQ predecessor admissibility, backtracking order, full emitted row sets, and all supported alphabet and coordinate widths;
3. finish reader integrity and strict-alphabet enforcement, then measure multi-record FASTQ and compressed workloads where bounded producer and consumer overlap has genuine independent work;
4. harden standalone writer durability with an explicit temporary-file, flush, synchronization, and atomic-publication protocol;
5. extend external providers with generated clients, cancellation-safe streaming, bounded retries, credential handling, and uniform structured SQL errors;
6. qualify the generated SQL and fmgr boundary across the intended PostgreSQL, operating-system, and architecture matrix; and
7. maintain benchmark qualifications as versioned evidence whose dataset, hardware, configuration, correctness predicate, and repository state are recorded together.

17.6. Closing perspective

BioSeqDB's most important result is architectural rather than a single speedup. A sequence database cannot obtain trustworthy high performance by placing an existing executable behind an SQL function. It must understand the representations, execution granularity, persistent state, and failure semantics that make the executable fast and correct. Conversely, an aligner embedded in a database cannot ignore planning, transactions, row multiplicity, memory contexts, and schema ownership.

The final system brings these concerns into one inspectable pipeline. It keeps the successful algorithmic structure of minimap2, replaces process-local assumptions with explicit sequence and index abstractions, adds measured SIMD and Vulkan routes, and gives PostgreSQL a planner-visible batch executor. The same emphasis on explicit ownership extends to IO, external providers, cache coordination, and generated SQL boundaries. The result is not the end of the engineering path. It is a sufficiently complete platform and a sufficiently candid record of its intermediate results to support the next round of scientific and systems work.